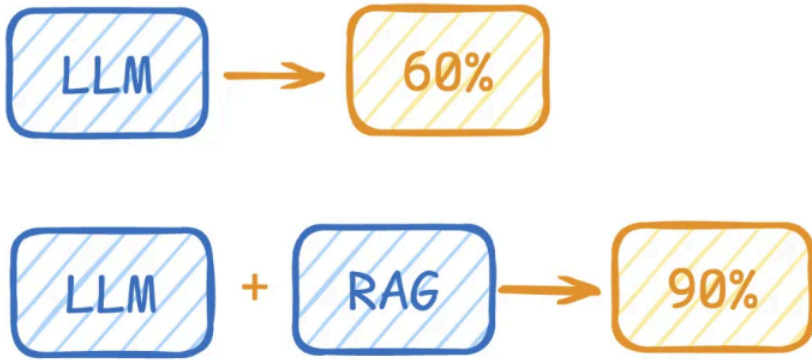


1. 项目简介

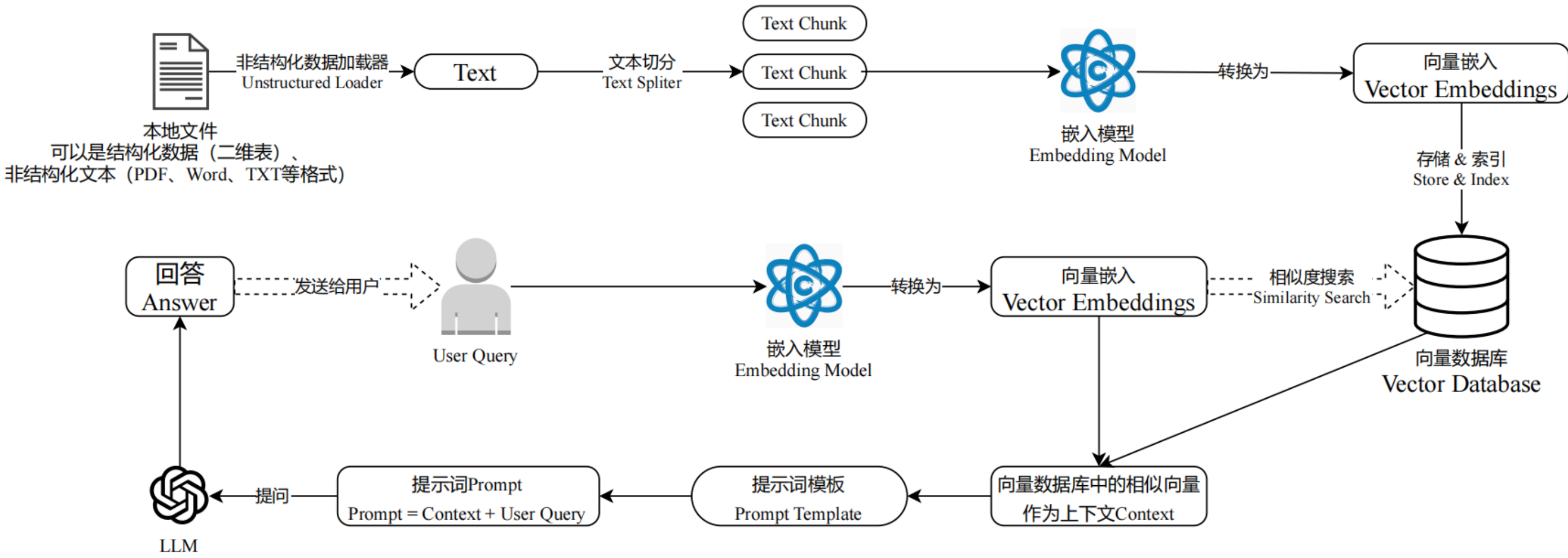
1.1 RAG检索增强生成 (Retrieval-Augmented Generation) 作用

LLM (大语言模型) 在面对训练数据之外的陌生领域知识时，容易产生“幻觉” (Hallucination)，导致回答不准确。RAG 技术通过外挂知识库的方式，在 LLM 生成回答前检索相关信息作为“提示” (Context)，引导 LLM 基于事实回答，从而显著提升回答的准确率和可信度。

RAG核心定义：通过检索外部知识库增强大语言模型的生成能力。



RAG 核心流程：



大模型在 RAG 中的关键应用：

- **Embedding (向量化)**：使用 Embedding 模型将文本转化为向量 (支持稠密和稀疏向量)，用于后续的相似度检索。
- **Rerank (重排序)**：使用 Rerank 模型对初步召回的切片 (如 Top 50) 进行精细排序，过滤无关信息，提升上下文的相关性。
- **LLM (生成)**：使用大语言模型基于检索到的上下文和用户问题生成最终答案。

RAG 核心场景：**企业知识库问答、智能客服、法律 / 金融 / 政务专业咨询、文档助手、教育答疑、搜索增强**

1.2 项目介绍

掌柜智库 (RAG) 是一款基于 **检索增强生成 (Retrieval-Augmented Generation)** 技术的企业级智能问答系统。本项目致力于构建一套集“私有知识库精准问答、实时联网信息补充、多维度结果优化”于一体的全流程智能客服解决方案，旨在实现核心知识的私有化管理、问答结果的高精准度以及业务场景的灵活适配。代码总行数： 5507

系统架构包含两大核心模块：

1. **数据处理流水线（准备）**：支持 PDF/Markdown 等多格式文档导入，执行文档结构化解析、智能切片、元数据提取及向量化存储的全链路预处理。（为智能客服提供数据支撑）

知识库文件导入



点击或拖拽文件到此处 (PDF/MD)

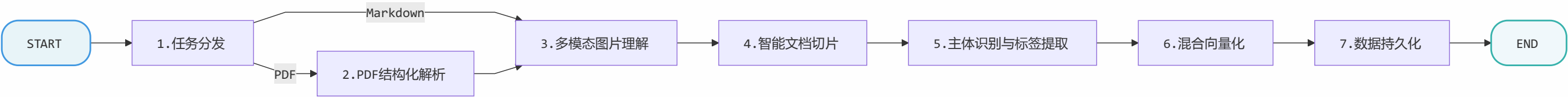
2. **智能检索系统 (查询)**: 集成混合检索 (稠密向量 + 稀疏向量)、假设性问题生成 (HyDE)、MCP 联网搜索及结果重排序 (Rerank) 等高级策略, 确保回答的准确性与时效性。



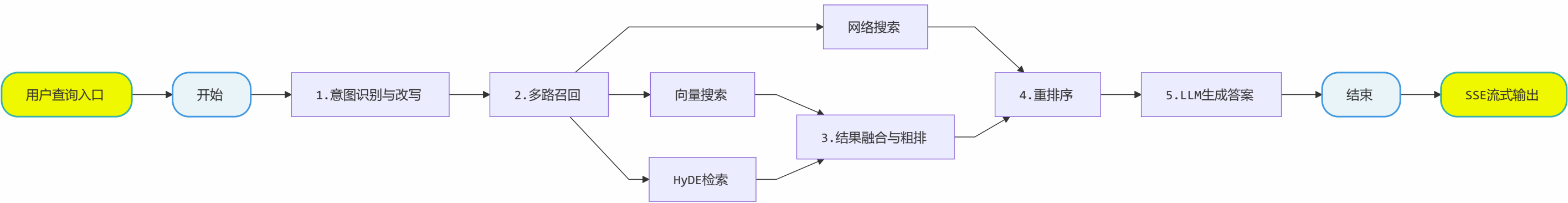
1.2 项目核心技术&流程

1.2.1 项目标准流程总结

■ **知识库构建 (Indexing)**: 原始数据 -> 文本解析 -> 文本切片 (Chunking) -> 向量化 (Embedding) -> 存入向量数据库。



■ **知识库检索 (Retrieval & Generation)**: 用户提问 -> 问题向量化 -> 多路检索相关切片 -> 重排序 (rrf+Rerank) -> 提示词增强 (Prompt Engineering) -> LLM 生成答案。



1.2.2 项目（RAG）确保准确率核心点

RAG 的准确率问题拆解为六个可工程化能力层：
数据结构化 → 语义切片 → 多路召回 → 混合检索 → 假设增强 → 精排裁决。
每一层都对应明确的误差来源与治理策略，形成端到端闭环。

1.2.2.1 非结构化文档语义重建(PDF)&图片识别

PDF 是排版格式，不是语义格式。直接抽取文本会出现结构坍塌、图文割裂、上下文断链。

专家级治理方案

- 版面语义重建：基于 MinerU 等引擎恢复标题层级、段落边界、表格与公式结构，输出语义化 Markdown。
- 图文联合解析：对图片/图表执行 OCR + VLM 双通路理解，将视觉知识转化为可检索文本与向量。
- 结构一致性校验：对标题树、段落归属、图文引用关系进行规则化校验，降低语义漂移。

将“展示型文档”升级为“可计算知识资产”，从源头提升召回上限与答案可解释性。

1.2.2.2 语义自治切片

固定长度切片会打断论证链路，导致“召回命中但证据不完整”。

切片级治理方案

- 按语义边界切分：优先使用章节、段落、主题变化等自然边界。
 - 动态长短控制：长块二次切分、短块同域合并，避免信息稀释与碎片噪声。
 - 上下文连续性增强：引入适度 overlap (重叠10~20%)与父标题继承，保持证据链完整。
- 技术价值

让每个 Chunk 成为“最小可解释知识单元”，显著提升检索相关性与生成连贯度。

1.2.2.3 多路召回引擎

RAG 多路召回（Multi-Path Retrieval） 是当前检索增强生成（RAG）系统中提升检索准确率（Recall）和鲁棒性的核心策略。

它的核心思想是：不依赖单一的检索方式，而是并行使用多种不同的检索策略，最后将结果合并。

单纯依靠向量检索（Dense Retrieval）已被证明存在盲区，因此“多路召回 + 融合重排序”已成为标准架构。



1.2.2.4 混合向量检索

核心痛点: 语义相关 ≠ 词项命中，关键词命中 ≠ 语义正确，单一检索无法兼顾。

专家级治理方案

- Dense Vector：负责语义理解、同义泛化、隐含关联。
- Sparse Vector：负责术语命中、实体精确、关键字段约束。
- 统一评分空间融合：通过归一化与权重机制消除量纲偏差，避免单路分数“劫持”结果。

实现“语义广覆盖 + 关键词高精度”的双重最优。

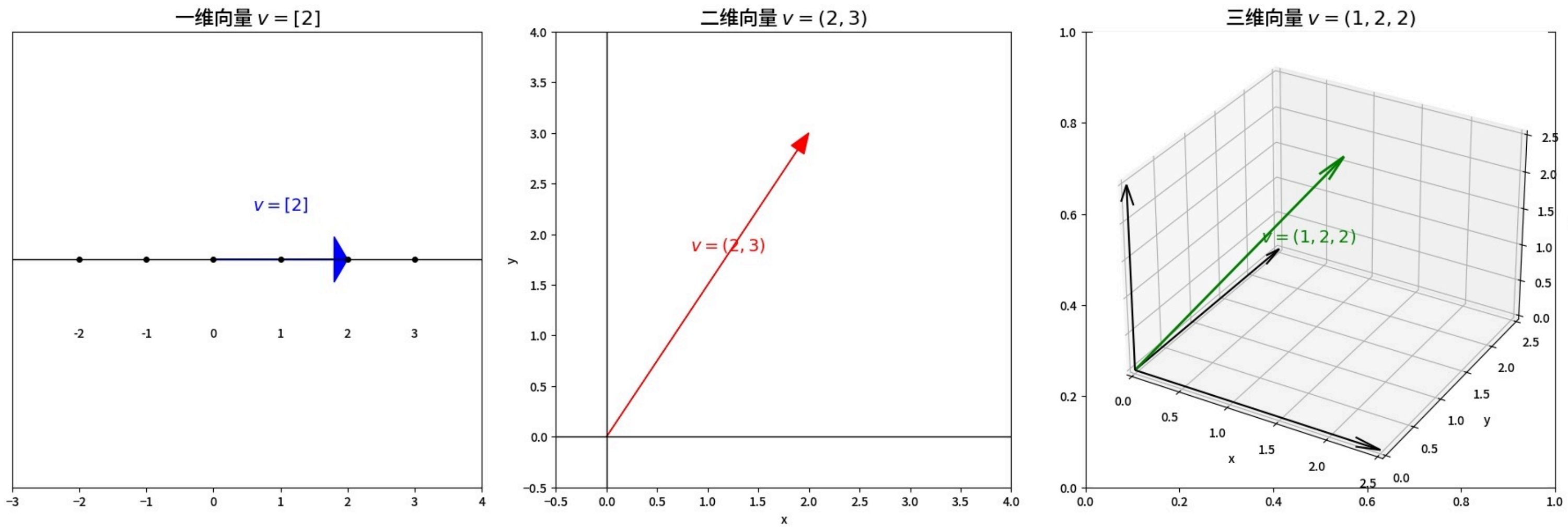
向量了解

向量是**既有大小、又有方向**的一组数字，可用来**量化描述事物特征**。

文本、图片、语音、用户行为等信息，都可以被转换成向量。

在向量空间中，**两个向量越接近，代表它们对应的内容越相似**。

注意：数学领域表示向量常用小括号，编程领域常用中括号



稠密向量(语义向量)

作用：用于表示**语义、含义、内容特征**，侧重“懂意思”。【1204】

特点：

- 向量中几乎没有 0，数值分布**稠密、饱满**
- 维度通常在几十到几千维

典型场景：

- 大模型生成的 Embedding（词嵌入 / 句嵌入）
- 语义搜索、智能推荐、相似度计算、内容理解

稀疏向量(统计向量)

作用：用于表示**词频、计数、关键词是否出现**，侧重“查字面”。

特点：

- 向量中**绝大多数是 0**，只保留少数非 0 数值
- 维度可以非常大（几万、几十万甚至更高）

典型场景：

- 关键词匹配、精确检索
- 词袋模型、TF-IDF、传统搜索召回

混合向量检索 = 稀疏 + 稠密协同使用

- **稀疏向量**：负责**快速、精准召回**，保证关键词匹配不遗漏
- **稠密向量**：负责**语义理解与泛化匹配**，能识别同义、近义、相关内容
- 结合优势：既保留关键词搜索的准确性，又具备语义搜索的全面性，让搜索结果又准、又全、更懂用户意图。

1.2.2.5 重排序裁决层

将“可用候选”升级为“高置信证据”，直接决定最终答案质量上限。

在 RAG 系统中，结果重排序（Rerank / 重排）是指：对初步检索到的候选文档（或文本片段）进行第二次、更精细的相关性排序，以选出最相关、最有用的内容输入给大语言模型



为什么需要重排序？

RAG 的基本流程是：

1. 用户提问
2. 系统从知识库（如向量数据库）中**快速检索**一批可能相关的文档（比如 Top-50）
3. 将这些文档拼接成上下文，交给 LLM 生成答案。

但问题在于：第一步的“快速检索”通常**不够精准**。原因包括：

- 初步检索追求高召回率，宁可多召回一些，也不漏掉，但会混入低相关性的噪声；
- LLM 的上下文窗口有限，不能把所有召回结果都塞进去，必须优中选优。

因此，需要对初步结果重新打分、排序——这就是 **重排序**。

2. 模块流程设计

2.1 导入核心业务流程

2.1.1 设计目标

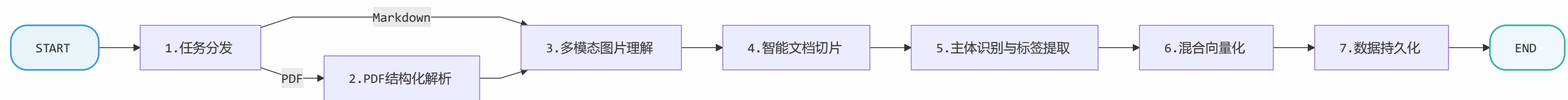
本模块旨在构建一套高效的数据处理流水线，将非结构化文档 (PDF, Markdown) 转化为计算机可处理、AI 可理解的结构化知识单元。重点解决以下技术痛点：

- 复杂文档解析**：针对 PDF 文档排版丢失、图文分离等问题，采用高精度解析工具完整保留文档层级结构 (标题、章节) 并关联图片上下文，确保信息无损还原。
- 语义完整性增强**：解决传统文本切片导致的“上下文缺失”问题。通过结构化处理，为每个切片附加元数据 (如产品名称、层级标题)，将孤立文本转化为具备独立语义的知识单元。

处理后的数据将经过向量化编码，存入 **Milvus 向量数据库**，为后续的 RAG 检索提供高保真、高关联的数据基座。

2.1.2 处理链路详解

本系统采用 **LangGraph** 进行流水线编排，通过状态机管理数据流转。主要处理节点如下：



1. Step 1: 任务分发 (node_entry)

- 功能**：作为流水线入口，依据文件类型决定处理路径。
- 逻辑**：PDF 文件路由至解析节点 (node_pdf_to_md)；Markdown 文件直接路由至图片处理节点 (node_md_img)。

2. Step 2: PDF 结构化解析 (node_pdf_to_md)

- 背景**：PDF 格式缺乏语义结构，直接提取文本易造成信息混淆。
- 方案**：集成 **MinerU (Magic-PDF)** 工具，精准识别文档布局，将 PDF 转换为保留多级标题及表格结构的 Markdown 格式，并提取图片占位符。

3. Step 3: 多模态图片理解 (node_md_img)

- 背景**：技术文档中的图表往往蕴含关键信息，需转化为文本以支持检索。
- 方案**：将 Markdown 中的图片上传至 MinIO 对象存储，并调用 **多模态大模型 (VLM)** 进行图像语义分析，生成详细的文本描述 (如“电路结构图，左侧标识为电源模块...”)，替换原始图片链接。
- 转换**：![原图名](本地路径) -> ![VL模型描述](MinIO URL)

4. Step 4: 智能文档切片 (node_document_split)

- 背景**：为适配 LLM 上下文窗口限制，需将长文档拆分为细粒度文本块 (Chunk)。
- 方案**：采用层级切分策略，优先基于**段落**进行语义切分，超长段落降级为**句子**切分。
- 上下文保留**：在每个 Chunk 前拼接所属的**标题路径** (如“【操作手册-故障排查-电源故障】...”)，消除切片歧义。

5. Step 5: 主体识别与标签提取 (node_item_name_recognition)

- 背景**：文档切片常省略主语 (如“重量为 200g”)，导致跨文档检索时指代不明。
- 方案**：提取文档头部关键信息输入 LLM，自动识别文档描述的主体对象 (如“iPhone 17 Pro Max”)，并将该 **Item Name** 作为全局元数据附加至该文档的所有切片。

6. Step 6: 混合向量化 (node_bge_embedding)

- 背景**：将自然语言转化为机器可计算的高维向量。
- 方案**：采用 **BGE-M3** 模型，将 item_name 与 content 拼接后进行编码。
- 能力**：同时生成 **稠密向量 (Dense Vector)** (用于语义模糊匹配) 和 **稀疏向量 (Sparse Vector)** (用于关键词精确匹配)，支持混合检索。

7. Step 7: 数据持久化 (node_import_milvus)

- 功能**：将结构化数据写入 **Milvus** 向量数据库。

- **Schema 设计:**
 - chunk_id (Int64): 全局唯一标识
 - content (String): 文本切片内容
 - title (String): 完整标题路径
 - file_title (String): 原始文件名
 - item_name (String): 实体/商品名称
 - dense_vector (Float Vector): 语义向量
 - sparse_vector (Sparse Float Vector): 稀疏关键词向量
 - parent_title, part: 辅助层级信息

2.1.3 核心技术栈

- **LangGraph:** 复杂的流程编排与状态管理。
- **MinerU:** 业界领先的高精度 PDF 解析工具。
- **BGE-M3:** 支持长文本、多语言及混合检索的旗舰级 Embedding 模型。
- **Milvus:** 高性能云原生向量数据库，支持海量数据低延迟检索。
- **Minio:** 图片文件云存储，避免切换md文字导致图片失效

2.1.4 向量数据库设计和说明

- **文档级索引 (kb_item_names)**

存储单个文档的基本信息，包含文档的文件名称（file_title）、文档描述的设备名称（item_name）以及设备名称对应的稠密和稀疏向量！

字段	类型
pk  auto id	Int64
file_title	VarChar(65535)
item_name	VarChar(65535)
dense_vector	FloatVector(1024)
sparse_vector	SparseFloatVector

- **切片级索引 (kb_chunks)**

存储文档中具体切片内容，主要思路根据段落进行切片，但是段落数据过大（添加max文本大小限制），还会进行二次切片！最终将切面存储到 kb_chunks集合中

字段	类型	索引名称
chunk_id  auto id	Int64	
content	VarChar(65535)	
title	VarChar(65535)	
parent_title	VarChar(65535)	
part	Int8	
file_title	VarChar(65535)	
item_name	VarChar(65535)	
sparse_vector	SparseFloatVector	sparse_vector_index
dense_vector	FloatVector(1024)	dense_vector_index

2.2 检索核心业务流程

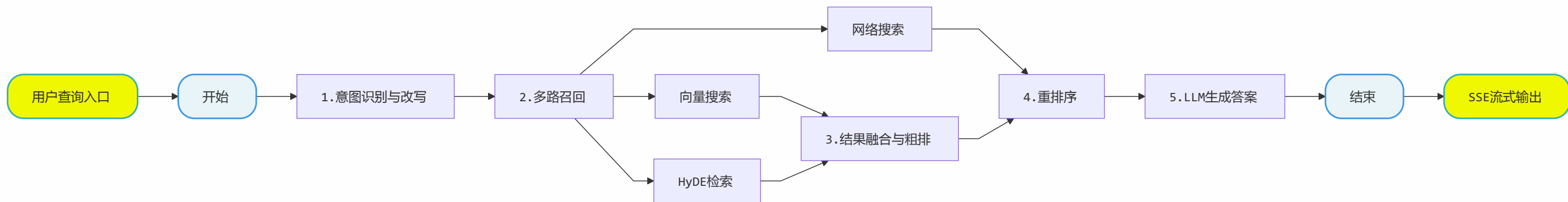
2.2.1 设计目标

本模块旨在构建一套高精度、低延迟的智能问答检索流水线，将用户的自然语言问题转化为精准的答案。重点解决以下技术痛点：

- 意图理解偏差**：针对用户提问模糊、指代不明（如“它多少钱”）的问题，通过多轮对话上下文和实体识别技术以及重写提问，精准还原用户真实意图。
- 召回率与准确率平衡**：单一的向量检索难以应对复杂语义，通过**多路召回（Multi-path Retrieval）**策略，结合语义检索、关键词匹配和网络搜索，确保关键信息不遗漏。
- 答案幻觉抑制**：通过**重排序（Rerank）**机制剔除无关噪声文档，并利用高质量的上下文提示工程，最大程度减少幻觉。

2.2.2 处理链路详解

本系统采用 **LangGraph** 进行检索流水线的编排，按照 **意图理解 -> 多路召回 -> 排序融合 -> 答案生成** 的流水线进行处理。主要处理节点如下：



1. Step 1: 意图识别与改写 (Item Name Confirm)

- 背景**：用户提问往往口语化且缺乏关键实体（如“这款手机续航多久？”），直接检索效果差。
- 方案**：调用 LLM 分析历史对话上下文，提取或补全关键实体名称（Item Name），并将问题改写为更适合检索的陈述句。

2. Step 2: 多路召回 (Multi-path Retrieval)

- 背景**：单一检索方式存在盲区，例如向量检索对专有名词不敏感，关键词检索无法理解语义。
- 方案**：并发执行多种检索策略：
 - 向量检索 (Vector Search)**：基于 BGE-M3 模型计算语义相似度，检索 Milvus 中的文档切片。
 - 假设性文档嵌入 (HyDE)**：利用 LLM 生成“虚构答案”，将其向量化后进行检索，提升对隐式意图的召回能力。
 - 网络搜索 (Web Search)**：通过搜索引擎获取外部信息。

3. Step 3: 结果融合与粗排 (Join & RRF)

- 背景**：多路召回返回的结果分数标准不一（如距离分 vs 匹配度），无法直接比较。
- 方案**：采用 **倒排秩融合 (RRF)** 算法，仅依据排名进行加权融合，生成统一的候选文档列表，去重并保留 Top-N。

4. Step 4: 精准重排序 (Rerank)

- 背景**：召回阶段为了覆盖率通常会引入部分噪声文档，直接输入 LLM 会消耗 Token 并引发幻觉。
- 方案**：引入高精度的 **Cross-Encoder** 模型（如 BGE-Reranker），对“问题-文档”对进行深度语义打分，只保留相关性最高的 Top-K 文档（如 Top 5）。

5. Step 5: 答案生成 (Answer Generation)

- 方案**：将精选的 Top-K 文档片段作为上下文（Context），配合精心设计的 Prompt 模板输入给大模型（LLM）。
- 流式输出**：通过 Server-Sent Events (SSE) 技术，将 LLM 生成的答案逐字实时推送到前端，提供丝滑的交互体验。

2.2.3 核心技术栈

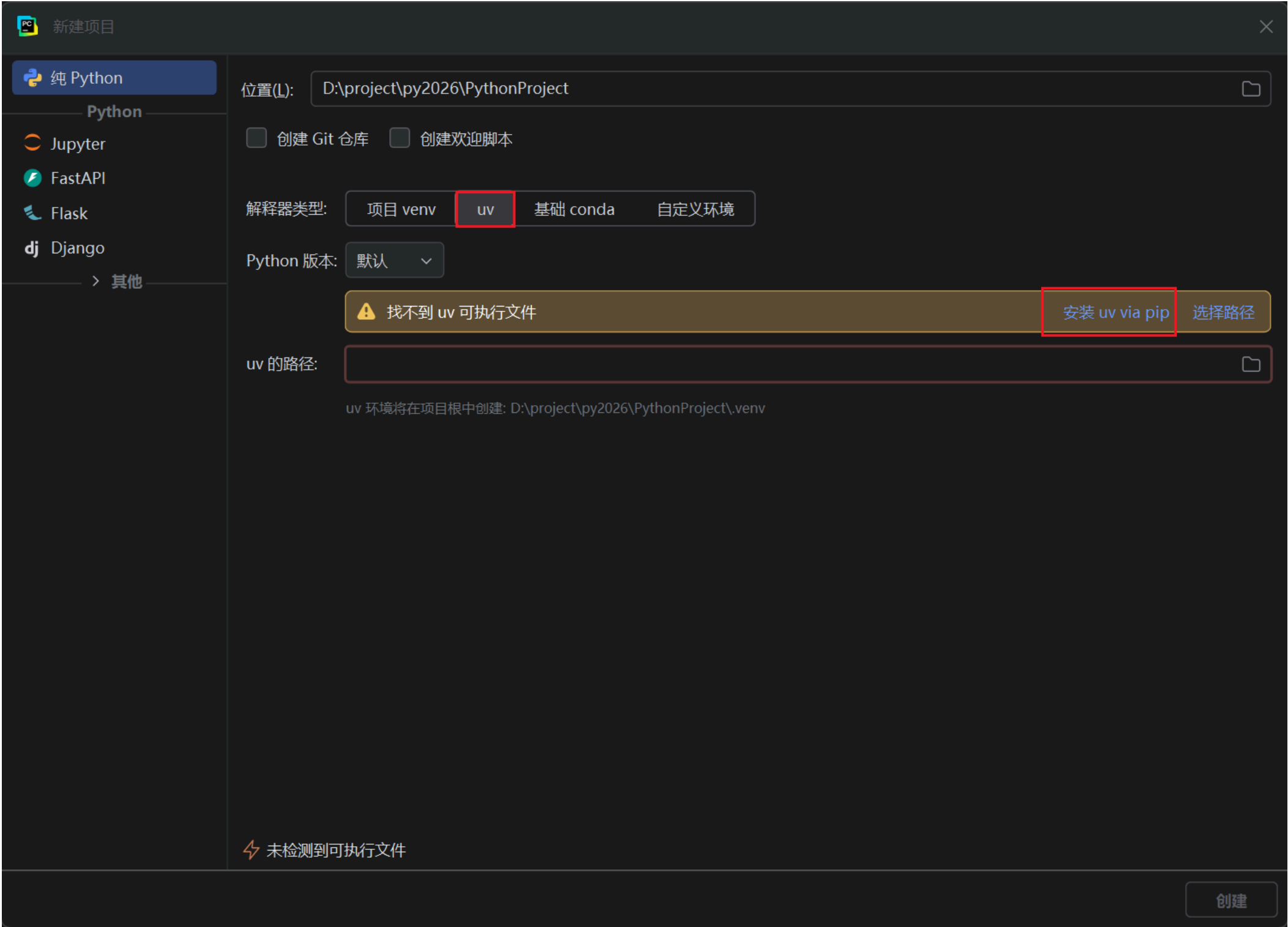
- LangGraph**: 编排复杂的检索与生成流程（RAG Pipeline）。
- Milvus**: 承载海量文档切片的向量检索。
- BGE-Reranker**: 高性能重排序模型，显著提升 RAG 系统的最终准确率。
- LLM (Qwen)**: 负责意图理解、HyDE 生成及最终答案合成。

3. 环境准备

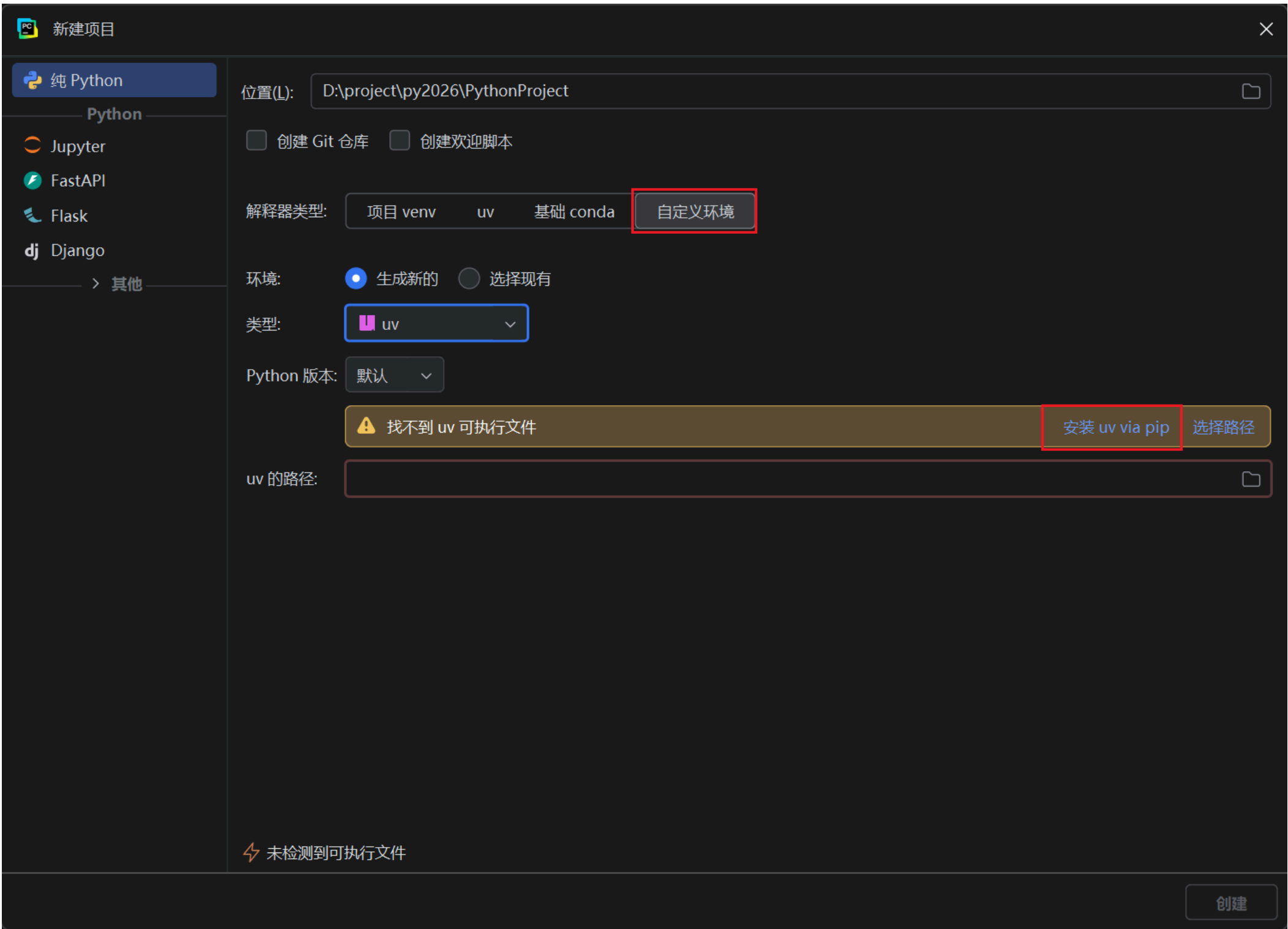
3.1 虚拟环境创建

3.1.1 安装uv

使用 uv 方式创建虚拟环境，如果之前用其他方式安装过 uv 则此处会自动识别出 uv 路径，如果没安装过 uv 直接点击 安装 uv via pip



PyCharm版本没有 uv 选项，则选择 自定义环境，类型选 uv，然后再点击 安装 uv via pip

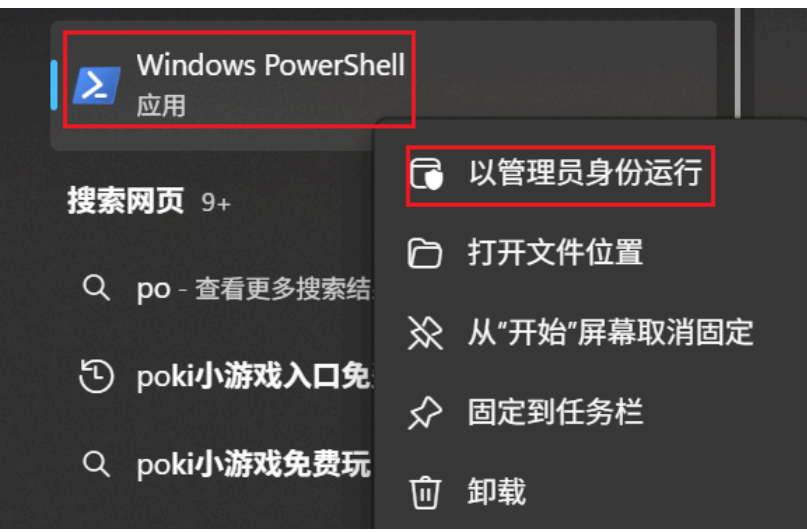


安装完uv 后将 uv 路径配置在系统的 Path 环境变量中：例如我的路径是 C:\Users\用户名\AppData\Roaming\Python\Scripts

3.1.2 配置镜像源

为uv命令配置国内镜像源。

以管理员身份运行Windows PowerShell



如果运行报如下错误：

```
. : 无法加载文件 C:\Users\55317\Documents\WindowsPowerShell\profile.ps1，因为在此系统上禁止运行脚本。有关详细信息，请参阅 https:/go.microsoft.com/fwlink/?LinkID=135170 中的 about_Execution_Policies。
所在位置 行:1 字符: 3
+ . 'C:\Users\55317\Documents\WindowsPowerShell\profile.ps1'
+
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

则在 Windows PowerShell 中设置安全策略允许本地脚本执行，方式如下：

```
1 Set-ExecutionPolicy Unrestricted -Scope CurrentUser
```

然后执行以下命令创建uv配置文件

```
1 # 在用户目录创建文件夹
2 mkdir -Force $env:APPDATA\uv
3 # 在文件夹中创建文件
4 New-Item -Path $env:APPDATA\uv\uv.toml -ItemType File
```

编辑 uv.toml 文件，添加以下内容

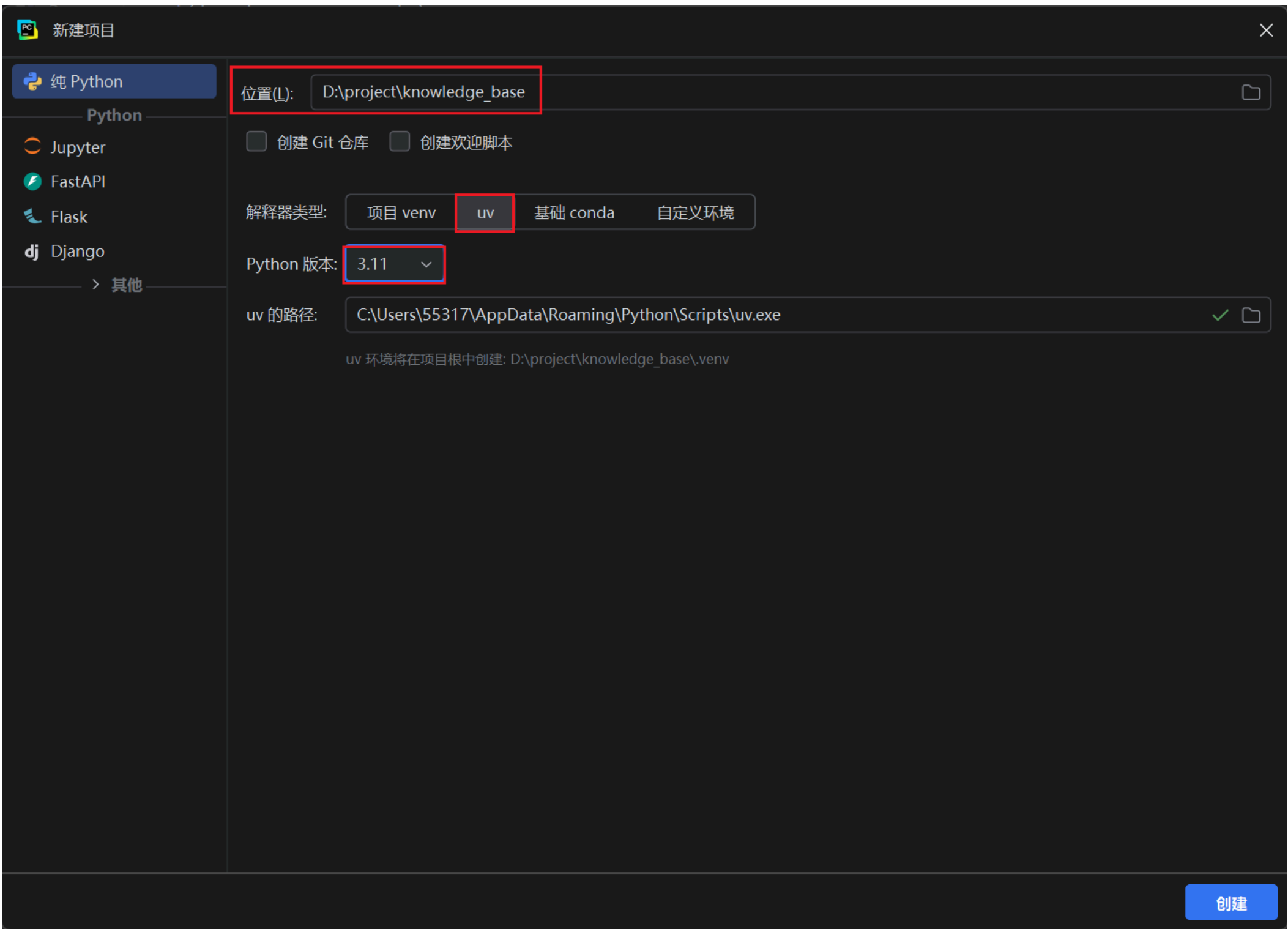
```
1 # 清华镜像（推荐）
2 index-url = "https://pypi.tuna.tsinghua.edu.cn/simple"
3 # 阿里镜像
4 # index-url = "https://mirrors.aliyun.com/pypi/simple/"
5
6 # 可选：添加信任源（避免安装报错）
7 # trusted-host = ["pypi.tuna.tsinghua.edu.cn"]
```

3.1.3 安装指定版本的Python解释器

打开命令行终端，执行以下命令安装 python 3.11

```
1 # 列出已安装版本
2 uv python list
3
4 uv python install 3.11
```

回到 PyCharm 创建项目的页面，选择 Python版本为 3.11，定义项目名称为 knowledge_base 然后创建项目



项目创建成功后，根目录下自动生成一个 `pyproject.toml` 文件。在 `uv` 虚拟环境中，`pyproject.toml` 是**项目依赖和配置的核心文件**，相当于项目的“身份卡 + 依赖清单”，`uv` 会通过这个文件统一管理项目的 Python 版本、依赖包、构建规则等，是 `uv` 实现“跨环境一致性”的关键。

3.1.4 激活当前虚拟环境

打开 PyCharm**的终端**，当前虚拟环境会自动被激活



3.2 基础结构搭建

3.2.1 配置文件导入

3.2.1.1 在项目的跟目录中放入以下配置文件

资料

- `.env` 文件（不提交到Git，配置apikey）
- `.env.example` 文件（提交到Git，不要配置正确的 `apikey`）

3.2.1.2 测试环境变量的读取

安装依赖（环境变量加载器）

```
1 uv add python-dotenv
2 或
3 pip uv install python-dotenv
```

使用 `uv add` 或 `uv pip install` 都以可为当前虚拟环境安装依赖。区别如下

- 使用 `uv add` 安装的依赖会自动添加到 `pyproject.toml` 中，但是不能在命令中直接使用 `--index-url` 参数指定镜像源，只能通过前面的`uv.toml` 文件指定
- 使用 `uv pip install` 则不会将依赖添加到 `pyproject.toml` 中，但是可以在命令中直接使用 `--index-url` 参数指定镜像源

测试环境变量的读取

```
1 # test/01-env和系统环境变量的优先级.py
2
3 import os
4 from dotenv import load_dotenv
5
6 # 加载.env文件
7 load_dotenv(override=True)
8
9 print(os.getenv("OPENAI_API_KEY"))
10
11 # 实际默认逻辑：override=False
12 # - 如果系统环境变量不存在 → 用.env里的值
13 # - 如果系统环境变量已存在 → 系统变量优先级更高
14 # 想让.env覆盖系统变量，需显式传 override=True
15 # load_dotenv(override=True)
16
17 # 示例：假设系统有环境变量 MY_KEY=system_val，.env里 MY_KEY=dotenv_val
18 print(os.getenv("MY_KEY"))
19 # load_dotenv() → 输出 system_val（系统优先级高）
20 # load_dotenv(override=True) → 输出 dotenv_val（.env覆盖系统）
```

3.2.2 统一日志

3.2.2.1 安装依赖

loguru 是 Python 中一款**开箱即用、功能强大且极简的日志工具**，核心作用是替代 Python 标准库的 logging 模块，解决原生日志配置繁琐、用法不友好的问题。我用新手能看懂的方式讲清楚它的核心作用和优势：

```
1 uv add loguru
2 或
3 pip uv install loguru
```

3.2.2.2 日志模块代码

资料
app/core/logger.py

3.2.2.3 测试日志记录

```
1 """
2 项目日志工具类
3 基于loguru实现，支持.env配置控制台/文件双输出，自动生成logs/app_年月日.log
4 特性：
5 1. 配置驱动：通过.env开关输出、修改日志级别
6 2. 自动路径：文件日志默认输出到 项目根/logs/app_YYYYMMDD.log
7 3. 自动清理：按配置保留日志，自动删除过期文件
8 4. 中文友好：utf-8编码，彻底解决中文乱码
9 5. 异步安全：开启异步入队，支持多线程/异步场景，避免日志错乱
10 6. 开箱即用：项目所有模块直接导入logger即可使用
11 7. 位置终极精准：穿透loguru内部+工具类自身，完美显示业务模块实际调用位置
12 """
13 import sys
14 import inspect
15 from pathlib import Path
16 import os
17 from dotenv import load_dotenv
18 from loguru import logger
19
20
21 # ----- 第一步：加载.env配置文件 -----
22 load_dotenv()
23
24 # ----- 第二步：读取.env配置（带默认值，防止配置缺失） -----
25 LOG_CONSOLE_ENABLE = os.getenv("LOG_CONSOLE_ENABLE", "True").lower() == "true"
26 LOG_CONSOLE_LEVEL = os.getenv("LOG_CONSOLE_LEVEL", "INFO").upper()
27 LOG_FILE_ENABLE = os.getenv("LOG_FILE_ENABLE", "True").lower() == "true"
28 LOG_FILE_LEVEL = os.getenv("LOG_FILE_LEVEL", "INFO").upper()
29 LOG_FILE_RETENTION = os.getenv("LOG_FILE_RETENTION", "7 days")
```

```
30
31 # ----- 第三步：定义日志路径（自动推导项目根） -----
32 PROJECT_ROOT = Path(__file__).resolve().parent.parent.parent
33 LOG_DIR = PROJECT_ROOT / "logs"
34 LOG_FILE_NAME = "app_{time:YYYYMMDD}.log"
35 LOG_FILE_PATH = LOG_DIR / LOG_FILE_NAME
36
37 # ----- 第四步：定义日志格式（彩色、结构化、易读） -----
38 LOG_FORMAT = (
39     "<green>{time:YYYY-MM-DD HH:mm:ss.SSS}</green> | "
40     "<level>{level: <8}</level> | "
41     "<cyan>{name: <20}</cyan>:<cyan>{function: <15}</cyan>:<cyan>{line: <4}</cyan> - "
42     "<level>{message}</level>"
43 )
44
45 # ----- 第五步：初始化日志配置（核心方法） -----
46 def init_logger():
47     """
48     初始化全局日志配置
49     1. 移除loguru默认控制台输出（避免重复打印）
50     2. 根据.env配置开启/关闭控制台输出
51     3. 根据.env配置开启/关闭文件输出（自动创建logs文件夹）
52     4. 配置日志格式、级别、分割、保留策略
53     :return: 配置完成的loguru logger实例
54     """
55     # 1. 移除loguru默认的控制台输出
56     logger.remove()
57
58     # 2. 配置控制台输出（若.env开启）
59     if LOG_CONSOLE_ENABLE:
60         logger.add(
61             sink=sys.stdout,
62             level=LOG_CONSOLE_LEVEL,
63             format=LOG_FORMAT,
64             colorize=True,
65             enqueue=True
66         )
67
68     # 3. 配置文件输出（若.env开启）
69     if LOG_FILE_ENABLE:
70         LOG_DIR.mkdir(parents=True, exist_ok=True)
71         logger.add(
72             sink=LOG_FILE_PATH,
73             level=LOG_FILE_LEVEL,
74             format=LOG_FORMAT,
75             rotation="00:00",
76             retention=LOG_FILE_RETENTION,
77             encoding="utf-8",
78             enqueue=True,
79             backtrace=True,
80             diagnose=True
81         )
82
83     return logger
84
85 # ----- 第六步：初始化并终极修正全局logger -----
86 base_logger = init_logger()
87
88 def fix_log_position(record):
89     """遍历调用栈，跳过loguru内部帧+工具类自身帧，提取业务代码实际调用位置"""
90     for frame in inspect.stack():
91         # 终极过滤：排除loguru内部 + 排除工具类logger.py自身，直接定位业务模块
92         if ("_logger.py" in frame.filename or frame.function == "_log") or "logger.py" in frame.filename:
93             continue
94         # 更新日志字段为业务代码实际位置
95         record.update(
96             name=frame.filename.split("/")[-1].split("\\")[-1],
97             function=frame.function,
98             line=frame.lineno
99         )
100     break
101
```

```
102 # 应用终极修复，导出全局可用的logger
103 logger = base_logger.patch(fix_log_position)
104
105 # ----- 测试代码（验证修复效果） -----
106 if __name__ == '__main__':
107     logger.info("【测试】logger.py内部调用（仅测试，业务模块调用会显示正确文件名）")
108     print(f"日志文件输出路径：{LOG_FILE_PATH}")
```

修改.env文件中的日志配置进行测试，例如 LEVEL、ENABLE

```
1 # ===== 日志配置 =====
2 # 控制台日志：True=开启/False=关闭，级别可选(DEBUG/INFO/WARNING/ERROR/CRITICAL)
3 LOG_CONSOLE_ENABLE=True
4 LOG_CONSOLE_LEVEL=INFO
5
6 # 文件日志：True=开启/False=关闭，级别与控制台可独立设置
7 LOG_FILE_ENABLE=True
8 LOG_FILE_LEVEL=INFO
9 # 日志文件保留天数（自动删除过期日志，避免磁盘占满）
10 LOG_FILE_RETENTION=7 days
```

3.2.4 准备提示词和导入工具

资料
prompts
app/core/load_prompt.py

3.2.5 导入utils和tools

资料
app/tool
app/utils

3.2.6 大模型&客户端

资料
app/clients
app/conf``
``app/lm

3.2.6 PDF文件

资料
设备手册放入项目根目录doc文件夹

3.3 安装torch

torch：PyTorch 深度学习框架，为向量模型（BGE-M3）提供运行环境，支撑模型的加载、推理、张量计算，是大部分深度学习模型的基础依赖。

安装哪个版本：

- cu124 源
 - 给 PyTorch 装上“GPU 加速器”，只有你的电脑 / 服务器有 NVIDIA GPU，且装了对应驱动，才能用上这个加速器；如果没有 GPU，装了也没用（PyTorch 会自动降级到 CPU 运行）。
 - cu124 对应 CUDA 12.4，你的 NVIDIA 驱动版本需≥545.23（可通过 nvidia-smi 查看驱动版本）；
 - 如果驱动版本低（比如只支持 CUDA 12.1），需换 cu121 源（https://download.pytorch.org/whl/cu121），否则会报错。

显卡架构	算力版本	显卡系列	具体型号
Blackwell	sm_100	RTX 50 系列	RTX 5090、5080、5070、5060、5050、5060 Ti、5070 Ti、5080 Ti、5090 Ti

显卡架构	算力版本	显卡系列	具体型号
Ada Lovelace	sm_89	RTX 40 系列	RTX 4090、4080、4070、4070 Ti、4060、4060 Ti、4050、RTX 4000 SFF、RTX 5000、RTX 6000
Ampere	sm_86	RTX 30 系列	RTX 3090 Ti、3090、3080 Ti、3080、3070 Ti、3070、3060 Ti、3060、3050、RTX A2000、A3000、A4000、A5000、A6000
Turing	sm_75	RTX 20 系列	RTX 2080 Ti、2080 Super、2080、2070 Super、2070、2060 Super、2060、Titan RTX、RTX 5000、6000、8000
Turing	sm_75	GTX 16 系列	GTX 1660 Ti、1660 Super、1660、1650 Super、1650
Pascal	sm_61	GTX 10 系列	GTX 1080 Ti、1080、1070 Ti、1070、1060、1050 Ti、1050、Titan Xp、Titan X
Maxwell	sm_52	GTX 9 系列	GTX 980 Ti、980、970、960、950、Titan X

▪ **cpu 源**

- PyTorch 的“基础版”，不管有没有 GPU，都只能用 CPU 干活，优点是体积小、不挑环境，缺点是速度慢。

下载PyTorch：

正常情况下使用 `uv add` 就可以安装依赖，但是对于PyTorch来说最好去官网下载和计算机对应的版本，因为国内普通镜像源大概率是**CPU-only 版本**，可能没有适配最新 CUDA 的版本。

```
1 # uv add torch # PyTorch核心库：张量计算+GPU/TPU加速+深度学习模型构建
2
3 # 包含 CUDA 12.4 加速模块，支持 NVIDIA GPU 运算
4 uv pip install --force-reinstall torch --index-url https://download.pytorch.org/whl/cu124
5
6 # 使用 CPU
7 uv add torch torchvision torchaudio
```

验证 PyTorch 是否正常加载：

```
1 # test/03-cuda测试.py
2 try:
3     import torch
4     print(f"✅ PyTorch 加载成功！版本：{torch.__version__}")
5     print(f"✅ CUDA 状态：{torch.cuda.is_available()} ( CPU版显示False正常 ) ")
6     print(f"✅ CUDA 设备数：{torch.cuda.device_count()}")
7     print(f"✅ CUDA 设备名称：{torch.cuda.get_device_name(0)}")
8 except Exception as e:
9     print(f"❌ PyTorch 加载失败：{e}")
```

3.4 其他相关依赖

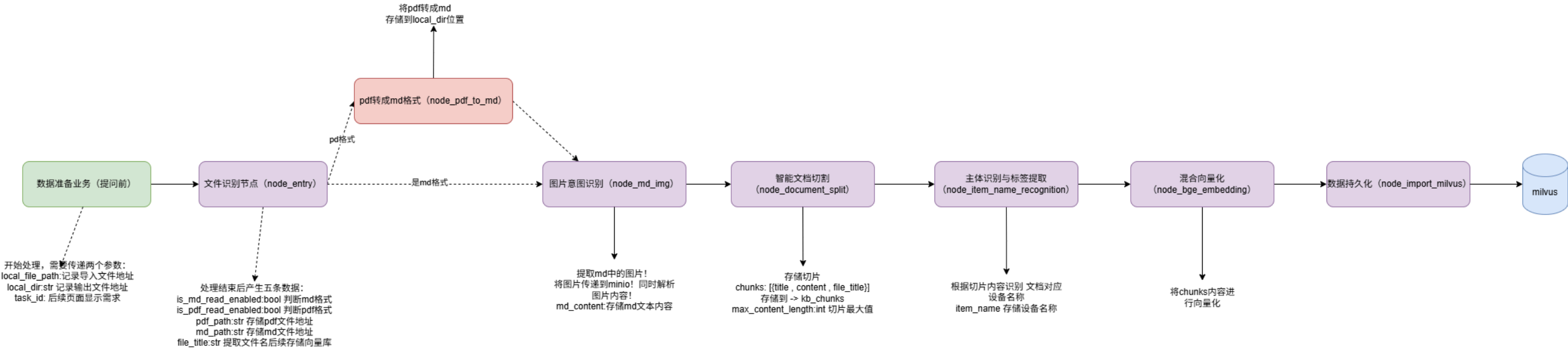
- langgraph**：LangChain 生态的工作流框架，用于搭建大模型应用的**任务流程编排**（如定义多步骤任务的执行顺序、分支逻辑、状态管理），**适配复杂的知识图谱 / 文档处理流程**。
- langchain**：大模型应用开发核心框架，封装了大模型调用、提示词管理、工具链整合、数据处理等通用能力，是连接业务代码与大模型的**核心中间层**。
- langchain_openai**：LangChain 对 OpenAI 系列模型的专属封装，兼容 OpenAI / 国产大模型（千问 / 即梦 AI 等）的 OpenAI 风格 API，**简化大模型客户端初始化与调用**。
- pymilvus**：Milvus 向量数据库的 Python 客户端，用于**向量的增删改查、数据库连接与操作**，是大模型应用中存储嵌入向量（Embedding）的核心依赖。
- minio**：MinIO 对象存储的 Python SDK，实现**文件（图片 / PDF / 文档）的上传、下载、删除、桶管理**，适配分布式文件存储场景。
- numpy**：Python 数值计算基础库，提供高性能数组操作，**支撑向量计算、数据预处理**等数值相关逻辑。
- pandas**：高效数据处理库，以 DataFrame 结构处理**结构化数据（如解析后的 PDF 文本、任务日志）**，支持数据筛选、清洗、转换。
- regex**：增强版正则表达式库，兼容 Python 原生 re 库，**支持更复杂的正则匹配**（如 Markdown 图片引用、PDF 文本提取后的内容匹配）。
- magic-pdf**：基于大模型的智能 PDF 解析库（MinerU 是其核心内核），**解决传统 PDF 解析的格式错乱、图片 / 表格提取失败、公式识别差等问题**，能精准提取 PDF 中的文本、图片、表格、公式，保留原文档排版结构，同时支持将解析结果输出为 Markdown/HTML 等易处理格式，是大模型应用中 PDF 文档预处理的核心工具。
- FlagEmbedding**：智源研究院开源的嵌入模型（Embedding）工具库，**封装了 BGE-M3 等优秀的向量模型**，一键实现文本 / 图片的向量化转换（将自然语言转为计算机可识别的向量），**适配检索增强生成（RAG）场景的向量检索需求**。

```
1 #uv add python-dotenv
2 #uv add loguru
3 #uv pip install --force-reinstall torch --index-url https://download.pytorch.org/whl/cu128
4
5 uv add numpy
6 uv add langgraph
7 uv add grandalf
8 uv add fastapi
9
10 uv add minio
11 #前面安装langgraph时已经关联安装的依赖
12 #uv add requests
13 #uv add pydantic
14
15 # LangChain & LangGraph
16 uv add langchain
17 uv add langchain_openai
18 uv add langchain_community
19
20 # Web Framework
21 uv add uvicorn
22 uv add python-multipart
23
24 # Database & Storage
25 #PyMilvus 团队在 2.4.0 版本后做了明确的包职责拆分，避免主包依赖冗余：
26 #pymilvus（主包）：仅包含 Milvus 向量数据库的核心操作（连接、增删改查、索引创建等）；
27 #pymilvus-model（独立包）：专门包含嵌入函数、模型封装、混合检索等功能（如BGEM3EmbeddingFunction、各类向量编码器均在此包中）。
28 uv add pymilvus
29 uv add pymilvus-model
30
31 uv add pymongo
32
33 # AI & Models
34 uv add pandas
35 uv add modelscope
36 uv add FlagEmbedding
37 uv add magic-pdf
38 uv add regex
39 -----
```

4. 导入数据图与状态定义

4.1 定义状态 (State)

所有节点共享同一个状态对象。我们需要定义它来存储处理过程中的数据（如 PDF 路径、MD 内容、切片列表、向量等）。



文件: app/import_process/agent/state.py

```
1 from typing import TypedDict
2 import copy
3 from app.core.logger import logger
4
5 class ImportGraphState(TypedDict):
6     """
7     图的状态定义，包含所有节点产生和消费的数据字段。
8     TypedDict 让我们在代码中能有自动补全和类型检查。
9     使用字典式访问（如state["session_id"]、state.get("embedding_chunks")）
```

```
10     """
11     task_id: str          # 任务唯一ID，用于追踪日志
12
13     # --- 流程控制标记 ---
14     is_md_read_enabled: bool    # 是否启用 Markdown 读取路径
15     is_pdf_read_enabled: bool   # 是否启用 PDF 读取路径
16
17
18     # --- 切块相关 --- 【没用】
19     is_normal_split_enabled: bool
20     is_silicon_flow_api_enabled: bool
21     is_advanced_split_enabled: bool
22     is_vllm_enabled: bool
23
24     # --- 路径相关 ---
25     local_dir: str          # 当前工作目录或输出目录
26     local_file_path: str    # 原始输入文件路径
27     file_title: str         # 文件标题（文件名去后缀）
28     pdf_path: str           # PDF 文件路径（如果输入是PDF）
29     md_path: str            # Markdown 文件路径（转换后或直接输入的）
30     split_path: str         # 分块后的文件路径 【没用】
31     embeddings_path: str    # 向量数据库文件路径【没用】
32
33     # --- 内容数据 ---
34     md_content: str         # Markdown 的全文内容
35     chunks: list           # 切片后的文本列表，包含 metadata
36     item_name: str          # 识别出的主体名称（如："万用表"），用于增强检索
37
38     # --- 数据库相关 ---
39     embeddings_content: list # 包含向量数据的列表，准备写入 Milvus
40
41
42 # 建议定一个初始化对象，方便后续使用
43 # 定义图状态的默认初始值
44 graph_default_state: ImportGraphState = {
45     "task_id": "",
46     "is_pdf_read_enabled": False,
47     "is_md_read_enabled": False,
48     "is_normal_split_enabled": True,
49     "is_silicon_flow_api_enabled": True,
50     "is_advanced_split_enabled": False,
51     "is_vllm_enabled": False,
52     "local_dir": "",
53     "local_file_path": "",
54     "pdf_path": "",
55     "md_path": "",
56     "file_title": "",
57     "split_path": "",
58     "embeddings_path": "",
59     "md_content": "",
60     "chunks": [],
61     "item_name": "",
62     "embeddings_content": []
63 }
64
65 def create_default_state(**overrides) -> ImportGraphState:
66     """
67     创建默认状态，支持覆盖
68
69     Args:
70         **overrides: 要覆盖的字段（关键字参数解包）
71
72     Returns:
73         新的状态实例
74
75     Examples:
76         state = create_default_state(task_id="task_001", local_file_path="doc.pdf")
77     """
78
79     # 默认状态
80     state = copy.deepcopy(graph_default_state)
81     # 用 overrides 覆盖默认值
```

```
82     state.update(overrides)
83     # 返回创建好的状态字典实例
84     return state
85
86 def get_default_state() -> ImportGraphState:
87     """
88     返回一个新的状态实例，避免全局变量污染
89     """
90     return copy.deepcopy(graph_default_state)
91
92
93 if __name__ == "__main__":
94     """
95     测试
96     """
97     # 创建默认状态
98     state = create_default_state(local_file_path="万用表RS-12的使用.pdf")
99     logger.info(state)
```

4.2 定义主图

为了确保整体流程设计的科学性与执行连贯性，我们采用 "Top-Down"（自顶向下）的开发模式，以“总指挥部”的全局视角统筹推进，具体实施步骤如下：

- 搭建节点骨架（Stubs）**：优先定义全流程所需的所有功能节点，仅保留核心日志打印能力（如节点进入 / 退出日志），暂不实现内部复杂业务逻辑，快速搭建起流程的“骨架结构”；
- 串联主图（Graph）**：基于预设的业务流转规则，编写主图逻辑将所有节点骨架按序串联，明确节点间的输入输出关系、分支判断条件（如文件格式分流逻辑），形成完整的流程链路；
- 验证流程通畅性**：启动端到端测试，验证节点间的调用链路是否通顺、数据流转是否符合预期、分支跳转是否准确，确保流程无阻塞、无逻辑漏洞；
- 填充节点核心逻辑**：在流程链路验证通过后，再逐一聚焦每个节点的内部实现，完成复杂业务逻辑的开发（如 PDF 结构化转换、向量编码、重排序算法等），实现“骨架”到“完整系统”的落地。

该模式的核心优势在于：先保障“流程走得通”，再聚焦“功能做得好”，避免因局部逻辑复杂导致整体流程设计偏差，大幅提升开发效率与流程稳定性。

4.2.1 第一步：创建节点骨架

我们需要先创建以下 8 个文件，每个文件里只写一个最简单的“空函数”，确保主图能导入它们。
请在 `app/import_process/agent/nodes` 目录下创建以下文件，并将对应代码复制进去。

(1) 入口节点: `node_entry.py`

```
1  import sys
2
3  from app.core.logger import logger
4  from app.import_process.agent.state import ImportGraphState
5
6
7  def node_entry(state: ImportGraphState) -> ImportGraphState:
8      """
9      节点：入口节点 (node_entry)
10     为什么叫这个名字：作为图的 Entry Point，负责接收外部输入并决定流程走向。
11     未来要实现：
12     1. 接收文件路径。
13     2. 判断文件类型（PDF/MD）。
14     3. 设置 state 中的路由标记（is_pdf_read_enabled / is_md_read_enabled）。
15     """
16     logger.info(f">>> [Stub] 执行节点: {sys._getframe().f_code.co_name}")
17
18     # 模拟简单的路由逻辑，防止报错（仅 node_entry 需要）
19     if "local_file_path" in state:
20         path = state["local_file_path"]
21         if path.endswith(".pdf"):
22             state["is_pdf_read_enabled"] = True
23         elif path.endswith(".md"):
24             state["is_md_read_enabled"] = True
25
```

```
26 |         return state
```

(2) PDF转换节点: node_pdf_to_md.py

```
1 | import sys
2 |
3 | from app.core.logger import logger
4 | from app.import_process.agent.state import ImportGraphState
5 |
6 |
7 | def node_pdf_to_md(state: ImportGraphState) -> ImportGraphState:
8 |     """
9 |     节点：PDF转Markdown (node_pdf_to_md)
10 |    为什么叫这个名字：核心任务是将 PDF 非结构化数据转换为 Markdown 结构化数据。
11 |    未来要实现：
12 |    1. 调用 MinerU (magic-pdf) 工具。
13 |    2. 将 PDF 转换成 Markdown 格式。
14 |    3. 将结果保存到 state["md_content"]。
15 |    """
16 |    logger.info(f">>> [Stub] 执行节点: {sys._getframe().f_code.co_name}")
17 |    return state
```

(3) 图片处理节点: node_md_img.py

```
1 | import sys
2 |
3 | from app.core.logger import logger
4 | from app.import_process.agent.state import ImportGraphState
5 |
6 | def node_md_img(state: ImportGraphState) -> ImportGraphState:
7 |     """
8 |     节点：图片处理 (node_md_img)
9 |     为什么叫这个名字：处理 Markdown 中的图片资源 (Image)。
10 |    未来要实现：
11 |    1. 扫描 Markdown 中的图片链接。
12 |    2. 将图片上传到 MinIO 对象存储。
13 |    3. (可选) 调用多模态模型生成图片描述。
14 |    4. 替换 Markdown 中的图片链接为 MinIO URL。
15 |    """
16 |    logger.info(f">>> [Stub] 执行节点: {sys._getframe().f_code.co_name}")
17 |    return state
```

(4) 文档切分节点: node_document_split.py

```
1 | import sys
2 |
3 | from app.core.logger import logger
4 | from app.import_process.agent.state import ImportGraphState
5 |
6 | def node_document_split(state: ImportGraphState) -> ImportGraphState:
7 |     """
8 |     节点：文档切分 (node_document_split)
9 |     为什么叫这个名字：将长文档切分成小的 Chunks (切片) 以便检索。
10 |    未来要实现：
11 |    1. 基于 Markdown 标题层级进行递归切分。
12 |    2. 对过长的段落进行二次切分。
13 |    3. 生成包含 Metadata (标题路径) 的 Chunk 列表。
14 |    """
15 |    logger.info(f">>> [Stub] 执行节点: {sys._getframe().f_code.co_name}")
16 |    return state
```

(5) 主体识别节点: node_item_name_recognition.py

```
1 | import sys
2 |
3 | from app.core.logger import logger
4 | from app.import_process.agent.state import ImportGraphState
5 |
6 | def node_item_name_recognition(state: ImportGraphState) -> ImportGraphState:
7 |     """
8 |     节点：主体识别 (node_item_name_recognition)
```

```
9      为什么叫这个名字：识别文档核心描述的物品/商品名称（Item Name）。
10     未来要实现：
11     1. 取文档前几段内容。
12     2. 调用 LLM 识别这篇文档讲的是什么东西（如："Fluke 17B+ 万用表"）。
13     3. 存入 state["item_name"] 用于后续数据幂等性清理。
14     """
15     logger.info(f">>> [Stub] 执行节点: {sys._getframe().f_code.co_name}")
16     return state
```

(6) 向量化节点: node_bge_embedding.py

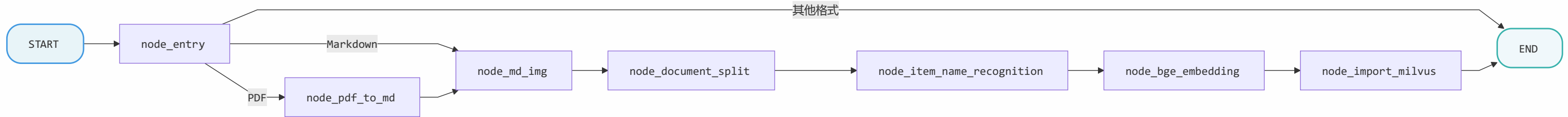
```
1 import sys
2
3 from app.core.logger import logger
4 from app.import_process.agent.state import ImportGraphState
5
6 def node_bge_embedding(state: ImportGraphState) -> ImportGraphState:
7     """
8     节点：向量化（node_bge_embedding）
9     为什么叫这个名字：使用 BGE-M3 模型将文本转换为向量（Embedding）。
10    未来要实现：
11    1. 加载 BGE-M3 模型。
12    2. 对每个 Chunk 的文本进行 Dense（稠密）和 Sparse（稀疏）向量化。
13    3. 准备好写入 Milvus 的数据格式。
14    """
15    logger.info(f">>> [Stub] 执行节点: {sys._getframe().f_code.co_name}")
16    return state
```

(7) 写入Milvus节点: node_import_milvus.py

```
1 import sys
2
3 from app.core.logger import logger
4 from app.import_process.agent.state import ImportGraphState
5
6 def node_import_milvus(state: ImportGraphState) -> ImportGraphState:
7     """
8     节点：导入向量库（node_import_milvus）
9     为什么叫这个名字：将处理好的向量数据写入 Milvus 数据库。
10    未来要实现：
11    1. 连接 Milvus。
12    2. 根据 item_name 删除旧数据（幂等性）。
13    3. 批量插入新的向量数据。
14    """
15    logger.info(f">>> [Stub] 执行节点: {sys._getframe().f_code.co_name}")
16    return state
```

4.2.2 第二步：编写主图代码

节点思路总结：



文件: app/import_process/agent/main_graph.py

```
1 # 加载环境变量：从 .env 文件读取配置（如Milvus地址、KG服务地址、BGE模型路径等）
2 from dotenv import load_dotenv
3 # 导入LangGraph核心依赖：StateGraph(状态图)、START/END(内置起始/结束节点常量)
4 from langgraph.graph import StateGraph, END, START
5
6 from app.core.logger import logger
7 # 导入自定义状态类：统一管理 workflow 全程的所有数据（各节点共享/修改）
8 from app.import_process.agent.state import ImportGraphState, create_default_state
9 # 导入所有自定义业务节点：每个节点对应知识库导入的一个具体步骤
10 from app.import_process.agent.nodes.node_entry import node_entry # 入口节点：初始化参数、校验输入
```

```
11 from app.import_process.agent.nodes.node_pdf_to_md import node_pdf_to_md # PDF转MD：解析PDF文件为markdown格式
12 from app.import_process.agent.nodes.node_md_img import node_md_img # MD图片处理：提取/下载markdown中的图片、修复图片路径
13 from app.import_process.agent.nodes.node_document_split import node_document_split # 文档分块：将长文档切分为符合模型要求的小片段
14 from app.import_process.agent.nodes.node_item_name_recognition import node_item_name_recognition # 项目名识别：从分块中提取核心项目名称（业务定制化）
15 from app.import_process.agent.nodes.node_bge_embedding import node_bge_embedding # BGE向量化：将文本分块转换为向量表示（适配Milvus向量库）
16 from app.import_process.agent.nodes.node_import_milvus import node_import_milvus # 导入Milvus：将向量数据写入Milvus向量数据库
17
18
19 # 初始化环境变量：必须在配置读取前执行，确保后续节点能获取到环境变量中的配置信息
20 load_dotenv()
21
22 # ===== 1. 初始化LangGraph状态图 =====
23 # 核心：StateGraph是LangGraph的核心类，用于构建有状态的工作流
24 # 参数ImportGraphState：自定义TypedDict类型，定义了工作流的**全量状态字段**
25 # 作用：所有节点的入参都是该状态对象，节点返回的键值对会自动合并回状态，实现节点间数据共享
26 workflow = StateGraph(ImportGraphState)
27
28 # ===== 2. 注册所有业务节点 =====
29 # 语法：add_node("节点唯一标识", 节点函数)
30 # 要求：节点函数必须接收「状态对象」作为入参，返回字典（用于更新状态）
31 # 所有节点按「知识库导入流程」先后顺序注册，节点标识与函数名保持一致，便于维护
32 workflow.add_node("node_entry", node_entry) # 流程入口：参数初始化、输入校验
33 workflow.add_node("node_pdf_to_md", node_pdf_to_md) # PDF转MD：非MD格式文件的前置处理
34 workflow.add_node("node_md_img", node_md_img) # MD图片处理：保证文档中图片的可访问性
35 workflow.add_node("node_document_split", node_document_split) # 文档分块：解决大文本无法向量化/推理的问题
36 workflow.add_node("node_item_name_recognition", node_item_name_recognition) # 项目名识别：业务定制化步骤，提取核心业务标识
37 workflow.add_node("node_bge_embedding", node_bge_embedding) # BGE向量化：文本→向量，为Milvus存储做准备
38 workflow.add_node("node_import_milvus", node_import_milvus) # 向量入库：将向量数据持久化到Milvus
39
40 # ===== 3. 设置工作流入口节点 =====
41 # 语法：set_entry_point("节点标识") → 推荐写法，直接指定流程起始节点
42 # 等效写法：workflow.add_edge(START, "node_entry")（START是LangGraph内置起始常量）
43 # 作用：指定工作流执行的第一个节点，替代手动添加START到目标节点的边，代码更简洁
44 workflow.set_entry_point("node_entry")
45
46 # ===== 4. 定义条件路由函数（入口节点后的分支逻辑） =====
47 # 核心：根据状态中的配置项，动态决定后续执行路径，实现「PDF导入」/「MD直接导入」分支
48 # 要求：接收状态对象为入参，返回「目标节点标识」或END（内置结束常量）
49 def route_after_entry(state: ImportGraphState) -> str:
50     """
51     入口节点后的条件路由逻辑
52     :param state: 工作流全量状态对象，包含所有配置项和中间结果
53     :return: 目标节点标识/END，LangGraph会自动跳转到对应节点
54     """
55     # 分支1：开启MD直接导入 → 跳过PDF转MD，直接执行MD图片处理
56     if state.get("is_md_read_enabled"):
57         return "node_md_img"
58     # 分支2：开启PDF导入 → 执行PDF转MD，再走后续流程
59     elif state.get("is_pdf_read_enabled"):
60         return "node_pdf_to_md"
61     # 分支3：未开启任何导入配置 → 直接终止工作流（END是LangGraph内置结束常量）
62     else:
63         return END
64
65 # 注册条件边：将入口节点与路由函数绑定
66 # 语法：add_conditional_edges("源节点标识", 路由函数)
67 # 作用：源节点执行完成后，调用路由函数，根据返回值动态跳转到目标节点
68 workflow.add_conditional_edges(
69     "node_entry",
70     route_after_entry,
71     {
72         "node_md_img": "node_md_img",
73         "node_pdf_to_md": "node_pdf_to_md",
74         END: END
75     }
76 )
77
78 # ===== 5. 注册静态顺序边（分支合并后的统一流程） =====
79 # 核心：所有分支最终合并为「固定顺序执行流程」，从MD图片处理到知识图谱入库，一步到底
80 # 语法：add_edge("源节点标识", "目标节点标识/END") → 静态边，固定路由关系，无分支逻辑
```

```
81 workflow.add_edge("node_pdf_to_md", "node_md_img")  # PDF转MD完成→ MD图片处理
82 workflow.add_edge("node_md_img", "node_document_split")  # MD处理完成 → 文档分块
83 workflow.add_edge("node_document_split", "node_item_name_recognition")  # 分块完成 → 项目名识别
84 workflow.add_edge("node_item_name_recognition", "node_bge_embedding")  # 项目名识别完成 → BGE向量化
85 workflow.add_edge("node_bge_embedding", "node_import_milvus")  # 向量化完成 → 导入Milvus向量库
86 workflow.add_edge("node_import_milvus", END)  # Milvus入库完成 →  workflow执行结束（END是内置结束节点）
87
88 # ===== 6. 编译 workflow 为可执行对象 =====
89 # 语法：compile() → 将StateGraph构建的流程编译为LangGraph的可执行应用
90 # 作用：生成可调用的kb_import_app，通过invoke()方法触发 workflow 执行
91 # 特性：编译后可重复调用，支持传入不同的初始状态，实现多任务执行
92 kb_import_app = workflow.compile()
```

4.2.3 第三步：验证图流程

在实现具体业务逻辑前，我们先跑一个测试脚本，看看图能不能跑通，路线对不对。

创建测试文件: test/04-test_graph_flow.py

```
1 import json
2
3 from app.import_process.agent.main_graph import kb_import_app
4 from app.import_process.agent.state import create_default_state
5 import sys
6 from app.core.logger import logger
7
8 logger.info("==== 开始测试 =====")
9
10 initial_state = create_default_state(local_file_path="万用表RS-12的使用.pdf")
11 final_state = None
12
13 # 只输出更最终的状态值（字典形式），不包含节点名称、执行日志、元数据等额外信息
14 for event in kb_import_app.stream(initial_state):
15     for key, value in event.items():
16         logger.info(f"节点: {key}")
17         final_state = value
18
19 # 格式化输出最终状态
20 logger.info(f"最终状态: {json.dumps(final_state, indent=4, ensure_ascii=False)}")
21
22 logger.info("图结构:")
23 # uv add grandalf
24 kb_import_app.get_graph().print_ascii()
25
26 logger.info("==== 测试结束 =====")
27
```

预期效果:

你应该能看到控制台依次打印出每个节点的 >>> [Stub] 执行节点：... 日志。

- PDF 流程应包含：node_entry -> node_pdf_to_md -> node_md_img -> ... -> node_import_milvus
- Markdown 流程应包含：node_entry -> node_md_img -> ... -> node_import_milvus (**跳过了 node_pdf_to_md**)
- 其他格式的文档只包含：node_entry (**直接结束**)

如果能看到这些日志，说明我们的图结构搭建成功！接下来就可以放心地去填充每个节点的具体代码了。

5. 导入数据节点实现与测试

5.1 入口与类型判断 (node_entry)

文件: app/import_process/agent/nodes/node_entry.py

相关工具类位置: app/utils/task_utils.py

节点作用与实现思路

节点作用: 作为数据加载流程的“总调度员”，负责接收外部输入的文件，识别文件类型（PDF或Markdown），并根据类型开启相应的处理分支。同时，它提取文件名作为全局元数据，并初始化任务追踪状态，确保后续流程可追溯、可监控。

实现思路:

- 路由分发:** 采用轻量级的条件判断逻辑, 通过文件后缀 (.pdf / .md) 决定激活 is_pdf_read_enabled 还是 is_md_read_enabled 状态位, 实现不同格式文件的差异化处理。
- 元数据提取:** 在入口处统一提取文件名 (file_title), 作为贯穿整个知识库构建流程的唯一标识, 避免后续节点重复解析。
- 任务监控初始化:** 集成 task_utils, 记录当前任务 ID 和初始状态, 为前端提供实时的进度反馈。

步骤分解

- 接收状态:** 获取 local_file_path。
- 判断类型:** 检查文件后缀是 .pdf 还是 .md。
- 设置标记:** 更新 state 中的 is_pdf_read_enabled 或 is_md_read_enabled, 供主图路由使用。
- 提取标题:** 从文件名中提取 file_title, 后续作为元数据。

工具类解读：任务追踪

文件: app/utils/task_utils.py

实现思路:

- 内存管理:** 使用简单的内存字典 _tasks_running_list 和 _tasks_done_list 记录任务状态, 轻量高效。
- 状态映射:** 维护 _NODE_NAME_TO_CN 字典, 将技术性的节点名称 (如 node_entry) 映射为用户友好的中文名称 (如 检查文件), 方便前端展示。
- SSE 集成:** 集成 SSE (Server-Sent Events) 推送机制, 允许实时将任务进度推送到前端。
- 操作封装:** 提供 add_running_task 和 add_done_task 接口, 方便各节点调用, 屏蔽底层状态管理细节。

代码实现

```
1 import os
2 import sys
3 from os.path import splitext
4
5 from app.core.logger import logger
6 from app.import_process.agent.state import ImportGraphState, create_default_state
7 from app.utils.format_utils import format_state
8 from app.utils.task_utils import add_running_task, add_done_task
9
10 def node_entry(state: ImportGraphState) -> ImportGraphState:
11     """
12     LangGraph知识库导入工作流 - 入口节点
13     核心职责：初始化参数校验 | 自动判断文件类型(PDF/MD) | 设置解析开关 | 提取业务标识
14     入参：ImportGraphState - 必须包含 local_file_path(文件路径)、task_id(任务ID)
15     出参：ImportGraphState - 新增/更新 is_pdf_read_enabled/is_md_read_enabled/pdf_path/md_path/file_title
16     执行链路：__start__ → 本节点 → route_after_entry(条件路由) → 对应解析节点/流程终止
17     """
18
19     # 动态获取函数名避免硬编码
20     func_name = sys._getframe().f_code.co_name
21
22     # 节点启动日志，打印当前工作流状态
23     logger.debug(f"【{func_name}】节点启动，\n当前工作流状态：{format_state(state)}")
24
25     # 开始：记录节点运行状态
26     add_running_task(state["task_id"], func_name)
27
28
29     # 1. 核心参数提取与非空校验
30     document_path = state.get("local_file_path", "")
31     if not document_path:
32         logger.error(f"【{func_name}】核心参数缺失：工作流状态中未配置local_file_path，文件路径为空")
33         return state
34
35     # 2. 根据文件后缀判断类型，设置对应解析开关
36     if document_path.endswith(".pdf"):
37         logger.info(f"【{func_name}】文件类型校验通过：{document_path} → PDF格式，开启PDF解析流程")
38         state["is_pdf_read_enabled"] = True
39         state["pdf_path"] = document_path
40     elif document_path.endswith(".md"):
41         logger.info(f"【{func_name}】文件类型校验通过：{document_path} → MD格式，开启MD解析流程")
```

```
42         state["is_md_read_enabled"] = True
43         state["md_path"] = document_path
44     else:
45         logger.warning(f" 【{func_name}】 文件类型校验失败：{document_path} → 不支持的格式，仅支持.pdf/.md")
46
47     # 3. 提取文件无后缀纯名称，作为全局业务标识
48     file_name = os.path.basename(document_path)
49     state["file_title"] = splitext(file_name)[0]
50     logger.info(f" 【{func_name}】 文件业务标识提取完成：file_title = {state['file_title']}")
51
52     # 结束：记录节点运行状态
53     add_done_task(state["task_id"], func_name)
54
55     # 节点完成日志，打印当前 workflow 状态
56     logger.debug(f" 【{func_name}】 节点执行完成，\n更新后 workflow 状态：{format_state(state)}")
57
58     return state
```

关键语法补充说明

语法 / 函数	具体作用	执行示例
os.path.basename(document_path)	从完整路径提取文件名	/data/kb/test.pdf → test.pdf p.stem
splitext(file_name)	拆分文件名和后缀	test.pdf → ('test', '.pdf')
state.get("key", "")	安全提取状态值，无 key 时返回默认值	state.get("a", "") → 无 "a" 则返回 ""
sys._getframe().f_code.co_name	动态获取当前函数名	本节点中返回 node_entry
add_running_task/add_done_task	记录任务的节点运行状态	用于任务监控面板，展示节点执行进度(fastapi使用)

单元测试

您可以在 node_entry.py 文件底部直接运行以下测试代码：

```
1 if __name__ == '__main__':
2
3     # 单元测试：覆盖不支持类型、MD、PDF三种场景
4     logger.info("==== 开始node_entry节点单元测试 =====")
5
6     # 测试1：不支持的TXT文件
7     test_state1 = create_default_state(
8         task_id="test_task_001",
9         local_file_path="联想海豚用户手册.txt"
10    )
11    node_entry(test_state1)
12
13    # 测试2：MD文件
14    test_state2 = create_default_state(
15        task_id="test_task_002",
16        local_file_path="小米用户手册.md"
17    )
18    node_entry(test_state2)
19
20    # 测试3：PDF文件
21    test_state3 = create_default_state(
22        task_id="test_task_003",
23        local_file_path="万用表的使用.pdf"
24    )
25    node_entry(test_state3)
26
27    logger.info("==== 结束node_entry节点单元测试 =====")
```

5.2 PDF 转 Markdown (node_pdf_to_md)

文件: app/import_process/agent/nodes/node_pdf_to_md.py

相关工具类位置: app/utils/task_utils.py

节点作用与实现思路

节点作用: 解决非结构化 PDF 文档难以直接处理的痛点，利用 MinerU 高精度解析能力，将 PDF 转换为结构清晰、包含图片引用的 Markdown 格式，为后续的文本切分和图片理解打下基础。

实现思路:

- 云端算力卸载:** 考虑到本地 OCR 和布局分析的资源消耗巨大，本节点采用“云端 API”模式，将繁重的解析任务卸载到 MinerU 服务端。
- 异步轮询机制:** 针对大文件解析耗时较长的问题，设计了“上传 -> 提交 -> 轮询 -> 下载”的异步交互流程，通过**指数退避**或固定间隔轮询状态，避免长连接超时。
- 完整性保障:** 解析完成后，不仅提取 Markdown 文本，还同步下载并解压关联的图片资源，保持文档的多模态完整性。

步骤分解

- 准备参数:** 获取 PDF 路径和输出目录。
- 请求上传:** 调用 MinerU 在线 API (/file-urls/batch) 获取上传链接。
- 上传文件:** 将 PDF 文件 PUT 到签名 URL。
- 轮询结果:** 循环查询任务状态 (/extract-results/batch/{batch_id})，直到完成。
- 获取结果:** 下载生成的 ZIP 包，解压并读取 .md 文件内容到 state。

注意: 需要在 .env 中配置 MINERU_BASE_API_TOKEN。

API申请位置：<https://mineru.net/apiManage/token> （有效期90天）

参考官方文档：<https://mineru.net/doc/docs/index.html?theme=light&v=1.0#%E6%89%B9%E9%87%8F%E6%96%87%E4%BB%B6%E8%A7%A3%E6%9E%90>

1. 导入与配置 (Imports & Config)

添加全局配置 .env 文件

```
1 # mineru api以及url处理地址！
2 MINERU_API_TOKEN=eyJ0eXB1IjoiSlIdUIiwiYWxnIjoiSFM1MTIiIjQ.eyJqdGkiOiI3NjUwMDY4NiIsInJvbCI6I1JPTeVfUkVHSVNUbGVzLCJpc3MiOiJPcGVuWEExhYiIsIm1hdCI6MTc3MzY3MDk2OSwiY2xpZW50SWQiOiJsa3pkeDU3bnZ5MjJqa3BxOXgydyIsInBob251IjoiIiwib3B1bk1kIjpudWxsLCJ1dWlkIjoiMTBkYTM3MTgtMzA3YS00ODdkLWJhNDEtZjNkYzlmMGQ3ZmE5IiwiaWZlhaWwiOiIiLCJleHAiOjE3ODU0NDY5Nj19.9DF0FHvQYaK6NRnOVG87V_IYToHPRcyHcQd-tuXETuPIJzVV6h6PMc3RtY5zeaIN7LiPRobziXqAKWoDNl3ueA
3 MINERU_BASE_URL=https://mineru.net/api/v4
```

定义配置类，读取配置文件

位置：app/import_process/config/mineru_config.py

```
1 # 导入核心依赖：数据类、环境变量读取、路径处理
2 from dataclasses import dataclass
3 import os
4 from dotenv import load_dotenv
5
6 # 提前加载.env配置文件（必须在读取环境变量前执行，确保os.getenv能获取到值）
7 # 若.env不在项目根目录，可指定路径：load_dotenv(dotenv_path=Path(__file__).parent / ".env")
8 load_dotenv()
9
10 # 定义minerU服务配置
11 @dataclass
12 class MinerUConfig:
13     base_url: str
14     api_token : str
15
16 mineru_config = MinerUConfig(
17     base_url=os.getenv("MINERU_BASE_URL"),
18     api_token=os.getenv("MINERU_API_TOKEN")
19 )
```

首先引入必要的库，并加载环境变量配置。

```
1 # 系统库
```

```
2 import os
3 import sys
4 import time
5 import requests
6 import zipfile
7 import shutil
8 from pathlib import Path
9
10 # 项目内部库
11 from app.import_process.agent.state import ImportGraphState
12 from app.utils.task_utils import add_running_task, add_done_task
13 from app.conf.mineru_config import mineru_config
14 from app.core.logger import logger # 统一日志工具
15
16 # MinerU配置（缓存配置信息）
17 MINERU_BASE_URL = mineru_config.base_url
18 MINERU_API_TOKEN = mineru_config.api_token
```

2. 主流程定义 (Main Flow)

定义 node_pdf_to_md 入口函数，清晰展示三个主要步骤的调用关系。

```
1 def node_pdf_to_md(state: ImportGraphState) -> ImportGraphState:
2     """
3     LangGraph workflow node: PDF to MD core processing node
4     Core flow: Path verification -> MinerU upload parsing -> Result download decompression -> Read MD content and update workflow status
5     Parameters: state - workflow status object, must contain pdf_path/local_dir/task_id
6     Returns: Updated workflow status, new md_path/md_content
7     """
8
9     # 动态获取函数名避免硬编码
10    func_name = sys._getframe().f_code.co_name
11
12    # 节点启动日志，打印当前 workflow 状态
13    logger.debug(f"【{func_name}】节点启动，\n当前 workflow 状态：{format_state(state)}")
14
15    # 开始：记录节点运行状态
16    add_running_task(state["task_id"], func_name)
17
18
19    try:
20        # 步骤1：校验PDF路径和输出目录
21        pdf_path_obj, output_dir_obj = step_1_validate_paths(state)
22
23        # 步骤2：上传PDF至MinerU并轮询解析结果
24        zip_url = step_2_upload_and_poll(pdf_path_obj, output_dir_obj)
25
26        # 步骤3：下载ZIP包并提取MD文件
27        md_path = step_3_download_and_extract(zip_url, output_dir_obj, pdf_path_obj.stem)
28
29        # 更新 workflow 状态：记录MD文件路径和内容
30        state["md_path"] = md_path
31        logger.info(f"【{func_name}】MD文件生成成功，路径：{md_path}")
32
33        # 读取MD文件内容，捕获异常仅警告不终止
34        try:
35            with open(md_path, "r", encoding="utf-8") as f:
36                state["md_content"] = f.read()
37                logger.debug(f"【{func_name}】MD文件内容读取成功，内容长度：{len(state['md_content'])}字符")
38        except Exception as e:
39            logger.error(f"【{func_name}】读取MD文件内容失败：{str(e)}")
40
41        logger.info(f"【{func_name}】节点执行完成，更新后 workflow 状态键：{list(state.keys())}")
42
43    except Exception as e:
44        # 异常日志分级，精准提示配置问题
45        logger.error(f"【{func_name}】PDF转MD流程执行失败：{str(e)}", exc_info=True)
46        raise # 抛出异常，终止 workflow
47    finally:
48
```

```
49         # 结束：记录节点运行状态
50         add_done_task(state["task_id"], func_name)
51
52         # 节点完成日志，打印当前 workflow 状态
53         logger.debug(f"【{func_name}】节点执行完成，\n更新后 workflow 状态：{format_state(state)}")
54
55     return state
```

3. 步骤 1: 校验路径 (Step 1: Validate Paths)

这一步负责检查 state 中的参数，并确保文件存在。

```
1 def step_1_validate_paths(state: ImportGraphState):
2     """
3     步骤1：校验PDF文件路径和输出目录
4     核心职责：参数非空校验 | PDF文件有效性校验 | 输出目录自动创建
5     返回：合法的PDF文件Path对象、输出目录Path对象
6     异常：ValueError(参数缺失)、FileNotFoundError(文件无效)
7     """
8     log_prefix = "[step_1_validate_paths] "
9     pdf_path = state.get("pdf_path", "").strip()
10    local_dir = state.get("local_dir", "").strip()
11
12    # 参数非空校验
13    if not pdf_path:
14        raise ValueError(f"{log_prefix}workflow状态缺失有效参数：pdf_path，当前值：{repr(pdf_path)}")
15    if not local_dir:
16        raise ValueError(f"{log_prefix}workflow状态缺失有效参数：local_dir，当前值：{repr(local_dir)}")
17
18    # 转换为Path对象统一处理路径
19    pdf_path_obj = Path(pdf_path)
20    output_dir_obj = Path(local_dir)
21
22    # PDF文件有效性校验（存在且为文件，非目录）
23    if not pdf_path_obj.exists():
24        raise FileNotFoundError(f"{log_prefix}PDF文件不存在，绝对路径：{pdf_path_obj.absolute()}")
25    if not pdf_path_obj.is_file():
26        raise FileNotFoundError(f"{log_prefix}指定路径非文件（是目录），绝对路径：{pdf_path_obj.absolute()}")
27
28    # 确保输出目录存在，不存在则递归创建
29    if not output_dir_obj.exists():
30        logger.info(f"{log_prefix}输出目录不存在，自动创建：{output_dir_obj.absolute()}")
31        output_dir_obj.mkdir(parents=True, exist_ok=True)
32
33    return pdf_path_obj, output_dir_obj
```

总结：from pathlib import Path

传统路径操作依赖os.path模块（如os.path.exists()/os.path.join()），需要对**字符串路径**做各种处理，而Path对象是**面向对象的路径操作方式**，把路径的「属性」和「操作方法」封装在对象中，相比字符串路径+os.path，优势极其明显，也是 Python3.4 + 官方推荐的路径操作方式：

1. 跨平台兼容性强，无需手动处理路径分隔符

不同系统的路径分隔符不同：Windows 用\，Linux/Mac 用/，用字符串路径时需要手动适配（如os.path.join()），而Path对象会**自动识别当前系统**，生成对应分隔符的路径，代码一次编写，多平台运行。

```
1 # 字符串路径（需手动用os.path.join适配）
2 import os
3 path1 = os.path.join("D:", "test", "pdf.pdf") # Windows→D:\test\pdf.pdf，Linux→D:/test/pdf.pdf
4
5 # Path对象（自动适配，更简洁）
6 from pathlib import Path
7 path2 = Path("D:") / "test" / "pdf.pdf" # 用/直接拼接，自动适配系统分隔符
```

2. 路径拼接更直观，支持「/」运算符

字符串路径拼接需要使用os.path.join()，语法繁琐且容易出错（如忘记加分隔符）；Path对象支持**直接用 / 运算符拼接路径**，和日常书写路径的习惯一致，代码更简洁、可读性更高。

3. 封装了路径的「属性」，直接获取路径信息，无需手动解析

字符串路径获取「文件名、后缀、纯名称、父目录」等信息，需要结合os.path.basename()/os.path.splitext()等多个函数，而Path对象内置了这些**只读属性**，直接点取即可，无需额外处理：

```
1 p = Path("D:/test/report_v1.2.pdf")
2 print(p.name)      # 获取完整文件名：report_v1.2.pdf
3 print(p.stem)      # 获取文件纯名称（无后缀）：report_v1.2（你代码中核心使用的属性）
4 print(p.suffix)    # 获取文件后缀：.pdf
5 print(p.parent)    # 获取父目录：D:/test
6 print(p.absolute())# 获取绝对路径：D:\test\report_v1.2.pdf (Windows)
```

4. 封装了路径的「操作方法」，直接调用，无需传参

os.path的所有操作函数（如判断存在、判断是文件 / 目录）都需要**将路径字符串作为参数传入**，而Path对象将这些操作封装为**自身方法**，直接调用即可，无需重复传参，语法更简洁：

```
1 p = Path("D:/test.pdf")
2 d = Path("D:/output")
3
4 # Path对象（直接调用方法）
5 p.exists()      # 判断路径是否存在，等价于os.path.exists(p)
6 p.is_file()     # 判断是否是文件，等价于os.path.isfile(p)
7 d.mkdir(parents=True, exist_ok=True) # 创建目录，等价于os.makedirs(d, exist_ok=True)
8
9 # 字符串路径+os.path（需传参，繁琐）
10 import os
11 os.path.exists(str(p))
12 os.path.isfile(str(p))
13 os.makedirs(str(d), exist_ok=True)
```

5. 方法链式调用，简化多步路径操作

Path对象的多数方法执行后，**返回的还是 Path 对象**，支持**链式调用**，可以把多个路径操作写在一行，相比os.path的多次函数调用，代码更简洁：

```
1 # 链式调用：创建父目录 → 拼接子文件路径 → 获取绝对路径
2 new_file = Path("D:/temp/test").parent.mkdir(parents=True, exist_ok=True) / "new.pdf"
```

6. 与 Python 内置函数无缝兼容，无需频繁类型转换

Python 的内置文件操作函数（如open()）、第三方库（如shutil/zipfile）都**直接支持 Path 对象作为参数**，无需手动转换为字符串，避免了str(path)的频繁类型转换，减少代码冗余：

```
1 p = Path("D:/test.md")
2 # 直接传入Path对象，无需str(p)
3 with open(p, "r", encoding="utf-8") as f:
4     content = f.read()
5
6 # 字符串路径（需直接传字符串）
7 with open(str(p), "r", encoding="utf-8") as f:
8     content = f.read()
```

4. 步骤 2: 上传与轮询 (Upload & Poll)

这一步实现了与 MinerU API 的交互，包括申请上传链接、上传文件、提交任务和轮询状态。

1. 配置校验 → **2. 获取上传链接** + batch_id（批量接口自动生成，无需手动提交） → **3. 文件上传**（含重试 + 代理禁用） → **4. 轮询状态**（根据 batch_id 持续查询，直至完成 / 失败 / 超时）

```
1 def step_2_upload_and_poll(pdf_path_obj: Path, output_dir_obj: Path):
2     """
3     步骤2：上传PDF至MinerU并轮询解析任务状态
4     核心流程：配置校验 → 获取上传链接 → 文件上传（含重试） → 任务轮询（直至完成/失败/超时）
5     参数：pdf_path_obj-已校验的PDF Path对象；output_dir_obj-输出目录Path对象
6     返回：解析结果ZIP包下载链接full_zip_url
7     异常：ValueError(配置缺失)、RuntimeError(请求/上传失败)、TimeoutError(任务超时)
8     """
9     # 前置配置校验，拦截无效配置
10    if not MINERU_BASE_URL or not MINERU_API_TOKEN:
11        raise ValueError("MinerU配置缺失：请在.env中正确配置MINERU_BASE_URL和MINERU_API_TOKEN")
12    logger.info(f"[配置校验] MinerU基础配置加载成功，开始处理文件：{pdf_path_obj.name}")
```

```
13
14 # 构造请求头 (符合HTTP规范, Bearer鉴权)
15 request_headers = {
16     "Content-Type": "application/json",
17     "Authorization": f"Bearer {MINERU_API_TOKEN}"
18 }
19
20 # 1. 调用批量接口, 获取上传Signed URL和任务batch_id
21 url_get_upload = f"{MINERU_BASE_URL}/file-urls/batch"
22 req_data = {
23     "files": [{"name": pdf_path_obj.name}],
24     "model_version": "vlm" # 官方推荐解析模型
25 }
26 logger.debug(f"[获取上传链接] 调用接口: {url_get_upload}, 请求参数: {req_data}")
27 resp = requests.post(url=url_get_upload, headers=request_headers, json=req_data, timeout=30)
28
29 # 响应校验: 先验HTTP状态, 再验业务返回码
30 if resp.status_code != 200:
31     raise RuntimeError(f"[获取上传链接] 网络请求失败, 状态码: {resp.status_code}, 响应内容: {resp.text}")
32
33 resp_data = resp.json()
34 if resp_data["code"] != 0:
35     raise RuntimeError(f"[获取上传链接] API业务错误, 返回数据: {resp_data}")
36
37 # 提取核心数据: 上传链接和任务唯一标识
38 signed_url = resp_data["data"]["file_urls"][0]
39 batch_id = resp_data["data"]["batch_id"]
40 logger.info(f"[获取上传链接] 成功, batch_id: {batch_id}, 上传链接已生成")
41
42 # 2. 读取PDF二进制数据, 准备上传
43 logger.info(f"[文件上传] 开始读取PDF文件: {pdf_path_obj.name}")
44 with open(pdf_path_obj, "rb") as f:
45     file_data = f.read()
46
47 # 创建Session (复用TCP连接, 禁用代理避免签名验证失败)
48 upload_session = requests.Session()
49 upload_session.trust_env = False
50
51 try:
52     # 首次上传: 自动识别文件类型
53     put_resp = upload_session.put(url=signed_url, data=file_data, timeout=60)
54     # 重试逻辑: 首次失败则强制指定PDF的Content-Type
55     if put_resp.status_code != 200:
56         logger.warning(f"[文件上传] 首次上传失败 (状态码: {put_resp.status_code}), 强制指定PDF类型重试")
57         pdf_headers = {"Content-Type": "application/pdf"}
58         put_resp = upload_session.put(url=signed_url, data=file_data, headers=pdf_headers, timeout=60)
59         # 重试仍失败则抛出异常
60         if put_resp.status_code != 200:
61             raise RuntimeError(f"[文件上传] 重试后仍失败, 状态码: {put_resp.status_code}, 响应内容: {put_resp.text}")
62         logger.info(f"[文件上传] 成功, 文件{pdf_path_obj.name}已存入云存储")
63 except Exception as e:
64     raise RuntimeError(f"[文件上传] 网络异常导致上传失败, 错误信息: {str(e)}")
65 finally:
66     # 无论成败, 关闭Session释放网络连接, 避免资源泄漏
67     upload_session.close()
68
69 # 3. 根据batch_id轮询任务状态, 直至完成/失败/超时
70 poll_url = f"{MINERU_BASE_URL}/extract-results/batch/{batch_id}"
71 start_time = time.time()
72 timeout_seconds = 600 # 最大超时时间10分钟 (适配600页内PDF)
73 poll_interval = 3 # 轮询间隔3秒 (平衡查询频率和服务端压力)
74 logger.info(f"[任务轮询] 开始监控任务状态, batch_id: {batch_id}, 最大超时: {timeout_seconds}s")
75
76 while True:
77     # 超时检查: 超过最大时间直接终止轮询
78     elapsed_time = time.time() - start_time
79     if elapsed_time > timeout_seconds:
80         raise TimeoutError(f"[任务轮询] 超时! 任务处理超{int(timeout_seconds)}秒, batch_id: {batch_id}")
81
82     # 发起轮询请求, 短超时10秒, 异常则重试
83     try:
84         poll_resp = requests.get(url=poll_url, headers=request_headers, timeout=10)
```

```
85     except Exception as e:
86         logger.warning(f"[任务轮询] 网络请求异常，{poll_interval}秒后重试：{str(e)}")
87         time.sleep(poll_interval)
88         continue
89
90     # 处理HTTP响应错误：5xx服务端繁忙则重试，其他错误直接抛出
91     if poll_resp.status_code != 200:
92         if 500 <= poll_resp.status_code < 600:
93             logger.warning(f"[任务轮询] 服务端繁忙（状态码：{poll_resp.status_code}），{poll_interval}秒后重试")
94             time.sleep(poll_interval)
95             continue
96         else:
97             raise RuntimeError(f"[任务轮询] HTTP请求失败，状态码：{poll_resp.status_code}，响应内容：{poll_resp.text}")
98
99     # 解析轮询结果，校验业务状态
100    poll_data = poll_resp.json()
101    if poll_data["code"] != 0:
102        raise RuntimeError(f"[任务轮询] API业务错误，返回数据：{poll_data}")
103
104    extract_results = poll_data["data"]["extract_result"]
105    # 结果暂空，继续轮询
106    if not extract_results:
107        logger.debug(f"[任务轮询] 结果暂为空，已耗时{int(elapsed_time)}s，继续等待")
108        time.sleep(poll_interval)
109        continue
110
111    # 解析任务状态，分支处理
112    result_item = extract_results[0]
113    state_status = result_item["state"]
114    # 状态1：任务完成，提取ZIP下载链接
115    if state_status == "done":
116        logger.info(f"[任务轮询] 解析任务完成！总耗时：{int(elapsed_time)}s，batch_id：{batch_id}")
117        full_zip_url = result_item.get("full_zip_url")
118        if not full_zip_url:
119            raise RuntimeError("[任务轮询] 任务完成但未返回ZIP包下载链接，batch_id：{batch_id}")
120        logger.info(f"[任务轮询] 结果ZIP包下载链接：{full_zip_url}...")
121        return full_zip_url
122    # 状态2：任务失败，提取错误信息抛出
123    elif state_status == "failed":
124        err_msg = result_item.get("err_msg", "未知错误，无具体信息")
125        raise RuntimeError(f"[任务轮询] 解析任务失败，batch_id：{batch_id}，错误信息：{err_msg}")
126    # 状态3：处理中，实时打印进度（覆盖当前行）
127    else:
128        logger.debug(
129            f"[任务轮询] 处理中（已耗时{int(elapsed_time)}s），状态：{state_status} | 刷新间隔{poll_interval}s",
130            end="\r"
131        )
132        time.sleep(poll_interval)
```

总结：为什么 PUT 需要 Session 而 POST/GET 不需要？

1. POST/GET 不需要：
- 它们是普通的业务接口，只要 Token 对了就能过。即使你的请求经过了代理（比如开了 VPN），Header 里多了一些代理信息（如 Via），服务端通常会忽略这些杂质，照样处理，所以直接用 requests.post/get 也没事。
2. PUT 必须需要 Session (trust_env=False)：
- 它使用的是预签名 URL (Presigned URL) 直连对象存储（如 OSS/S3）。这种接口有极其严格的签名校验机制：
- 不开 Session (trust_env=True)：

如果你的电脑有代理，代理服务器可能会偷偷往 Header 里加东西（如 Via , Connection: close）。服务端收到后，发现请求头和签名里约定的不一样，判定为请求被篡改，直接报错 403 SignatureDoesNotMatch。

■ 开了 Session (trust_env=False)：

强制绕过系统代理，发送纯净无杂质的原始请求。服务端校验签名一致，上传成功。
- 一句话：POST/GET 是宽进宽出，怎么发都行；PUT 是字节级校验，必须用 Session 确保请求头一个标点符号都不被代理修改。

5. 步骤 3: 下载与解压 (Step 3: Download & Extract)

这一步负责下载 MinerU 返回的 ZIP 包，解压并提取目标 Markdown 文件。

```
1 def step_3_download_and_extract(zip_url: str, output_dir_obj: Path, pdf_stem: str) -> str:
2     """
3     步骤3：下载MinerU解析结果ZIP包并解压，提取目标MD文件（重命名统一规范）
4     核心流程：下载ZIP → 清理旧目录并解压 → 查找MD文件（按优先级） → 重命名统一为PDF同名
5     参数：zip_url-ZIP包下载链接；output_dir_obj-输出目录Path；pdf_stem-PDF无后缀纯名称
```

```
6  返回：最终MD文件的字符串格式绝对路径
7  异常：RuntimeError(下载失败)、FileNotFoundError(无MD文件)
8  """
9  logger.info(f"===== 开始处理[{pdf_stem}]的MinerU解析结果 =====")
10
11  # 1. 下载解析结果ZIP包，120秒超时适配大文件
12  logger.info(f"[步骤1/4] 开始下载ZIP包，链接：{zip_url}...")
13  resp = requests.get(zip_url, timeout=120)
14  if resp.status_code != 200:
15      raise RuntimeError(f"[步骤1/4] ZIP包下载失败，HTTP状态码：{resp.status_code}")
16
17  # 拼接ZIP包保存路径，按PDF名称唯一命名
18  zip_save_path = output_dir_obj / f"{pdf_stem}_result.zip"
19  with open(zip_save_path, "wb") as f:
20      f.write(resp.content)
21  logger.info(f"[步骤1/4] ZIP包下载成功，保存路径：{zip_save_path}")
22
23  # 2. 清理旧解压目录并解压ZIP包（避免旧文件干扰，为每个PDF创建专属目录）
24  logger.info(f"[步骤2/4] 开始解压ZIP包...")
25  extract_target_dir = output_dir_obj / pdf_stem
26
27  # 清理旧目录，异常则警告不终止
28  if extract_target_dir.exists():
29      try:
30          # 递归删除整个目录树，包括目录本身及其所有子目录和文件。
31          shutil.rmtree(extract_target_dir)
32          logger.info(f"[步骤2/4] 已清理旧的解压目录：{extract_target_dir}")
33      except Exception as e:
34          logger.warning(f"[步骤2/4] 清理旧目录失败，可能不影响新文件解压：{str(e)}")
35
36  # 重新创建解压目录
37  extract_target_dir.mkdir(parents=True, exist_ok=True)
38
39  # 核心解压操作，保留原目录结构
40  with zipfile.ZipFile(zip_save_path, 'r') as zip_file_obj:
41      zip_file_obj.extractall(extract_target_dir)
42  logger.info(f"[步骤2/4] ZIP包解压完成，解压目录：{extract_target_dir}")
43
44  # 3. 递归查找解压目录下所有MD文件（适配子目录结构）
45  logger.info(f"[步骤3/4] 开始查找解压目录中的MD文件...")
46  md_file_list = list(extract_target_dir.rglob("*.md"))
47  if not md_file_list:
48      raise FileNotFoundError(f"[步骤3/4] 解压目录中未找到任何.md格式文件：{extract_target_dir}")
49  logger.info(f"[步骤3/4] 共找到{len(md_file_list)}个MD文件，按优先级匹配目标文件")
50
51  # 4. 按优先级匹配目标MD文件（同名→full.md→第一个，兜底避免流程中断）
52  target_md_file = None
53  # 优先级1：与PDF纯名称完全同名的MD文件
54  for md_file in md_file_list:
55      if md_file.stem == pdf_stem:
56          target_md_file = md_file
57          logger.info(f"[步骤4/4] 匹配到优先级1目标：与PDF同名的MD文件 {target_md_file.name}")
58          break
59  # 优先级2：MinerU默认生成的full.md（不区分大小写）
60  if not target_md_file:
61      for md_file in md_file_list:
62          if md_file.name.lower() == "full.md":
63              target_md_file = md_file
64              logger.info(f"[步骤4/4] 匹配到优先级2目标：MinerU默认文件 {target_md_file.name}")
65              break
66  # 优先级3：兜底取第一个MD文件
67  if not target_md_file:
68      target_md_file = md_file_list[0]
69      logger.info(f"[步骤4/4] 未匹配到前两级目标，兜底取第一个MD文件 {target_md_file.name}")
70
71  # 重命名MD文件：统一为PDF纯名称，便于后续流程处理（仅不同名时执行）
72  if target_md_file.stem != pdf_stem:
73      logger.info(f"[步骤4/4] 开始重命名MD文件，统一为PDF同名：{pdf_stem}.md")
74      new_md_path = target_md_file.with_name(f"{pdf_stem}.md")
75      try:
76          # 将磁盘上的文件进行重命名
77          target_md_file.rename(new_md_path)
```

```
78         # 更新变量引用
79         target_md_file = new_md_path
80         logger.info(f"[步骤4/4] MD文件重命名成功：{pdf_stem}.md")
81     except OSError as e:
82         logger.warning(f"[步骤4/4] MD文件重命名失败，将使用原文件名继续流程：{str(e)}")
83
84     # 转换为字符串绝对路径返回，适配后续仅支持字符串路径的函数
85     final_md_path = str(target_md_file.absolute())
86     logger.info(f"==== [{pdf_stem}]解析结果处理完成，最终MD文件路径：{final_md_path} =====")
87     return final_md_path
```

技术细节说明：

函数 / 方法	所属模块 / 对象	核心作用	关键特点
zipfile.ZipFile(..., 'r')	zipfile	以只读模式打开 ZIP 压缩包，创建 ZIP 操作对象	需配合with语句使用，自动释放文件句柄，避免资源泄漏
zip_file_obj.extractall()	zipfile.ZipFile	将 ZIP 包内 所有文件 / 目录 解压至指定目录，保留原内部目录结构	一键解压，无需遍历文件，适配 MinerU 返回的 ZIP 包格式
shutil.rmtree()	shutil	递归删除指定目录及所有子文件 / 子目录	比os.rmdir更彻底（os.rmdir 仅删除空目录），适合清理旧解压目录
Path.rglob("*.md")	pathlib.Path	递归遍历当前目录及 所有子目录 ，匹配指定通配符的文件	相比glob()（仅当前目录），更适配解压后可能的多层子目录结构
Path.with_name(新文件名)	pathlib.Path	基于原路径， 仅修改文件名 ，保留父目录，返回新的 Path 对象	不修改实际文件，仅生成路径；自动保留父目录，无需手动拼接路径
Path.rename(新路径)	pathlib.Path	将文件 / 目录从原路径 重命名 / 移动 至新路径，实际修改文件系统	若新路径已存在会报错；需捕获异常，处理文件被占用 / 权限不足等场景

6. 单元测试 (Unit Test)

您可以在 node_pdf_to_md.py 文件底部直接运行以下测试代码：

```
1  if __name__ == "__main__":
2
3      # 单元测试：验证PDF转MD全流程
4      logger.info("==== 开始node_pdf_to_md节点单元测试 =====")
5
6      from app.utils.path_util import PROJECT_ROOT
7      logger.info(f"测试获取根地址：{PROJECT_ROOT}")
8
9      test_pdf_name = os.path.join("doc", "hak180产品安全手册.pdf")
10     test_pdf_path = os.path.join(PROJECT_ROOT, test_pdf_name)
11
12     # 构造测试状态
13     test_state = create_default_state(
14         task_id="test_pdf2md_task_001",
15         pdf_path=test_pdf_path,
16         local_dir=os.path.join(PROJECT_ROOT, "output")
17     )
18
19     node_pdf_to_md(test_state)
20
21     logger.info("==== 结束node_pdf_to_md节点单元测试 =====")
```

测试结果：



5.3 图片处理 (node_md_img)

文件: app/import_process/agent/nodes/node_md_img.py

相关工具类位置: app/clients/minio_utils.py, app/utils/task_utils.py

节点作用: 实现文档的“多模态语义对齐”。将 Markdown 中的本地图片路径转换为在线可访问的 MinIO 对象存储链接，并利用多模态大模型（VLM）生成图片内容的文字摘要，填入 alt 属性，使纯文本检索模型也能理解图片内容，将图片处理成需要的格式[多模态模型对图片的识别](自己文件服务器url地址，避免相对地址切换无法访问问题)

实现思路:

- 图文解耦与持久化:** 将图片从本地文件系统迁移到 MinIO 对象存储，实现计算节点与存储分离，确保知识库在不同环境下的可访问性。
- 语义增强:** 引入 qwen3-vl-flash 或其他 VLM 模型（多模态模型），对每张图片进行“看图说话”，将视觉信息转化为文本摘要。这使得原本不可被搜索的图片，能够通过文本语义被检索到（如搜索“架构图”能找到对应的图片）。
- 速率控制:** 在调用 VLM API 时加入速率限制（Rate Limit），防止因图片过多触发并发风控。

步骤分解

本节点负责处理 Markdown 文件中的图片，实现多模态信息的融合。

- 初始化与上下文获取 (Step 1):** 从 state 中读取 Markdown 文件路径和内容。
- 扫描与筛选图片 (Step 2):** 扫描 Markdown 中引用的本地图片，校验图片是否存在。
- 图片内容总结 (Step 3):** 使用多模态大模型（如 qwen3-vl-flash 或其他 VL 模型）对图片生成中文摘要。为了避免 API 限流，实现了令牌桶算法进行速率控制（Rate Limit）。
- 上传与替换 (Step 4):**
 - 清理 MinIO 中对应的旧图片目录。
 - 将图片批量上传到 MinIO 对象存储。
 - 将 Markdown 中的本地图片路径替换为 MinIO 的 HTTP URL，并将生成的图片摘要填入 Markdown 图片的 Alt 文本中。
- 备份与保存 (Step 5):** 将处理后的内容保存为 _new.md 文件，并更新 state 中的路径。

基于LangChain大模型的配置和类

添加全局配置 .env文件

```
1 # api_key申请地址：https://bailian.console.aliyun.com/cn-beijing/?spm=5176.29597918.J_SEsSjsNv72yRuRFS2Vkn0.2.4d877b08ThdGtP&tab=model#/api-key
2 # 大模型的文档地址：https://bailian.console.aliyun.com/cn-beijing/?spm=5176.29597918.J_SEsSjsNv72yRuRFS2Vkn0.2.4d877b08ThdGtP&tab=doc#/doc
3 # Qwen3系列Flash模型，实现思考模式和非思考模式的有效融合，可在对话中切换模式。复杂推理类任务性能优秀，指令遵循、文本理解等能力显著提高。支持1M上下文长度，按照上下文长度进行阶梯计费。
4 LLM_DEFAULT_MODEL=qwen-flash
5 #Qwen3系列小尺寸视觉理解模型，实现思考模式和非思考模式的有效融合，效果优于开源版Qwen3-VL-30B-A3B，响应速度快。全面升级图像/视频理解，支持长视频长文档等超长上下文、
6 #空间感知与万物识别；具备视觉2D/3D定位能力，胜任复杂现实任务。
7 VL_MODEL=qwen3-vl-flash
8 OPENAI_API_KEY=your key
9 OPENAI_API_BASE=https://dashscope.aliyuncs.com/compatible-mode/v1
10 LLM_DEFAULT_TEMPERATURE=0.1
```

定义配置类，读取配置文件

位置：app/config/lm_config.py 【通用配置放在app下】

```
1 # 导入核心依赖：数据类、环境变量读取、路径处理
2 from dataclasses import dataclass
3 import os
4 from dotenv import load_dotenv
5
6 # 提前加载.env配置文件（必须在读取环境变量前执行，确保os.getenv能获取到值）
7 # 若.env不在项目根目录，可指定路径：load_dotenv(dotenv_path=Path(__file__).parent / ".env")
8 load_dotenv()
9
10
11 # 定义minerU服务配置
12 @dataclass
13 class LLMConfig:
14     base_url: str
```

```
15     api_key : str
16     lv_model: str
17     llm_model: str
18     llm_temperature: float
19
20 lm_config = LLMConfig(
21     base_url=os.getenv("OPENAI_API_BASE"),
22     api_key=os.getenv("OPENAI_API_KEY"),
23     lv_model=os.getenv("VL_MODEL"),
24     llm_model=os.getenv("LLM_DEFAULT_MODEL"),
25     llm_temperature=float(os.getenv("LLM_DEFAULT_TEMPERATURE"))
26 )
```

首先引入必要的库，并加载环境变量配置。

定义大模型工具类

位置：app/lm/llm_utils.py

```
1  # 环境配置与依赖导入
2  import os
3  from dotenv import load_dotenv
4  from langchain_openai import ChatOpenAI
5  from langchain_core.exceptions import LangChainException
6  from typing import Optional
7
8  # 项目内部依赖
9  from app.conf.lm_config import lm_config
10 from app.core.logger import logger
11
12 # 全局缓存：键为(模型名, JSON输出模式)元组，值为ChatOpenAI实例
13 # 作用：避免重复初始化客户端，提升性能，统一实例管理
14 _llm_client_cache = {}
15
16
17 def get_llm_client(model: Optional[str] = None, json_mode: bool = False) -> ChatOpenAI:
18     """
19     获取带全局缓存的LangChain ChatOpenAI客户端实例
20     适配OpenAI/千问/即梦AI等**OpenAI兼容API**，支持自定义模型和JSON标准化输出
21     核心特性：缓存机制+配置统一加载+异常精准捕获+国产模型参数适配
22
23     :param model: 模型名称，优先级：传入参数 > 配置文件lm_config.llm_model > 内置默认qwen3-32b
24     :param json_mode: 是否开启JSON输出模式，开启后返回标准json_object格式（适配结构化数据解析）
25     :return: 初始化完成的ChatOpenAI实例（优先从全局缓存获取，未命中则新建并缓存）
26     :raise ValueError: 缺失API密钥/基础地址等核心配置
27     :raise Exception: 模型初始化失败（LangChain封装层异常）
28     """
29     # 1. 确定目标模型（优先级递减，保证模型名非空）
30     target_model = model or lm_config.llm_model or "qwen3-32b"
31     # 缓存键：模型名+JSON模式，唯一标识不同配置的客户端
32     cache_key = (target_model, json_mode)
33
34     # 2. 缓存命中：直接返回已初始化的实例，避免重复创建
35     if cache_key in _llm_client_cache:
36         logger.debug(f"[LLM客户端] 缓存命中，直接返回实例：模型={target_model}，JSON模式={json_mode}")
37         return _llm_client_cache[cache_key]
38
39     # 3. 核心配置校验：拦截缺失的API关键配置，提前抛出明确异常
40     if not lm_config.api_key:
41         raise ValueError("[LLM客户端] 配置缺失：请在.env中配置OPENAI_API_KEY（大模型API密钥）")
42     if not lm_config.base_url:
43         raise ValueError("[LLM客户端] 配置缺失：请在.env中配置OPENAI_API_BASE（API接口基础地址）")
44     logger.info(f"[LLM客户端] 开始初始化新实例：模型={target_model}，JSON模式={json_mode}")
45
46     # 4. 配置参数组装：区分「国产模型私有参数」和「OpenAI通用参数」
47     # extra_body：千问/即梦等国产模型专属私有参数（LangChain透传至API）
48     extra_body = {"enable_thinking": False} # 千问专属：关闭思考链输出，减少冗余内容
49     # model_kwargs：OpenAI通用参数，所有兼容API均支持
50     model_kwargs = {}
51     if json_mode:
52         # 开启JSON标准输出模式，强制模型返回可解析的json_object
53         model_kwargs["response_format"] = {"type": "json_object"}
```

```
54         logger.debug(f"[LLM客户端] 已开启JSON输出模式，模型将返回标准JSON结构")
55
56     # 5. 客户端初始化：捕获LangChain封装层异常，抛出更友好的提示
57     try:
58         llm_client = ChatOpenAI(
59             model=target_model, # 目标模型名
60             temperature=lm_config.llm_temperature or 0.1, # 低温度保证输出确定性（0~1）
61             api_key=lm_config.api_key, # API密钥
62             base_url=lm_config.base_url, # API基础地址（适配国产模型代理地址）
63             extra_body=extra_body, # 国产模型私有参数透传
64             model_kwargs=model_kwargs, # OpenAI通用参数
65         )
66     except LangChainException as e:
67         raise Exception(f"[LLM客户端] 模型【{target_model}】初始化失败（LangChain层）：{str(e)}") from e
68
69     # 6. 新实例存入全局缓存，供后续调用复用
70     _llm_client_cache[cache_key] = llm_client
71     logger.info(f"[LLM客户端] 实例初始化成功并缓存：模型={target_model}，JSON模式={json_mode}")
72
73     return llm_client
74
75
76 # 测试示例：验证客户端创建、缓存机制及日志输出
77 if __name__ == "__main__":
78     logger.info("===== 开始执行LLM客户端工具测试 =====")
79     try:
80         # 测试1：默认配置（默认模型+普通模式）
81         client1 = get_llm_client()
82         logger.info("✅ 测试1通过：默认配置客户端创建成功")
83
84         # 测试2：指定多模态模型（qwen-vl-plus）+ 普通模式
85         client2 = get_llm_client(model="qwen-vl-plus")
86         logger.info("✅ 测试2通过：指定多模态模型客户端创建成功")
87
88         # 测试3：同一模型+模式，验证缓存命中
89         client3 = get_llm_client(model="qwen-vl-plus")
90         logger.info(f"✅ 测试3通过：缓存机制验证成功，client2与client3为同一实例：{client2 is client3}")
91
92         # 测试4：开启JSON输出模式
93         client4 = get_llm_client(model="qwen3-32b", json_mode=True)
94         logger.info("✅ 测试4通过：JSON输出模式客户端创建成功")
95
96     except Exception as e:
97         logger.error(f"❌ LLM客户端工具测试失败：{str(e)}", exc_info=True)
98     finally:
99         logger.info("===== LLM客户端工具测试结束 =====")
```

工具类支持: MinIO 客户端

封装 MinIO 客户端的初始化过程，支持从环境变量读取配置。
特别之处在于，初始化时会自动检查并创建默认的 Bucket（如果不存在），减少手动运维成本。
采用模块级变量 minio_client 作为单例，避免重复建立连接。

步骤1：使用Docker启动Minio

```
1 docker run -d --name minio \
2     -p 9000:9000 -p 9001:9001 \
3     -e "MINIO_ROOT_USER=minioadmin" \
4     -e "MINIO_ROOT_PASSWORD=minioadmin" \
5     -v $(pwd)/volumes/minio/data:/data \
6     quay.io/minio/minio server /data --console-address ":9001"
7
8 docker stop minio
9 docker start minio
10 # 实时查看日志（按 Ctrl+C 退出）
11 docker logs -f minio
```

MinIO 在 2025 年 5 月之后的社区版 中，完全移除了 Web 控制台（9001 端口）的权限管理入口（包括 Identity、Policies 等菜单），仅保留「文件 / 桶的基础浏览上传功能」，这是官方对社区版的功能精简（商业版仍保留完整控制台权限功能）。

步骤2：编写minio使用工具类

Java代码回忆：

```
1 public class App {
2     public static void main(String[] args) throws IOException, NoSuchAlgorithmException, InvalidKeyException {
3
4         try {
5             //构造MinIO Client （登录）
6             MinioClient minioClient = MinioClient.builder()
7                 .endpoint("http://192.168.10.101:9000")
8                 .credentials("minioadmin", "minioadmin")
9                 .build();
10
11             //创建hello-minio桶
12             boolean found = minioClient.bucketExists(BucketExistsArgs.builder().bucket("hello-minio").build());
13             if (!found) {
14                 //创建hello-minio桶
15                 minioClient.makeBucket(MakeBucketArgs.builder().bucket("hello-minio").build());
16                 //设置hello-minio桶的访问权限
17                 String policy = ""
18                     {
19                     "Statement" : [ {
20                         "Action" : "s3:GetObject",
21                         "Effect" : "Allow",
22                         "Principal" : "*",
23                         "Resource" : "arn:aws:s3:::hello-minio/*"
24                     } ],
25                     "Version" : "2012-10-17"
26                 }"";
27                 minioClient.setBucketPolicy(SetBucketPolicyArgs.builder().bucket("hello-minio").config(policy).build());
28             } else {
29                 System.out.println("Bucket 'hello-minio' already exists.");
30             }
31
32             //上传图片
33             minioClient.uploadObject(
34                 UploadObjectArgs.builder()
35                     .bucket("hello-minio")
36                     .object("公寓-外观.jpg")
37                     .filename("D:\\workspace\\hello-minio\\src\\main\\resources\\公寓-外观.jpg")
38                     .build());
39             System.out.println("上传成功");
40         } catch (MinioException e) {
41             System.out.println("Error occurred: " + e);
42         }
43     }
44 }
```

添加minio全局配置 .env文件

```
1 #minio客户端
2 MINIO_ENDPOINT=http://47.94.86.115:9000
3 MINIO_ACCESS_KEY=minioadmin
4 MINIO_SECRET_KEY=minioadmin
5 MINIO_BUCKET_NAME=knowledge-base-files
6 MINIO_IMG_DIR=/upload-images
```

定义配置类，读取配置文件

位置： app/config/minio_config.py

```
1 # 导入核心依赖：数据类、环境变量读取、路径处理
2 from dataclasses import dataclass
3 import os
4 from dotenv import load_dotenv
5
6 # 提前加载.env配置文件（确保os.getenv能获取到MinIO相关配置）
7 load_dotenv()
8
9
10 # 定义MinIO对象存储服务配置（与LLMConfig风格一致，字段对应.env配置项）
11 @dataclass
12 class MinIOConfig:
```

```
13     endpoint: str      # MinIO服务地址（含http/https和端口）
14     access_key: str    # MinIO访问密钥（对应MINIO_ACCESS_KEY）
15     secret_key: str    # MinIO秘钥（对应MINIO_SECRET_KEY）
16     bucket_name: str   # MinIO默认存储桶名（知识库文件专用）
17     minio_img_dir: str #Minio存储图片的文件夹
18
19
20 # 实例化MinIO配置对象，自动从.env读取配置并绑定
21 minio_config = MinIOConfig(
22     endpoint=os.getenv("MINIO_ENDPOINT"),
23     access_key=os.getenv("MINIO_ACCESS_KEY"),
24     secret_key=os.getenv("MINIO_SECRET_KEY"),
25     bucket_name=os.getenv("MINIO_BUCKET_NAME"),
26     minio_img_dir=os.getenv("MINIO_IMG_DIR")
27 )
```

工具类文件: app/clients/minio_utils.py

```
1  # 导入Python内置模块
2  import os
3  import json
4  # 导入MinIO官方Python SDK核心类
5  from minio import Minio
6  # 项目内部配置与日志
7  from app.conf.minio_config import minio_config
8  from app.core.logger import logger
9
10 # 全局MinIO客户端对象，初始化后供全项目调用
11 minio_client = None
12
13 try:
14     # 初始化MinIO客户端实例
15     minio_client = Minio(
16         endpoint=minio_config.endpoint,
17         access_key=minio_config.access_key,
18         secret_key=minio_config.secret_key,
19         secure=False # 内网/本地部署用HTTP，公网部署需改为True并配置SSL
20     )
21     bucket_name = minio_config.bucket_name
22
23     # 检查存储桶是否存在，不存在则自动创建
24     if not minio_client.bucket_exists(bucket_name):
25         logger.info(f"MinIO存储桶[{bucket_name}]不存在，开始创建")
26         minio_client.make_bucket(bucket_name)
27         logger.info(f"MinIO存储桶[{bucket_name}]创建成功")
28     else:
29         logger.info(f"MinIO存储桶[{bucket_name}]已存在，无需重复创建")
30
31     # 配置存储桶公网只读策略：允许匿名用户通过URL直接访问桶内文件
32     bucket_policy = {
33         "Version": "2012-10-17",
34         "Statement": [{
35             "Effect": "Allow",
36             "Principal": {"AWS": ["*"]}, # *表示所有匿名用户（S3兼容标识）
37             "Action": ["s3:GetObject"], # 仅授权文件获取/访问操作
38             "Resource": [f"arn:aws:s3:::{bucket_name}/*"]
39         }]
40     }
41     minio_client.set_bucket_policy(bucket_name, json.dumps(bucket_policy))
42     logger.info(f"MinIO存储桶[{bucket_name}]已配置公网只读策略，支持匿名URL访问")
43
44 except Exception as e:
45     # 捕获初始化异常，记录错误日志并置空客户端
46     logger.error(f"MinIO客户端初始化失败，错误信息：{str(e)}", exc_info=True)
47     minio_client = None
48
49
50 def get_minio_client():
51     """
52     获取全局初始化的MinIO客户端实例
53     :return: 已初始化的Minio对象 / None（初始化失败时）
54     """
```

55	<code>return minio_client</code>
----	----------------------------------

1. 导入与配置

首先引入必要的库，并加载环境变量配置。

```
1 import os
2 import re
3 import sys
4 import base64
5 from pathlib import Path
6 from typing import Dict, List, Tuple
7 from collections import deque
8
9 # MinIO相关依赖
10 from minio import Minio
11 from minio.deleteobjects import DeleteObject
12
13 # 【核心改造1：移除原生OpenAI，导入LangChain工具类和多模态消息模块】
14 from app.clients.minio_utils import get_minio_client
15 from app.import_process.agent.state import ImportGraphState
16 from app.utils.task_utils import add_running_task
17 # LLM客户端工具类（核心复用，替换原生OpenAI调用）
18 from app.lm.lm_utils import get_llm_client
19 # LangChain多模态依赖（消息构造+异常捕获）
20 from langchain.messages import HumanMessage
21 from langchain_core.exceptions import LangChainException
22 # 项目配置
23 from app.conf.minio_config import minio_config
24 from app.conf.lm_config import lm_config
25 # 项目日志工具（统一使用）
26 from app.core.logger import logger
27 # api访问限速工具
28 from app.utils.rate_limit_utils import apply_api_rate_limit
29 # 提示词加载工具
30 from app.core.load_prompt import load_prompt
31
32 # MinIO支持的图片格式集合（小写后缀，统一匹配标准）
33 IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".gif", ".bmp", ".webp"}
34
35 def is_supported_image(filename: str) -> bool:
36     """
37     判断文件是否为MinIO支持的图片格式（后缀不区分大小写）
38     :param filename: 文件名（含后缀）
39     :return: 支持返回True，否则False
40     """
41     return os.path.splitext(filename)[1].lower() in IMAGE_EXTENSIONS
```

2. 主流程定义

node_md_img 是本节点的入口函数，它定义了整个图片处理的流水线。我们先定义好步骤，具体的实现细节在后续部分展开。

```
1 def node_md_img(state: ImportGraphState) -> ImportGraphState:
2     """
3     MD文件图片处理核心节点 - 五步法完成图片全流程处理
4     核心流程：
5     1. 初始化获取MD内容、文件路径、图片文件夹路径
6     2. 扫描图片文件夹，筛选MD中实际引用的支持格式图片
7     3. 调用多模态大模型为图片生成内容摘要
8     4. 将图片上传至MinIO，替换MD中本地图片路径为MinIO访问URL，并填充图片摘要
9     5. 备份原MD文件，保存处理后的新MD文件并更新状态
10    :param state: 导入流程全局状态对象，包含task_id、md_path、md_content等核心参数
11    :return: 更新后的全局状态对象（md_content/md_path为处理后新值）
12    """
13    # 记录当前运行任务，用于任务监控和状态追踪
14    add_running_task(state["task_id"], sys._getframe().f_code.co_name)
15
16    # 步骤1：初始化数据，获取MD核心信息
17    md_content, path_obj, images_dir = step_1_get_content(state)
18    state["md_content"] = md_content
19
```

```
20 # 无图片文件夹，直接跳过所有图片处理逻辑
21 if not images_dir.exists():
22     logger.info(f"图片文件夹不存在，跳过图片处理：{images_dir.absolute()}")
23     return state
24
25 # 初始化MinIO客户端，失败则终止流程
26 minio_client = get_minio_client()
27 if not minio_client:
28     logger.warning("MinIO客户端初始化失败，已跳过图片处理全流程")
29     return state
30
31 # 步骤2：扫描并筛选MD中引用的支持格式图片
32 # (image_file, img_path, context_list[0])
33 targets = step_2_scan_images(md_content, images_dir)
34 if not targets:
35     logger.info("未检测到MD中引用的支持格式图片，跳过后续处理")
36     return state
37
38 # 步骤3：调用多模态大模型生成图片摘要（修复原代码传参错误：使用文件主名而非MD内容）
39 summaries = step_3_generate_summaries(path_obj.stem, targets)
40
41 # 步骤4：上传图片至MinIO，替换MD图片路径并填充摘要
42 new_md_content = step_4_upload_and_replace(minio_client, path_obj.stem, targets, summaries, md_content)
43 state["md_content"] = new_md_content
44
45 # 步骤5：备份并保存新MD文件，更新状态中的文件路径
46 new_md_file_name = step_5_backup_new_md_file(state['md_path'], new_md_content)
47 state["md_path"] = new_md_file_name
48 logger.info(f"MD图片处理完成，新文件已保存：{new_md_file_name}")
49
50 return state
```

3. 步骤 1: 获取内容与路径

这一步负责从 state 中获取文件路径，读取文件内容，并确定图片存放的目录。

```
1 # 步骤1：初始化MD核心数据，获取内容、文件路径、图片文件夹路径
2 def step_1_get_content(state: ImportGraphState) -> Tuple[str, Path, Path]:
3     """
4     从全局状态中提取并初始化MD处理所需核心数据
5     :param state: 导入流程全局状态对象
6     :return: 三元组(MD文件内容，MD文件路径对象，图片文件夹路径对象)
7     :raise FileNotFoundError: 当状态中无有效MD文件路径时抛出
8     """
9     md_file_path = state["md_path"]
10    # 校验MD文件路径有效性
11    if not md_file_path:
12        raise FileNotFoundError(f"全局状态中无有效MD文件路径：{state['md_path']}")
13
14    path_obj = Path(md_file_path)
15    # 优先使用状态中已存在的MD内容，无则从文件读取
16    if not state["md_content"]:
17        with open(path_obj, "r", encoding="utf-8") as f:
18            md_content = f.read()
19        logger.debug(f"从文件读取MD内容完成，文件大小：{len(md_content)} 字符")
20    else:
21        md_content = state["md_content"]
22        logger.debug(f"从全局状态获取MD内容完成，内容大小：{len(md_content)} 字符")
23
24    # 图片文件夹固定为MD文件同级的images目录
25    images_dir = path_obj.parent / "images"
26    return md_content, path_obj, images_dir
```

4. 步骤 2: 图片扫描

这一步扫描 Markdown 文件中引用的本地图片，并检查文件是否存在。同时提取图片的上下文（前后文），用于后续生成摘要。

```
1 def is_supported_image(filename: str) -> bool:
2     """
3     判断文件是否为MinIO支持的图片格式（后缀不区分大小写）
4     :param filename: 文件名（含后缀）
```

```
5         :return: 支持返回True , 否则False
6         """
7         return os.path.splitext(filename)[1].lower() in IMAGE_EXTENSIONS
8
9
10 def find_image_in_md(md_content: str, image_filename: str, context_len: int = 100) -> List[Tuple[str, str]]:
11     """
12     查找MD内容中指定图片的所有引用位置，并返回每个位置的上下文文本
13     :param md_content: MD文件完整内容
14     :param image_filename: 图片文件名（含后缀）
15     :param context_len: 上下文截取长度，默认前后各100字符
16     :return: 上下文列表，每个元素为(上文，下文)元组，无匹配则返回空列表
17     """
18     # 转义图片文件名特殊字符，避免正则语法错误；编译正则提升匹配效率
19     # r 全称是 raw string（原始字符串），作用是：告诉 Python 解释器：不要处理字符串里的转义字符（如 \、\n、\t 等），按字面意思解析。
20     pattern = re.compile(r"!\[.*?\]\(.*?" + re.escape(image_filename) + r".*?\)")
21     results = []
22
23     # 迭代查找所有MD图片标签匹配项
24     for m in pattern.finditer(md_content):
25         start, end = m.span()
26         # 截取匹配位置的上文和下文（防止索引越界）
27         pre_text = md_content[max(0, start - context_len):start]
28         post_text = md_content[end:min(len(md_content), end + context_len)]
29         # 打印图片上下文，便于调试
30         logger.debug(f"图片[{image_filename}]匹配到引用，上文：{pre_text.strip()}")
31         logger.debug(f"图片[{image_filename}]匹配到引用，下文：{post_text.strip()}")
32         results.append((pre_text, post_text))
33
34     if not results:
35         logger.debug(f"MD内容中未找到图片[{image_filename}]的引用")
36     return results
37
38
39 # 步骤2：扫描图片文件夹，筛选MD中实际引用的支持格式图片
40 def step_2_scan_images(md_content: str, images_dir: Path) -> List[Tuple[str, str, Tuple[str, str]]]:
41     """
42     扫描图片文件夹，过滤出「支持格式+MD中实际引用」的图片，组装处理元数据
43     :param md_content: MD文件完整内容
44     :param images_dir: 图片文件夹路径对象
45     :return: 待处理图片列表，每个元素为(图片文件名，图片完整路径，图片上下文)元组
46     """
47     targets = []
48     # 遍历图片文件夹所有文件
49     for image_file in os.listdir(images_dir):
50         # 过滤非支持格式的图片
51         if not is_supported_image(image_file):
52             logger.debug(f"图片格式不支持，跳过：{image_file}")
53             continue
54
55         # 组装图片完整路径
56         img_path = str(images_dir / image_file)
57         # 查找图片在MD中的引用上下文
58         context_list = find_image_in_md(md_content, image_file)
59
60         # 过滤MD中未引用的图片
61         if not context_list:
62             logger.warning(f"图片未在MD中引用，跳过处理：{image_file}")
63             continue
64
65         # 组装待处理图片元数据，取第一个匹配的上下文
66         targets.append((image_file, img_path, context_list[0]))
67         logger.info(f"图片加入待处理列表：{image_file}")
68
69     logger.info(f"图片扫描完成，共筛选出待处理图片：{len(targets)} 张")
70     return targets
```

为避免与其他re方法混淆，整理关键方法差异，现在场景用finditer是最优选择：

方法	核心作用	返回值	适用场景
compile	预编译正则，提升效率	Pattern 对象	多次调用同一正则时（推荐）

方法	核心作用	返回值	适用场景
finditer	迭代查找所有非重叠匹配项	Match 迭代器	大文本 / 需获取所有匹配结果（你的场景）
findall	查找所有匹配项，返回字符串列表	列表（[str, ...]）	简单场景，仅需匹配字符串
search	查找第一个匹配项	单个 Match 对象 / None	仅需第一个匹配结果

明确 2 个基础概念

1. **.* 贪婪匹配**：.* 表示**匹配任意字符（.）任意次数（*，0 次或多次）**，**贪婪特性会让它尽可能多的匹配字符**，直到字符串末尾，再回头验证是否符合正则整体规则；
2. **.*? 非贪婪匹配**：在*后加?就变成了非贪婪模式，**会让它尽可能少的匹配字符**，只要满足正则整体规则，就立刻停止匹配，不会继续向后延伸。

匹配 Markdown 图片语法：!\[.*?\]\(.*?图片名.*?\)，对应的 Markdown 图片格式是 ![图片描述](图片路径)，比如一段包含**多张图片**的 Markdown 内容：

```
1 | 这是第一张图![风景](a.jpg)，这是第二张图![人物](b.jpg)，这是第三张图![动物](c.jpg)
```

用贪婪匹配 **.***（会出现**匹配过度**，完全不符合需求），如果把你的正则写成贪婪模式（去掉?）：!\[.*\]\(.*a.jpg.*\)，匹配结果会是：

```
1 | ![风景](a.jpg)，这是第二张图![人物](b.jpg)，这是第三张图![动物](c.jpg)
```

贪婪的.*会「贪心」的尽可能多匹配：

- 第一个.*匹配从[开始，一直到**最后一个**]（而不是第一个]）；
- 第二个.* 匹配从(开始，一直到最后一个)；

最终把从第一张图开始到最后一张图结束的所有内容都匹配成了「一个结果」，这就是**匹配过度**，完全无法精准获取单个图片标签。

假设有字符串：ab123ab456ab，正则匹配a.*b（贪婪）和a.*?b（非贪婪）：

- 贪婪a.*b：匹配结果是ab123ab456ab（从第一个 a 到最后一个 b，尽可能多匹配）；
- 非贪婪a.*?b：匹配结果是ab（从第一个 a 到**第一个**b，尽可能少匹配）。

5. 步骤 3: 图片摘要

提取提示词，建议将所有的提示词提取放在外部，统一管理。我们存储的文件是根路径下 prompts文件夹

文件：prompts/image_summary.prompt

```
1 | 这是“{root_folder}”文件中的一张图片，图片上文部分为“{image_content[0]}”，
2 | 下文部分为“{image_content[1]}”，请用中文简要总结这张图片的内容，用于 Markdown 图片标题，控制在50字以内。
```

加载提示词和格式化工具类

文件：app/core/load_prompt.py

```
1 | from pathlib import Path
2 | from app.utils.path_util import PROJECT_ROOT
3 | from app.core.logger import logger # 可选，加日志更友好
4 |
5 | def load_prompt(name: str, **kwargs) -> str:
6 |     """
7 |     加载提示词并渲染变量占位符
8 |     :param name: 提示词文件名（不带.prompt后缀，如image_summary）
9 |     :param **kwargs: 需渲染的变量键值对（如root_folder="测试文件", image_content=("上文内容", "下文内容"））
10 |    :return: 渲染后的最终提示词字符串
11 |    """
12 |    # 1. 拼接提示词路径（你的原有逻辑，完全保留）
13 |    prompt_path = PROJECT_ROOT / 'prompts' / f'{name}.prompt'
14 |
15 |    # 2. 校验文件是否存在（可选，避免文件不存在直接报错）
16 |    if not prompt_path.exists():
17 |        raise FileNotFoundError(f"提示词文件不存在：{prompt_path.absolute()}")
18 |
19 |    # 3. 读取纯文本提示词（你的原有逻辑）
20 |    raw_prompt = prompt_path.read_text(encoding='utf-8')
21 |
22 |    # 4. 核心：如果传了参数，渲染占位符；没传参，直接返回原文本
23 |    if kwargs:
24 |        rendered_prompt = raw_prompt.format(**kwargs)
```

```
25     logger.debug(f"提示词渲染成功，替换变量：{list(kwargs.keys())}")
26     return rendered_prompt
27 return raw_prompt
28
29
30
31 if __name__ == '__main__':
32     # 测试：传入参数渲染占位符（和业务代码中实际使用方式一致）
33     root_folder = "h13070使用说明书" # 要替换的文件名称
34     image_content = ("这是图片的上文内容", "这是图片的下文内容") # 要替换的上下文
35     # 调用时传入所有需要渲染的变量（键名必须和.prompt中的占位符完全一致）
36     final_prompt = load_prompt(
37         name='image_summary',
38         root_folder=root_folder, # 对应{root_folder}
39         image_content=image_content # 对应{image_content[0]}、{image_content[1]}
40     )
41     print("✅ 渲染后的最终提示词：")
42     print(final_prompt)
```

这一步调用多模态大模型（如 GPT-4o 或 Qwen-VL）来生成图片的中文摘要，作为 Markdown 图片的 Alt Text。为了避免触发 API 速率限制，我们实现了简单的令牌桶算法。

```
1 def encode_image_to_base64(image_path: str) -> str:
2     """
3     将本地图片文件编码为Base64字符串（用于多模态大模型输入）
4     :param image_path: 图片本地完整路径
5     :return: 图片的Base64编码字符串（UTF-8解码）
6     """
7     with open(image_path, "rb") as img_file:
8         base64_str = base64.b64encode(img_file.read()).decode("utf-8")
9     logger.debug(f"图片Base64编码完成，文件：{image_path}，编码后长度：{len(base64_str)}")
10    return base64_str
11
12
13 def summarize_image(image_path: str, root_folder: str, image_content: Tuple[str, str]) -> str:
14     """
15     调用多模态大模型生成图片内容摘要（适配LangChain工具类，复用项目统一LLM客户端）
16     生成的摘要用于Markdown图片标题，严格控制50字以内中文描述
17     :param image_path: 图片本地完整路径
18     :param root_folder: 文档所属文件夹/主名，为大模型提供上下文
19     :param image_content: 图片在MD中的上下文元组，格式(上文文本，下文文本)
20     :return: 图片内容摘要（异常时返回默认值"图片描述"）
21     """
22     # 将图片编码为Base64，适配多模态大模型输入要求
23     base64_image = encode_image_to_base64(image_path)
24     try:
25         # 1. 获取项目统一LLM客户端（自动缓存，传入多模态模型名）
26         lvm_client = get_llm_client(model=lm_config.lv_model)
27
28         # 加载并渲染提示词（核心：传入所有占位符对应的变量）
29         prompt_text = load_prompt(
30             name="image_summary", # 提示词文件名（不带.prompt）
31             root_folder=root_folder, # 对应{root_folder}
32             image_content=image_content # 对应{image_content[0]}、{image_content[1]}
33         )
34
35         # 2. 构造LangChain标准多模态HumanMessage（兼容千问/OpenAI等视觉模型）
36         messages = [
37             HumanMessage(
38                 content=[
39                     # 文本提示词：携带上下文，限定摘要规则
40                     {
41                         "type": "text",
42                         "text": prompt_text
43                     },
44                     # 多模态核心：Base64编码图片数据
45                     {
46                         "type": "image_url",
47                         "image_url": {
48                             "url": f"data:image/jpeg;base64,{base64_image}"
49                         }
50                     }
51                 ]
52             )
53         ]
```

```
51         ]
52     )
53 ]
54
55 # 3. LangChain标准调用：invoke方法（工具类已封装超时/重试等参数）
56 response = lvm_client.invoke(messages)
57
58 # 4. 解析响应（LangChain统一返回content字段，统一格式无需多层解析）
59 summary = response.content.strip().replace("\n", "")
60 logger.info(f"图片摘要生成成功：{image_path}，摘要：{summary}")
61 return summary
62
63 except LangChainException as e:
64     logger.error(f"图片摘要生成失败（LangChain框架异常）：{image_path}，错误信息：{str(e)}")
65     return "图片描述"
66 except Exception as e:
67     logger.error(f"图片摘要生成失败（系统异常）：{image_path}，错误信息：{str(e)}")
68     return "图片描述"
69
70
71 def step_3_generate_summaries(doc_stem: str, targets: List[Tuple[str, str, Tuple[str, str]]],
72                               requests_per_minute: int = 9) -> Dict[str, str]:
73     """
74     步骤3：批量为待处理图片生成内容摘要，带API速率限制防止触发大模型限流
75     :param doc_stem: 文档文件名（不含后缀），作为大模型prompt上下文
76     :param targets: 待处理图片列表，元素为(图片文件名，图片完整路径，图片上下文)
77     :param requests_per_minute: 每分钟最大API请求数，默认9次（按大模型限制调整）
78     :return: 图片摘要字典，键：图片文件名，值：图片内容摘要
79     """
80     summaries = {}
81     request_times = deque() # 外部初始化请求时间队列，跨循环复用
82
83     for img_file, image_path, context in targets:
84         # 直接调用抽离的公共工具方法，参数和原逻辑完全一致
85         apply_api_rate_limit(request_times, requests_per_minute, window_seconds=60)
86         logger.debug(f"开始生成图片摘要：{image_path}")
87         summaries[img_file] = summarize_image(image_path, root_folder=doc_stem, image_content=context)
88
89     logger.info(f"图片摘要批量生成完成，共处理{len(summaries)}张图片")
90     return summaries
```

6. 步骤 4: 上传与替换

这一步将图片上传到 MinIO 对象存储，并将 Markdown 中的本地图片路径替换为 MinIO 的 URL，同时填入生成的摘要。

<

knowledge-base-files / upload-images

Create new path

☐	▲ Name	Last Modified	Size
☐	华为擎云B530用户指南-(PUCZ,Windows11_03,...		-
☐	华为擎云W525用户指南-(PGUW-WBX0,KOS&U...		-
☐	华为显示器B3-211H用户指南-(NSN-21BZ,02,z...		-
☐	AolynkCB304nCable网桥用户手册-5W100-整...		-
☐	H3CLA2608室内无线网关用户手册-6W100-整...		-
☐	hak180产品安全手册		-
☐	hl3040网络说明书		-

```
1 def step_4_upload_and_replace(minio_client: Minio, doc_stem: str, targets: List[Tuple[str, str, Tuple[str, str]]],
2     summaries: Dict[str, str], md_content: str) -> str:
3     """
4     步骤4：核心流程-图片上传MinIO + 合并摘要&URL + 替换MD图片引用
5     完整流程：清理MinIO旧目录 → 批量上传新图片 → 合并摘要和URL → 替换MD内容
6     :param minio_client: 初始化完成的MinIO客户端对象
7     :param doc_stem: 文档文件名（不含后缀），作为MinIO上传子目录名（按文档隔离）
8     :param targets: 待处理图片列表，元素为(图片文件名，图片完整路径，图片上下文)
9     :param summaries: 图片摘要字典，键：图片文件名，值：内容摘要
10    :param md_content: 原始MD文件内容
11    :return: 图片引用替换后的新MD内容
12    """
13    # 构造MinIO上传目录：配置根目录 + 文档主名（去除空格，避免路径问题）
14    minio_img_dir = minio_config.minio_img_dir
15    upload_dir = f"{minio_img_dir}/{doc_stem}".replace(" ", "")
16
17    # 步骤1：清理该文档对应的MinIO旧目录，保证幂等性
18    clean_minio_directory(minio_client, upload_dir)
19    # 步骤2：批量上传图片至MinIO，获取URL映射
20    urls = upload_images_batch(minio_client, upload_dir, targets)
21    # 步骤3：合并图片摘要和URL，过滤上传失败的图片
22    image_info = merge_summary_and_url(summaries, urls)
23    # 步骤4：替换MD内容中的本地图片引用为MinIO远程引用
24    if image_info:
25        md_content = process_md_file(md_content, image_info)
26
27    return md_content
28
29 def clean_minio_directory(minio_client: Minio, prefix: str) -> None:
30     """
31     幂等性清理MinIO指定目录下的所有旧文件，防止重名文件内容混淆和垃圾文件堆积
32     幂等性：多次调用结果一致，无文件时不报错
33     :param minio_client: 初始化完成的MinIO客户端对象
34     :param prefix: MinIO目录前缀（要清理的目录路径）
35     """
36     try:
37         # 列出指定前缀下的所有对象（递归遍历子目录）
38         objects_to_delete = minio_client.list_objects(
```

```
39         bucket_name=minio_config.bucket_name,
40         prefix=prefix,
41         recursive=True
42     )
43     # 构造删除对象列表
44     delete_list = [DeleteObject(obj.object_name) for obj in objects_to_delete]
45
46     if delete_list:
47         logger.info(f"开始清理MinIO旧文件，待删除文件数：{len(delete_list)}，目录：{prefix}")
48         # 批量删除对象
49         errors = minio_client.remove_objects(minio_config.bucket_name, delete_list)
50         # 遍历删除错误信息，记录异常
51         for error in errors:
52             logger.error(f"MinIO文件删除失败：{error}")
53     else:
54         logger.debug(f"MinIO目录无旧文件，无需清理：{prefix}")
55 except Exception as e:
56     logger.error(f"MinIO目录清理失败：{prefix}，错误信息：{str(e)}")
57
58
59 def upload_images_batch(minio_client: Minio, upload_dir: str, targets: List[Tuple[str, str, Tuple[str, str]]]) -> Dict[
60     str, str]:
61     """
62     批量上传待处理图片至MinIO，返回图片文件名与访问URL的映射关系
63     :param minio_client: 初始化完成的MinIO客户端对象
64     :param upload_dir: MinIO上传根目录
65     :param targets: 待处理图片列表，元素为(图片文件名，图片完整路径，图片上下文)
66     :return: 图片URL字典，键：图片文件名，值：MinIO访问URL
67     """
68     urls = {}
69     for img_file, img_path, _ in targets:
70         # 构造MinIO对象名称
71         object_name = f"{upload_dir}/{img_file}"
72         logger.debug(f"构造MinIO对象名称完成：{object_name}")
73         # 上传单张图片并获取URL
74         """
75         := 是 Python 3.8+ 引入的海象运算符 (Walrus Operator)，核心作用是 **「表达式内赋值 + 结果判断」一体化**：
76         在执行判断、循环等逻辑的同一个表达式中，完成变量赋值和赋值结果的使用 / 判断，替代传统「先赋值、后判断」的两行代码，让逻辑更简洁。
77         """
78         if img_url := upload_to_minio(minio_client, img_path, object_name):
79             urls[img_file] = img_url
80     logger.info(f"图片批量上传完成，成功上传{len(urls)}/{len(targets)}张图片")
81     return urls
82
83 def upload_to_minio(minio_client: Minio, local_path: str, object_name: str) -> str | None:
84     """
85     将单张本地图片上传至MinIO对象存储，并返回公网可访问URL
86     :param minio_client: 初始化完成的MinIO客户端对象
87     :param local_path: 图片本地完整路径
88     :param object_name: MinIO中要存储的对象名称（带目录）
89     :return: 图片MinIO访问URL（上传失败返回None）
90     """
91     try:
92         logger.info(f"开始上传图片至MinIO：本地路径={local_path}，MinIO对象名={object_name}")
93         # 上传本地文件至MinIO（fput_object：文件流上传，适合大文件）
94         minio_client.fput_object(
95             bucket_name=minio_config.bucket_name, # MinIO存储桶名（从配置读取）
96             object_name=object_name, # MinIO对象名称
97             file_path=local_path, # 本地文件路径
98             # 自动推断图片Content-Type（如image/png、image/jpeg）
99             # 入参：文件路径字符串（可带目录，如/a/b/test.jpg、demo.tar.gz）；
100             # 返回值：元组(root, ext)，其中：
101             # root：文件主名（含目录，去掉最后一个后缀的完整部分）；
102             # ext：文件后缀（以.开头，仅包含最后一个扩展名，如.jpg、.gz，无后缀则为空字符串""）；
103             # 关键规则：仅识别 ** 最后一个.** 作为后缀分隔符，多后缀文件仅拆分最后一个（如test.tar.gz拆分为("test.tar", ".gz")）。
104             content_type=f"image/{os.path.splitext(local_path)[1][1:]}"
105         )
106
107         # 处理路径特殊字符，避免URL解析错误
108         object_name = object_name.replace("\\", "%5C")
109         # 根据配置选择HTTP/HTTPS协议
110         protocol = "https" if minio_config.minio_secure else "http"
```

```
111     # 构造MinIO基础访问URL
112     base_url = f"{protocol}://{minio_config.endpoint}/{minio_config.bucket_name}"
113     # 拼接完整图片访问URL base_url 后面带 / 中间直接两个字符串拼接即可
114     img_url = f"{base_url}{object_name}"
115     logger.info(f"图片上传成功，访问URL：{img_url}")
116     return img_url
117 except Exception as e:
118     logger.error(f"图片上传MinIO失败：{local_path}，错误信息：{str(e)}")
119     return None
120
121 def merge_summary_and_url(summaries: Dict[str, str], urls: Dict[str, str]) -> Dict[str, Tuple[str, str]]:
122     """
123     合并图片摘要字典和URL字典，过滤掉上传失败无URL的图片
124     :param summaries: 图片摘要字典，键：图片文件名，值：内容摘要
125     :param urls: 图片URL字典，键：图片文件名，值：MinIO访问URL
126     :return: 合并后的图片信息字典，键：图片文件名，值：(摘要，URL)元组
127     """
128     image_info = {}
129     # 遍历摘要字典，仅保留有对应URL的图片
130     for image_file, summary in summaries.items():
131         if url := urls.get(image_file):
132             image_info[image_file] = (summary, url)
133     logger.info(f"图片摘要与URL合并完成，有效图片信息{len(image_info)}条")
134     return image_info
135
136 def process_md_file(md_content: str, image_info: Dict[str, Tuple[str, str]]) -> str:
137     """
138     核心功能：替换MD内容中的本地图片引用为MinIO远程引用
139     替换规则：![原描述](本地路径) → ![图片摘要](MinIO访问URL)
140     :param md_content: 原始MD文件内容
141     :param image_info: 合并后的图片信息字典，键：图片文件名，值：(摘要，URL)
142     :return: 替换后的新MD内容
143     """
144     for img_filename, (summary, new_url) in image_info.items():
145         # 正则匹配MD图片标签，忽略大小写，兼容不同路径写法
146         # 正则规则：![任意描述](任意路径+图片文件名+任意后缀)
147         pattern = re.compile(
148             r"!\[.*?\]\(.*?" + re.escape(img_filename) + r".*?\)",
149             re.IGNORECASE
150         )
151         # 替换匹配内容：使用新摘要作为图片描述，新URL作为图片路径
152         # - 如果你的 summary 和 new_url 是完全可控的纯文本（不含反斜杠）：这两种写法确实 一模一样 。
153         # - 如果你想写出“防御性代码”（Defensive Code），防止未来某天被特殊字符坑：请坚持使用 Lambda 写法。它是最稳健、最安全的做法。
154         # md_content = pattern.sub(lambda m: f"![{summary}]({new_url})", md_content)
155         md_content = pattern.sub( f"![{summary}]({new_url})", md_content)
156         logger.debug(f"完成MD图片引用替换：{img_filename} → {new_url}")
157
158     logger.info(f"MD文件图片引用替换完成，共替换{len(image_info)}处图片引用")
159     logger.debug(f"替换后MD内容：{md_content[:500]}..." if len(md_content) > 500 else f"替换后MD内容：{md_content}")
160     return md_content
```

7. 步骤 5: 备份文件

最后，我们将处理后的 Markdown 内容保存为一个新的文件，通常命名为 *_new.md。

```
1 def step_5_backup_new_md_file(origin_md_path: str, md_content: str) -> str:
2     """
3     步骤5：将处理后的MD内容保存为新文件（原文件不变，避免数据丢失）
4     新文件命名规则：原文件名 + _new.md（如test.md → test_new.md）
5     :param origin_md_path: 原始MD文件完整路径
6     :param md_content: 处理后的新MD内容
7     :return: 新MD文件的完整路径
8     """
9     # 构造新文件路径：替换原后缀为 _new.md
10    new_md_file_name = os.path.splitext(origin_md_path)[0] + "_new.md"
11
12    # 写入新MD内容（覆盖写入，若文件已存在则更新）
13    with open(new_md_file_name, "w", encoding="utf-8") as f:
14        f.write(md_content)
15
16    logger.info(f"处理后MD文件已保存，新文件路径：{new_md_file_name}")
17    return new_md_file_name
```

8. 单元测试 (Unit Test)

您可以在 node_md_img.py 文件底部直接运行以下测试代码：

```
1 if __name__ == "__main__":
2     """本地测试入口：单独运行该文件时，执行MD图片处理全流程测试"""
3     from app.utils.path_util import PROJECT_ROOT
4     logger.info(f"本地测试 - 项目根目录：{PROJECT_ROOT}")
5
6     # 测试MD文件路径（需手动将测试文件放入对应目录）
7     test_md_name = os.path.join(r"output\h13040网络说明书", "h13040网络说明书.md")
8     test_md_path = os.path.join(PROJECT_ROOT, test_md_name)
9
10    # 校验测试文件是否存在
11    if not os.path.exists(test_md_path):
12        logger.error(f"本地测试 - 测试文件不存在：{test_md_path}")
13        logger.info("请检查文件路径，或手动将测试MD文件放入项目根目录的output目录下")
14    else:
15        # 构造测试状态对象，模拟流程入参
16        test_state = {
17            "md_path": test_md_path,
18            "task_id": "test_task_123456",
19            "md_content": ""
20        }
21        logger.info("开始本地测试 - MD图片处理全流程")
22        # 执行核心处理流程
23        result_state = node_md_img(test_state)
24        logger.info(f"本地测试完成 - 处理结果状态：{result_state}")
```

5.4 文档切分 (node_document_split)

文件: kb/import_process/node_document_split.py

节点作用与实现思路

节点作用: 将长文档转化为适合向量检索的微观语义单元（Chunk）。通过智能切分策略，在保持语义完整性的前提下，控制切片长度，最大化检索的召回率和准确率。

切分思路:RAG 文档切块可按文档类型采用“四类策略 + 参数模板”统一落地：

第一类说明书/手册/技术文档，采用“标题层级切分 + 超长二次切分”，建议块大小 700 ~ 1200 token、重叠 80 ~ 120 token，目标是保证步骤和操作链条完整；

第二类论文/研究报告/长叙述文本，采用“语义切分 + 适度重叠”，建议块大小 800 ~ 1500 token、重叠 120 ~ 200 token，重点保持论点与论据的连续性（语义识别可以使用标题或者配合大模式识别后切割）；

第三类对话记录/日志/工单/表格，采用“固定窗口 + 强重叠”或按轮次/行切分，建议块大小 300 ~ 800 token、重叠 100 ~ 150 token，优先保证上下文承接；

第四类法律/合同/制度文档，采用“按条款整段切分”，建议块大小 500 ~ 1500 token、重叠 50 ~ 100 token，原则是不拆条款、不破坏引用关系。

通用控制线可设为：最小块不低于 300 token，常规块 600 ~ 1200 token，硬上限不超过 1500 token；若单块超限，先按语义边界二次拆分，再补充重叠，不做句中硬截断。

实现思路:

- 结构化优先:** 摒弃简单的固定字符数切分，优先依据 Markdown 标题层级（Headers）进行逻辑分块，确保同一知识点的连贯性。
- 递归优化:** 对于超长段落，采用“段落 -> 句子”的递归切分策略。先按换行符切，若仍超长则按句号切，确保切片不会在句子中间截断。
- 上下文保留:** 在每个切片中保留 file_title 和 parent_title（父级标题），为后续的 LLM 回答提供丰富的上下文背景。

步骤分解

本节点负责将 Markdown 文本切分为适合检索的 Chunk（片段）。采用“先按标题切分，再按长度优化”的策略。

- 获取输入 (Step 1):** 从 state 中提取 Markdown 内容、文件标题等输入数据。
- 标题初切 (Step 2):** 基于 Markdown 标题语法 (#) 进行第一轮粗略切分，保留层级结构。

3. **无标题兜底** (Step 3): 处理无标题的纯文本文件，避免丢失内容。
4. **精细化处理** (Step 4): 对过长的 Chunk 进行递归切分（按段落/句子），对过短的 Chunk 进行合并。
5. **打印统计** (Step 5): 输出切分结果的统计信息。
6. **备份与更新** (Step 6): 将切分结果更新到 state 并备份到本地 JSON 文件。

1. 导入与配置

引入必要的正则表达式、JSON 处理库，并定义切分相关的阈值常量。

```
1 import re
2 import json
3 import os
4 import sys
5 # 统一类型注解，避免混用any/Any
6 from typing import List, Dict, Any, Tuple
7 # LangChain文本分割器（标注核心用途，便于理解）
8 from langchain_text_splitters import RecursiveCharacterTextSplitter
9
10 # 项目内部工具/状态/日志导入（保持原有路径）
11 from app.utils.task_utils import add_running_task
12 from app.import_process.agent.state import ImportGraphState
13 from app.core.logger import logger # 项目统一日志工具，核心替换print
14
15 # --- 配置参数 (Configuration) ---
16 # 单个Chunk最大字符长度：超过则触发二次切分（适配大模型上下文窗口）
17 DEFAULT_MAX_CONTENT_LENGTH = 2000
18 # 短Chunk合并阈值：同父标题的短Chunk会被合并，减少碎片化
19 MIN_CONTENT_LENGTH = 500
```

2. 主流程定义

定义 LangGraph 的节点入口函数，串联所有步骤。

```
1 def node_document_split(state: ImportGraphState) -> ImportGraphState:
2     """
3     【核心节点】文档切分主节点（node_document_split）
4     整体流程：加载输入→按MD标题初切→无标题兜底→长切短合→统计输出→结果备份
5     核心目的：将长MD文档切分为长度适中的Chunk，适配大模型上下文窗口和向量检索
6     后续扩展点：可在各步骤间新增Chunk元信息补充、自定义切分规则、向量入库前置处理等
7     :param state: 项目状态字典（ImportGraphState），必须包含md_content/task_id；可选local_dir/max_content_length/file_title
8     :return: 更新后的状态字典，新增chunks键（存储最终处理后的Chunk列表，每个Chunk为含title/content/parent_title的字典）
9     """
10    # 初始化当前节点信息，用于任务监控和日志溯源
11    node_name = sys._getframe().f_code.co_name
12    logger.info(f">>> 开始执行核心节点：【文档切分】{node_name}")
13    # 将当前节点加入运行中任务，更新全局任务状态
14    add_running_task(state["task_id"], node_name)
15
16    try:
17        # ===== 步骤1：加载并标准化输入数据 =====
18        # 作用：从状态字典提取MD内容/文件标题/Chunk最大长度，统一换行符消除系统差异，做空值兜底
19        # 输出：标准化后的md_content、文件标题、单个Chunk最大长度；无有效MD内容则直接终止节点执行
20        content, file_title, max_len = step_1_get_inputs(state)
21        if content is None:
22            logger.info(f">>> 节点执行终止：{node_name}（无有效MD内容）")
23            return state
24
25        # ===== 步骤2：按MD标题进行初次切分 =====
26        # 作用：基于Markdown标题（#/#/#/#/#）切分文档为独立章节，自动跳过代码块内的伪标题，保证章节语义完整 #
27        # 输出：初切后的章节列表、识别到的有效标题数量、MD原始文本总行数（为后续统计/日志使用）
28        sections, title_count, lines_count = step_2_split_by_titles(content, file_title)
29
30        # ===== 步骤3：无标题场景兜底处理 =====
31        # 作用：解决MD文档无任何标题的边界情况，避免后续切分逻辑异常
32        # 输出：有标题则返回步骤2的章节列表；无标题则将全文封装为单个「无标题」章节，保证数据格式统一
33        sections = step_3_handle_no_title(content, sections, title_count, file_title)
34
35        # ===== 步骤4：Chunk精细化处理（长切短合） =====
36        # 作用：核心切分逻辑，先将超长章节按「段落→句子」二次切分，再合并同父标题的过短章节，减少碎片化
37        # 额外处理：对所有Chunk做parent_title兜底，适配Milvus向量库必填字段要求
```

```
38         # 输出：长度适中、语义完整、低碎片化的最终Chunk列表（可直接用于向量入库/大模型调用）
39         sections = step_4_refine_chunks(sections, max_len)
40
41         # ===== 步骤5：输出文档切分统计信息 =====
42         # 作用：打印核心统计数据，便于监控切分效果、调试问题（原始行数/最终Chunk数/首个Chunk预览）
43         # 输出：无返回值，仅通过logger输出标准化统计日志
44         step_5_print_stats(lines_count, sections)
45
46         # ===== 步骤6：Chunk结果本地JSON备份 + 状态更新 =====
47         # 作用1：将最终Chunk列表备份到local_dir目录的chunks.json，便于后续问题排查、数据复用
48         # 作用2：将Chunk列表写入状态字典，传递给下一个节点（如向量入库、大模型摘要等）
49         # 输出：状态字典新增chunks键；无local_dir则跳过备份，不影响主流程
50         state["chunks"] = sections
51         step_6_backup(state, sections)
52
53         # 节点执行完成日志
54         logger.info(f">>> 核心节点执行完成：【文档切分】{node_name}，已生成{len(sections)}个有效Chunk，结果已写入状态字典")
55
56     except Exception as e:
57         # 全局异常捕获：保证节点执行失败不崩溃整个流程，记录详细错误日志便于排查
58         logger.error(f">>> 核心节点执行失败：【文档切分】{node_name}，错误信息：{str(e)}", exc_info=True)
59
60     # 返回更新后的状态字典，传递Chunk结果到下游节点
61     return state
```

3. 步骤 1: 获取输入 (Step 1: Get Inputs)

从 State 中提取必要的数据，并进行基础清洗。

```
1 def step_1_get_inputs(state: ImportGraphState) -> Tuple[Any, str, int]:
2     """
3     【步骤1】获取并预处理输入数据
4     功能：从状态字典中提取MD内容/文件标题/最大长度，做基础标准化
5     :param state: 项目状态字典（ImportGraphState），包含md_content等核心键
6     :return: 标准化后的MD内容/文件标题/单个Chunk最大长度（无内容则返回None,None,None）
7     """
8     # 从状态中提取MD原始内容
9     content = state.get("md_content")
10    # 空内容兜底：无MD内容则直接返回，终止后续处理
11    if not content:
12        logger.warning("状态字典中无有效MD内容，终止文档切分")
13        return None, None, None
14
15    # 基础标准化：统一换行符，避免Windows/Linux换行符差异导致的后续处理异常
16    # 原始混合换行："# HL3070说明书\r\n## 产品概述\nHL3070是扫描枪\r\n\r\n### 操作步骤"
17    # 统一后："# HL3070说明书\n## 产品概述\nHL3070是扫描枪\n\n### 操作步骤"
18    content = content.replace("\r\n", "\n").replace("\r", "\n")
19    # 提取文件标题：有则用，无则默认"Unknown File"
20    file_title = state.get("file_title", "Unknown File")
21    # 提取最大Chunk长度：有则用状态中的配置，无则用全局默认值
22    max_len = DEFAULT_MAX_CONTENT_LENGTH
23
24    logger.info(f"步骤1：输入数据加载完成，文件标题：{file_title}，最大Chunk长度：{max_len}")
25    return content, file_title, max_len
```

4. 步骤 2: 标题初切 (Step 2: Split by Titles)

基于 Markdown 的标题语法（#）进行第一轮粗略切分。

```
1 def step_2_split_by_titles(content: str, file_title: str) -> Tuple[List[Dict[str, Any]], int, int]:
2     """
3     【步骤2】按Markdown标题初次切分（核心：按#分级切分，跳过代码块内标题）
4     LangChain前置预处理：将整份MD按标题拆分为独立章节，为后续精细化切分做基础
5     :param content: 标准化后的MD完整内容（字符串）
6     :param file_title: 所属文件标题，用于标记章节归属
7     :return: 切分后的章节列表/有效标题数量/原始文本总行数
8     """
9     # 正则匹配Markdown 1-6级标题（核心规则，适配缩进/标准格式）
10    # ^\s*：行首允许0/多个空格/Tab（兼容缩进的标题）
11    # #{1,6}：匹配1-6个#（对应MD1-6级标题）
12    # \s+：#后必须有至少1个空格（区分#是标题还是普通文本）
```

```
13 # .+ : 标题文字至少1个字符（避免空标题）
14 title_pattern = r'^\s*#{1,6}\s+.'
```

5. 步骤 3: 无标题兜底 (Step 3: Handle No Title)

处理那些没有任何 Markdown 标题的纯文本文件。

```
1 def step_3_handle_no_title(content: str, sections: List[Dict[str, Any]], title_count: int, file_title: str) -> List[Dict[str, Any]]:
2     """
3     【步骤3】无标题兜底处理
4     功能：若MD中未识别到任何标题，将全文作为一个整体处理，避免后续逻辑异常
5     :param content: 标准化后的MD完整内容
6     :param sections: 步骤2切分后的章节列表
7     :param title_count: 步骤2识别的有效标题数量
8     :param file_title: 所属文件标题
9     :return: 兜底后的章节列表
10    """
11    if title_count == 0:
12        # 无标题情况：替换为单章节，标题为"无标题"
13        logger.warning(f"步骤3：未识别到任何MD标题，将全文作为单个章节处理，文件：{file_title}")
14        return [{"title": "无标题", "content": content, "file_title": file_title}]
15    # 有标题情况：直接返回步骤2的结果
16    logger.debug(f"步骤3：检测到{title_count}个有效标题，无需兜底处理")
17    return sections
```

6. 步骤 4: 精细化 (Step 4: Refine Chunks)

调用辅助函数，对过长或过短的 Chunk 进行二次处理。

```
1 def _split_long_section(section: Dict[str, Any], max_length: int = DEFAULT_MAX_CONTENT_LENGTH) -> List[Dict[str, Any]]:
2     """
3     【辅助函数】超长章节二次切分（核心适配LangChain分割器）
4     功能：单个章节内容超限时，按「段落→句子→空格」从粗到细切分，保留语义
5     切分规则：1.先按空行(段落) 2.再按换行 3.最后按中英文标点/空格
6     :param section: 原始章节字典，必须包含content键，可选title/file_title等
7     :param max_length: 单个Chunk最大字符长度，默认使用全局配置
8     :return: 切分后的子章节列表，每个子章节带父标题/序号等元信息
9     """
10    # 内容空值兜底：无内容直接返回原章节
11    content = section.get("content", "") or ""
12    # 长度未超限，无需切分，直接返回原章节（列表格式保持统一）
13    if len(content) <= max_length:
14        return [section]
15
16    # 标准化预处理：统一换行符，避免不同系统(\r\n\n)导致的切分异常
17    content = content.replace("\r\n", "\n").replace("\r", "\n")
18    # 提取章节标题，用于组装子Chunk前缀（保留标题上下文）
19    title = section.get("title", "") or ""
20    # 标题前缀：带空行分隔，与正文区分开
21    prefix = f"{title}\n\n" if title else ""
22    # 计算正文可用长度：总长度 - 标题前缀长度（避免标题占满Chunk额度）
23    available_len = max_length - len(prefix)
24    # 极端情况：标题长度超过阈值，无法切分，返回原章节
25    if available_len <= 0:
26        logger.warning(f"章节标题过长，无法切分：{title[:20]}...")
27        return [section]
28
29    # 清理正文重复标题：避免原章节中正文开头重复标题，导致子Chunk内容冗余
30    body = content
31    if title and body.lstrip().startswith(title):
32        body = body[body.find(title) + len(title):].lstrip()
33
34    # 初始化LangChain递归分割器（核心工具：按优先级分隔符切分，保留语义）
35    # separators：分隔符优先级（从粗到细），优先按大语义单元切分，最后才硬拆
36    splitter = RecursiveCharacterTextSplitter(
37        chunk_size=available_len, # 正文部分最大长度（已扣除标题）
38        chunk_overlap=0, # 无重叠：按标题切分后语义完整，无需重叠
39        # 分隔符优先级：空行(段落)→换行→中文标点→英文标点→空格，最后硬拆
40        separators=["\n\n", "\n", "。", "！", "？", "；", ".", "!", "?", ";", " "],
41    )
42
43    # 切分正文并组装子章节（带完整元信息，便于溯源）
44    sub_sections = []
45    for idx, chunk in enumerate(splitter.split_text(body), start=1):
46        # 清理空内容：跳过切分后的空字符串
47        text = chunk.strip()
48        if not text:
49            continue
50        # 组装子Chunk完整内容：标题前缀 + 切分后的正文
51        full_text = (prefix + text).strip()
52        # 子章节元信息：保留父级关联，添加序号，便于后续检索/溯源
53        sub_sections.append({
54            "title": f"{title}-{idx}" if title else f"chunk-{idx}", # 子Chunk标题（带序号）
55            "content": full_text, # 切分后的完整内容
56            "parent_title": title, # 父章节标题（用于后续合并）
57            "part": idx, # 子Chunk序号
58            "file_title": section.get("file_title"), # 所属文件标题
59        })
60
61    logger.debug(f"超长章节切分完成：{title} → 生成{len(sub_sections)}个子Chunk")
62    return sub_sections
63
64 def _merge_short_sections(sections: List[Dict[str, Any]], min_length: int = MIN_CONTENT_LENGTH) -> List[Dict[str, Any]]:
65     """
66     【辅助函数】过短章节合并（减少碎片化，提升检索效果）
67     核心规则：仅合并「同父标题」且「当前块长度不足阈值」的相邻Chunk，避免跨章节合并
68     :param sections: 待合并的Chunk列表（通常是_split_long_section切分后的结果）
```

```
69 :param min_length: 最小长度阈值，低于此值的Chunk会被合并
70 :return: 合并后的Chunk列表，长度适中，保留元信息
71 """
72 # 边界处理：空列表直接返回，避免后续索引报错
73 if not sections:
74     logger.debug("待合并Chunk列表为空，直接返回")
75     return []
76
77 merged_sections = [] # 最终合并结果
78 current_chunk = None # 迭代累加器：保存当前待合并的Chunk
79
80 for sec in sections:
81     # 初始化：第一个Chunk直接作为当前待合并块
82     if current_chunk is None:
83         current_chunk = sec
84         continue
85
86     # 合并条件：1.当前块长度不足阈值 2.与下一块同父标题（同属一个原章节）
87     is_current_short = len(current_chunk["content"]) < min_length
88     is_same_parent = current_chunk.get("parent_title") == sec.get("parent_title")
89
90     if is_current_short and is_same_parent:
91         # 合并前清理：去掉下一块开头重复的父标题，避免内容冗余
92         parent_title = sec.get("parent_title", "")
93         next_content = sec["content"]
94         if parent_title and next_content.startswith(parent_title):
95             next_content = next_content[len(parent_title):].lstrip()
96         # 合并内容：空行分隔，保证格式整洁
97         current_chunk["content"] += "\n\n" + next_content
98         # 更新子Chunk序号：保留最新序号，便于溯源
99         if "part" in sec:
100             current_chunk["part"] = sec["part"]
101         logger.debug(f"合并短Chunk：{current_chunk.get('parent_title')} → 累计长度{len(current_chunk['content'])}")
102     else:
103         # 不满足合并条件：将当前块加入结果，切换为新的待合并块
104         merged_sections.append(current_chunk)
105         current_chunk = sec
106
107 # 循环结束后，将最后一个待合并块加入结果
108 if current_chunk is not None:
109     merged_sections.append(current_chunk)
110
111 logger.debug(f"短Chunk合并完成：原{len(sections)}个 → 合并后{len(merged_sections)}个")
112 return merged_sections
113
114 def step_4_refine_chunks(sections: List[Dict[str, Any]], max_len: int) -> List[Dict[str, Any]]:
115     """
116     【步骤4】Chunk精细化处理（核心：长切短合，适配大模型/检索）
117     执行流程：1.切分超长章节 2.合并过短章节 3.父标题兜底（适配Milvus向量库schema）
118     :param sections: 步骤3处理后的章节列表
119     :param max_len: 单个Chunk最大字符长度
120     :return: 长度适中、低碎片化的最终Chunk列表
121     """
122     # 边界处理：最大长度无效（空/≤0），直接返回原章节，避免切分异常
123     if not max_len or max_len <= 0:
124         logger.warning(f"步骤4：Chunk最大长度配置无效（{max_len}），跳过精细化处理")
125         return sections
126
127     # 阶段1：切分超长章节 → 所有章节长度控制在max_len内
128     refined_split = []
129     for sec in sections:
130         # 对每个章节执行超长切分，结果平铺加入列表（避免嵌套）
131         # extend 的作用就是：把另一个列表（或可迭代对象）里的“元素”，一个个拆出来，直接追加到当前列表的尾部
132         refined_split.extend(_split_long_section(sec, max_len))
133     logger.info(f"步骤4-1：超长章节切分完成，共生成{len(refined_split)}个初始子Chunk")
134
135     # 阶段2：合并过短章节 → 减少碎片化，提升后续检索/大模型调用效果
136     final_sections = _merge_short_sections(refined_split)
137     logger.info(f"步骤4-2：过短章节合并完成，最终得到{len(final_sections)}个Chunk")
138
139     # 阶段3：父标题兜底 → 适配Milvus向量库schema（parent_title为必填字段）
140     # 兜底规则：无parent_title则用自身title，title也无则填空字符串
```

```
141     for sec in final_sections:
142         if not isinstance(sec, dict):
143             continue
144
145         # 补全缺失的part字段（默认0），适配Milvus schema
146         if "part" not in sec:
147             sec["part"] = 0
148
149         if not sec.get("parent_title"):
150             sec["parent_title"] = sec.get("title") or ""
151     logger.debug(f"步骤4-3：父标题兜底完成，所有Chunk均包含parent_title字段")
152
153     return final_sections
```

7. 步骤 5: 打印统计 (Step 5: Print Stats)

在控制台输出切分结果的简要统计。

```
1 def step_5_print_stats(lines_count: int, sections: List[Dict[str, Any]]) -> None:
2     """
3     【步骤5】输出文档切分统计信息（日志记录，便于监控/调试）
4     :param lines_count: MD原始文本总行数
5     :param sections: 最终处理后的Chunk列表
6     """
7     chunk_num = len(sections)
8     # 输出核心统计信息：原始行数/最终Chunk数/首个Chunk预览
9     logger.info("-" * 50 + " 文档切分统计信息 " + "-" * 50)
10    logger.info(f"MD原始文本总行数：{lines_count}")
11    logger.info(f"最终生成Chunk数量：{chunk_num}")
12    if sections:
13        first_title = sections[0].get("title", "无标题")
14        logger.info(f"首个Chunk标题预览：{first_title}")
15    logger.info("-" * 110)
```

8. 步骤 6: 备份与更新 (Step 6: Backup & Update)

更新 State 并将结果备份到本地。

```
1 def step_6_backup(state: ImportGraphState, sections: List[Dict[str, Any]]) -> None:
2     """
3     【步骤6】Chunk结果本地JSON备份（便于调试/问题排查，保留处理结果）
4     :param state: 项目状态字典，需包含local_dir（备份目录）
5     :param sections: 最终处理后的Chunk列表
6     """
7     # 提取备份目录：无则直接返回，不执行备份
8     local_dir = state.get("local_dir")
9     if not local_dir:
10        logger.warning("步骤6：未配置备份目录（local_dir），跳过Chunk结果备份")
11        return
12
13    try:
14        # 创建备份目录：已存在则不报错（exist_ok=True）
15        os.makedirs(local_dir, exist_ok=True)
16        # 拼接备份文件路径：local_dir + chunks.json（固定文件名，便于查找）
17        backup_path = os.path.join(local_dir, "chunks.json")
18        # 写入JSON文件：保留中文/格式化缩进，便于人工查看
19        with open(backup_path, "w", encoding="utf-8") as f:
20            """
21            sections是Python 嵌套数据结构（List[Dict[str, Any]]，列表里装字典，字典里可能嵌套字符串 / 数字等），而普通文件写入
22            （如f.write(sections)）仅支持写入字符串，直接写 Python 数据结构会报错。
23            json.dump的核心作用就是：将 Python 原生数据结构（列表、字典、字符串、数字等）直接序列化并写入 JSON 文件，无需手动转换为字符串，
24            同时保证数据格式规范、可跨语言 / 跨场景读取，完美适配「Chunk 列表备份」的需求。
25            """
26            json.dump(
27                sections,
28                f,
29                #开启 True："title": "\u4e00\u7ea7\u6807\u9898"（乱码，无法直接看）；
30                #开启 False："title": "一级标题"（正常中文，人工可直接阅读）。
31                ensure_ascii=False, # 保留中文，不转义为\u编码
32                indent=2             # 格式化缩进，便于阅读
33            )
```

```
34         logger.info(f"步骤6：Chunk结果备份成功，备份文件路径：{backup_path}")
35     except Exception as e:
36         # 备份失败仅记录日志，不终止主流程
37         logger.error(f"步骤6：Chunk结果备份失败，错误信息：{str(e)}", exc_info=False)
38
```

9. 单元测试 (Unit Test)

您可以在 node_document_split.py 文件底部直接运行以下测试代码。包含了**模拟数据测试**和**联合 Step 3 的真实文件测试**。

```
1  if __name__ == '__main__':
2      """
3      单元测试：联合node_md_img（图片处理节点）进行集成测试
4      测试条件：1.已配置.env（MinIO/大模型环境） 2.存在测试MD文件 3.能导入node_md_img
5      测试流程：先运行图片处理→再运行文档切分，验证端到端流程
6      """
7
8      """本地测试入口：单独运行该文件时，执行MD图片处理全流程测试"""
9      from app.utils.path_util import PROJECT_ROOT
10     from app.import_process.agent.nodes.node_md_img import node_md_img
11
12     logger.info(f"本地测试 - 项目根目录：{PROJECT_ROOT}")
13
14     # 测试MD文件路径（需手动将测试文件放入对应目录）
15     test_md_name = os.path.join(r"output\h13040网络说明书", "h13040网络说明书.md")
16     test_md_path = os.path.join(PROJECT_ROOT, test_md_name)
17
18     # 校验测试文件是否存在
19     if not os.path.exists(test_md_path):
20         logger.error(f"本地测试 - 测试文件不存在：{test_md_path}")
21         logger.info("请检查文件路径，或手动将测试MD文件放入项目根目录的output目录下")
22     else:
23         # 构造测试状态对象，模拟流程入参
24         test_state = {
25             "md_path": test_md_path,
26             "task_id": "test_task_123456",
27             "md_content": "",
28             "file_title": "h13040网络说明书",
29             "local_dir":os.path.join(PROJECT_ROOT, "output"),
30         }
31         logger.info("开始本地测试 - MD图片处理全流程")
32         # 执行核心处理流程
33         result_state = node_md_img(test_state)
34         logger.info(f"本地测试完成 - 处理结果状态：{result_state}")
35         logger.info("\n=== 开始执行文档切分节点集成测试 ===")
36
37         logger.info(">> 开始运行当前节点：node_document_split（文档切分）")
38         final_state = node_document_split(result_state)
39         final_chunks = final_state.get("chunks", [])
40         logger.info(f"✅ 测试成功：最终生成{len(final_chunks)}个有效Chunk{final_chunks}")
```

5.5 主体识别 (node_item_name_recognition)

文件: app/import_process/agent/nodes/node_item_name_recognition.py

1. 节点作用与实现思路

节点作用: 提取文档的核心实体（如产品名称、特定概念），用于后续的精确过滤和去重。这是构建“实体-切片”关联的关键一步，支持基于实体的结构化查询。

实现思路:

- 上下文压缩:** 取文档的前 K 个切片（通常包含标题和摘要）构建 Prompt，利用 LLM 强大的理解能力提取核心实体名称。
- 错误鲁棒性:** 考虑到 LLM 输出的不确定性，增加了空结果处理和默认值兜底机制，确保流程不会因识别失败而中断。
- 向量化准备:** 识别出的实体名称也会被向量化，用于在 Milvus 中进行基于实体的语义对齐（如搜索“iPhone”能关联到“苹果手机”的切片）。

2. 步骤分解

1. **导入与配置:** 引入必要的库（LangChain, Milvus, etc.）及配置参数。
2. **核心辅助函数:** 包含字符串安全转义等辅助逻辑。
3. **主流程定义:** LangGraph 节点的入口函数，串联各个步骤。
4. **步骤 1: 获取输入:** 校验 State 中的 file_title 和 chunks。
5. **步骤 2: 构建上下文:** 截取前 K 个切片作为 LLM 的识别素材。
6. **步骤 3: 调用 LLM:** 使用大模型识别商品名称，包含错误重试与兜底。
7. **步骤 4: 回填数据:** 将识别结果更新回 State 和 Chunks 元数据。
8. **步骤 5: 生成向量:** 调用 Embedding 模型生成 Dense/Sparse 向量。
9. **步骤 6: 保存结果:** 将数据写入 Milvus 向量库，并处理幂等性。
10. **单元测试:** 独立运行的测试代码，验证核心流程。

3. 准备Embedding模型和工具

3.1 什么是“生成词向量”？

词向量（Word Vector/Embedding）就是把**文字（比如“苏泊尔 5000W 大功率电磁炉”）转换成计算机能理解的数字列表（向量）**的过程。

打个比方

- 人类理解文字：“苹果手机”= 品牌（苹果）+ 品类（手机）；
- 计算机理解文字：没法直接懂“苹果手机”，但能懂 [0.23, -0.56, 1.89, ...] 这样的数字列表；
- 词向量的作用：把文字的**语义信息**（含义、特征、关联度）编码成数字，让计算机能“计算文字相似度”“分类文字”“检索相似内容”。

举个简单例子

文字	对应的词向量（简化版，实际是几百 / 几千维）
苹果手机	[0.23, -0.56, 1.89, 0.78]
华为手机	[0.21, -0.58, 1.91, 0.76]
苹果笔记本	[0.22, -0.55, 0.87, 0.79]

计算机通过对比这些数字列表的相似度，就能判断：

- “苹果手机”和“华为手机”更像（数字差异小）；
- “苹果手机”和“苹果笔记本”相似度低（数字差异大）。

3.2 “稀疏向量 + 稠密向量”

代码是基于 BGE-M3 模型生成两种词向量（这是当前主流的多模态嵌入方案）拆解：

类型	特点	用途
稠密向量（Dense Vector）	长度固定（比如 768 维 / 1024 维），每个位置都是连续数值（如 0.23、-0.56）	捕捉文字的 语义信息 （比如“苹果手机”的核心含义），适合相似度计算
稀疏向量（Sparse Vector）	长度极长（比如几十万维），但只有少数位置有非 0 值，其余都是 0	捕捉文字的 关键词 / 字面特征 （比如“苹果”“5000W”“电磁炉”），适合精准检索

BGE-M3 模型同时输出这两种向量，结合使用能兼顾“语义理解”和“精准匹配”。

3.3 安装Python依赖库

在使用模型之前，需要安装相关的 Python 依赖库。

```
1 # ===== 环境安装命令（适配BGE-M3+Milvus，GPU/CPU版区分） =====
2 # 【优先推荐-GPU版】安装CUDA 12.4版PyTorch（含torchvision/torchaudio，NVIDIA显卡GPU加速必备）
3 # 适配：有NVIDIA独显且驱动≥551.61，后续BGE-M3可开启FP16半精度推理
4 uv pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
5
6 # 【备用-CPU版】无NVIDIA显卡（AMD/Intel集显）请用此命令，直接安装CPU版PyTorch
7 # 注释掉上方GPU版命令，取消注释下方即可
8 uv add torch torchvision torchaudio
9
10 # 安装Milvus和BGE-M3核心依赖（所有环境必装，无GPU/CPU区分）
11 # pymilvus[model]：Milvus Python客户端（带模型相关依赖，适配向量入库/检索）
```

```
12 # FlagEmbedding：BGE-M3向量生成模型的核心依赖（不可替代）
13 # transformers：FlagEmbedding底层依赖，Hugging Face模型运行库
14 uv add pymilvus[model] FlagEmbedding transformers
15
16 # ⚠️：安装FlagEmbedding的时候会自动安装一个cpu版本的torch替换掉之前的gpu版本的torch，
17 # 要解决这个问题需要做以下几个步骤
18 # 步骤1 先删除已经安装的FlagEmbedding（先在pyproject.toml中确定一下自己安装的版本）
19 uv remove FlagEmbeddin
20
21 # 步骤2 将以下内容配置在pyproject.toml中
22 dependencies = [
23     其他之前安装过的配置，
24     "flagembedding>=v1.3.5",
25     "torch>=2.10.0",
26     "torchvision>=0.25.0",
27     "torchaudio>=2.10.0",
28 ]
29
30 [tool.uv.sources]
31 # 强制从 NVIDIA 源安装
32 torch = { index = "pytorch-cuda" }
33 torchvision = { index = "pytorch-cuda" }
34 torchaudio = { index = "pytorch-cuda" }
35
36 [[tool.uv.index]]
37 name = "pytorch-cuda"
38 url = "https://download.pytorch.org/whl/cu128"
39 explicit = true
40
41 # 步骤3 删除锁文件并重新锁定
42 rm uv.lock
43 uv lock
44
45 # 步骤4：重新同步环境
46 uv sync --reinstall
47
48 # 步骤5：验证
49 uv run python -c "import torch; print('GPU:', torch.cuda.is_available())"
```

CUDA 每个版本都有**最低算力要求**，CUDA 12.4 要求显卡的**CUDA 算力≥3.5**（几乎 2016 年之后的 NVIDIA 独显都满足，老款如 GTX 750 Ti 也达标），**主流显卡（RTX30/40 系、GTX16/20 系）全兼容**，几乎不用担心里程碑。

直接打开 NVIDIA 官方算力表，搜索自己的显卡型号，看对应的**Compute Capability（算力）**数值：

[NVIDIA 显卡 CUDA 算力官方查询地址](#)

- 桌面显卡看GeForce栏，笔记本显卡看GeForce Notebook栏；
- 示例：RTX 3060 算力 8.6、GTX 1650 算力 7.5、RTX 4090 算力 8.9，都远大于 3.5，完美适配 CUDA 12.4。

3.4 Embedding下载模型

如果访问 HuggingFace 较慢，可以使用阿里云(阿里巴巴通义实验室（原达摩院）)的 ModelScope 社区下载。

<https://www.modelscope.cn/models/BAAI/bge-m3>

1. 安装 modelscope 库：

```
1 uv add modelscope
```

2. 运行 Python 脚本下载：

创建一个临时的 Python 脚本（例如 download_bge.py）并运行：

```
1 from modelscope.hub.snapshot_download import snapshot_download
2
3 # 下载模型到当前目录下的 models/bge-m3 文件夹
4 model_dir = snapshot_download('BAAI/bge-m3', cache_dir='D:/ai_models/modelscope_cache/models')
5 print(f"模型已下载到: {model_dir}")
```

3. .env配置

```
1 #embedding配置
2 # BGE-M3模型本地缓存/部署路径（本地加载模型时使用，指向ModelScope下载
3 BGE_M3_PATH=D:\ai_models\modelscope_cache\models\BAAI\bge-m3
4 # BGE-M3模型官方标识（ModelScope/HuggingFace通用，拉取模型时使用）
5 BGE_M3=BAAI/bge-m3
6 # BGE-M3运行设备，cuda:0表示使用第1块GPU，cpu表示使用CPU，cuda:N表示第N+1块GPU
7 BGE_DEVICE=cuda:0
8 # BGE-M3是否开启FP16半精度推理，1=开启（GPU加速更高效），0=关闭（兼容低版本GPU/CPU）
9 BGE_FP16=1
```

4. 配置参数读取

文件：app.config.embedding_config.py

```
1 # 导入核心依赖：数据类、环境变量读取、路径处理
2 from dataclasses import dataclass
3 import os
4 from dotenv import load_dotenv
5
6 # 提前加载.env配置文件（保持和原代码一致，只需执行一次）
7 load_dotenv()
8
9 # 定义Embedding配置（适配BGE-M3的所有配置，类名embedding_config）
10 @dataclass
11 class EmbeddingConfig:
12     bge_m3_path: str # 本地模型路径
13     bge_m3: str      # 模型仓库标识
14     bge_device: str  # 运行设备(cuda:0/cpu)
15     bge_fp16: bool   # 是否开启半精度（1=True/0=False）
16
17 # 实例化配置对象，和原代码lm_config风格保持一致
18 embedding_config = EmbeddingConfig(
19     bge_m3_path=os.getenv("BGE_M3_PATH"),
20     bge_m3=os.getenv("BGE_M3"),
21     bge_device=os.getenv("BGE_DEVICE"),
22     # 特殊处理：将.env中的1/0转为布尔值，兼容常见的数字/字符串格式
23     bge_fp16=os.getenv("BGE_FP16") in ("1", "True", "true", 1)
24 )
```

3.5 工具代码导入

文件：app.lm.embedding_utils.py

```
1 from pymilvus.model.hybrid import BGEM3EmbeddingFunction
2 from app.core.logger import logger
3 from app.conf.embedding_config import embedding_config
4
5 # 模型单例对象，避免重复初始化
6 _bge_m3_ef = None
7
8 def get_bge_m3_ef():
9     """
10     获取BGE-M3模型单例对象，自动加载环境变量配置
11     :return: 初始化完成的BGEM3EmbeddingFunction实例
12     """
13     global _bge_m3_ef
14     # 单例模式：已初始化则直接返回，避免重复加载模型
15     if _bge_m3_ef is not None:
16         logger.debug("BGE-M3模型单例已存在，直接返回实例")
17         return _bge_m3_ef
18
19     # 从环境变量加载配置，无配置则使用默认值
20     # 本地有可以使用本地地址！没有使用 "BAAI/bge-m3" 会自动下载！如果云端部署也可以使用url地址！
21     model_name = embedding_config.bge_m3_path or "BAAI/bge-m3"
22     device = embedding_config.bge_device or "cpu"
23     use_fp16 = embedding_config.bge_fp16 or False
24
25     # 打印模型初始化配置，便于问题排查
26     logger.info(
27         "开始初始化BGE-M3模型",
28         extra={
29             "model_name": model_name,
```

```
30         "device": device,
31         "use_fp16": use_fp16,
32         "normalize_embeddings": True
33     }
34 )
35
36 try:
37     # 初始化BGE-M3模型，开启原生L2归一化（适配Milvus IP内积检索）
38     _bge_m3_ef = BGEM3EmbeddingFunction(
39         model_name=model_name,
40         device=device,
41         use_fp16=use_fp16,
42         normalize_embeddings=True # 模型原生对稠密+稀疏向量做L2归一化
43     )
44     logger.success("BGE-M3模型初始化成功，已开启原生L2归一化")
45     # “它把所有向量拉伸到统一长度（模长为1），让我们能在数据库中放心使用最快的内积（IP）检索，既提速又不丢精度。”
46     return _bge_m3_ef
47 except Exception as e:
48     logger.error(f"BGE-M3模型初始化失败：{str(e)}", exc_info=True)
49     raise # 向上抛出异常，由调用方处理
50
51
52 def generate_embeddings(texts):
53     """
54     为文本列表生成稠密+稀疏混合向量嵌入（模型原生L2归一化）
55     :param texts: 要生成嵌入的文本列表，单文本也需封装为列表
56     :return: 字典格式的向量结果，key为dense/sparse，对应嵌套列表/字典列表
57     :raise: 向量生成过程中的异常，由调用方捕获处理
58     """
59     # 入参合法性校验
60     if not isinstance(texts, list) or len(texts) == 0:
61         logger.warning("生成向量入参不合法，texts必须为非空列表")
62         raise ValueError("参数texts必须是包含文本的非空列表")
63
64     logger.info(f"开始为{len(texts)}条文本生成混合向量嵌入")
65     try:
66         # 加载BGE-M3模型单例
67         model = get_bge_m3_ef()
68         # 模型编码生成向量，返回dense（稠密向量）+sparse（CSR格式稀疏向量）
69         embeddings = model.encode_documents(texts)
70         logger.debug(f"模型编码完成，开始解析稀疏向量格式，共{len(texts)}条")
71
72         # 初始化稀疏向量处理结果，解析为字典格式（适配序列化/存储）
73         processed_sparse = []
74         for i in range(len(texts)):
75             # 提取第i个文本的稀疏向量索引：np.int64 → Python int（满足字典key可哈希要求）
76             sparse_indices = embeddings["sparse"].indices[
77                 embeddings["sparse"].indptr[i]:embeddings["sparse"].indptr[i + 1]
78             ].tolist()
79             # 提取第i个文本的稀疏向量权重：np.float32 → Python float（适配JSON序列化/接口返回）
80             sparse_data = embeddings["sparse"].data[
81                 embeddings["sparse"].indptr[i]:embeddings["sparse"].indptr[i + 1]
82             ].tolist()
83             # 构造{特征索引: 归一化权重}的稀疏向量字典
84             sparse_dict = {k: v for k, v in zip(sparse_indices, sparse_data)}
85             processed_sparse.append(sparse_dict)
86
87         # 构造最终返回结果，稠密向量转列表（解决numpy数组不可序列化问题）
88         result = {
89             "dense": [emb.tolist() for emb in embeddings["dense"]], # 嵌套列表，与输入文本一一对应
90             "sparse": processed_sparse # 字典列表，模型已做L2归一化
91         }
92         logger.success(f"{len(texts)}条文本向量生成完成，格式已适配工业级使用")
93         return result
94
95     except Exception as e:
96         logger.error(f"文本向量生成失败：{str(e)}", exc_info=True)
97         raise # 不吞异常，向上传递让调用方做重试/降级处理
98
99
100 """
101 核心设计亮点&适配说明：
```

1021. 模型原生归一化：开启normalize_embeddings = True，自动对稠密+稀疏向量做L2归一化，完美适配Milvus IP内积检索（单位化后IP等价于余弦，计算更快）；

1032. 彻底解决NumPy类型做key问题：sparse_indices加.tolist()，将np.int64转为Python原生int，满足字典key的可哈希要求，无报错风险；

1043. 稀疏值适配序列化：sparse_data加.tolist()，将np.float32转为Python原生float，支持JSON写入/接口返回/Milvus入库等所有场景；

1054. 单例模式优化：模型仅初始化一次，避免重复加载耗时耗资源，提升批量处理效率；

1065. 格式匹配业务调用：返回dense嵌套列表、sparse字典列表，与vector_result["dense"][0]/sparse_vector["sparse"][0]取值逻辑完美契合；

1076. 分级日志覆盖：从模型初始化、向量生成到异常报错，全流程日志记录，便于生产环境问题排查；

1087. 入参合法性校验：防止空列表/非列表入参导致的内部报错，提升工具类健壮性。

109"""

4. 准备Milvues向量数据库和工具

4.1 安装 Milvus 并完成连接测试

- 操作系统：Linux（CentOS/RHEL/Ubuntu 通用，yum 命令适用于 CentOS/RHEL，Ubuntu 需替换为 apt-get）
- 核心目标：编译安装 Python3.8（Milvus 客户端适配）+ 部署 Milvus 2.4.11 单机版 + 解决 MinIO 端口冲突 + 部署 Attu 可视化客户端
- 前置要求：服务器已安装 docker + docker compose（未安装可先执行：yum install -y docker docker-compose-plugin && systemctl start docker && systemctl enable docker）

第一步：Python3.8 环境编译安装（客户端基础！使用Docker安装无需要此步骤）

```
1 # 1. 安装 Python 编译必备的工具和依赖库
2 yum install -y zlib-devel bzip2-devel openssl-devel ncurses-devel sqlite-devel readline-devel tk-devel gcc make libffi-devel
3
4 # 2. 创建 Python3.8 专属安装目录，避免和系统Python冲突
5 mkdir -p /usr/local/python3.8
6
7 # 3. 下载 Python3.8.16 源码包（稳定版，适配Milvus客户端）
8 wget https://www.python.org/ftp/python/3.8.16/Python-3.8.16.tgz
9
10 # 4. 解压源码包
11 tar -zxvf Python-3.8.16.tgz
12
13 # 5. 进入解压目录，配置编译参数（指定安装路径+关联openssl）
14 cd Python-3.8.16
15 ./configure --prefix=/usr/local/python3.8 --with-openssl=/usr/local/openssl
16
17 # 6. 多线程编译并安装（-j $(nproc) 调用所有CPU核心，加速编译）
18 make -j $(nproc) && make install
19
20 # 7. 验证安装成功（输出版本号即成功，若提示命令不存在，需配置环境变量：ln -s /usr/local/python3.8/bin/python3 /usr/bin/python3）
21 python3 --version
```

注：Ubuntu 系统替换第一步命令为：apt-get update && apt-get install -y zlib1g-dev libbz2-dev libssl-dev libncurses5-dev libsqlite3-dev libreadline-dev libtk8.6 libgdm-dev libdb4o-cil-dev libffi-dev gcc make

第二步：部署 Milvus 2.4.11 单机版（从旧版本升级 / 全新部署通用）

```
1 # 1. 创建并进入Milvus工作目录（无则新建，有则进入原有目录）
2 mkdir -p ~/milvus && cd ~/milvus
3
4 # 2. 停止当前运行的旧版Milvus（全新部署执行此命令无影响）
5 docker compose down
6
7 # 3. 备份原有数据（关键！防止数据丢失，若为全新部署，无volumes目录可跳过此步）
8 mv volumes volumes.bak_2.3.5
9
10 # 4. 下载 Milvus 2.4.11 官方单机版docker-compose配置文件（覆盖原有文件）
11 wget https://github.com/milvus-io/milvus/releases/download/v2.4.11/milvus-standalone-docker-compose.yml -O docker-compose.yml
12
13 # 5. 启动 Milvus 2.4.11（-d 后台运行）
14 docker compose up -d
15
16 # 6. 检查Milvus运行状态（等待3-5秒，所有容器状态为 Up 即启动成功）
17 docker compose ps
18
19 # 【常用运维命令】后续管理Milvus可使用
20 # 停止Milvus：docker compose stop
21 # 重启Milvus：docker compose restart
22 # 查看运行日志（排查问题用）：docker compose logs milvus-standalone
```

第三步：解决 MinIO 端口冲突问题（修改 docker-compose.yml）

Milvus 内置 MinIO 用于存储向量数据，默认端口 9000/9001 若被本地服务占用，需修改映射端口，编辑 ~/milvus/docker-compose.yml，找到 minio 节点，修改 ports 配置：

```
1 minio:
2   container_name: milvus-minio
3   image: minio/minio:RELEASE.2023-03-20T20-16-18Z
4   environment:
5     MINIO_ACCESS_KEY: minioadmin # 内置账号，无需修改
6     MINIO_SECRET_KEY: minioadmin # 内置密码，无需修改
7   ports:
8     - "9003:9001" # 控制台端口：原9001→改为9003（避开占用）
9     - "9002:9000" # 数据端口：原9000→改为9002（避开占用）
10  volumes:
11    - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/minio:/minio_data # 数据持久化目录
12  command: minio server /minio_data --console-address ":9001" # 容器内端口不变，仅改宿主机映射
13  healthcheck:
14    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
15    interval: 30s
16    timeout: 20s
17    retries: 3
```

改完端口后，需重启 Milvus 生效：cd ~/milvus && docker compose restart

第四步：部署 Attu 2.4.0 可视化客户端（适配 Milvus 2.4.11）

```
1 # 1. 先删除旧版Attu（若未安装，此命令无影响）
2 docker rm -f attu
3
4 # 2. 启动新版Attu 2.4.0（适配Milvus 2.4.11，后台运行）
5 # 关键：MILVUS_URL 填写Milvus服务器的IP+默认端口19530（本地部署填127.0.0.1:19530，远程服务器填公网/内网IP）
6 docker run -d --name attu \
7   -p 8000:3000 \ # 宿主机8000端口映射容器3000端口
8   -e MILVUS_URL=47.94.86.115:19530 \
9   zilliz/attu:v2.4.0
```

第五步：Attu 连接测试（验证 Milvus 部署成功）

1. 服务器开放端口：若为云服务器 / 防火墙开启状态，需放行 8000 端口（Milvus 默认 19530 端口无需外部访问，Attu 用 8000 端口）

```
1 # CentOS 放行8000端口
2 firewall-cmd --add-port=8000/tcp --permanent
3 firewall-cmd --reload
```

2. 浏览器访问：打开本地浏览器，输入地址

```
1 http://<Linux服务器IP>:8000
```

- 本地虚拟机 / 服务器本地访问：http://127.0.0.1:8000
- 远程服务器访问：http://服务器公网/内网IP:8000

3. 连接 Milvus：页面无需输入账号密码，直接点击「Connect」按钮，若能进入 Attu 可视化界面，即Milvus 部署 + Attu 连接全部成功。

4.2 定义Milvus客户端工具类

步骤1：定义.env配置参数

文件：.env

```
1 # Milvus 配置
2 # 切换成你milvus的url地址
3 MILVUS_URL=http://47.94.86.115:19530
4 # 存储切片集合
5 CHUNKS_COLLECTION=kb_chunks
6 # 预留
7 ENTITY_NAME_COLLECTION=kb_graph_entity_names
8 # 存储每个文档对应的实体类
9 ITEM_NAME_COLLECTION=kb_item_names
```

步骤2：读取配置参数

文件：app.config.milvus_config.py

```
1 # 导入核心依赖（和其他配置类共用，只需导入一次）
2 from dataclasses import dataclass
3 import os
4 from dotenv import load_dotenv
5
6 # 提前加载.env配置文件（全局执行一次即可，无需重复写）
7 load_dotenv()
8
9 # ===== 其他配置类（LLM/Embedding）可放在上方，保持原有代码不变 =====
10 # ... 你的LLMConfig、EmbeddingConfig代码 ...
11
12 # 定义Milvus向量数据库配置类
13 @dataclass
14 class MilvusConfig:
15     milvus_url: str # Milvus服务端连接地址
16     chunks_collection: str # 存储切片的集合名称
17     entity_name_collection: str # 预留-实体名称集合
18     item_name_collection: str # 存储文档对应实体类的集合名称
19
20 # 实例化Milvus配置对象（和其他配置对象命名风格统一）
21 milvus_config = MilvusConfig(
22     milvus_url=os.getenv("MILVUS_URL"),
23     chunks_collection=os.getenv("CHUNKS_COLLECTION"),
24     entity_name_collection=os.getenv("ENTITY_NAME_COLLECTION"),
25     item_name_collection=os.getenv("ITEM_NAME_COLLECTION")
26 )
```

步骤3：定义Milvus客户端类

```
1 import os
2 from pymilvus import MilvusClient, AnnSearchRequest, WeightedRanker
3 from app.conf.milvus_config import milvus_config
4 from app.core.logger import logger
5
6 # 全局Milvus客户端实例，实现单例复用
7 _milvus_client = None
8
9
10 def get_milvus_client():
11     """
12     Milvus客户端单例获取方法
13     实现客户端连接复用，避免重复创建连接消耗资源
14     :return: MilvusClient实例，连接失败返回None
15     """
16     try:
17         global _milvus_client
18         # 单例判断：未初始化则创建新连接
19         if _milvus_client is None:
20             milvus_uri = milvus_config.milvus_url
21             # 校验Milvus连接地址配置
22             if not milvus_uri:
23                 logger.error("Milvus客户端连接失败：缺少MILVUS_URL环境变量配置")
24                 return None
25             # 初始化Milvus客户端
26             _milvus_client = MilvusClient(uri=milvus_uri)
27             logger.info("Milvus客户端连接成功")
28             return _milvus_client
29     except Exception as e:
30         logger.error(f"Milvus客户端连接异常：{str(e)}", exc_info=True)
31         return None
32
33
34 def _coerce_int64_ids(ids):
35     """
36     转换chunk_id为Milvus要求的INT64类型（主键字段schema为INT64）
37     过滤无效ID，分离可转换/不可转换的ID
38     :param ids: 待转换的chunk_id列表
39     :return: 元组(ok_ids, bad_ids)，ok_ids为可转换的int64类型ID列表，bad_ids为无效ID列表
40     """
41     ok, bad = [], []
```

```
42     for x in (ids or []):
43         if x is None:
44             continue
45         try:
46             ok.append(int(x))
47         except Exception:
48             bad.append(x)
49     return ok, bad
50
51
52 def fetch_chunks_by_chunk_ids(
53     client,
54     collection_name: str,
55     chunk_ids,
56     *,
57     output_fields=None,
58     batch_size: int = 100,
59 ):
60     """
61     通过chunk_id主键批量查询Milvus中的切片数据
62     用于补全「仅拥有chunk_id无文本内容」场景的切片信息
63     优先使用get方法（主键直查，性能最优），失败则回退query过滤查询
64     :param client: MilvusClient实例
65     :param collection_name: 集合名称
66     :param chunk_ids: 待查询的chunk_id列表
67     :param output_fields: 需要返回的字段列表，默认返回核心切片字段
68     :param batch_size: 分批查询大小，避免单次查询数据量过大，默认100
69     :return: List[dict], Milvus实体字典列表，查询失败返回空列表
70     """
71     # 前置校验：客户端/集合名无效直接返回空
72     if client is None:
73         return []
74     if not collection_name:
75         return []
76     # 默认返回字段：核心切片标识与内容字段
77     if output_fields is None:
78         output_fields = ["chunk_id", "content", "title", "parent_title", "item_name"]
79
80     # 转换ID为INT64类型，分离有效/无效ID
81     ok_ids, bad_ids = _coerce_int64_ids(chunk_ids)
82     if bad_ids:
83         # 记录无效ID，跳过查询
84         logger.warning(f"存在无法转换为INT64的chunk_id，将跳过查询：{bad_ids}")
85
86     # 无有效ID直接返回空
87     if not ok_ids:
88         return []
89
90     results = []
91     # 分批查询：按batch_size切分有效ID，循环查询
92     for i in range(0, len(ok_ids), batch_size):
93         batch = ok_ids[i: i + batch_size]
94
95         # 方式1：优先使用主键get方法查询（性能最优）
96         if hasattr(client, "get"):
97             try:
98                 got = client.get(collection_name=collection_name, ids=batch, output_fields=output_fields)
99                 if got:
100                     results.extend(got)
101                 continue
102             except Exception as e:
103                 logger.warning(f"Milvus get方法查询失败，将回退至query方法：{str(e)}")
104
105         # 方式2：get方法失败，回退使用filter过滤查询
106         try:
107             expr = f"chunk_id in [{', '.join(str(x) for x in batch)}]"
108             q = client.query(collection_name=collection_name, filter=expr, output_fields=output_fields)
109             if q:
110                 results.extend(q)
111         except Exception as e:
112             logger.error(f"Milvus query方法批量查询chunk_id失败：{str(e)}", exc_info=True)
113
```

```
114     return results
115
116
117 def create_hybrid_search_requests(dense_vector, sparse_vector, dense_params=None, sparse_params=None, expr=None,
118                                   limit=5):
119     """
120     构建Milvus混合搜索请求对象
121     分别创建稠密/稀疏向量的搜索请求，用于后续混合搜索融合
122     :param dense_vector: 文本生成的稠密向量
123     :param sparse_vector: 文本生成的稀疏向量
124     :param dense_params: 稠密向量搜索参数，默认使用余弦相似度
125     :param sparse_params: 稀疏向量搜索参数，默认使用内积相似度
126     :param expr: 搜索过滤表达式，用于精准筛选数据
127     :param limit: 单向量搜索返回结果数量，默认5
128     :return: 搜索请求列表，包含[dense_req, sparse_req]
129     """
130     # 稠密向量默认搜索参数：余弦相似度（COSINE），适配BGE-M3稠密向量
131     if dense_params is None:
132         dense_params = {"metric_type": "COSINE"}
133     # 稀疏向量默认搜索参数：内积（IP），适配BGE-M3稀疏向量
134     if sparse_params is None:
135         sparse_params = {"metric_type": "IP"}
136
137     # 构建稠密向量搜索请求，关联Milvus的dense_vector字段
138     dense_req = AnnSearchRequest(
139         data=[dense_vector],
140         anns_field="dense_vector",
141         param=dense_params,
142         expr=expr,
143         limit=limit
144     )
145
146     # 构建稀疏向量搜索请求，关联Milvus的sparse_vector字段
147     sparse_req = AnnSearchRequest(
148         data=[sparse_vector],
149         anns_field="sparse_vector",
150         param=sparse_params,
151         expr=expr,
152         limit=limit
153     )
154
155     return [dense_req, sparse_req]
156
157
158 def hybrid_search(client, collection_name, reqs, ranker_weights=(0.5, 0.5), norm_score=False, limit=5,
159                  output_fields=None, search_params=None):
160     """
161     执行Milvus稠密+稀疏向量混合搜索
162     基于WeightedRanker实现双向量搜索结果加权融合，提升检索准确性
163     :param client: MilvusClient实例
164     :param collection_name: 集合名称
165     :param reqs: 搜索请求列表，固定为[dense_req, sparse_req]
166     :param ranker_weights: 加权融合权重，默认(0.5,0.5)，依次对应稠密/稀疏向量
167     :param norm_score: 是否归一化评分后再融合，避免评分量级差异导致权重失效
168     :param limit: 混合搜索最终返回结果数量，默认5
169     :param output_fields: 需要返回的字段列表，默认返回item_name
170     :param search_params: 搜索参数，如ef/topk等，默认None
171     :return: 混合搜索结果列表，搜索失败返回None
172     """
173     try:
174         # 初始化加权排名器：按权重融合稠密/稀疏向量的搜索结果
175         # norm_score=True：先将两个向量评分归一化到0~1区间，再加权计算
176         rerank = WeightedRanker(ranker_weights[0], ranker_weights[1], norm_score=norm_score)
177
178         # 默认返回字段：文档标识字段
179         if output_fields is None:
180             output_fields = ["item_name"]
181
182         # 执行混合搜索：融合稠密+稀疏向量结果，按权重重新排序
183         res = client.hybrid_search(
184             collection_name=collection_name,
185             reqs=reqs,
```

```
186         ranker=rerank,
187         limit=limit,
188         output_fields=output_fields,
189         search_params=search_params
190     )
191
192     logger.info(f"Milvus混合搜索完成，集合[{collection_name}]共检索到{len(res[0])}条结果")
193     return res
194 except Exception as e:
195     logger.error(f"Milvus混合搜索执行失败，集合[{collection_name}]: {str(e)}", exc_info=True)
196     return None
```

5. 导入与配置

引入必要的依赖库，并定义默认配置参数。

```
1 # 导入基础库：系统、路径、类型注解（类型注解提升代码可读性和可维护性）
2 import os
3 import sys
4 from typing import List, Dict, Any, Tuple
5
6 # 导入Milvus客户端（向量数据库核心操作）、数据类型枚举（定义集合Schema）
7 from pymilvus import MilvusClient, DataType
8 # 导入LangChain消息类（标准化大模型对话消息格式）
9 from langchain_core.messages import SystemMessage, HumanMessage
10
11 # 导入自定义模块：
12 # 1. 流程状态载体：ImportGraphState为LangGraph流程的统一状态管理对象
13 from app.import_process.agent.state import ImportGraphState
14 # 2. Milvus工具：获取单例Milvus客户端，实现连接复用
15 from app.clients.milvus_utils import get_milvus_client
16 # 3. 大模型工具：获取大模型客户端，统一模型调用入口
17 from app.lm.lm_utils import get_llm_client
18 # 4. 向量工具：BGE-M3模型实例、向量生成方法（稠密+稀疏向量）
19 from app.lm.embedding_utils import get_bge_m3_ef, generate_embeddings
20 # 5. 稀疏向量工具：归一化处理，保证向量长度为1，提升检索准确性
21 from app.utils.normalize_sparse_vector import normalize_sparse_vector
22 # 6. 任务工具：更新任务运行状态，用于任务监控和管理
23 from app.utils.task_utils import add_running_task
24 # 7. 日志工具：项目统一日志入口，分级输出（info/warning/error）
25 from app.core.logger import logger
26 # 8. 提示词工具：加载本地prompt模板，实现提示词与代码解耦
27 from app.core.load_prompt import load_prompt
28
29 from app.utils.escape_milvus_string_utils import escape_milvus_string
30
31 # --- 配置参数（Configuration） ---
32 # 大模型识别商品名称的上下文切片数：取前5个切片，避免上下文过长导致大模型输入超限
33 DEFAULT_ITEM_NAME_CHUNK_K = 5
34 # 单个切片内容截断长度：防止单切片内容过长，占满大模型上下文
35 SINGLE_CHUNK_CONTENT_MAX_LEN = 800
36 # 大模型上下文总字符数上限：适配主流大模型输入限制，默认2500
37 CONTEXT_TOTAL_MAX_CHARS = 2500
```

6. 核心辅助函数

处理 Milvus 字符串转义等底层逻辑。

```
1 from app.utils.escape_milvus_string_utils import escape_milvus_string
```

7. 主流程定义

LangGraph 节点的入口函数，负责串联所有步骤。

```
1 def node_item_name_recognition(state: ImportGraphState) -> ImportGraphState:
2     """
3     【核心节点】商品主体名称识别（node_item_name_recognition）
4     整体流程：提取输入→构建上下文→大模型识别→回填数据→生成向量→存入Milvus
5     核心目的：利用大模型从文档切片中精准识别商品/主体名称，并生成双路向量（稠密+稀疏）存入数据库
6     后续扩展点：支持多主体识别、增加商品属性提取、对接其他向量库等
7     :param state: 项目状态字典（ImportGraphState），必须包含chunks/file_title/task_id
```

```
8      :return: 更新后的状态字典，新增item_name键，且chunks列表中每个元素新增item_name字段
9      """
10     # 初始化当前节点信息，用于任务监控和日志溯源
11     node_name = sys._getframe().f_code.co_name
12     logger.info(f">>> 开始执行核心节点：【商品名称识别】{node_name}")
13     # 将当前节点加入运行中任务，更新全局任务状态
14     add_running_task(state.get("task_id", ""), node_name)
15
16     try:
17         # ===== 步骤1：提取并校验输入数据 =====
18         # 作用：从状态字典提取文件标题和切片列表，校验数据完整性
19         # 输出：文件标题、切片列表；若无切片则抛出异常或终止
20         file_title, chunks = step_1_get_inputs(state)
21         if not chunks:
22             logger.warning(f">>> 节点执行警告：{node_name}（无有效切片数据），跳过识别")
23             return state
24
25         # ===== 步骤2：构建大模型识别上下文 =====
26         # 作用：截取前N个切片的内容，拼接成大模型可阅读的上下文，用于辅助识别
27         # 输出：拼接后的上下文字符串
28         context = step_2_build_context(chunks)
29
30         # ===== 步骤3：调用大模型识别商品名称 =====
31         # 作用：构造Prompt，调用LLM从上下文和标题中提取最核心的商品名称
32         # 输出：识别出的商品名称字符串（如 "iPhone 15 Pro"）
33         item_name = step_3_call_llm(file_title, context)
34
35         # ===== 步骤4：回填商品名称到状态和切片 =====
36         # 作用：将识别结果写入状态字典，并同步更新到每一个Chunk对象的元数据中
37         # 输出：状态字典新增item_name，chunks列表被就地修改
38         step_4_update_chunks(state, chunks, item_name)
39
40         # ===== 步骤5：生成双路向量（稠密+稀疏） =====
41         # 作用：调用BGE-M3模型，为商品名称生成稠密语义向量和稀疏关键词向量
42         # 输出：dense_vector (List[float])、sparse_vector (Dict[int, float])
43         dense_vector, sparse_vector = step_5_generate_vectors(item_name)
44
45         # ===== 步骤6：存入Milvus向量数据库 =====
46         # 作用：将商品名称及其双路向量存入Milvus的 item_names 集合，用于后续检索
47         # 输出：无返回值，数据已持久化
48         step_6_save_to_milvus(state, file_title, item_name, dense_vector, sparse_vector)
49
50         # 节点执行完成日志
51         logger.info(f">>> 核心节点执行完成：【商品名称识别】{node_name}，识别结果：{item_name}，已存入Milvus")
52
53     except Exception as e:
54         # 全局异常捕获：保证节点执行失败不崩溃整个流程，记录详细错误日志便于排查
55         logger.error(f">>> 核心节点执行失败：【商品名称识别】{node_name}，错误信息：{str(e)}", exc_info=True)
56         # 可选：失败时设置默认值或标记状态
57         state["item_name"] = "未知商品"
58
59     # 返回更新后的状态（供下游节点使用）
60     return state
```

8. 步骤 1: 获取输入

从 State 中提取文件名和切片数据，并进行基础校验。

```
1 def step_1_get_inputs(state: ImportGraphState) -> Tuple[str, List[Dict]]:
2     """
3     步骤 1：接收并校验流程输入（商品名称识别的前置数据处理）
4     核心作用：
5         1. 从流程状态中提取文件标题、文本切片核心数据
6         2. 做多层空值兜底，避免后续流程因空值报错
7         3. 基础数据类型校验，保证下游流程输入有效性
8     依赖的状态数据（上游节点产出）：
9         - state["file_title"]：上游提取的文件标题（优先使用）
10        - state["file_name"]：原始文件名（file_title为空时兜底）
11        - state["chunks"]：文本切片列表（每个切片为字典，含title/content等字段）
12    返回值：
13        Tuple[str, List[Dict]]：(处理后的文件标题，校验后的文本切片列表)
14    """
```

```
15 # 多层兜底获取文件标题：优先file_title → 其次file_name → 空字符串
16 file_title = state.get("file_title", "") or state.get("file_name", "")
17 # 获取文本切片列表：空值时返回空列表，避免后续遍历报错
18 chunks = state.get("chunks") or []
19
20 # 二次兜底：file_title仍为空时，尝试从第一个有效切片中提取
21 if not file_title:
22     if chunks and isinstance(chunks[0], dict):
23         file_title = chunks[0].get("file_title", "")
24         logger.warning("state中无有效file_title，已从第一个切片中提取兜底标题")
25
26 # 空值日志提示：文件标题为空时不中断流程，仅记录警告
27 if not file_title:
28     logger.warning("state中缺少file_title和file_name，后续大模型识别可能精度下降")
29
30 # 数据类型校验：确保chunks为有效非空列表，否则返回空列表
31 if not isinstance(chunks, list) or not chunks:
32     logger.warning("state中chunks为空或非列表类型，无法进行商品名称识别")
33     return file_title, []
34
35 logger.info(f"步骤1：输入校验完成，获取到{len(chunks)}个有效文本切片")
36 return file_title, chunks
```

9. 步骤 2: 构建上下文

截取文档的前 K 个切片，拼接成用于 LLM 识别的 Context。

```
1 def step_2_build_context(chunks: List[Dict], k: int = DEFAULT_ITEM_NAME_CHUNK_K, max_chars: int = CONTEXT_TOTAL_MAX_CHARS) -> str:
2     """
3     步骤 2：构造大模型商品名称识别的标准化上下文
4     核心作用：
5         1. 限制切片数量：仅取前k个切片，避免上下文过长
6         2. 限制字符长度：单切片+总上下文双重字符限制，适配大模型输入上限
7         3. 格式化内容：带序号的结构化格式，提升大模型识别精度
8         4. 过滤无效切片：跳过空内容/非字典类型切片，保证上下文有效性
9     参数说明：
10         chunks: 文本切片列表（每个元素为字典，需包含"title"和"content"键）
11         k: 最大取片数，默认5个（可通过配置调整）
12         max_chars: 上下文总字符数上限，默认2500（适配大模型输入限制）
13     返回值：
14         str: 格式化后的上下文字符串（直接传给大模型，空切片时返回空字符串）
15     """
16     # 空切片直接返回空字符串，无需后续处理
17     if not chunks:
18         return ""
19
20     # 存储格式化后的切片片段，保证上下文结构化
21     parts: List[str] = []
22     # 统计已拼接字符数，用于控制总长度不超限
23     total_chars = 0
24
25     # 遍历前k个切片，避免上下文过长
26     for idx, chunk in enumerate(chunks[:k]):
27         # 跳过非字典类型切片，防止键取值报错
28         if not isinstance(chunk, dict):
29             logger.debug(f"第{idx+1}个切片非字典类型，已过滤")
30             continue
31
32         # 提取切片标题和内容，去首尾空格，过滤无效字符
33         chunk_title = chunk.get("title", "").strip()
34         chunk_content = chunk.get("content", "").strip()
35
36         # 标题和内容均为空，跳过该无效切片
37         if not (chunk_title or chunk_content):
38             logger.debug(f"第{idx+1}个切片为空白内容，已过滤")
39             continue
40
41         # 单切片内容截断：防止单个切片内容过长占满上下文
42         if len(chunk_content) > SINGLE_CHUNK_CONTENT_MAX_LEN:
43             chunk_content = chunk_content[:SINGLE_CHUNK_CONTENT_MAX_LEN]
44             logger.debug(f"第{idx+1}个切片内容过长，已截断至{SINGLE_CHUNK_CONTENT_MAX_LEN}字符")
45
```

```
46     # 结构化格式化切片：带序号+标题+内容，提升大模型识别效率
47     piece = f"【切片{idx + 1}】\n标题：{chunk_title} \n内容：{chunk_content}"
48     parts.append(piece)
49     # 累计字符数，包含分隔符
50     total_chars += len(piece)
51
52     # 总字符数超限时立即停止拼接，避免大模型输入超限
53     if total_chars > max_chars:
54         logger.info(f"上下文总字符数即将超限（{max_chars}），已停止拼接后续切片")
55         break
56
57     # 用空行分隔切片片段，拼接为最终上下文，最后一次去重空格
58     context = "\n\n".join(parts).strip()
59     # 最终二次截断，确保绝对不超限
60     final_context = context[:max_chars]
61     logger.info(f"步骤2：上下文构建完成，最终长度{len(final_context)}字符")
62     return final_context
```

10. 步骤 3: 调用

构造 Prompt 并调用大模型，识别商品名称。

```
1 def step_3_call_llm(file_title: str, context: str) -> str:
2     """
3     步骤 3：调用大模型实现商品名称/型号精准识别
4     核心逻辑：
5         1. 上下文为空 → 直接返回file_title（兜底，无需调用大模型）
6         2. 上下文非空 → 加载标准化prompt模板，构建大模型对话消息
7         3. 调用大模型后对返回结果做清洗，过滤无效字符
8         4. 大模型返回空/调用异常 → 均返回file_title兜底，保证流程不中断
9     核心特性：
10        - 提示词解耦：通过load_prompt加载本地模板，无需硬编码
11        - 格式兼容：兼容不同LLM客户端返回格式，防止属性报错
12        - 异常兜底：全异常捕获，大模型服务不可用时不影响主流程
13    参数：
14        file_title: 处理后的文件标题（异常/空值时的兜底值）
15        context: 步骤2构建的结构化切片上下文（大模型识别的核心依据）
16    返回值：
17        str: 清洗后的商品名称（异常/空值时返回原始file_title）
18    """
19    logger.info("开始执行步骤3：调用大模型识别商品名称")
20
21    # 上下文为空时，直接返回文件标题，跳过大模型调用
22    if not context:
23        logger.warning("上下文为空，跳过大模型调用，直接使用文件标题作为商品名称")
24        return file_title
25
26    try:
27        # 加载商品名称识别prompt模板，动态传入文件标题和上下文
28        human_prompt = load_prompt("item_name_recognition", file_title=file_title, context=context)
29        # 加载系统提示词，定义大模型角色（商品识别专家，仅返回纯结果）
30        system_prompt = load_prompt("product_recognition_system")
31        logger.debug(f"大模型调用提示词构建完成，系统提示词长度{len(system_prompt)}，人类提示词长度{len(human_prompt)}")
32
33        # 获取大模型客户端：json_mode=False，要求返回纯文本而非JSON格式
34        llm = get_llm_client(json_mode=False)
35        if not llm:
36            logger.error("大模型客户端获取失败，使用文件标题兜底")
37            return file_title
38
39        # 标准化构建大模型对话消息：SystemMessage定义角色 + HumanMessage传递业务请求
40        messages = [
41            SystemMessage(content=system_prompt),
42            HumanMessage(content=human_prompt)
43        ]
44        # 调用大模型并获取返回结果
45        resp = llm.invoke(messages)
46
47        # 兼容不同LLM客户端返回格式：优先取content字段，无则返回空字符串
48        item_name = getattr(resp, "content", "").strip()
49        # 清洗返回结果：过滤空格、换行、回车、制表符等无效字符
50        item_name = item_name.replace(" ", "").replace("\n", "").replace("\r", "").replace("\t", "")
```

```
51
52     # 清洗后结果为空，使用文件标题兜底
53     if not item_name:
54         logger.warning("大模型返回空内容，使用文件标题作为商品名称兜底")
55         return file_title
56
57     logger.info(f"步骤3：大模型识别商品名称成功，结果为：{item_name}")
58     return item_name
59
60 # 捕获所有异常：大模型调用超时、网络错误、格式错误等，均不中断主流程
61 except Exception as e:
62     logger.error(f"步骤3：大模型调用失败，原因：{str(e)}", exc_info=True)
63     # 异常时返回文件标题兜底，保证流程继续执行
64     return file_title
```

11. 步骤 4: 回填数据

将识别到的 item_name 回填到 State 和 Chunks 中。

```
1 def step_4_update_chunks(state: ImportGraphState, chunks: List[Dict], item_name: str):
2     """
3     步骤 4：回填商品名称到流程状态和所有文本切片
4     核心作用：
5         1. 全局状态更新：将item_name存入state，供下游所有节点直接使用
6         2. 切片数据补全：为每个切片添加item_name字段，保证数据一致性
7         3. 状态同步：更新state中的chunks，确保切片修改全局生效
8     设计思路：
9         所有切片关联同一商品名称，保证后续向量入库、检索时的维度一致性
10    参数：
11        state: 流程状态对象（ ImportGraphState ），全局数据载体
12        chunks: 校验后的文本切片列表（ 步骤1输出 ）
13        item_name: 步骤3识别并清洗后的商品名称
14    """
15    # 将商品名称存入全局状态，供下游节点调用
16    state["item_name"] = item_name
17    # 遍历所有切片，为每个切片添加商品名字段，保证数据全链路一致
18    for chunk in chunks:
19        chunk["item_name"] = item_name
20    # 同步更新state中的切片列表，确保修改全局生效
21    state["chunks"] = chunks
22    logger.info(f"步骤4：商品名称回填完成，共为{len(chunks)}个切片添加item_name字段，值为：{item_name}")
```

12. 步骤 5: 生成向量

使用 Embedding 模型为商品名称生成向量。

BGE-M3 模型同时输出这两种向量，结合使用能兼顾“语义理解”和“精准匹配”。

```
1 def step_5_generate_vectors(item_name: str) -> Tuple[Any, Any]:
2     """
3     步骤 5：为商品名称生成BGE-M3稠密+稀疏双向量（ Milvus向量检索核心 ）
4     核心说明：
5         - 稠密向量（ dense_vector ）：BGE-M3固定1024维，记录文本深层语义信息
6         - 稀疏向量（ sparse_vector ）：变长键值对，记录文本关键词/特征位置信息
7     依赖工具：
8         generate_embeddings：封装BGE-M3模型，批量生成双向量，兼容单条/批量输入
9     参数：
10        item_name: 步骤3识别的商品名称（ 非空，空值时直接返回空向量 ）
11    返回值：
12        Tuple[Any, Any]: （稠密向量列表，稀疏向量字典），空值/异常时返回(None, None)
13    """
14    logger.info(f"开始执行步骤5：为商品名称[{item_name}]生成BGE-M3双向量")
15
16    # 商品名称为空，直接返回空向量，跳过模型调用
17    if not item_name:
18        logger.warning("商品名称为空，跳过向量生成，返回空向量")
19        return None, None
20
21    try:
22        # 调用向量生成工具：传入列表支持批量生成，单条数据仍用列表保证格式统一
23        vector_result = generate_embeddings([item_name])
```

```
24
25     # 向量生成结果非空，才进行后续解析
26     if vector_result and "dense" in vector_result and "sparse" in vector_result:
27         # 稠密向量解析：取批量结果第一个，为Python列表（Milvus存储要求）
28         dense_vector = vector_result["dense"][0]
29         # 稀疏向量解析：取批量结果第一个，CSR矩阵解析为字典格式
30         sparse_vector = vector_result["sparse"][0]
31         logger.info("步骤5：BGE-M3稠密+稀疏向量生成成功")
32     else:
33         logger.warning("步骤5：向量生成工具返回空结果，无法提取双向量")
34         dense_vector, sparse_vector = None, None
35
36     # 捕获所有异常：模型加载失败、向量生成超时、格式错误等
37     except Exception as e:
38         logger.error(f"步骤5：向量生成失败，原因：{str(e)}", exc_info=True)
39         dense_vector, sparse_vector = None, None
40
41     return dense_vector, sparse_vector
```

13. 步骤 6: 保存结果

将识别结果及向量保存到 Milvus 数据库。

```
1 def step_6_save_to_milvus(state: ImportGraphState, file_title: str, item_name: str, dense_vector, sparse_vector):
2     """
3     步骤 6：将商品名称、文件标题、双向量持久化到Milvus向量数据库
4     核心逻辑：
5         1. 配置校验：检查Milvus连接地址和集合名配置，缺失则跳过
6         2. 客户端获取：获取单例Milvus客户端，连接失败则跳过
7         3. 集合初始化：无集合则创建（定义Schema+索引），有集合则直接使用（保留原有配置）
8         4. 幂等性处理：删除同名商品数据，避免重复存储
9         5. 数据插入：构造符合Schema的数据，非空向量才添加
10        6. 集合加载：插入后强制加载集合，确保数据立即可查/Attu可见
11    参数：
12        state: 流程状态对象，用于最终状态同步
13        file_title: 处理后的文件标题
14        item_name: 识别后的商品名称（主键去重依据）
15        dense_vector: 步骤5生成的稠密向量（1024维列表）
16        sparse_vector: 步骤5生成的稀疏向量（字典格式）
17    """
18    # 从环境变量读取Milvus核心配置，与MilvusConfig配置类保持一致
19    milvus_uri = os.environ.get("MILVUS_URL")
20    collection_name = os.environ.get("ITEM_NAME_COLLECTION")
21
22    # 配置缺失校验：任一配置为空则跳过Milvus存储，记录警告
23    if not all([milvus_uri, collection_name]):
24        logger.warning("Milvus配置缺失（MILVUS_URL/ITEM_NAME_COLLECTION），跳过数据保存")
25        return
26
27    logger.info(f"开始执行步骤6：将商品名称[{item_name}]保存到Milvus集合[{collection_name}]")
28
29    try:
30        # 获取Milvus单例客户端，连接失败则直接返回
31        client = get_milvus_client()
32        if not client:
33            logger.error("无法获取Milvus客户端（连接失败），跳过数据保存")
34            return
35
36        # 集合初始化：不存在则创建（定义Schema+索引），存在则直接使用
37        if not client.has_collection(collection_name=collection_name):
38            logger.info(f"Milvus集合[{collection_name}]不存在，开始创建Schema和索引")
39            # 创建集合Schema：自增主键+动态字段，适配灵活的数据存储
40            schema = client.create_schema(auto_id=True, enable_dynamic_field=True)
41            # 添加自增主键字段：INT64类型，唯一标识每条数据
42            schema.add_field(
43                field_name="pk",
44                datatype=DataType.INT64,
45                is_primary=True,
46                auto_id=True
47            )
48            # 添加文件标题字段：VARCHAR类型，最大长度65535，适配长标题
49            schema.add_field(
```

```
50         field_name="file_title",
51         datatype=DataType.VARCHAR,
52         max_length=65535
53     )
54     # 添加商品名字段：VARCHAR类型，最大长度65535，去重依据
55     schema.add_field(
56         field_name="item_name",
57         datatype=DataType.VARCHAR,
58         max_length=65535
59     )
60     # 添加稠密向量字段：FLOAT_VECTOR，1024维（BGE-M3固定维度）
61     schema.add_field(
62         field_name="dense_vector",
63         datatype=DataType.FLOAT_VECTOR,
64         dim=1024
65     )
66     # 添加稀疏向量字段：SPARSE_FLOAT_VECTOR，变长
67     schema.add_field(
68         field_name="sparse_vector",
69         datatype=DataType.SPARSE_FLOAT_VECTOR
70     )
71
72     # 构建索引参数：为向量字段创建索引，提升检索性能
73     index_params = client.prepare_index_params()
74     # 优化版稠密向量索引：HNSW + COSINE（恢复最佳性能配置）
75     index_params.add_index(
76         field_name="dense_vector",
77         index_name="dense_vector_index",
78         # HNSW (Hierarchical Navigable Small World) 是目前性能最好、最常用的基于图的索引，检索速度极快，精度极高。
79         index_type="HNSW",
80         # 使用 COSINE 作为稠密向量相似度计算方式
81         metric_type="COSINE",
82         # M：图中每个节点的最大连接数(常用16-64)
83         # efConstruction：构建索引时的搜索范围(越大建索引越慢，但精度越高，常用100-200)
84         # 不同数据体量的推荐建议(万级)：
85         # 10000 条数据：M=16, efConstruction=200
86         # 50000 条数据：M=32, efConstruction=300
87         # 100000 条数据：M=64, efConstruction=400
88         params={"M": 16, "efConstruction": 200}
89     )
90
91     # 稀疏向量索引：专用SPARSE_INVERTED_INDEX+IP，关闭量化保证精度
92     index_params.add_index(
93         field_name="sparse_vector",
94         index_name="sparse_vector_index",
95         # 稀疏倒排索引 专门为稀疏向量（比如文本的 TF-IDF 向量、关键词权重向量，特点是大部分元素为 0，只有少数维度有值）设计的倒排索引，是稀疏向量检索的标配索引类型。
96         index_type="SPARSE_INVERTED_INDEX",
97         # IP（内积，Inner Product）如果向量是“文本语义向量 + 关键词权重”，长度代表文本与主题的关联强度，此时用 IP 能同时体现“语义匹配度”和“关联强度”。
98         metric_type="IP",
99         # DAAT_MAXSCORE：稀疏向量检索时，只计算可能得高分的维度，跳过大量0值，速度更快。
100        # quantization="none"：稀疏向量里的权重是小数，不做压缩，保证精度不丢。
101        params={"inverted_index_algo": "DAAT_MAXSCORE", "quantization": "none"}
102    )
103
104    # 创建集合：Schema + 索引参数
105    client.create_collection(collection_name=collection_name, schema=schema, index_params=index_params)
106    logger.info(f"Milvus集合[{collection_name}]创建成功，包含Schema和向量索引")
107
108    # 幂等性处理：删除同名商品数据，避免重复存储（核心：先加载集合才能删除）
109    clean_item_name = (item_name or "").strip()
110    if clean_item_name:
111        client.load_collection(collection_name=collection_name)
112        # 商品名称转义，防止特殊字符导致过滤表达式解析失败
113        safe_item_name = escape_milvus_string(clean_item_name)
114        filter_expr = f'item_name=="{safe_item_name}"'
115        # 执行删除操作
116        client.delete(collection_name=collection_name, filter=filter_expr)
117        logger.info(f"Milvus幂等性处理完成，已删除集合中[{clean_item_name}]的历史数据")
118
119    # 构造插入Milvus的数据：基础字段+非空向量字段
```

```
120         data = {
121             "file_title": file_title,
122             "item_name": item_name
123         }
124         # 稠密向量非空才添加，避免空值入库报错
125         if dense_vector is not None:
126             data["dense_vector"] = dense_vector
127         # 稀疏向量非空则归一化后添加，保证检索准确性
128         if sparse_vector is not None:
129             data["sparse_vector"] = sparse_vector
130
131         # 插入数据：列表格式支持批量插入，单条数据保持格式统一
132         client.insert(collection_name=collection_name, data=[data])
133         # 插入后强制加载集合，确保数据立即可查、Attu可视化界面可见
134         client.load_collection(collection_name=collection_name)
135
136         # 最终同步商品名称到全局状态
137         state["item_name"] = item_name
138         logger.info(f"步骤6：商品名称[{item_name}]成功存入Milvus集合[{collection_name}]，数据：{list(data.keys())}")
139
140         # 捕获所有Milvus操作异常：连接中断、入库失败、索引错误等，不中断主流程
141         except Exception as e:
142             logger.error(f"步骤6：数据存入Milvus失败，原因：{str(e)}", exc_info=True)
```

15. 单元测试

模拟数据测试核心流程。

```
1  # ===== 本地测试方法（直接运行调试，无需启动LangGraph） =====
2  def test_node_item_name_recognition():
3      """
4      商品名称识别节点本地测试方法
5      功能：模拟LangGraph流程输入，独立测试node_item_name_recognition节点全链路逻辑
6      适用场景：本地开发、调试、单节点功能验证，无需启动整个LangGraph流程
7      测试前准备：
8          1. 确保项目环境变量配置完成（MILVUS_URL/ITEM_NAME_COLLECTION等）
9          2. 确保大模型、Milvus、BGE-M3服务均可正常访问
10         3. 确保prompt模板（item_name_recognition/product_recognition_system）已存在
11     使用方法：
12         直接运行该函数：if __name__ == "__main__": test_node_item_name_recognition()
13     """
14     logger.info("=== 开始执行商品名称识别节点本地测试 ===")
15     try:
16         # 1. 构造模拟的ImportGraphState状态（模拟上游节点产出数据）
17         mock_state = ImportGraphState({
18             "task_id": "test_task_123456", # 测试任务ID
19             "file_title": "华为Mate60 Pro手机使用说明书", # 模拟文件标题
20             "file_name": "华为Mate60Pro说明书.pdf", # 模拟原始文件名（兜底用）
21             # 模拟文本切片列表（上游切片节点产出，含title/content字段）
22             "chunks": [
23                 {
24                     "title": "产品简介",
25                     "content": "华为Mate60 Pro是华为公司2023年发布的旗舰智能手机，搭载麒麟9000S芯片，支持卫星通话功能，屏幕尺寸6.82英寸，分辨率2700×1224。"
26                 },
27                 {
28                     "title": "拍照功能",
29                     "content": "华为Mate60 Pro后置5000万像素超光变摄像头+1200万像素超广角摄像头+4800万像素长焦摄像头，支持5倍光学变焦，100倍数字变焦。"
30                 },
31                 {
32                     "title": "电池参数",
33                     "content": "电池容量5000mAh，支持88W有线超级快充，50W无线超级快充，反向无线充电功能。"
34                 }
35             ]
36         })
37
38         # 2. 调用商品名称识别核心节点
39         result_state = node_item_name_recognition(mock_state)
40
41         # 3. 打印测试结果（调试用）
42         logger.info("=== 商品名称识别节点本地测试完成 ===")
```

```
43     logger.info(f"测试任务ID：{result_state.get('task_id')}")
44     logger.info(f"最终识别商品名称：{result_state.get('item_name')}")
45     logger.info(f"切片数量：{len(result_state.get('chunks', []))}")
46     logger.info(f"第一个切片商品名称：{result_state.get('chunks', [{}])[0].get('item_name')}")
47
48     # 4. 验证Milvus存储（可选）
49     milvus_client = get_milvus_client()
50     collection_name = os.environ.get("ITEM_NAME_COLLECTION")
51     if milvus_client and collection_name:
52         milvus_client.load_collection(collection_name)
53         # 检索测试结果
54         item_name = result_state.get('item_name')
55         safe_name = _escape_milvus_string(item_name)
56         res = milvus_client.query(
57             collection_name=collection_name,
58             filter=f'item_name=="{safe_name}"',
59             output_fields=["file_title", "item_name"]
60         )
61         logger.info(f"Milvus中检索到的数据：{res}")
62
63     except Exception as e:
64         logger.error(f"商品名称识别节点本地测试失败，原因：{str(e)}", exc_info=True)
65
66
67 # 测试方法运行入口：直接执行该文件即可触发测试
68 if __name__ == "__main__":
69     # 执行本地测试
70     test_node_item_name_recognition()
```

5.6 向量化 (node_bge_embedding)

文件: app/import_process/agent/nodes/node_bge_embedding.py
相关工具类位置: app/lm/embedding_utils.py

节点作用与实现思路

节点作用: 将人类可读的文本切片转化为机器可计算的向量表示。采用 BGE-M3 模型，同时生成稠密向量（Dense）和稀疏向量（Sparse），为“混合检索”提供底层数据支持。

实现思路:

- 1. **双路编码:** 利用 BGE-M3 的特性，一次推理同时产出语义向量（Dense，捕获语义相似度）和词汇向量（Sparse，捕获关键词匹配），兼顾语义理解和精确匹配。
- 2. **归一化处理:** 对稀疏向量进行 L2 归一化，消除文本长度带来的权重偏差，确保检索评分的公平性。
- 3. **批处理优化:** 针对大量切片，采用 Batch 处理模式调用模型，大幅提升 GPU/CPU 的计算利用率。

1. 导入与配置

引入必要的依赖库，配置 BGE-M3 向量化流程。

```
1 import sys
2 import os
3 from typing import Any, List, Dict
4
5 from app.import_process.agent.state import ImportGraphState
6 from app.lm.embedding_utils import get_bge_m3_ef, generate_embeddings
7 from app.utils.task_utils import add_running_task,add_done_task
8 from app.core.logger import logger
```

2. 主流程定义

定义 node_bge_embedding 函数，串联各个步骤。

```
1 # =====
2 # BGE-M3向量化核心节点
3 # 核心能力：将文本切片转换为稠密/稀疏双向量，为Milvus向量检索提供数据基础
4 # 依赖模型：BAAI/bge-m3（多语言、多粒度，同时支持语义/关键词检索）
```

```
5 # 向量说明：
6 #     1. 稠密向量：1024维固定长度，记录文本深层语义信息，用于语义相似度匹配
7 #     2. 稀疏向量：变长键值对，记录文本关键词/特征位置，用于关键词精准匹配
8 # 核心设计：
9 #     - 单例模型：避免重复加载模型，节省显存/时间
10 #     - 批量处理：分批生成向量，防止大批次导致的显存溢出
11 #     - 文本增强：拼接商品名+切片内容，强化核心特征，提升检索准确性
12 # =====
13 def node_bge_embedding(state: ImportGraphState) -> ImportGraphState:
14     """
15     LangGraph核心节点：BGE-M3文本向量化处理
16     主流程（串行执行，全流程异常隔离）：
17         1. 输入校验：验证chunks有效性，核心数据缺失则终止当前节点
18         2. 模型初始化：获取BGE-M3单例模型实例，避免重复加载
19         3. 批量向量化：分批拼接文本、生成双向量，为切片绑定向量字段
20         4. 状态更新：将带向量的chunks更新回全局状态，供下游Milvus入库节点使用
21     参数：
22         state: ImportGraphState - 流程全局状态对象，包含上游传入的chunks、task_id等数据
23     返回：
24         ImportGraphState - 更新后的状态对象，chunks字段新增dense_vector/sparse_vector
25     异常处理：
26         节点内所有异常均捕获，不终止整体LangGraph流程，仅记录错误日志
27     """
28     # 获取当前节点名称，用于日志和任务状态记录
29     current_node = sys._getframe().f_code.co_name
30     logger.info(f">>> 开始执行LangGraph节点：{current_node}")
31
32     # 标记任务运行状态，用于任务监控/前端进度展示
33     add_running_task(state.get("task_id", ""), current_node)
34     logger.info("--- BGE-M3 文本向量化处理启动 ---")
35
36     try:
37         # 步骤1：输入数据校验，核心chunks无效则抛出异常
38         texts_to_embed = step_1_validate_input(state)
39
40         # 步骤2：初始化BGE-M3模型（单例模式，仅加载一次）
41         bge_m3_ef = step_2_init_model()
42
43         # 步骤3：批量生成双向量，为切片绑定向量字段
44         output_data = step_3_generate_embeddings(texts_to_embed, bge_m3_ef)
45
46         # 步骤4：更新全局状态，将带向量的chunks回传下游
47         state['chunks'] = output_data
48         logger.info(f "--- BGE-M3 向量化处理完成，共处理 {len(output_data)} 条文本切片 --- ")
49         add_done_task(state.get("task_id", ""), current_node)
50     except Exception as e:
51         # 捕获节点所有异常，记录错误堆栈，不中断整体流程
52         logger.error(f"BGE-M3向量化节点执行失败：{str(e)}", exc_info=True)
53
54     # 返回更新后的状态对象，传递至下游节点
55     return state
```

4. 步骤 1: 校验输入

检查输入数据是否有效。

```
1 def step_1_validate_input(state: ImportGraphState) -> List[Dict[str, Any]]:
2     """
3     向量化前置步骤1：输入数据有效性校验
4     核心作用：
5         1. 从全局状态提取待向量化的chunks切片列表
6         2. 严格校验chunks类型和非空性，无有效数据则终止向量化
7     参数：
8         state: ImportGraphState - 流程全局状态对象
9     返回：
10         List[Dict[str, Any]] - 校验通过的文本切片列表
11     异常：
12         若chunks非列表/为空，抛出ValueError，终止当前向量化流程
13     """
14     # 从状态中提取切片数据
15     texts_to_embed = state.get("chunks")
16     # 校验：必须是非空列表，否则无法进行向量化
```

```
17         if not isinstance(texts_to_embed, list) or not texts_to_embed:
18             logger.error("向量化输入校验失败：chunks字段为空或非有效列表")
19             raise ValueError("错误：无有效文本切片数据，无法执行向量化处理")
20
21     logger.info(f"向量化输入校验通过，待处理文本切片数量：{len(texts_to_embed)}")
22     return texts_to_embed
```

5. 步骤 2: 初始化模型

加载 BGE-M3 模型客户端。（可以忽略）

```
1 def step_2_init_model():
2     """
3     向量化步骤2：初始化BGE-M3模型实例（单例模式）
4     核心作用：
5         1. 调用单例函数get_bge_m3_ef，确保模型全局仅加载一次
6         2. 校验模型实例有效性，加载失败则抛出明确异常
7     返回：
8         Any - 有效BGE-M3模型实例（embedding function）
9     异常：
10        模型加载失败（路径错误/显存不足/依赖缺失）时，抛出ValueError并提示配置问题
11    """
12    try:
13        # 获取单例模型实例，避免重复加载浪费资源
14        ef = get_bge_m3_ef()
15        # 校验模型实例是否有效
16        if ef is None:
17            raise ValueError("BGE-M3模型实例为None：pymilvus.model模块未找到或模型加载失败")
18
19        logger.info("BGE-M3模型实例初始化成功（单例模式）")
20        return ef
21    except Exception as e:
22        # 包装异常信息，明确错误原因和排查方向
23        error_msg = f"BGE-M3模型初始化失败：{e}，请检查模型路径/环境变量配置是否正确"
24        logger.error(error_msg)
25        raise ValueError(error_msg)
```

6. 步骤 3: 批量生成向量

分批次为文本生成稠密和稀疏向量，并进行归一化处理。

```
1 def step_3_generate_embeddings(texts_to_embed: List[Dict[str, Any]], bge_m3_ef: Any) -> List[Dict[str, Any]]:
2     """
3     向量化核心步骤3：批量生成稠密/稀疏双向量
4     核心逻辑（分批执行，每批独立异常处理）：
5         1. 文本拼接：item_name（商品名）+ 换行 + content（切片内容），强化核心特征
6         2. 批量调用：传入拼接后的文本，生成批量双向量
7         3. 向量绑定：为每个切片复制原数据，新增dense_vector/sparse_vector字段
8         4. 异常兜底：单批次失败则保留原切片数据，继续处理下一批次
9     参数：
10        texts_to_embed: List[Dict[str, Any]] - 校验通过的文本切片列表，含item_name/content字段
11        bge_m3_ef: Any - 步骤2初始化的BGE-M3模型实例
12    返回：
13        List[Dict[str, Any]] - 带向量字段的文本切片列表，异常批次保留原数据
14    关键配置：
15        batch_size: 每批处理5条，可根据服务器显存大小调整（显存大则调大，反之调小）
16    """
17    # 初始化结果列表，存储带向量的切片数据
18    output_data = []
19    # 批次大小配置：平衡显存占用和处理效率，建议根据实际环境调整
20    batch_size = 5
21
22    # 按批次遍历，避免一次性处理过多数据导致显存溢出（OOM）
23    total = len(texts_to_embed)
24    for i in range(0, total, batch_size):
25        # 截取当前批次的切片，最后一批自动适配剩余数量【每次获取5个】
26        batch_texts = texts_to_embed[i:i + batch_size]
27        # 计算当前批次的起止索引，用于日志展示（方便看从1开始，也不获取下标，没有影响）
28        start_idx, end_idx = i + 1, min(i + len(batch_texts), total)
29
30        try:
```

```
31         # 构造模型输入文本：拼接商品名+切片内容，增强核心特征
32         input_texts = []
33         for doc in batch_texts:
34             item_name = doc["item_name"]
35             content = doc["content"]
36             # 有商品名则拼接（换行分隔提升模型识别效率），无则直接使用内容
37             # 几乎所有的 Embedding 模型（尤其是基于 BERT 架构的），对前 128 个 token 的注意力是最集中的。越往后的词，对最终向量方向的拉
            扯力越弱。
38             # ***“核心词前置”**的原则
39             # 方案 1：用强标点代替换行（最简单、最推荐）
40             # 优化前：苹果手机\n性能很好...
41             # 优化后：苹果手机。性能很好...
42             # 方案2：加一点“微量”的语义胶水（适合属性明确的场景）
43             text = f"商品：{item_name}，介绍：{content}" if item_name else content
44             # Embedding 模型是个强迫症，你给它喂中文，就用全套中文标点伺候；给它喂英文，就用全套英文标点。保持 语境纯粹 ，生成的向量质量最
            高！
45             input_texts.append(text)
46
47
48         # 调用封装函数生成批量向量，返回格式：{"dense": [稠密向量列表], "sparse": [稀疏向量列表]}
49         docs_embeddings = generate_embeddings(input_texts)
50         if not docs_embeddings:
51             logger.warning(f"第{start_idx}-{end_idx}条切片：向量生成返回空，保留原数据")
52             output_data.extend(batch_texts)
53             continue
54
55         # 为当前批次每个切片绑定对应向量，复制原数据避免修改上游源数据
56         for j, doc in enumerate(batch_texts):
57             item = doc.copy()
58             item["dense_vector"] = docs_embeddings["dense"][j] # 绑定稠密向量
59             item["sparse_vector"] = docs_embeddings["sparse"][j] # 绑定稀疏向量（已归一化）
60             output_data.append(item)
61
62         logger.info(f"第{start_idx}-{end_idx}条切片：双向量生成成功")
63
64     except Exception as e:
65         # 捕获单批次所有异常，记录错误堆栈，不终止整体批量处理
66         logger.error(
67             f"第{start_idx}-{end_idx}条切片：向量生成失败，保留原数据 | 错误原因：{str(e)}",
68             exc_info=True
69         )
70         # 异常批次保留原切片数据，保证数据完整性，后续可人工排查
71         output_data.extend(batch_texts)
72         continue
73
74     return output_data
```

7. 单元测试

您可以在 node_bge_embedding.py 文件底部直接运行以下测试代码：

```
1  # =====
2  # 本地单元测试入口
3  # 功能：独立验证向量化节点全链路逻辑，无需启动整个LangGraph流程
4  # 适用场景：本地开发、调试、模型有效性验证
5  # =====
6  if __name__ == '__main__':
7      # 加载环境变量：定位项目根目录下的.env，读取模型路径/设备等配置
8      current_dir = os.path.dirname(os.path.abspath(__file__))
9      project_root = os.path.dirname(os.path.dirname(current_dir))
10     load_dotenv(os.path.join(project_root, ".env"))
11
12     # 构造模拟测试状态：模拟上游节点输出的chunks数据，贴合真实业务场景
13     test_state = ImportGraphState({
14         "task_id": "test_task_embedding_001", # 测试任务ID
15         "chunks": [ # 模拟带item_name的文本切片（上游商品名称识别节点产出）
16             {
17                 "content": "这是一个测试文档的内容，用于验证向量化是否成功。",
18                 "title": "测试文档标题",
19                 "item_name": "测试项目",
20                 "file_title": "测试文件.pdf"
21             },
```

```
22         {
23             "content": "这是第二个测试文档的内容，用于验证批量处理逻辑。",
24             "title": "测试文档标题2",
25             "item_name": "测试项目",
26             "file_title": "测试文件.pdf"
27         }
28     ]
29 })
30
31 # 执行本地测试
32 logger.info("=== BGE-M3向量化节点本地单元测试启动 ===")
33 try:
34     # 调用核心节点函数
35     result_state = node_bge_embedding(test_state)
36     # 提取测试结果
37     result_chunks = result_state.get("chunks", [])
38
39     # 打印测试结果统计
40     logger.info(f"=== 向量化节点本地测试完成 ===")
41     logger.info(f"测试任务ID：{test_state.get('task_id')}")
42     logger.info(f"待处理切片数：2 | 实际处理切片数：{len(result_chunks)}")
43
44     # 验证向量生成结果（打印向量字段是否存在）
45     for idx, chunk in enumerate(result_chunks):
46         has_dense = "dense_vector" in chunk
47         has_sparse = "sparse_vector" in chunk
48         logger.info(
49             f"第{idx + 1}条切片：稠密向量生成{' ' if has_dense else '未'}成功 | 稀疏向量生成{' ' if has_sparse else '未'}成功")
50
51 except Exception as e:
52     logger.error(f"=== 向量化节点本地测试失败 ===" f"错误原因：{str(e)}", exc_info=True)
53     # 新手友好提示：给出核心排查方向
54     logger.warning("排查提示：请检查BGE-M3模型路径、显存是否充足、环境变量配置是否正确")
```

5.7 存入 Milvus (node_import_milvus)

文件: app/import_process/node_import_milvus.py

节点作用与实现思路

节点作用: 数据加载流程的终点，负责将处理好的结构化数据（切片内容、元数据、向量）持久化存储到向量数据库中，构建可供即时查询的索引。

实现思路:

- 幂等性设计:** 在插入新数据前，根据 item_name 或文件 ID 清理旧数据，防止重复导入导致的数据污染。
- Schema 适配:** 严格按照 Milvus 集合的 Schema 定义（主键、Dense字段、Sparse字段、JSON元数据字段）组织数据，确保插入成功率。
- 混合索引构建:** 确保存入的数据能够支持 Milvus 的 Hybrid Search（Dense + Sparse 加权），最大化检索效果。

1. 导入与配置

- 目的:** 导入必要的库（如 pymilvus）和项目工具类，配置 Milvus 集合名称。
- 关键点:**
 - CHUNKS_COLLECTION_NAME: 从环境变量获取集合名。
 - add_running_task: 记录任务执行状态。

```
1 import os
2 import sys
3 from typing import List, Dict, Any
4 # 导入Milvus相关依赖
5 from pymilvus import DataType
6 # 导入自定义模块
7 from app.import_process.agent.state import ImportGraphState
8 from app.clients.milvus_utils import get_milvus_client
9 from app.utils.task_utils import add_running_task
10 from app.core.logger import logger
11 from app.conf.milvus_config import milvus_config
12
```

```
13 # 从配置文件读取切片集合名称，与配置解耦，便于环境切换
14 CHUNKS_COLLECTION_NAME = milvus_config.chunks_collection
```

2. 核心辅助函数

功能: 处理幂等性清理（删除旧数据）和字符串转义。

```
1 from app.utils.escape_milvus_string_utils import escape_milvus_string
```

3. 主流程定义

函数: node_import_milvus

逻辑:

- 1. **Step 1:** 检查输入 (step_1_check_input)。
- 2. **Step 2:** 准备环境 (step_2_prepare_collection)。
- 3. **Step 3:** 清理旧数据 (step_3_clean_old_data)。
- 4. **Step 4:** 插入数据 (step_4_insert_data)。

```
1 # =====
2 # Milvus切片数据入库核心节点
3 # 核心能力：将上游向量化后的文本切片批量存入Milvus，实现幂等性写入
4 # 核心设计：
5 #     1. 幂等性：插入前删除同item_name旧数据，避免重复存储
6 #     2. 自动建表：集合不存在时自动创建Schema和向量索引，无需手动初始化
7 #     3. 数据校验：前置校验切片有效性、向量字段完整性，避免脏数据入库
8 #     4. 主键回填：将Milvus自增的chunk_id回填到切片，供下游业务使用
9 # 依赖上游：BGE-M3向量化节点（提供dense_vector/sparse_vector字段）
10 # =====
11 def node_import_milvus(state: Dict[str, Any]) -> Dict[str, Any]:
12     """
13     LangGraph核心节点：Milvus切片数据入库主流程
14     执行流程（串行执行，一步一校验，保证数据一致性）：
15         1. 输入校验：验证切片有效性、向量字段完整性，提取向量维度
16         2. 环境准备：连接Milvus，集合不存在则自动创建Schema+索引
17         3. 幂等清理：删除同item_name旧数据，避免重复存储
18         4. 批量插入：预处理数据后批量入库，回填Milvus自增chunk_id
19         5. 状态更新：将回填了chunk_id的切片更新回全局状态，供下游使用
20     参数：
21         state: Dict[str, Any] - 流程全局状态对象，包含chunks、task_id等数据
22     返回：
23         Dict[str, Any] - 更新后的状态对象，chunks字段回填chunk_id
24     异常处理：
25         任一步骤失败抛出ValueError，终止节点执行，保证数据不脏写
26     """
27     # 获取当前节点名称，用于任务监控和日志标识
28     current_node = sys._getframe().f_code.co_name
29     logger.info(f">>> 开始执行LangGraph节点：{current_node}（Milvus切片数据入库）")
30     # 标记任务运行状态，用于前端进度展示/任务监控
31     add_running_task(state["task_id"], current_node)
32     logger.info("--- Milvus切片数据入库流程启动 ---")
33
34     try:
35         # 步骤1：输入数据有效性校验
36         chunks_json_data, vector_dimension = step_1_check_input(state)
37         # 步骤2：Milvus客户端连接+集合准备（自动建表）
38         client = step_2_prepare_collection(vector_dimension)
39         # 步骤3：幂等性处理 - 清理同item_name旧数据
40         step_3_clean_old_data(client, chunks_json_data)
41         # 步骤4：批量插入数据+主键chunk_id回填
42         updated_chunks = step_4_insert_data(client, chunks_json_data)
43         # 步骤5：更新全局状态，将回填后的切片回传下游
44         state["chunks"] = updated_chunks
45
46         logger.info("--- Milvus切片数据入库流程完成 ---")
47     except Exception as e:
48         logger.error(f"Milvus切片数据入库节点执行失败：{str(e)}", exc_info=True)
49         raise ValueError(f"Milvus 导入过程中发生错误：{e}")
50
```

```
51     return state
52
53 def step_1_check_input(state: Dict[str, Any]) -> tuple[List[Dict[str, Any]], int]:
54     """
55     步骤1：输入数据有效性校验（入库前置必检）
56     核心校验项：
57         1. chunks非空且为列表类型
58         2. 切片包含dense_vector核心字段（上游向量化节点必输）
59         3. 提取向量维度，为集合创建/索引构建提供依据
60     参数：
61         state: Dict[str, Any] - 流程状态对象，包含上游传入的chunks数据
62     返回：
63         tuple - （校验通过的切片列表，稠密向量维度）
64     异常：
65         任一校验项不通过，抛出ValueError终止入库流程，避免脏数据处理
66     """
67     # 提取待入库的切片数据
68     chunks_json_data = state.get("chunks")
69     # 校验1：chunks非空
70     if not chunks_json_data:
71         logger.error("Milvus入库校验失败：state中chunks字段为空")
72         raise ValueError("错误：chunks为空，无法执行Milvus入库")
73     # 校验2：chunks为非空列表
74     if not isinstance(chunks_json_data, list) or len(chunks_json_data) == 0:
75         logger.error("Milvus入库校验失败：chunks非列表类型或为空列表")
76         raise ValueError("错误：chunks数据格式不正确，必须为非空列表")
77     # 校验3：切片包含dense_vector字段（向量化节点核心产出）
78     first_chunk = chunks_json_data[0]
79     if 'dense_vector' not in first_chunk:
80         logger.error("Milvus入库校验失败：切片缺失dense_vector字段，上游向量化节点可能执行失败")
81         raise ValueError("错误：数据中缺失dense_vector字段，请检查上游向量化节点执行状态")
82
83     # 提取向量维度和商品名称，用于后续集合创建/日志展示
84     vector_dimension = len(first_chunk['dense_vector'])
85     item_name = first_chunk.get('item_name', '未知商品名')
86     logger.info(
87         f"Milvus入库校验通过，待入库切片数：{len(chunks_json_data)} | 向量维度：{vector_dimension} | 商品名称：{item_name}")
88
89     return chunks_json_data, vector_dimension
```

4. 步骤 1: 检查输入

功能: 验证 chunks 是否存在，并提取 dense_vector 维度和 item_name。

```
1 def step_1_check_input(state: Dict[str, Any]) -> tuple[List[Dict[str, Any]], int]:
2     """
3     步骤1：输入数据有效性校验（入库前置必检）
4     核心校验项：
5         1. chunks非空且为列表类型
6         2. 切片包含dense_vector核心字段（上游向量化节点必输）
7         3. 提取向量维度，为集合创建/索引构建提供依据
8     参数：
9         state: Dict[str, Any] - 流程状态对象，包含上游传入的chunks数据
10    返回：
11        tuple - （校验通过的切片列表，稠密向量维度）
12    异常：
13        任一校验项不通过，抛出ValueError终止入库流程，避免脏数据处理
14    """
15    # 提取待入库的切片数据
16    chunks_json_data = state.get("chunks")
17    # 校验1：chunks非空
18    if not chunks_json_data:
19        logger.error("Milvus入库校验失败：state中chunks字段为空")
20        raise ValueError("错误：chunks为空，无法执行Milvus入库")
21    # 校验2：chunks为非空列表
22    if not isinstance(chunks_json_data, list) or len(chunks_json_data) == 0:
23        logger.error("Milvus入库校验失败：chunks非列表类型或为空列表")
24        raise ValueError("错误：chunks数据格式不正确，必须为非空列表")
25    # 校验3：切片包含dense_vector字段（向量化节点核心产出）
26    first_chunk = chunks_json_data[0]
27    if 'dense_vector' not in first_chunk:
28        logger.error("Milvus入库校验失败：切片缺失dense_vector字段，上游向量化节点可能执行失败")
```

```
29         raise ValueError("错误：数据中缺失dense_vector字段，请检查上游向量化节点执行状态")
30
31     # 提取向量维度和商品名称，用于后续集合创建/日志展示
32     vector_dimension = len(first_chunk['dense_vector'])
33     item_name = first_chunk.get('item_name', '未知商品名')
34     logger.info(
35         f"Milvus入库校验通过，待入库切片数：{len(chunks_json_data)} | 向量维度：{vector_dimension} | 商品名称：{item_name}")
36
37     return chunks_json_data, vector_dimension
```

5. 步骤 2: 准备集合

功能: 获取 Milvus 客户端，如果集合不存在则创建。

```
1 def create_collection(client, collection_name: str, vector_dimension: int):
2     """
3     辅助函数：Milvus集合+索引自动创建
4     核心逻辑：
5         1. 定义集合Schema：包含业务字段+双向量字段，自增主键chunk_id
6         2. 构建向量索引：稠密向量用AUTOINDEX（Milvus自动选最优索引），稀疏向量用专用索引
7     参数：
8         client - MilvusClient实例（已连接）
9         collection_name: str - 要创建的集合名称
10        vector_dimension: int - 稠密向量维度（与向量化模型保持一致）
11    """
12    # 1. 创建Schema：自增主键+支持动态字段，适配灵活的业务扩展
13    schema = client.create_schema(auto_id=True, enable_dynamic_fields=True)
14
15    # 2. 新增字段：业务字段+主键+双向量字段，字段类型/长度适配业务场景
16    schema.add_field(field_name="chunk_id", datatype=DataType.INT64, is_primary=True, auto_id=True)
17    schema.add_field(field_name="content", datatype=DataType.VARCHAR, max_length=65535) # 切片内容
18    schema.add_field(field_name="title", datatype=DataType.VARCHAR, max_length=65535) # 切片标题
19    schema.add_field(field_name="parent_title", datatype=DataType.VARCHAR, max_length=65535) # 父标题
20    schema.add_field(field_name="part", datatype=DataType.INT8) # 分片编号
21    schema.add_field(field_name="file_title", datatype=DataType.VARCHAR, max_length=65535) # 源文件标题
22    schema.add_field(field_name="item_name", datatype=DataType.VARCHAR, max_length=65535) # 商品名称（幂等性依据）
23    schema.add_field(field_name="sparse_vector", datatype=DataType.SPARSE_FLOAT_VECTOR) # 稀疏向量
24    schema.add_field(field_name="dense_vector", datatype=DataType.FLOAT_VECTOR, dim=vector_dimension) # 稠密向量
25    # 对于 BGE-M3 模型：
26    # 它的输出维度是固定的 1024 。
27    # 所以你的代码里必须是：
28    # ```
29    # vector_dimension=必须是1024，不能改！
30    # schema.add_field(...,dim=vector_dimension)
31    # ```（如果你用的是 BGE-base，那就是 768；BGE-small 是 384。这完全由模型架构决定。）
32    # 3. 构建索引参数：为向量字段创建索引，提升检索性能
33    index_params = client.prepare_index_params()
34    # 优化版稠密向量索引：HNSW + COSINE（恢复最佳性能配置）
35    index_params.add_index(
36        field_name="dense_vector",
37        index_name="dense_vector_index",
38        # HNSW (Hierarchical Navigable Small World) 是目前性能最好、最常用的基于图的索引，检索速度极快，精度极高。
39        index_type="HNSW",
40        # 使用 COSINE 作为稠密向量相似度计算方式
41        metric_type="COSINE",
42        # M：图中每个节点的最大连接数(常用16-64)
43        # efConstruction：构建索引时的搜索范围(越大建索引越慢，但精度越高，常用100-200)
44        # 不同数据体量的推荐建议万级：
45        # 10000 条数据：M=16, efConstruction=200
46        # 50000 条数据：M=32, efConstruction=300
47        # 100000 条数据：M=64, efConstruction=400
48        params={"M": 16, "efConstruction": 200}
49    )
50
51    # 稀疏向量索引：专用SPARSE_INVERTED_INDEX+IP，关闭量化保证精度
52    index_params.add_index(
53        field_name="sparse_vector",
54        index_name="sparse_vector_index",
55        # 稀疏倒排索引 专门为稀疏向量（比如文本的 TF-IDF 向量、关键词权重向量，特点是大部分元素为 0，只有少数维度有值）设计的倒排索引，是稀疏
56        # 向量检索的标配索引类型。
57        index_type="SPARSE_INVERTED_INDEX",
```

```
57         # IP ( 内积 , Inner Product ) 如果向量是 “文本语义向量 + 关键词权重”，长度代表文本与主题的关联强度，此时用 IP 能同时体现 “语义匹配度”
和 “关联强度”。
58         metric_type="IP",
59         # DAAT_MAXSCORE：稀疏向量检索时，只计算可能得高分的维度，跳过大量0值，速度更快。
60         # quantization="none"：稀疏向量里的权重是小数，不做压缩，保证精度不丢。
61         params={"inverted_index_algo": "DAAT_MAXSCORE", "quantization": "none"}
62     )
63
64     # 4. 创建集合：Schema+索引参数结合，一次性完成初始化
65     client.create_collection(collection_name=collection_name, schema=schema, index_params=index_params)
66     logger.info(f"Milvus集合创建成功：{collection_name}，向量维度：{vector_dimension}")
67
68
69 def step_2_prepare_collection(vector_dimension: int):
70     """
71     步骤2：Milvus客户端连接+集合准备
72     核心逻辑：
73         1. 获取Milvus单例客户端，验证连接有效性
74         2. 集合不存在则自动创建（Schema+索引），存在则直接复用
75     参数：
76         vector_dimension: int - 稠密向量维度（步骤1提取）
77     返回：
78         MilvusClient - 已连接、集合准备完成的客户端实例
79     异常：
80         客户端获取失败/集合名称未配置，抛出ValueError终止流程
81     """
82     logger.info(f"开始准备Milvus环境，目标集合：{CHUNKS_COLLECTION_NAME}")
83     # 1. 获取Milvus单例客户端，验证连接
84     client = get_milvus_client()
85     if client is None:
86         logger.error("Milvus客户端获取失败：get_milvus_client()返回空，连接可能异常")
87         raise ValueError("Milvus 连接失败：get_milvus_client() 返回空")
88     # 2. 验证集合名称配置
89     if not CHUNKS_COLLECTION_NAME:
90         logger.error("Milvus集合名称未配置：CHUNKS_COLLECTION_NAME为空")
91         raise ValueError("未配置CHUNKS_COLLECTION集合名称")
92
93     # 3. 集合不存在则自动创建
94     if not client.has_collection(collection_name=CHUNKS_COLLECTION_NAME):
95         logger.info(f"Milvus集合{CHUNKS_COLLECTION_NAME}不存在，开始自动创建Schema和索引")
96         create_collection(client, CHUNKS_COLLECTION_NAME, vector_dimension)
97     else:
98         logger.info(f"Milvus集合{CHUNKS_COLLECTION_NAME}已存在，直接复用")
99
100     return client
```

6. 步骤 3: 清理旧数据

功能: 根据 item_name 删除已存在的切片，确保幂等性。

```
1 def step_3_clean_old_data(client, chunks_json_data: List[Dict[str, Any]]):
2     """
3     步骤3：幂等性处理 - 基于item_name清理旧数据
4     核心设计：
5         插入新数据前删除同item_name的所有旧切片，确保多次执行仅保留最新数据
6         支持多item_name批量清理，自动去重避免重复操作
7     参数：
8         client - MilvusClient实例
9         chunks_json_data: List[Dict[str, Any]] - 待入库的切片列表
10    """
11    # 提取并去重item_name，避免重复清理同一商品数据
12    # - 顺序：先循环（for）-> 再判断（if）-> 最后产出（name）。
13    # - 海象操作符（:=）的作用：它在第②步判断的时候，顺手把处理好的字符串塞进了name变量里。如果name是空字符串""
14    #（在Python里等同于False），if条件不成立，第③步就不会执行，这个空值就被扔掉了。
15    item_names = sorted(
16    { name # ③ 最后一步：如果没被if拦住，把name丢进篮子里
17      for x in chunks_json_data or [] # ① 第一步：开始循环，拿到x
18      if (name := str(x.get("item_name", "")).strip()) # ② 第二步：提取 -> 去空格 -> 赋值给name -> 判断name是否为空
19    })
20
21    # 无有效item_name则跳过清理
22    if not item_names:
```

```
23         logger.warning("Milvus幂等性清理跳过：切片中无有效item_name")
24         return
25     # 多item_name提示日志
26     if len(item_names) > 1:
27         logger.warning(f"Milvus幂等性清理：本次检测到多个item_name，将逐个清理：{item_names}")
28
29     # 遍历item_name，逐个清理旧数据
30     for i_name in item_names:
31         _clear_chunks_by_item_name(client, CHUNKS_COLLECTION_NAME, i_name)
32
33
34 def _clear_chunks_by_item_name(client, collection_name: str, item_name: str):
35     """
36     内部核心函数：根据item_name删除Milvus中的旧切片数据
37     参数：
38         client - MilvusClient实例
39         collection_name: str - 集合名称
40         item_name: str - 要清理的商品名称
41     异常：
42         清理失败抛出ValueError，终止整个入库流程（保证幂等性）
43     """
44     # 预处理：去除空格，空值直接返回
45     i_name = (item_name or "").strip()
46     if not i_name:
47         logger.warning("Milvus单商品清理跳过：item_name为空")
48         return
49     if not collection_name:
50         logger.warning("Milvus单商品清理跳过：集合名称未配置")
51         return
52
53     try:
54         # 集合不存在则无需清理
55         if not client.has_collection(collection_name=collection_name):
56             logger.info(f"Milvus单商品清理跳过：集合{collection_name}不存在")
57             return
58
59         # 1. 商品名称安全转义，避免filter表达式报错
60         safe_item_name = escape_milvus_string(i_name)
61         filter_expr = f'item_name == "{safe_item_name}"'
62         logger.info(f"Milvus幂等性清理：开始删除集合{collection_name}中item_name={i_name}的旧数据")
63
64         # 2. 执行删除操作
65         client.delete(collection_name=collection_name, filter=filter_expr)
66
67         # 3. 强制flush，确保删除操作立即生效（避免Milvus异步延迟）
68         if hasattr(client, "flush"):
69             try:
70                 client.flush(collection_name=collection_name)
71             except Exception as e:
72                 logger.warning(f"Milvus幂等性清理：flush操作失败，不影响主流程 | 错误：{str(e)}")
73
74         logger.info(f"Milvus幂等性清理完成：成功删除item_name={i_name}的旧数据")
75     except Exception as e:
76         logger.error(f"Milvus幂等性清理失败：item_name={i_name} | 错误：{str(e)}", exc_info=True)
77         raise ValueError(f"幂等清理失败 ( item_name={i_name} ) : {e}")
```

7. 步骤 4: 插入数据

功能: 移除临时 chunk_id，批量插入数据，并回填生成的 ID。

```
1 def step_4_insert_data(client, chunks_json_data: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
2     """
3     步骤4：批量插入切片数据到Milvus+主键回填
4     核心逻辑：
5         1. 移除手动chunk_id：因auto_id=True，Milvus自动生成主键，避免冲突
6         2. 批量插入数据：提升入库效率，减少Milvus连接次数
7         3. 回填chunk_id：将Milvus生成的自增主键回填到切片，供下游业务使用
8     参数：
9         client - MilvusClient实例
10        chunks_json_data: List[Dict[str, Any]] - 待入库的切片列表
11    返回：
12        List[Dict[str, Any]] - 回填了chunk_id的切片列表
```

```
13 """
14 # 1. 预处理数据：移除手动chunk_id，避免与Milvus自增主键冲突
15 data_to_insert = []
16 for item in chunks_json_data:
17     item_copy = item.copy()
18     if isinstance(item_copy, dict) and "chunk_id" in item_copy:
19         item_copy.pop("chunk_id", None)
20     data_to_insert.append(item_copy)
21
22 logger.info(f"Milvus数据插入：准备{len(data_to_insert)}条切片数据，开始批量插入")
23 # 2. 执行批量插入
24 insert_result = client.insert(collection_name=CHUNKS_COLLECTION_NAME, data=data_to_insert)
25 insert_count = insert_result.get('insert_count', 0)
26 logger.info(f"Milvus数据插入完成：成功插入{insert_count}条数据，插入结果：{insert_result}")
27
28 # 3. 主键回填：将Milvus生成的chunk_id回填到原始切片
29 inserted_ids = insert_result.get('ids', [])
30 if inserted_ids and len(inserted_ids) == len(chunks_json_data):
31     logger.info(f"Milvus主键回填：开始将{len(inserted_ids)}个自增chunk_id回填到切片")
32     for idx, item in enumerate(chunks_json_data):
33         item['chunk_id'] = str(inserted_ids[idx])
34     logger.info("Milvus主键回填完成：所有切片已绑定chunk_id")
35 else:
36     logger.warning(f"Milvus主键回填失败：生成ID数量({len(inserted_ids)})与切片数量({len(chunks_json_data)})不一致")
37
38 return chunks_json_data
```

8. 单元测试

您可以在 node_import_milvus.py 文件底部直接运行以下测试代码：

```
1 if __name__ == '__main__':
2     # --- 单元测试 ---
3     # 目的：验证 Milvus 导入节点的完整流程，包括连接、创建集合、清理旧数据和插入新数据。
4     import sys
5     import os
6     from dotenv import load_dotenv
7
8     # 加载环境变量（自动寻找项目根目录的 .env）
9     current_dir = os.path.dirname(os.path.abspath(__file__))
10    project_root = os.path.dirname(os.path.dirname(current_dir))
11    load_dotenv(os.path.join(project_root, ".env"))
12
13    # 构造测试数据
14    dim = 1024
15    test_state = {
16        "task_id": "test_milvus_task",
17        "chunks": [
18            {
19                "content": "Milvus 测试文本 1",
20                "title": "测试标题",
21                "item_name": "测试项目_Milvus", # 必须有 item_name，用于幂等清理
22                "parent_title": "test.pdf",
23                "part": 1,
24                "file_title": "test.pdf",
25                "dense_vector": [0.1] * dim, # 模拟 Dense Vector
26                "sparse_vector": {1: 0.5, 10: 0.8} # 模拟 Sparse Vector
27            }
28        ]
29    }
30
31    print("正在执行 Milvus 导入节点测试...")
32    try:
33        # 检查必要的环境变量
34        if not os.getenv("MILVUS_URL"):
35            print("✗ 未设置 MILVUS_URL，无法连接 Milvus")
36        elif not os.getenv("CHUNKS_COLLECTION"):
37            print("✗ 未设置 CHUNKS_COLLECTION")
38        else:
39            # 执行节点函数
40            result_state = node_import_milvus(test_state)
41
```

```
42         # 验证结果
43         chunks = result_state.get("chunks", [])
44         if chunks and chunks[0].get("chunk_id"):
45             print(f"✅ Milvus 导入测试通过，生成 ID: {chunks[0]['chunk_id']}")
46         else:
47             print(f"❌ 测试失败：未能获取 chunk_id")
48
49     except Exception as e:
50         print(f"❌ 测试失败：{e}")
```

5.8 进行主图调用测试 (main_graph)

主图添加测试代码，进行流程完成测试！

```
1  if __name__ == "__main__":
2      from app.utils.path_util import PROJECT_ROOT
3      import os
4
5      # 全流程测试：验证PDF导入→Milvus入库→KG导入完整链路
6      logger.info("===== 开始执行知识图谱导入全流程测试 =====")
7      # 1. 构造测试文件路径（复用你项目的doc目录，和pdf2md测试文件一致）
8      test_pdf_name = os.path.join("doc", "万用表RS-12的使用.pdf")
9      test_pdf_path = os.path.join(PROJECT_ROOT, test_pdf_name)
10     # 2. 构造输出目录（存放MD/图片等中间文件）
11     test_output_dir = os.path.join(PROJECT_ROOT, "output")
12     os.makedirs(test_output_dir, exist_ok=True) # 不存在则创建
13
14     # 3. 校验测试PDF文件是否存在
15     if not os.path.exists(test_pdf_path):
16         logger.error(f"全流程测试失败：测试PDF文件不存在，路径：{test_pdf_path}")
17         logger.info("请检查文件路径，或手动将测试文件放入项目根目录的doc文件夹中")
18     else:
19         # 4. 构造测试状态（贴合实际业务入参，开启PDF解析开关）
20         test_state = ImportGraphState({
21             "task_id": "test_kg_import_workflow_001", # 测试任务ID
22             "user_id": "test_user", # 测试用户ID
23             "local_file_path": test_pdf_path, # 测试PDF文件路径
24             "local_dir": test_output_dir, # 中间文件输出目录
25             "is_pdf_read_enabled": False, # 开启PDF解析（核心开关）
26             "is_md_read_enabled": False # 关闭MD解析
27         })
28         try:
29             logger.info(f"测试任务启动，PDF文件路径：{test_pdf_path}")
30             logger.info(f"中间文件输出目录：{test_output_dir}")
31             logger.info("开始执行全流程节点，依次执行：entry→pdf2md→md_img→split→item_name→embedding→milvus→kg")
32
33             # 5. 执行LangGraph全流程（流式执行，打印节点执行进度）
34             final_state = None
35             for step in kb_import_app.stream(test_state, stream_mode="values"):
36                 # 打印当前执行完成的节点（流式输出更直观）
37                 current_node = list(step.keys())[-1] if step else "未知节点"
38                 logger.info(f"✅ 节点执行完成：{current_node}")
39                 final_state = step # 保存最终状态
40
41             # 6. 全流程执行完成，结果预览和核心指标打印
42             if final_state:
43                 logger.info("-" * 80)
44                 logger.info("===== 全流程测试执行成功，核心结果预览 =====")
45                 # 提取核心结果指标
46                 chunks = final_state.get("chunks", [])
47                 chunk_count = len(chunks)
48                 md_content = final_state.get("md_content", "")[:150] # MD内容前150字符
49                 has_embedding = all("dense_vector" in c and "sparse_vector" in c for c in chunks) if chunks else False
50                 has_chunk_id = all("chunk_id" in c for c in chunks) if chunks else False
51                 kg_id = final_state.get("kg_id", "未生成") # KG导入生成的ID（按实际业务字段调整）
52
53                 # 打印核心指标
54                 logger.info(f"📄 PDF转MD内容预览（前150字符）：{md_content}...")
55                 logger.info(f"📄 文档切分总切片数：{chunk_count}")
```

```
56         logger.info(f"🔍 所有切片是否完成向量化：{'是' if has_embedding else '否'}")
57         logger.info(f"📄 所有切片是否完成Milvus入库（含chunk_id）：{'是' if has_chunk_id else '否'}")
58         logger.info(f"🧠 知识图谱导入ID：{kg_id}")
59         logger.info(f"📁 最终状态包含的核心键：{list(final_state.keys())}")
60         logger.info("-" * 80)
61     except Exception as e:
62         logger.exception(f"===== 全流程测试运行失败 =====")
63     logger.info("===== 知识图谱导入全流程测试结束 =====")
```

输出日志：

---识别完成: HAK 180 烫金机---

[Stub]执行节点：node_bge_embedding

---开始 BGE-M3 向量化处理 (node_bge_embedding)---

成功获取第 1-5 项的嵌入。

成功获取第 6-6 项的嵌入。

--- 向量化处理完成，共处理 6 条数据 ---

[Stub] 执行节点：node_import_milvus

---开始导入 Milvus---

从数据中检测到向量维度为: 1024

从数据中检测到 item_name 为: HAK 180 烫金机

正在连接到 Milvus 并准备集合 'kb_chunks'...

幂等清理：正在删除 Milvus 集合 'kb_chunks' 中 item_name=HAK 180 烫金机的旧切片...

幂等清理完成：item_name=HAK 180 烫金机

准备了 6 条数据，正在执行插入操作...

成功插入 6 条数据。

[Stub]执行节点：node_import_kg

= 流程执行结束 =

- ✅ 任务ID: full_graph_test_001
- ✅ 文件标题: hak180产品安全手册
- ✅ 识别商品: HAK 180 烫金机
- ✅ 切片数量: 6
- ✅ 向量化结果: 成功 (检测到 6 条带向量的数据)

效果体现：

default

kb_chunks (6)

kb_item_names (1)

my_collection (2)

概览

向量搜索

数据

分区

数据段(Segments)

导入文件

插入样本数据

清空数据

请输入你的数据查询，例如 id > 0

一致性 Bounded

重置

查询

pk	file_title	item_name	content
463773328655778103	hak180产品安全手册	HAK 180 烫金机	# HAK 180 烫金机 # 产品安全手册（简体中文）• 请勿将本设备放在化学品附近，或者将本设备放
463773328655778104	hak180产品安全手册	HAK 180 烫金机	# 设备 • 对于保养、调整或维修事宜，请联系 Brother 呼叫中心或您当地的Brother 经销商。• 如果
463773328655778105	hak180产品安全手册	HAK 180 烫金机	# 注意 不遵守说明和警告可能导致人员中度或严重受伤。 遵守这些指引以避免人员受伤。• 将本设
463773328655778106	hak180产品安全手册	HAK 180 烫金机	# 电源线 • 如果您长时间不会使用本设备，请从电源插座中拔掉电源线以确保安全。• 本设备必须安
463773328655778107	hak180产品安全手册	HAK 180 烫金机	# 警告标签 请勿撕下或损坏设备上的任何注意/ 警告标签以及序列号标签。 # 设备保修和责任 本手
463773328655778108	hak180产品安全手册	HAK 180 烫金机	# 设备和电源线 • 请仅使用本设备随附的电源线。• 不要在本设备周围放置任何物体。在紧急情况

6. 导入web服务集成和测试

本章节介绍如何将上述实现的 LangGraph 知识库导入流程集成到 FastAPI Web 服务中，并提供可视化界面供用户上传文件和查看导入进度。

6.1 FastAPI 基础讲解

FastAPI 是一个现代、快速（高性能）的 Web 框架，用于构建 APIs with Python 3.7+。本节内容完全独立于本项目，旨在帮助初学者快速掌握 FastAPI 的核心用法。

1. 为什么选择 FastAPI?

- **极速性能**：基于 **Starlette** (负责路由和异步) 和 **Pydantic** (负责数据验证)，性能在 Python web 框架中名列前茅。
- **开发快**：简捷的语法减少了约 40% 的代码量，自动补全支持极佳。
- **原生异步**：完美支持 `async / await`，轻松应对高并发场景。
- **交互式文档**：启动服务后，访问 `/docs` 即可获得自动生成的 **Swagger UI** 接口文档，直接在浏览器调试接口。

2. 核心概念与代码示例

示例1：安装和快速体验

安装 FastAPI 很简单，这里我们使用 **pip** 命令来安装。

```
1 | uv add install fastapi
```

另外我们还需要一个 ASGI 服务器，生产环境可以使用 **Uvicorn** 或者 **Hypercorn**：

```
1 | uv add "uvicorn[standard]"
```

运行第一个 FastAPI 应用

创建一个名为 `main.py` 的文件，添加以下代码：

```
1 | from fastapi import FastAPI
2 |
3 | app = FastAPI()
4 |
5 | @app.get("/")
6 | def read_root():
7 |     return {"Hello": "World"}
```

在命令行中运行以下命令以启动应用：

```
1 | uvicorn main:app --reload
```

现在，打开浏览器并访问 **http://127.0.0.1:8000**，你应该能够看到 FastAPI 自动生成的交互式文档，并在根路径 (`/`) 返回的 JSON 响应。

代码解析：

- `from fastapi import FastAPI`： 这行代码从`fastapi` 模块中导入了 `FastAPI` 类。`FastAPI` 类是 FastAPI 框架的核心，用于创建 FastAPI 应用程序实例。
- `app = FastAPI()`： 这行代码创建了一个 FastAPI 应用实例。与 `Flask` 不同，FastAPI 不需要传递 `__name__` 参数，因为它默认使用当前模块。
- `@app.get("/")`： 这是一个装饰器，用于告诉 FastAPI 哪个 URL 应该触发下面的函数，并且指定了 HTTP 方法为 `GET`。在这个例子中，它指定了根 URL（即网站的主页）。
- `def read_root():`： 这是定义了一个名为 `read_root` 的函数，它将被调用当用户使用 `GET` 方法访问根 URL 时。
- `return {"Hello": "World"}`： 这行代码是 `read_root` 函数的返回值。当用户使用 `GET` 方法访问根 URL 时，这个 JSON 对象将被发送回用户的浏览器或 API 客户端。

示例 2: 最简单的 API 与 参数解析

展示如何定义 `GET` 请求，以及如何自动解析路径参数和查询参数。

```
1 | from fastapi import FastAPI
2 |
3 | app = FastAPI()
4 |
5 | # 访问 http://127.0.0.1:8000/
6 | @app.get("/")
7 | async def root():
8 |     return {"message": "Hello World"}
9 |
10 | # 访问 http://127.0.0.1:8000/items/5?q=somequery
11 | # item_id: 路径参数 (自动转为 int)
12 | # q: 查询参数 (可选, 默认 None)
13 | @app.get("/items/{item_id}")
14 | async def read_item(item_id: int, q: str | None = None):
15 |     return {"item_id": item_id, "q": q}
```

```
16
17 # 接收? skip=? & limit = ?
18 @app.get("/items/")
19 def read_item(skip: int = 0, limit: int = 10):
20     return {"skip": skip, "limit": limit}
21
22 # 展示如何定义数据结构，FastAPI 会自动进行类型检查和错误提示。
23 from pydantic import BaseModel
24
25 # 定义数据模型
26 class Item(BaseModel):
27     name: str
28     price: float
29     is_offer: bool = None
30
31 # 使用 Header 和 Cookie 类型注解获取请求头和 Cookie 数据。
32 # POST 请求接收 JSON 数据
33 @app.post("/items/")
34 def create_item(item: Item):
35     # item 已经是验证过的 Item 对象
36     # 如果客户端传来的 price 是字符串 "abc"，FastAPI 会自动报错
37     return {"item_name": item.name, "item_price": item.price}
38
39 from fastapi import Header, Cookie
40 from fastapi import FastAPI
41
42 app = FastAPI()
43
44 @app.get("/items/")
45 def read_item(user_agent: str = Header(None), session_token: str = Cookie(None)):
46     return {"User-Agent": user_agent, "Session-Token": session_token}
```

函数接受两个参数：

- `item_id` --是路径参数，指定为整数类型。
- `q` -- 是查询参数，指定为字符串类型或空（None）。

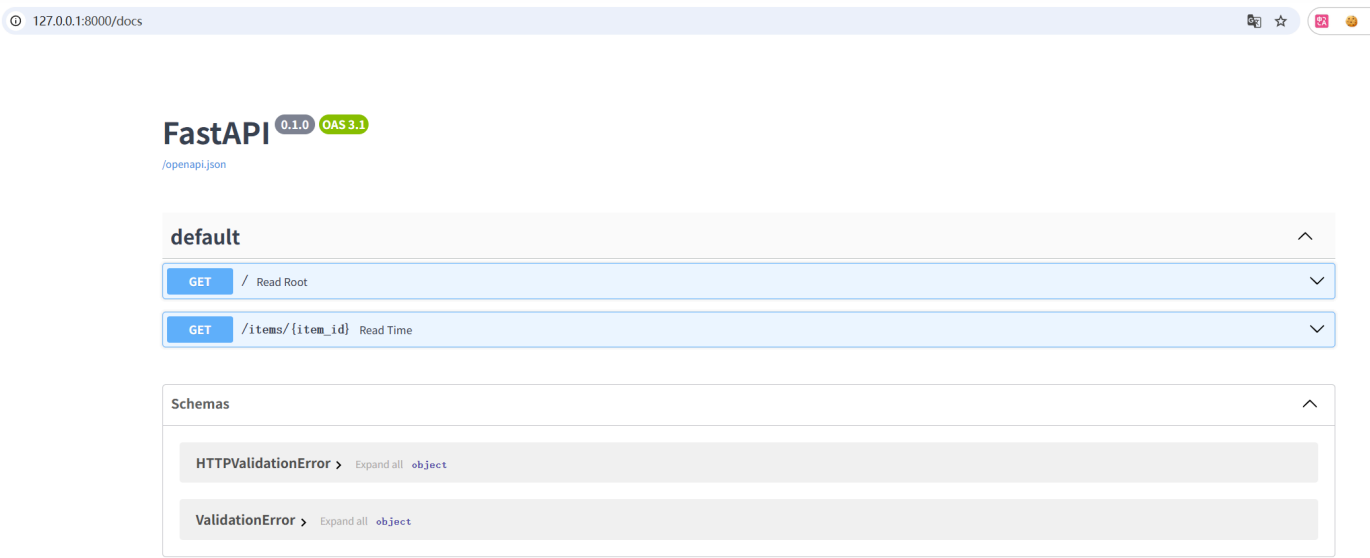
FastAPI 提供了内置的交互式 API 文档，使开发者能够轻松了解和测试 API 的各个端点。

这个文档是自动生成的，基于 OpenAPI 规范，支持 Swagger UI 和 ReDoc 两种交互式界面。

通过 FastAPI 的交互式 API 文档，开发者能够更轻松地理解和使用 API，提高开发效率

在运行 FastAPI 应用时，Uvicorn 同时启动了交互式 API 文档服务。

默认情况下，你可以通过访问 <http://127.0.0.1:8000/docs> 来打开 Swagger UI 风格的文档：



示例 3: 响应JSON数据

展示如何定义数据结构，FastAPI 会自动进行类型检查和错误提示。

```
1 # 路由处理函数返回一个 Pydantic 模型实例，FastAPI 将自动将其转换为 JSON 格式，并作为响应发送给客户端：
2 @app.post("/items/return")
3 def create_item(item: Item):
4     return item
5
6 #使用 RedirectResponse 实现重定向，将客户端重定向到 /items/ 路由。
7 from fastapi.responses import RedirectResponse
8
9 @app.get("/redirect")
10 def redirect():
11     return RedirectResponse(url="/items/")
```

```
12
13 #使用 HTTPException 抛出异常，返回自定义的状态码和详细信息。
14 #以下实例在 item_id 为 42 会返回 404 状态码：
15 from fastapi import HTTPException
16
17 app = FastAPI()
18
19 @app.get("/items/{item_id}")
20 def read_item(item_id: int):
21     if item_id == 42:
22         raise HTTPException(status_code=404, detail="Item not found")
23     return {"item_id": item_id}
```

FastAPI 从 `fastapi.responses` 中提供了多种响应类，覆盖不同的使用场景，下面按「常用程度」整理，每个都附简单示例：

1. JSONResponse（最常用）

- **作用：**返回 JSON 格式的数据（FastAPI 默认的响应类型）；
- **场景：**接口返回普通数据（列表、字典、状态信息等）；
- 示例

```
1 from fastapi.responses import JSONResponse
2
3 @app.get("/api/user")
4 def get_user():
5     # 等价于直接 return {"name": "张三", "age": 20} (FastAPI 自动转 JSONResponse)
6     return JSONResponse(
7         content={"name": "张三", "age": 20},
8         status_code=200, # 可选，默认 200
9         headers={"X-Custom-Header": "custom-value"} # 可选，自定义响应头
10    )
```

2. FileResponse（文件专用）

- **作用：**返回文件（支持大文件、静态文件、下载文件）；
- **场景：**返回 HTML / 图片 / 视频 / Excel 等文件；
- 关键参数
 - `path`：文件路径（必填）；
 - `filename`：下载时显示的文件名（可选）；
 - `media_type`：手动指定 MIME 类型（比如 `media_type="application/pdf"`）；
- 示例:

```
1 from fastapi.responses import FileResponse
2
3 @app.get("/download/excel")
4 def download_excel():
5     excel_path = "./data/report.xlsx"
6     # 返回文件并指定下载文件名
7     return FileResponse(
8         path=excel_path,
9         filename="月度报表.xlsx",
10        media_type="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
11    )
```

3. HTMLResponse

- **作用：**返回 HTML 字符串（直接渲染页面）；
- **场景：**动态生成 HTML 内容（比如拼接变量到 HTML 中）；
- 示例:

```
1 from fastapi.responses import HTMLResponse
2
3 @app.get("/hello")
4 def hello(name: str = "游客"):
5     html_content = f"""
6     <html>
7         <body>
8             <h1>你好, {name} ! </h1>
9         </body>
10    </html>
11    """
12    return HTMLResponse(content=html_content, status_code=200)
```

注意：如果是返回静态 HTML 文件，优先用 FileResponse；动态生成 HTML 用 HTMLResponse。

4. PlainTextResponse

- **作用：**返回纯文本格式的数据（非 JSON、非 HTML）；
- **场景：**返回简单的文本提示、日志内容等；
- **示例：**

```
1 from fastapi.responses import PlainTextResponse
2
3 @app.get("/text")
4 def get_text():
5     return PlainTextResponse(content="这是纯文本响应", status_code=200)
```

5. RedirectResponse

- **作用：**实现页面重定向；
- **场景：**登录成功后跳转到首页、旧接口重定向到新接口等；
- **示例：**

```
1 from fastapi.responses import RedirectResponse
2
3 @app.get("/old-path")
4 def redirect_old_path():
5     # 重定向到 /new-path, 状态码 307 表示临时重定向
6     return RedirectResponse(url="/new-path", status_code=307)
7
8 @app.get("/new-path")
9 def new_path():
10    return {"message": "这是新接口"}
```

6. StreamingResponse（流式响应）

- **作用：**返回流式数据（逐块传输，不一次性加载到内存）；
- **场景：**返回大文件、LLM 流式输出（比如 ChatGPT 逐字回复）、实时日志流等；
- **示例（LLM 流式输出）：**

```
1 from fastapi.responses import StreamingResponse
2 import asyncio
3
4 async def generate_stream():
5     # 模拟流式输出（逐字返回）
6     words = ["你", "好", "，", "，", "这", "，", "是", "，", "流", "，", "式", "，", "响", "，", "应"]
7     for word in words:
8         await asyncio.sleep(0.5)
9         yield word.encode("utf-8") # 流式输出需返回字节流
10
11 @app.get("/stream")
12 async def stream_response():
13     return StreamingResponse(generate_stream(), media_type="text/plain")
```

7. Response（基础响应类）

- **作用：**所有响应类的父类，用于自定义任意格式的响应；
- **场景：**需要高度定制响应（比如自定义 MIME 类型、响应体格式）；
- **示例：**

```
1 from fastapi.responses import Response
2
3 @app.get("/custom")
4 def custom_response():
5     # 返回二进制数据，指定自定义 MIME 类型
6     return Response(
7         content=b"custom binary data",
8         media_type="application/octet-stream",
9         status_code=200
10
```

总结

- 1. FileResponse 是 FastAPI 专门用于**返回文件**的响应类，支持流式传输，适合静态文件 / 下载场景；
- 2. FastAPI 核心响应类型按场景分类：
 - 常规数据：JSONResponse（默认）；
 - 文件：FileResponse；
 - 动态 HTML：HTMLResponse；
 - 纯文本：PlainTextResponse；
 - 重定向：RedirectResponse；
 - 流式数据：StreamingResponse；
 - 自定义响应：Response（父类）；
- 3. 选择响应类型的核心原则：**匹配返回数据的格式和业务场景**（比如静态文件用 FileResponse，流式输出用 StreamingResponse）。

示例 4: 文件上传 (UploadFile)

展示如何接收上传的文件，这是本项目最常用的功能。

```
1 from fastapi import FastAPI, File,UploadFile,HTTPException
2 from fastapi.responses import JSONResponse
3 import os
4
5 from rich import status
6
7 app = FastAPI()
8
9 UPLOAD_FOLDER = 'uploads'
10 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
11
12 @app.post("/upload",summary="单个文件上传接口")
13 async def upload(file: UploadFile = File(
14     ...,
15     description="需要上传的文件（支持图片/文档等）",
16     alias="upload_file", #前端参数的别名，默认文件名是file
17     media_type="application/octet-stream"
18 ),remark:str = None
19 ):
20     try:
21         # 1. 校验文件类型（示例：只允许上传图片）
22         ALLOWED_TYPES = ["image/jpeg", "image/png", "image/gif"]
23         if file.content_type not in ALLOWED_TYPES:
24             raise HTTPException(
25                 status_code=400,
26                 detail=f"仅支持上传{ALLOWED_TYPES}类型的文件，当前文件类型：{file.content_type}"
27             )
28         file_path = os.path.join(UPLOAD_FOLDER, file.filename)
29         CHUNK_SIZE = 1024 * 1024 # 1MB分块
30         with open(file_path, "wb") as f:
31             # 分块读取大文件（避免内存溢出）
32             while True:
33                 chunk = await file.read(CHUNK_SIZE) # 每次读1MB
34                 if not chunk:
35                     break
36                 f.write(chunk)
37         """
38         await的必要性：file.read()是异步函数，必须用await才能拿到文件内容，否则程序报错；
39         分块读取的核心：给read()传一个字节数（如 1MB），循环读取 + 写入，避免大文件占满内存；
40         用法选择：小文件可以用await file.read()一次性读取，大文件必须分块读取；
```

```
41         """
42         return JsonResponse(
43             status_code=200,
44             content={
45                 "code": 0,
46                 "msg": "文件上传成功",
47                 "data": {
48                     "filename": file.filename,
49                     "content_type": file.content_type,
50                     "file_size": f"{file.size} 字节", # 文件大小
51                     "save_path": file_path,
52                     "remark": remark or "无备注"
53                 }
54             }
55         )
56     except Exception as e:
57         # 异常捕获：返回友好的错误信息
58         raise HTTPException(
59             status_code=500,
60             detail=f"文件上传失败: {str(e)}"
61         )
62
63 if __name__ == "__main__":
64     import uvicorn
65     uvicorn.run(app, host="0.0.0.0", port=8000)
```

File()的参数和Form()/Query()逻辑一致，核心常用配置如下：

参数	作用	示例
...	表示 必填项 （必须传文件，不传则返回 422 错误）	File(...)
None	表示 可选项 （不传文件也能调用接口）	File(None)
description	接口文档中显示的描述（提升可读性，FastAPI 自动生成文档）	File(..., description="上传图片")
alias	前端传参的别名（比如前端用file_upload传，后端用file接收）	File(..., alias="file_upload")
media_type	限定文件的 MIME 类型（比如只允许图片）	File(..., media_type="image/*")
min_length	限制文件内容最小字节数（很少用，一般在校验逻辑里做）	File(..., min_length=1024)
max_length	限制文件内容最大字节数（比如限制 5MB：max_length=5*1024*1024）	File(..., max_length=5242880)

示例 5: 后台任务 (BackgroundTasks)

展示如何在返回响应后继续执行耗时操作（如发送邮件、处理数据），避免阻塞用户。

```
1 from fastapi import BackgroundTasks
2 import time
3
4 # 定义一个模拟的耗时任务
5 def write_log(message: str):
6     time.sleep(2) # 模拟耗时 2 秒
7     with open("log.txt", "a") as log:
8         log.write(message + "\n")
9
10 @app.post("/send-notification/{email}")
11 async def send_notification(email: str, background_tasks: BackgroundTasks):
12     # 1. 添加任务到后台队列
13     background_tasks.add_task(write_log, f"Notification sent to {email}")
14     # 2. 立即返回响应给用户，不需要等待 write_log 执行完毕
15     return {"message": "Notification sent in the background"}
```

示例 6: 静态文件与跨域配置 (CORS)

Web 开发必备：托管 HTML 页面和解决浏览器跨域限制。

```
1 from fastapi.staticfiles import StaticFiles
2 from fastapi.middleware.cors import CORSMiddleware
3
4 # 1. 静态文件挂载
5 # 假设当前目录下有个 static 文件夹，里面有 index.html
6 # 访问 http://127.0.0.1:8000/static/index.html 即可
```

```
7 app.mount("/static", StaticFiles(directory="static"), name="static")
8
9 # 访问 /go-to-index → 跳转到本地static/index.html
10 @app.get("/go-to-index")
11 async def redirect_to_index():
12     return RedirectResponse(url="/static/index.html")
13
14 # 2. CORS 配置
15 # 允许前端页面（如运行在 5500 端口）访问后端 API（8000 端口）
16 app.add_middleware(
17     CORSMiddleware,
18     allow_origins=["*"], # 允许所有来源（生产环境建议指定具体域名）
19     allow_credentials=True,
20     allow_methods=["*"], # 允许所有 HTTP 方法
21     allow_headers=["*"], # 允许所有请求头
22 )
```

6.2 web服务实现过程

文件位置:app/import_process/api/file_import_service.py

本节将展示完整的 Web 服务实现代码。该服务基于 FastAPI 构建，提供了文件上传、后台任务调度、状态查询以及静态页面服务功能。

1. 依赖库

在运行本服务前，请确保安装以下 Python 依赖库（已包含在 requirements.txt 中）：

```
1 uv add fastapi uvicorn python-multipart python-dotenv minio
```

此外，本项目依赖 LangGraph 和 LangChain 相关库来执行后台任务。

2. 代码实现详解

我们将 app/import_process/api/file_import_service.py 的实现拆分为 6 个核心部分进行讲解，以便理解每个模块的作用。

(1) 引入依赖与环境配置

首先，导入必要的系统库、FastAPI 组件以及我们的工具类 (minio_utils, task_utils, main_graph)。同时加载 .env 环境变量。

```
1 from pathlib import Path
2 import uuid
3 import uvicorn
4 from fastapi import FastAPI, BackgroundTasks, HTTPException, Request
5 from fastapi.responses import FileResponse, StreamingResponse
6 from pydantic import BaseModel, Field
7 from starlette.middleware.cors import CORSMiddleware
8 from app.core.logger import logger
9
10 from app.utils.task_utils import *
11 from app.utils.sse_utils import create_sse_queue, SSEEvent, sse_generator
12 from app.clients.mongo_history_utils import *
13 from app.query_process.agent.main_graph import query_app
```

(2) 应用初始化与跨域配置

初始化 FastAPI 应用，并配置 CORS（跨域资源共享）以允许前端调用。

```
1 # 定义fastapi对象
2 app = FastAPI(title="query service", description="掌柜智库查询服务！")
3
4 # 跨域配置
5 app.add_middleware(
6     CORSMiddleware,
7     allow_origins=["*"],
8     allow_methods=["*"],
9     allow_headers=["*"],
10 )
11
```

```
12 # -----
13 # 静态页面路由：返回文件导入前端页面import.html
14 # 访问地址：http://localhost:8000/import.html
15 # -----
16 @app.get("/import.html", response_class=FileResponse)
17 async def get_import_page():
18     """返回文件导入前端页面：import.html"""
19     # 拼接HTML文件绝对路径，基于项目根目录定位
20     html_abs_path = PROJECT_ROOT / "app/import_process/page/import.html"
21     # 日志记录页面访问的文件路径，方便排查文件不存在问题
22     logger.info(f"前端页面访问，文件绝对路径：{html_abs_path}")
23
24     # 校验文件是否存在，不存在则抛出404异常
25     if not os.path.exists(html_abs_path):
26         logger.error(f"前端页面文件不存在，路径：{html_abs_path}")
27         raise HTTPException(status_code=404, detail="import.html page not found")
28
29     # 以FileResponse返回HTML文件，浏览器自动渲染
30     return FileResponse(
31         path=html_abs_path,
32         media_type="text/html" # 显式指定媒体类型为HTML，确保浏览器正确解析
33     )
```

(3) 后台任务逻辑

这是连接 Web 服务与 LangGraph 的桥梁。run_graph_task 函数会在后台运行（不阻塞 HTTP 响应），它监听图的执行事件，并实时更新任务状态。

```
1 # -----
2 # 后台任务：LangGraph全流程执行
3 # 独立于主请求线程，由BackgroundTasks触发，避免阻塞接口响应
4 # -----
5 def run_graph_task(task_id: str, local_dir: str, local_file_path: str):
6     """
7     LangGraph全流程执行后台任务
8     核心流程：初始化状态 → 流式执行图节点 → 实时更新任务状态 → 异常捕获
9     任务状态更新：pending → processing → completed/failed
10    节点进度更新：每完成一个节点，将节点名加入done_list，供前端轮询查看
11
12    :param task_id: 全局唯一任务ID，关联单个文件的全流程处理
13    :param local_dir: 该任务的本地文件存储目录（含临时文件/解析结果）
14    :param local_file_path: 上传文件的本地绝对路径
15    """
16    try:
17        # 1. 更新任务全局状态为：处理中
18        update_task_status(task_id, "processing")
19        logger.info(f"[{task_id}] 开始执行LangGraph全流程，本地文件路径：{local_file_path}")
20
21        # 2. 初始化LangGraph状态：加载默认状态 + 注入当前任务的核心参数
22        init_state = get_default_state()
23        init_state["task_id"] = task_id # 任务ID关联
24        init_state["local_dir"] = local_dir # 任务本地目录
25        init_state["local_file_path"] = local_file_path # 上传文件本地路径
26
27        # 3. 流式执行LangGraph全流程（stream模式：实时获取每个节点的执行结果）
28        for event in kb_import_app.stream(init_state):
29            for node_name, node_result in event.items():
30                # 记录每个节点完成的日志，包含任务ID和节点名，方便追踪执行顺序
31                logger.info(f"[{task_id}] LangGraph节点执行完成：{node_name}")
32                # 将完成的节点名加入【已完成列表】，前端轮询/status/{task_id}可实时获取
33                add_done_task(task_id, node_name)
34
35        # 4. 全流程执行完成，更新任务全局状态为：已完成
36        update_task_status(task_id, "completed")
37        logger.info(f"[{task_id}] LangGraph全流程执行完毕，任务完成")
38
39    except Exception as e:
40        # 5. 捕获全流程异常，更新任务全局状态为：失败，并记录错误日志（含堆栈）
41        update_task_status(task_id, "failed")
42        logger.error(f"[{task_id}] LangGraph全流程执行失败，异常信息：{str(e)}", exc_info=True)
```

(4) 文件上传接口 (/upload)

处理文件上传请求。它负责：

- 1. 生成全局唯一的 task_id。
- 2. 将文件保存到本地临时目录。
- 3. 启动后台任务 (run_graph_task) 开始处理。

```
1 # -----
2 # 核心接口：多文件上传接口（不上传 MinIO）
3 # 支持多文件批量上传，核心流程：接收文件 → 本地保存 → 启动后台任务
4 # 访问地址：http://localhost:8000/upload （POST请求，form-data格式传参）
5 # -----
6 from pathlib import Path
7
8 @app.post("/upload", summary="文件上传接口", description="支持多文件批量上传，自动触发知识库导入全流程")
9 async def upload_files(
10     background_tasks: BackgroundTasks,
11     files: List[UploadFile] = File(...)
12 ):
13     """
14     文件上传核心接口（不上传 MinIO）
15     1. 接收前端上传的多文件（PDF/MD为主）
16     2. 按「日期/任务ID」分层保存到本地输出目录，避免文件冲突
17     3. 为每个文件生成唯一TaskID，启动独立的LangGraph后台处理任务
18     4. 实时更新任务状态，供前端轮询监控进度
19
20     :param background_tasks: FastAPI后台任务对象，用于异步执行LangGraph流程
21     :param files: 前端上传的文件列表（form-data格式）
22     :return: 包含上传结果和所有任务ID的JSON响应
23     """
24     # 1. 构建本地存储根目录：项目根目录/output/YYYYMMDD（按日期分层，方便管理）
25     today_str = datetime.now().strftime("%Y%m%d")
26     date_based_root_dir: Path = PROJECT_ROOT / "output" / today_str
27
28     # 初始化任务ID列表，用于返回给前端（一个文件对应一个TaskID）
29     task_ids = []
30
31     # 2. 遍历处理每个上传的文件（多文件批量处理，各自独立生成TaskID）
32     for file in files:
33         # 生成全局唯一TaskID（UUID4），作为单个文件的全流程标识
34         task_id = str(uuid.uuid4())
35         task_ids.append(task_id)
36         logger.info(f"[{task_id}] 开始处理上传文件，文件名：{file.filename}，文件类型：{file.content_type}")
37
38         # 3. 标记「文件上传」阶段为「运行中」，前端轮询可查
39         add_running_task(task_id, "upload_file")
40
41         # 4. 构建该任务的本地独立目录：output/YYYYMMDD/TaskID，避免多文件重名冲突
42         task_local_dir: Path = date_based_root_dir / task_id
43         task_local_dir.mkdir(parents=True, exist_ok=True)
44
45         # 5. 构建上传文件的本地保存绝对路径
46         local_file_abs_path: Path = task_local_dir / file.filename
47
48         # 6. 将上传的文件保存到本地临时目录
49         with local_file_abs_path.open("wb") as file_buffer:
50             shutil.copyfileobj(file.file, file_buffer)
51         logger.info(f"[{task_id}] 文件已保存至本地，路径：{local_file_abs_path}")
52
53         # 7. 标记「文件上传」阶段为「已完成」
54         add_done_task(task_id, "upload_file")
55
56         # 8. 将LangGraph全流程处理加入FastAPI后台任务
57         background_tasks.add_task(
58             run_graph_task,
59             task_id,
60             str(task_local_dir),
61             str(local_file_abs_path)
62         )
63         logger.info(f"[{task_id}] 已将LangGraph全流程加入后台任务，任务已启动")
64
```

```
65     # 9. 所有文件处理完毕，返回上传成功信息和所有TaskID
66     logger.info(f"多文件上传处理完毕，共处理{len(files)}个文件，生成TaskID列表：{task_ids}")
67     return {
68         "code": 200,
69         "message": f"Files uploaded successfully, total: {len(files)}",
70         "task_ids": task_ids
71     }
```

(5) 任务状态查询接口 (/status)

前端轮询此接口以获取进度。它直接从内存中读取由 task_utils 维护的任务状态。

```
1  # -----
2  # 核心接口：任务状态查询接口
3  # 前端轮询此接口获取单个任务的处理进度和状态
4  # 访问地址：http://localhost:8000/status/{task_id} （GET请求）
5  # -----
6  @app.get("/status/{task_id}", summary="任务状态查询", description="根据TaskID查询单个文件的处理进度和全局状态")
7  async def get_task_progress(task_id: str):
8      """
9      任务状态查询接口
10     前端轮询此接口（如每秒1次），获取任务的实时处理进度
11     返回数据均来自内存中的任务管理字典（task_utils.py），高性能无IO
12
13     :param task_id: 全局唯一任务ID（由/upload接口返回）
14     :return: 包含任务全局状态、已完成节点、运行中节点的JSON响应
15     """
16     # 构造任务状态返回体
17     task_status_info: Dict[str, Any] = {
18         "code": 200,
19         "task_id": task_id,
20         "status": get_task_status(task_id), # 任务全局状态：pending/processing/completed/failed
21         "done_list": get_done_task_list(task_id), # 已完成的节点/阶段列表
22         "running_list": get_running_task_list(task_id) # 正在运行的节点/阶段列表
23     }
24     # 记录状态查询日志，方便追踪前端轮询情况
25     logger.info(
26         f"[{task_id}] 任务状态查询，当前状态：{task_status_info['status']}，已完成节点：{task_status_info['done_list']}"
27     )
28     return task_status_info
```

(6) 启动入口

配置 Uvicorn 服务器启动参数。包含一段针对 Windows/PyCharm 环境下 asyncio 事件循环的兼容性补丁。

```
1  # -----
2  # 服务启动入口
3  # 直接运行此脚本即可启动FastAPI服务，无需额外执行uvicorn命令
4  # -----
5  if __name__ == "__main__":
6      """服务启动入口：本地开发环境直接运行"""
7      logger.info("File Import Service 服务启动中...")
8      # 启动uvicorn服务，绑定本地IP和8000端口，关闭自动重载（生产环境建议用workers多进程）
9      uvicorn.run(
10         app=app,
11         host="127.0.0.1", # 仅本地访问，生产环境改为0.0.0.0（允许所有IP访问）
12         port=8000 # 服务端口
13     )
```

3. 接口与 Task 列表关系详解

本服务通过 kb.utils.task_utils 模块维护了一个内存中的任务状态列表，实现了前后端的状态同步。

1. 上传阶段 (/upload):
- 请求到达时，生成 task_id。
 - 调用 add_running_task(task_id, "upload_file")：此时前端查询状态，会看到 "开始上传文件" 正在运行。
 - 文件上传 MinIO 成功后，调用 add_done_task(task_id, "upload_file")：此时前端会看到 "开始上传文件" 变为已完成。
2. 处理阶段 (run_graph_task):
- 后台任务启动，调用 update_task_status(task_id, "processing")：标记任务整体正在处理中。

- LangGraph 每执行完一个节点（如 PDF 转 Markdown）, 流式输出会捕获到事件。
- 调用 `add_done_task(task_id, node_name)`: 将该节点标记为已完成。前端轮询时, 进度条或日志列表会相应更新。

3. 完成阶段:

- 图执行结束, 调用 `update_task_status(task_id, "completed")`。
- 前端收到 `completed` 状态, 提示用户导入成功。

6.3 页面导入配置

文件位置: `app/import_process/page/import.html`

这是一个简洁的原生 HTML/JS 页面, 实现了以下功能:

1. **文件拖拽/选择:** 支持 PDF 和 MD 文件。
2. **上传进度条:** 显示文件上传到服务器的进度。
3. **状态轮询:** 上传成功后, 通过 `setInterval` 每 2 秒请求一次 `/status` 接口。
4. **日志展示:** 根据后端返回的 `done_list` 和 `running_list`, 动态渲染任务执行日志 (如 "node_entry已完成", "node_pdf_to_md正在进行...") 。

6.4 服务启动与测试流程

1. 启动服务

在项目根目录下 (`atguigu_knowledge_base`) , 运行以下命令:

```
1 # 方式一: 直接运行 Python 模块
2 python -m kb.web.file_import_service
3
4 # 方式二: 使用 uvicorn 命令
5 uvicorn kb.web.file_import_service:app --host 0.0.0.0 --port 8000 --reload
```

看到 `Application startup complete` 即表示启动成功。

2. 页面测试

1. 打开浏览器访问: `http://127.0.0.1:8000/import.html`
2. 点击上传区域, 选择一个测试文件 (如 `doc/hak180产品安全手册.pdf`) 。
3. **观察页面交互:**
 - 状态变为 "上传中..." -> "处理中..."。
 - 点击 "日志 (点击展开) ", 可以看到节点逐个执行的过程。
 - 等待所有节点执行完毕, 状态变为 "已完成" (绿色)。

3. 验证结果

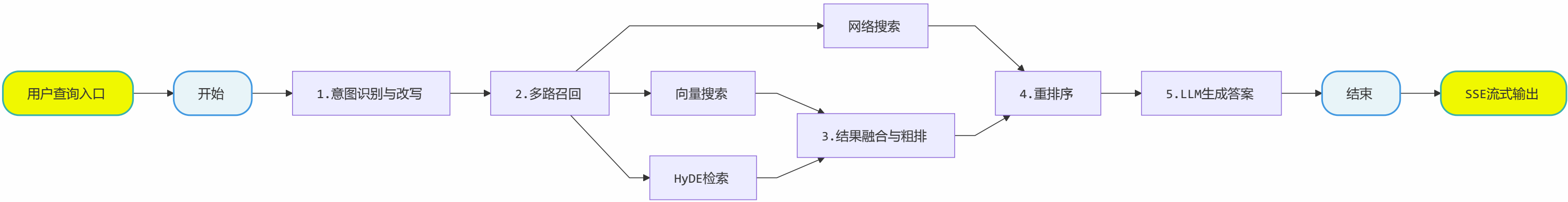
- **MinIO:** 检查文件是否已上传到对应日期的 bucket 中。
- **Milvus:** 使用 `Attu` 查看 `kb_chunks` 集合, 确认是否新增了该文件的向量数据。

通过以上步骤, 我们完成了从后端逻辑实现到 Web 服务集成, 再到前端可视化操作的完整闭环!

7. 检索数据图与状态定义

7.1 定义状态 (State)

所有节点共享同一个状态对象。我们需要定义它来存储处理过程中的数据 (如 查询原始问题、改写问题、历史对话、不同方向切片结果等) 。



文件: app/query_process/agent/state.py

```
1 from typing_extensions import TypedDict
2 from typing import List
3 import copy
4
5 class QueryGraphState(TypedDict):
6     """
7     QueryGraphState 定义了整个查询流程中流转的数据结构。
8     """
9     session_id: str # 会话唯一标识
10    original_query: str # 用户原始问题
11
12    # 检索过程中的中间数据
13    embedding_chunks: list # 普通向量检索回来的切片
14    hyde_embedding_chunks: list # HyDE 检索回来的切片
15    web_search_docs: list # 网络搜索回来的文档
16
17    # 排序过程中的数据
18    rrf_chunks: list # RRF 融合排序后的切片
19    reranked_docs: list # 重排序后的最终 Top-K 文档
20
21    # 生成过程中的数据
22    prompt: str # 组装好的 Prompt
23    answer: str # 最终生成的答案
24
25    # 辅助信息
26    item_names: List[str] # 提取出的商品名称
27    rewritten_query: str # 改写后的问题
28    history: list # 历史对话记录
29    is_stream: bool # 是否流式输出标记
30
31
32 # =====
33 # 默认状态 (全部为空)
34 # =====
35 query_graph_default_state: QueryGraphState = {
36     "session_id": "",
37     "original_query": "",
38     "embedding_chunks": [],
39     "hyde_embedding_chunks": [],
40     "web_search_docs": [],
41     "rrf_chunks": [],
42     "reranked_docs": [],
43     "prompt": "",
44     "answer": "",
45     "item_names": [],
46     "rewritten_query": "",
47     "history": [],
48     "is_stream": False
49 }
50
51
52 # =====
53 # 创建默认状态 (可覆盖)
54 # =====
55 def create_query_default_state(**overrides) -> QueryGraphState:
56     """
57     创建查询流程的默认状态，支持覆盖字段
58     """
59     state = copy.deepcopy(query_graph_default_state)
60     state.update(overrides)
61     return state
62
63
```

```
64 # =====
65 # 获取干净状态
66 # =====
67 def get_query_default_state() -> QueryGraphState:
68     return copy.deepcopy(query_graph_default_state)
69
70
71 # =====
72 # ✅ 状态复制函数（你要的）
73 # =====
74 def copy_query_state(state: QueryGraphState, **overrides) -> QueryGraphState:
75     """
76     复制现有状态并可覆盖字段，深拷贝，不污染原数据
77     """
78     new_state = copy.deepcopy(state)
79     new_state.update(overrides)
80     return new_state
81
82
83 if __name__ == "__main__":
84     # 测试
85     state = create_query_default_state(
86         session_id="test_001",
87         original_query="华为P60怎么样?",
88         is_stream=False
89     )
90     print("初始化状态：", state)
91
92     # 复制状态
93     new_state = copy_query_state(
94         state,
95         original_query="修改后的问题"
96     )
97     print("复制后的状态：", new_state)
```

7.2 定义主图

为了确保整体流程设计的科学性与执行连贯性，我们采用 "Top-Down"（自顶向下）的开发模式，以“总指挥部”的全局视角统筹推进，具体实施步骤如下：

- 1. **搭建节点骨架（Stubs）**： 优先定义全流程所需的所有功能节点，仅保留核心日志打印能力（如节点进入 / 退出日志），暂不实现内部复杂业务逻辑，快速搭建起流程的“骨架结构”；
- 2. **串联主图（Graph）**： 基于预设的业务流转规则，编写主图逻辑将所有节点骨架按序串联，明确节点间的输入输出关系、分支判断条件（如文件格式分流逻辑），形成完整的流程链路；
- 3. **验证流程通畅性**： 启动端到端测试，验证节点间的调用链路是否通顺、数据流转是否符合预期、分支跳转是否准确，确保流程无阻塞、无逻辑漏洞；
- 4. **填充节点核心逻辑**： 在流程链路验证通过后，再逐一聚焦每个节点的内部实现，完成复杂业务逻辑的开发（如 查询重写、定向查询、结果重排处理等），实现“骨架”到“完整系统”的落地。

该模式的核心优势在于：先保障“流程走得通”，再聚焦“功能做得好”，避免因局部逻辑复杂导致整体流程设计偏差，大幅提升开发效率与流程稳定性。

7.2.1 第一步：创建节点骨架

我们需要先创建以下 8 个文件，每个文件里只写一个最简单的“空函数”，确保主图能导入它们。
请在 app/query_process/agent/nodes 目录下创建以下文件，并将对应代码复制进去。

(1) 意图确认: node_item_name_confirm.py

这个节点主要干了 4 件事：

- 1. 提取与改写： 结合历史对话提取商品名，并将模糊问题改写为完整独立的精准问题。
- 2. 向量化检索： 将提取出的商品名在 Milvus 向量库中进行混合搜索。
- 3. 标准化对齐： 根据评分高低自动对齐标准型号，或生成反问让用户手动确认。
- 4. 同步历史记录： 将改写后的问题、确认的商品名和处理状态实时写入 MongoDB 数据库。

```
1 import time
```

```
2 import sys
3
4 from app.utils.task_utils import add_running_task, add_done_task
5
6
7 def node_item_name_confirm(state):
8     """
9     节点功能：确认用户问题中的核心商品名称。
10    输入：state['original_query']
11    输出：更新 state['item_names']
12    """
13    print(f"---node_item_name_confirm---开始处理")
14    # 记录任务开始
15    add_running_task(state["session_id"], sys._getframe().f_code.co_name, state["is_stream"])
16
17    # 后面会调用大模型，进行逻辑处理
18    time.sleep(1)
19    # 记录任务结束
20    add_done_task(state["session_id"], sys._getframe().f_code.co_name, state["is_stream"])
21
22    print(f"---node_item_name_confirm---处理结束")
23
24    return {"item_names": ["示例商品"]}
```

(2) 向量检索: node_search_embedding.py

这个节点 node_search_embedding 负责根据 改写后的用户问题，在 限定的商品范围内，利用 BGEM3 混合检索（稠密+稀疏）技术，从 Milvus 向量数据库中召回 Top5 最相关的知识切片。

```
1 import time
2 import sys
3 from app.utils.task_utils import add_done_task, add_running_task
4
5 def node_search_embedding(state):
6     """
7     节点功能：进行向量内容检索
8     """
9     print("---量内容检索 开始处理---")
10    add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
11
12    # 搜索假设性答案
13    print("量内容检索答案！！")
14    time.sleep(1)
15
16    # ...
17    add_done_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
18
19    print("---量内容检索 处理结束---")
20    return state
```

(3) HyDE 假设检索节点: node_search_embedding_hyde.py

这个节点 node_search_embedding_hyde 实现了 HyDE (Hypothetical Document Embeddings) 策略，核心逻辑是 先让 LLM 虚构一个“理想答案”，再用这个答案去向量库检索真实的文档。

一句话总结：它通过“LLM 生成假设性答案”来增强原始问题的语义信息，再进行混合向量检索，从而大幅提升对“语义匹配但字面不匹配”问题的召回能力。

```
1 import time
2 import sys
3 from app.utils.task_utils import add_done_task, add_running_task
4
5 def node_search_embedding_hyde(state):
6     """
7     节点功能：HyDE (Hypothetical Document Embedding)
8     先让 LLM 生成假设性答案，再对答案进行向量检索，提高召回率。
9     """
10    print("---HyDE 开始处理---")
11    add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
12
13    # 搜索假设性答案
14    print("搜索架设性答案！！")
```

```
15     time.sleep(1)
16
17     # ...
18     add_done_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
19
20     print("---HyDE 处理结束---")
21     return state
```

(4) 网络搜索节点: node_web_search_mcp.py

这个节点 node_web_search_mcp 负责调用 百炼 MCP (Model Context Protocol) 联网搜索服务，获取互联网上的实时信息。

一句话总结： 它通过 MCP 协议异步调用百炼联网搜索接口，将用户的查询转化为实时的、结构化的网络搜索结果（包含标题、链接和摘要）。

```
1 import time
2 import sys
3 from app.utils.task_utils import add_done_task,add_running_task
4
5 def node_web_search_mcp(state):
6     """
7     节点功能，调用外部搜索引擎补充信息
8     :param state:
9     :return:
10    """
11    add_running_task(state["session_id"], sys._getframe().f_code.co_name,state["is_stream"])
12    print("---node-web-search-mcp处理---")
13
14    add_done_task(state["session_id"],sys._getframe().f_code.co_name,state["is_stream"])
15    time.sleep(1)
16    # 调用mcp外部引擎
17    print(f"调用外部mcp引擎")
18
19    print("---node-web-search-mcp处理结束---")
20    return state
```

(5) 倒排融合排序节点: node_rrf.py

这个节点 node_rrf 负责 将来自不同检索源（如向量检索、HyDE 检索等）的文档列表进行统一合并和重新排序。它消除了不同检索方式的分数差异，给出一个综合排名最高的文档列表（Top 10）。

什么是 RRF (Reciprocal Rank Fusion, 倒排排序融合)?

RRF 是一种 不需要知道具体分数，只关心排名 的融合算法。

通俗解释： 假设你在参加一场全能比赛：

- 裁判 A（向量检索） 觉得你是第 1 名。
- 裁判 B（HyDE 检索） 觉得你是第 3 名。
- 裁判 C（关键字检索） 觉得你是第 10 名。

RRF 不管裁判打具体多少分（因为不同裁判打分标准不同，有的满分100，有的满分10），它只看 排名。

计算公式：

$$Score = \sum \frac{1}{k + rank}$$

- rank 是你在某个裁判那里的排名（第1名就是1，第2名就是2）。
- k 是一个常数（通常取 60），用来防止排名靠前的差距过大。

RRF 的核心思想：

- 奖励多面手：如果你在多个榜单上都排在前面，你的总分就会很高。
- 平滑差异：即使你在某个榜单上稍微落后，只要其他榜单表现好，总分依然能上去。
- 去重：同一个文档在不同榜单出现多次，RRF 会把它合并，只算一次总分。

举个栗子：假设我们有两个检索来源：

- 来源 A (向量检索) 找出了前 3 名：
 - 第 1 名：文档 X
 - 第 2 名：文档 Y

- 第 3 名： 文档 Z
2. 来源 B (HyDE 检索) 找出了前 3 名：
- 第 1 名： 文档 Y (注意： 它觉得 Y 比 X 好)
 - 第 2 名： 文档 Z
 - 第 3 名： 文档 W (新面孔)

我们来算一下每个文档的总分：

1. 文档 X
- 在 A 中排第 1 $\rightarrow$ 得分 = $1 / (60 + 1) = 0.0164$
 - 在 B 中没出现 $\rightarrow$ 得分 = 0
 - 总分 = $0.0164 + 0 = 0.0164$
2. 文档 Y (重点看这个!)
- 在 A 中排第 2 $\rightarrow$ 得分 = $1 / (60 + 2) = 0.0161$
 - 在 B 中排第 1 $\rightarrow$ 得分 = $1 / (60 + 1) = 0.0164$
 - 总分 = $0.0161 + 0.0164 = 0.0325$
3. 文档 Z
- 在 A 中排第 3 $\rightarrow$ 得分 = $1 / (60 + 3) = 0.0159$
 - 在 B 中排第 2 $\rightarrow$ 得分 = $1 / (60 + 2) = 0.0161$
 - 总分 = $0.0159 + 0.0161 = 0.0320$
4. 文档 W
- 在 A 中没出现 $\rightarrow$ 得分 = 0
 - 在 B 中排第 3 $\rightarrow$ 得分 = $1 / (60 + 3) = 0.0159$
 - 总分 = $0 + 0.0159 = 0.0159$

最终排名结果：

1. 第一名： 文档 Y (0.0325) (虽然在 A 排第二， 但在 B 排第一， 综合实力最强)
2. 第二名： 文档 Z (0.0320) (两个榜单都上榜， 表现稳定)
3. 第三名： 文档 X (0.0164) (偏科， 只在 A 表现好)
4. 第四名： 文档 W (0.0159)

总结： RRF 就是通过这种方式， 让 “在多个榜单都靠前” 的文档排到最前面， 比单看某一个榜单更靠谱！

```
1 import time
2 import sys
3 from app.utils.task_utils import add_running_task, add_done_task
4
5 def node_rrf(state):
6     """
7     节点功能：Reciprocal Rank Fusion
8     将多路召回的结果（向量、HyDE、Web、KG）进行加权融合排序。
9     """
10    print("---RRF---")
11    add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
12    time.sleep(1)
13    # ...
14    add_done_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))
15    return state
```

(6) 重排序节点: node_rerank.py

这个节点 node_rerank 是知识库检索的“精修师”，它对之前所有步骤召回的文档进行 二次精细排序。

核心流程分为三步：

1. 合并文档：将来自 RRF（本地检索）和 Web Search（联网搜索）的文档合并到一个池子中。
2. 精确打分：使用重排序模型计算每个文档与用户问题的相关性得分。
3. 动态截断：根据得分的“断崖式下跌”点，智能截取 TopK（最多 10 条），只保留高质量结果，过滤凑数的低分文档。

该节点使用的是 BGE Reranker (BAAI/bge-reranker-large) 模型。

- 模型类型：Cross-Encoder（交叉编码器）。
- 工作原理：它不像向量检索那样分别计算向量再比对，而是直接把“问题”和“文档”拼接在一起扔给模型，让模型像阅读理解一样，深入分析两者的语义匹配度。
- 特点：
 - 精度极高：能识别微小的语义差异（如“苹果手机”和“苹果公司”的区别）。
 - 速度较慢：因为计算量大，所以通常只用于对初筛后的少量文档（如 Top 20-50）进行精排，不适合全库检索。

```
1 import time
2 import sys
3 from app.utils.task_utils import add_running_task, add_done_task
4
5 def node_rerank(state):
6     """
7     节点功能：使用 Cross-Encoder 模型对 RRF 后的结果进行精确打分重排。
8     """
9     print("---Rerank处理---")
10    add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
11
12    time.sleep(1)
13    # ...
14    add_done_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))
15    return state
```

(7) 答案生成: node_answer_output.py

这个节点 node_answer_output 是知识库查询的“最后一公里”，负责 生成最终回答 并 交付给用户。

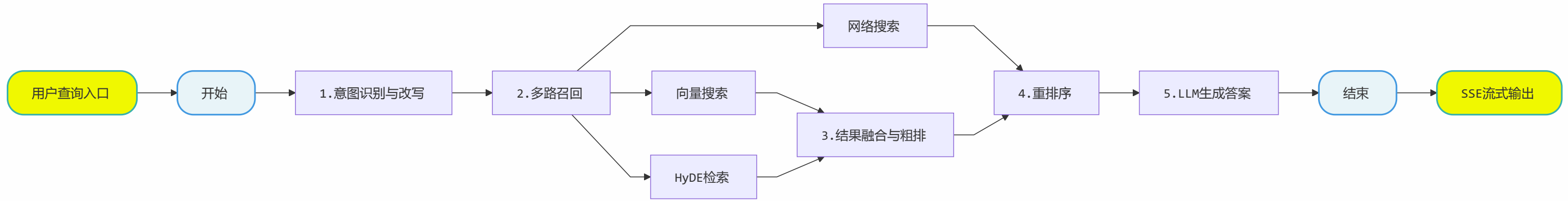
它整合了之前所有步骤的成果，通过以下 5 个核心动作完成任务：

1. 检查前置答案：如果之前步骤（如商品名确认节点）已经生成了追问或拒绝回答，直接输出，跳过 LLM 生成。
2. 构建 Prompt：将用户问题、历史对话、以及 Rerank 后的 TopK 高质量文档片段（包含元数据）组装成一段严谨的提示词。
3. LLM 生成与流式推送：调用大模型生成最终答案。如果是流式模式，会**逐字推送（Delta）**给前端，实现打字机效果。
4. 图片提取与增强：从引用文档中自动提取图片 URL（包括网页链接和本地 Markdown 图片），为纯文本答案补充视觉信息。
5. 收尾与存档：将最终答案和提取的图片写入 MongoDB 历史记录，并向前端发送包含完整信息（答案+图片）的 FINAL 信号。

```
1 import time
2 import sys
3 from app.core.logger import logger
4 from app.utils.sse_utils import push_to_session, SSEEvent
5 from app.utils.task_utils import add_running_task, add_done_task
6
7 def node_answer_output(state):
8     """
9     节点功能：进行过处理可以是流式输出可以整体输出！
10    """
11    print("---node_answer_output 节点处理开始---")
12    add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
13
14    session_id = state["session_id"]
15    is_stream = state.get("is_stream", True)
16    base_answer = state.get("answer") or f"这是关于 [{state.get('original_query', '当前问题')}] 的测试回答，正在演示打字机流式输出效果。"
17    final_text = ""
18
19    if is_stream:
20        for ch in base_answer:
21            final_text += ch
22            push_to_session(session_id, SSEEvent.DELTA, {"delta": ch})
23            time.sleep(0.03)
24
25    image_urls = ["https://example.com/demo-1.png", "https://example.com/demo-2.png"]
26    push_to_session(
27        session_id,
28        SSEEvent.FINAL,
29        {
30            "answer": final_text,
31            "status": "completed",
32            "image_urls": image_urls
```

```
33         }
34     )
35     logger.info(f"流式输出完成，总长度: {len(final_text)}")
36 else:
37     final_text = base_answer
38
39 add_done_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))
40 print("---node_answer_output 节点处理结束---")
41 return {"answer": final_text}
```

7.2.2 第二步：编写主图代码



文件: app/query_process/agent/main_graph.py

1) 初始化图与节点注册

```
1 from langgraph.graph import StateGraph, END
2
3 from app.query_process.agent.nodes.node_answer_output import node_answer_output
4 from app.query_process.agent.nodes.node_item_name_confirm import node_item_name_confirm
5 from app.query_process.agent.nodes.node_rerank import node_rerank
6 from app.query_process.agent.nodes.node_rrf import node_rrf
7 from app.query_process.agent.nodes.node_search_embedding import node_search_embedding
8 from app.query_process.agent.nodes.node_search_embedding_hyde import node_search_embedding_hyde
9 from app.query_process.agent.nodes.node_web_search_mcp import node_web_search_mcp
10 from app.query_process.agent.state import QueryGraphState
11
12 builder = StateGraph(QueryGraphState)
13
14 # 注册节点（已删除虚拟节点）
15 builder.add_node("node_item_name_confirm", node_item_name_confirm)
16 builder.add_node("node_search_embedding", node_search_embedding)
17 builder.add_node("node_search_embedding_hyde", node_search_embedding_hyde)
18 builder.add_node("node_web_search_mcp", node_web_search_mcp)
19 builder.add_node("node_rrf", node_rrf)
20 builder.add_node("node_rerank", node_rerank)
21 builder.add_node("node_answer_output", node_answer_output)
22
23 # 入口
24 builder.set_entry_point("node_item_name_confirm")
25
26 # 条件路由
27 def route_after_item_confirm(state: QueryGraphState):
28     if state.get("answer"):
29         """
30         这主要发生在 node_item_name_confirm 节点无法直接确定唯一的商品型号，从而需要“反问用户”或“拒绝回答”的场景。
31         具体来说，有以下两种情况会导致 state 中直接出现 answer，从而跳过后续的检索流程，直接输出：
32         1. 多选一（反问用户）：
33             - 场景：用户问得太模糊（比如“华为P60”），系统发现数据库里有“华为P60 128G”和“华为P60 Art”两个型号，且置信度都不足以直接确认。
34             - 处理：节点会生成一条反问句作为 answer，例如：“您是想问以下哪个产品：华为P60 128G、华为P60 Art？请明确一下型号。”
35             - 结果：此时不需要再去检索文档了，直接把这句话发给用户让他选。
36         2. 查无此人（拒绝回答）：
37
38             - 场景：用户问了一个系统里压根没有的商品（比如“小米15”，但库里只有华为的数据），或者评分过低（<0.6）。
39             - 处理：节点会生成一条拒绝句作为 answer，例如：“抱歉，未找到相关产品，请提供准确型号以便我为您查询。”
40             - 结果：同样不需要后续检索，直接结束流程。
41         """
42         return "node_answer_output"
43     # 直接进入并发搜索
44     return "node_search_embedding", "node_search_embedding_hyde", "node_web_search_mcp"
45
46 builder.add_conditional_edges(
47     "node_item_name_confirm",
48     route_after_item_confirm,
49     # 显示添加否则！不完整
```

```
50 {
51     "node_answer_output": "node_answer_output",
52     "node_search_embedding": "node_search_embedding",
53     "node_search_embedding_hyde": "node_search_embedding_hyde",
54     "node_web_search_mcp": "node_web_search_mcp",
55 }
56 )
57
58 # 三个搜索 → 直接汇合到 RRF
59 builder.add_edge("node_search_embedding", "node_rrf")
60 builder.add_edge("node_search_embedding_hyde", "node_rrf")
61 builder.add_edge("node_web_search_mcp", "node_rrf")
62
63 # 正常流程
64 builder.add_edge("node_rrf", "node_rerank")
65 builder.add_edge("node_rerank", "node_answer_output")
66 builder.add_edge("node_answer_output", END)
67
68 query_app = builder.compile()
```

7.2.3 第三步：验证图流程

在实现具体业务逻辑前，我们先跑一个测试脚本，看看图能不能跑通，路线对不对。

创建测试文件: test_query_main_graph.py (在项目根目录)

```
1 import json
2
3 from app.query_process.agent.main_graph import query_app
4 from app.query_process.agent.state import create_query_default_state
5 from app.core.logger import logger
6
7 logger.info("==== 开始测试 =====")
8
9 initial_state = create_query_default_state(session_id="test_001",
10     original_query="华为P60怎么样?")
11 final_state = None
12
13 # 只输出更最终的状态值（字典形式），不包含节点名称、执行日志、元数据等额外信息
14 for event in query_app.stream(initial_state):
15     for key, value in event.items():
16         logger.info(f"节点: {key}")
17         final_state = value
18
19 # 格式化输出最终状态
20 logger.info(f"最终状态: {json.dumps(final_state, indent=4, ensure_ascii=False)}")
21
22 logger.info("图结构:")
23 # uv add grandalf
24 query_app.get_graph().print_ascii()
25
26 logger.info("==== 测试结束 =====")
```

如果能看到这些日志，说明我们的图结构搭建成功！接下来就可以放心地去填充每个节点的具体代码了。

8. 检索web服务端搭建

8.1 SSE快速入门

SSE (Server-Sent Events) 是一种 **基于 HTTP 的服务端单向推送** 技术：浏览器用一个长连接（通常是 GET）订阅，服务端持续向这个连接 **按事件 (event) 流式写数据**，浏览器按事件回调接收。

- **单向推送**：服务端 → 客户端（客户端发消息仍走普通 HTTP 请求）
- **事件化**：每条消息带 event 类型 + data 数据
- **自动重连**：浏览器 EventSource 断线会自动重连（可配合 retry）
- **轻量**：基于 HTTP，不需要 WebSocket 的双工协议栈

8.1.1 应用场景

- **任务进度/阶段状态**：文件上传处理、图执行流程、批处理进度条
- **LLM 流式输出**：边生成边展示（token/片段增量 delta）
- **日志/监控流**：持续输出日志、告警、指标变化
- **通知推送**：轻量通知、状态变化（但不适合高频双向交互）

8.1.2 数据格式

SSE 协议规定了服务端向前端推送数据的**固定核心格式**，前端需按此解析，具体规则如下：

1	[可选] event: <事件名> 用于分类数据（如progress进度、result答案、error报错），前端可按事件名单独处理；
2	必填 data: <数据内容>\n\n 必填：\n\n（两个连续换行符，作为单条数据的结束标识）
3	（扩展）id: <唯一ID>/retry: <重连毫秒数>：可选，用于断点续传、自定义重连时间。

其中 event 把数据按事件分类，比如进度变更、答案输出、报错等等。data 是自定义数据。

8.1.3 SSE基础入门

场景一：最基础的 SSE

目标：后端每秒推 1 条固定消息，前端实时显示

步骤1：后端代码（sse_step1.py）

```
1 import asyncio
2 from fastapi import FastAPI
3 from fastapi.responses import StreamingResponse
4 from fastapi.middleware.cors import CORSMiddleware
5
6 # 1. 初始化+跨域（最基础配置）
7 app = FastAPI()
8 app.add_middleware(
9     CORSMiddleware,
10     allow_origins=["*"], # 仅测试用
11     allow_methods=["*"],
12     allow_headers=["*"],
13 )
14
15 # 2. 核心：SSE接口（只推固定数据）
16 @app.get("/simple_stream")
17 async def simple_stream():
18     async def event_generator():
19         # 模拟推5条消息，每秒1条
20         for i in range(5):
21             # ✅ 核心：SSE固定格式 data: 内容\n\n
22             # yield f"event:自定义\ndata:xxxx\n\n"
23             yield f"data: 这是第{i+1}条测试消息\n\n"
24             await asyncio.sleep(1) # 每秒推1条
25
26     # ✅ 核心：StreamingResponse + media_type=text/event-stream
27     return StreamingResponse(
28         event_generator(),
29         media_type="text/event-stream"
30     )
31
32 if __name__ == "__main__":
33     import uvicorn
34     uvicorn.run(app, host="127.0.0.1", port=8001)
```

步骤2：前端代码（sse_step1.html）

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <meta charset="UTF-8">
5     <title>步骤1：基础SSE</title>
6 </head>
7 <body>
8     <h3>步骤1：接收后端固定消息</h3>
```

```
9      <div id="result"></div>
10
11      <script>
12          //  ✅ 核心：创建EventSource连接SSE接口
13          const eventSource = new EventSource("http://127.0.0.1:8001/simple_stream");
14          const resultDom = document.getElementById("result");
15
16          // 监听SSE消息（实时接收）
17          eventSource.onmessage = function(event) {
18              resultDom.innerHTML += event.data + "<br>";
19          };
20      </script>
21 </body>
22 </html>
```

步骤3：运行演示

- 1. 启动后端：python sse_step1.py;
- 2. 打开sse_step1.html，能看到页面每秒显示 1 条消息：

```
1 这是第1条测试消息
2 这是第2条测试消息
3 ...
4 这是第5条测试消息
```

核心知识点拆解：

- 1. async def 异步函数
FastAPI 支持异步接口，async 标识该函数可以执行异步操作（比如 await asyncio.sleep(1)），不会阻塞整个服务的其他请求，性能更好。
- 2. 异步生成器 event_generator()
 - 用 async def 定义，内部通过 yield 逐次返回数据（而非 return 一次性返回）；
 - 每次 yield 都会向客户端推送一段数据，直到循环结束；
 - await asyncio.sleep(1) 是异步休眠，区别于 time.sleep(1)（同步休眠会阻塞），保证服务能同时处理其他请求。
- 3. StreamingResponse 流式响应
FastAPI 提供的专门用于“流式返回数据”的响应类，接收一个生成器（或异步生成器）作为参数，会逐次把生成器 yield 的内容发送给客户端。
- 4. SSE 协议核心规则
 - 响应的 media_type 必须设为 text/event-stream，客户端（比如浏览器）才能识别这是 SSE 流；
 - 推送的每条消息必须遵循 data: 内容\n\n 格式（\n\n 是消息结束的分隔符，缺一不可）；
 - SSE 是**单向通信**（服务器→客户端），适合实时推送通知、日志、进度等场景。

场景二：动态传参

目标：前端传会话 ID，后端按 ID 返回专属消息

步骤1：后端代码（sse_step2.py）

```
1  import asyncio
2  from fastapi import FastAPI
3  from fastapi.responses import StreamingResponse
4  from fastapi.middleware.cors import CORSMiddleware
5
6  app = FastAPI()
7  app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
8
9  # 新增：接口接收session_id参数
10 @app.get("/stream/{session_id}")
11 async def stream_by_session(session_id: str):
12     async def event_generator():
13         for i in range(5):
14             # 按session_id定制消息
15             yield f"data: 会话{session_id} - 第{i+1}条消息\n\n"
16             await asyncio.sleep(1)
17
18     return StreamingResponse(event_generator(), media_type="text/event-stream")
19
```

```
20 if __name__ == "__main__":
21     import uvicorn
22     uvicorn.run(app, host="127.0.0.1", port=8001)
```

步骤2：前端代码（sse_step2.html）

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <meta charset="UTF-8">
5     <title>步骤2：按会话ID推数据</title>
6 </head>
7 <body>
8     <h3>步骤2：输入会话ID，接收专属消息</h3>
9     <input type="text" id="sessionIdInput" placeholder="输入会话ID（如123）" value="123">
10    <button onclick="connectSSE()">连接SSE</button>
11    <div id="result"></div>
12
13    <script>
14        let eventSource = null;
15        const resultDom = document.getElementById("result");
16
17        function connectSSE() {
18            const sessionId = document.getElementById("sessionIdInput").value;
19            resultDom.innerHTML = "";
20
21            // ✅ 新增：URL带session_id参数
22            eventSource = new EventSource(`http://127.0.0.1:8001/stream/${sessionId}`);
23            eventSource.onmessage = function(event) {
24                resultDom.innerHTML += event.data + "<br>";
25            };
26        }
27    </script>
28 </body>
29 </html>
```

步骤3：运行演示

启动后端，打开前端，输入“abc123” 点击连接，页面显示：

```
1 会话abc123 - 第1条消息
2 会话abc123 - 第2条消息
3 ...
```

核心知识点

- 后端：通过 URL 路径参数（session_id）接收前端传参；
- 前端：SSE 连接的 URL 可动态拼接参数，实现“一对一”推送。

场景三：异步任务

目标：后端先接收查询请求，后台处理，SSE 推处理结果

步骤1：后端代码（sse_step3.py）

```
1 import asyncio
2 from fastapi import FastAPI, BackgroundTasks
3 from fastapi.responses import StreamingResponse
4 from fastapi.middleware.cors import CORSMiddleware
5
6 app = FastAPI()
7 app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
8
9 # 核心优化：用异步队列存储每个会话的待推送数据（替列表+轮询）
10 task_queues = {}
11
12
13 # 异步耗时任务：直接往队列丢数据，不用列表累加
14 async def long_task(session_id: str):
15     # 为当前会话创建专属队列
16     queue = asyncio.Queue()
17     task_queues[session_id] = queue
```

```
18
19     # 模拟5秒处理，每秒生成1条结果并丢进队列
20     for i in range(5):
21         msg = f"会话{session_id}处理结果{i + 1}"
22         await queue.put(msg)  # 把数据丢进队列
23         await asyncio.sleep(1)
24
25     # 关键：丢一个"结束标记"，告诉SSE可以停止了
26     await queue.put(None)
27
28
29 # 提交任务接口（逻辑不变）
30 @app.get("/submit/{session_id}")
31 async def submit_task(session_id: str, background_tasks: BackgroundTasks):
32     background_tasks.add_task(long_task, session_id)
33     return {"message": "任务已启动", "session_id": session_id}
34
35
36 # 简化后的SSE接口：直接从队列取数据，没有轮询！
37 @app.get("/stream/{session_id}")
38 async def stream_result(session_id: str):
39     async def event_generator():
40         # 获取当前会话的队列（没有则等待任务创建）
41         while session_id not in task_queues:
42             await asyncio.sleep(0.1)
43         queue = task_queues[session_id]
44
45         # 核心：循环从队列取数据，有数据就推，收到结束标记就停
46         while True:
47             msg = await queue.get()  # 阻塞等待队列数据（比轮询高效）
48             if msg is None:  # 收到结束标记，退出循环
49                 break
50             yield f"data: {msg}\n\n"  # 推送数据
51
52     return StreamingResponse(event_generator(), media_type="text/event-stream")
53
54
55 if __name__ == "__main__":
56     import uvicorn
57
58     uvicorn.run(app, host="127.0.0.1", port=8001)
```

`asyncio.Queue` 是 Python 异步编程（`asyncio` 框架）里的**异步队列**，专门解决异步场景下“生产者 - 消费者”的通信问题，你可以把它理解成一个「异步版的消息中转站」—— 生产者（比如你的后台任务）往里面丢数据，消费者（比如你的 SSE 接口）从里面取数据，全程不阻塞、不轮询，比你之前用的“列表 + 轮询”高效得多。

先讲核心特点（和普通列表 / 同步队列的区别）：

特性	普通列表	<code>asyncio.Queue</code> （异步队列）
取数据的方式	主动轮询（每 0.5s 查一次）	被动等待（有数据才唤醒）
是否阻塞	不阻塞（空列表也能查）	异步阻塞（没数据就暂停，不占 CPU）
线程 / 协程安全	不安全（多协程操作易出错）	天然安全（专为异步场景设计）
代码复杂度	高（要判断长度、处理重复）	低（只需要 <code>put/get</code> ）

核心用法（通俗解释）：

`asyncio.Queue` 的用法特别简单，核心就 3 个方法，且都需要用 `await` 调用（因为是异步操作）：

创建队列

```
1 # 创建一个无界异步队列（能装无限多数据）
2 queue = asyncio.Queue()
3 # 也可以指定最大容量（比如最多装10条，满了之后put会等待）
4 queue = asyncio.Queue(maxsize=10)
```

生产者：往队列里放数据（`put`）

```
1 await queue.put("要推送的消息")  # 把数据丢进队列
2 # 如果队列满了（指定了maxsize），这行代码会暂停，直到队列有空闲位置
```

对应你代码里的后台任务：await queue.put(msg)，每秒往队列丢一条消息。

消费者：从队列里取数据（get）

```
1 msg = await queue.get() # 从队列取数据
2 # 如果队列为空，这行代码会「异步暂停」，直到队列里有新数据才唤醒
```

对应你代码里的 SSE 接口：msg = await queue.get()，没有数据就等着，有数据就立刻取，不用你手动轮询。

步骤2：前端代码（sse_step3.html）

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="UTF-8">
5   <title>步骤3：异步任务+SSE推送</title>
6 </head>
7 <body>
8   <h3>步骤3：提交任务，SSE接收处理结果</h3>
9   <input type="text" id="sessionIdInput" placeholder="会话ID" value="test001">
10  <button onclick="submitTask()">提交任务</button>
11  <div id="result"></div>
12
13  <script>
14    let eventSource = null;
15    const resultDom = document.getElementById("result");
16
17    // 提交任务（触发后端异步处理）
18    async function submitTask() {
19      const sessionId = document.getElementById("sessionIdInput").value;
20      resultDom.innerHTML = "";
21
22      // 1. 调用提交接口
23      await fetch(`http://127.0.0.1:8001/submit/${sessionId}`);
24
25      // 2. 立即建立SSE连接，等结果
26      eventSource = new EventSource(`http://127.0.0.1:8001/stream/${sessionId}`);
27      eventSource.onmessage = function(event) {
28        resultDom.innerHTML += event.data + "<br>";
29      };
30    }
31  </script>
32 </body>
33 </html>
```

步骤3：运行演示

- 1. 启动后端，打开前端，点击“提交任务”；
- 2. 页面每秒显示 1 条后端处理结果，5 秒后停止。

核心知识点

- 后端：BackgroundTasks 实现异步任务，避免阻塞 SSE 连接；
- 核心逻辑：“提交任务→后台处理→SSE 监听结果→实时推送”。

场景四：前端输入查询内容

目标：前端输入查询内容，后端按查询词返回结果

步骤1：后端代码（sse_step4.py）

```
1 import asyncio
2 from fastapi import FastAPI, BackgroundTasks
3 from fastapi.middleware.cors import CORSMiddleware
4 from fastapi.responses import StreamingResponse
5 from pydantic import BaseModel
6
7 app = FastAPI()
8 # 跨域配置（保持不变）
9 app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
10
11 # 替换列表：用异步队列存储每个会话的待推送数据
```

```
12 task_queues = {}
13
14 # 新增：定义请求体模型（保持不变）
15 class QueryRequest(BaseModel):
16     query: str
17     session_id: str
18
19 # 重构异步任务：往队列丢数据（替代列表累加）
20 async def long_task(session_id: str, query: str):
21     # 为当前会话创建专属异步队列
22     queue = asyncio.Queue()
23     task_queues[session_id] = queue
24
25     # 按查询词生成5条结果，每秒1条丢进队列
26     for i in range(5):
27         msg = f"【{query}】的第{i+1}段回答：xxx{i+1}"
28         await queue.put(msg) # 数据入队
29         await asyncio.sleep(1)
30
31     # 关键：放入结束标记，告诉SSE停止推送
32     await queue.put(None)
33
34 # POST接口（逻辑不变，仅任务内部实现变了）
35 @app.post("/submit_query")
36 async def submit_query(req: QueryRequest, background_tasks: BackgroundTasks):
37     # 把查询词和会话ID传给后台任务
38     background_tasks.add_task(long_task, req.session_id, req.query)
39     return {"message": "任务已启动", "session_id": req.session_id}
40
41 # 简化SSE接口：从队列取数据，无轮询
42 @app.get("/stream/{session_id}")
43 async def stream_result(session_id: str):
44     async def event_generator():
45         # 等待当前会话的队列创建（防止SSE比任务先启动）
46         while session_id not in task_queues:
47             await asyncio.sleep(0.1)
48         queue = task_queues[session_id]
49
50         # 循环取队列数据，有数据就推，收到结束标记就停
51         while True:
52             msg = await queue.get() # 异步阻塞等待数据（无轮询）
53             if msg is None: # 收到结束标记，退出循环
54                 break
55             yield f"data: {msg}\n\n" # 推送SSE数据
56
57     return StreamingResponse(event_generator(), media_type="text/event-stream")
58
59 if __name__ == "__main__":
60     import uvicorn
61     uvicorn.run(app, host="127.0.0.1", port=8001)
```

步骤2：前端代码（sse_step4.html）

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <meta charset="UTF-8">
5     <title>步骤4：输入查询内容</title>
6 </head>
7 <body>
8     <h3>步骤4：输入查询内容，SSE接收专属回答</h3>
9     <input type="text" id="queryInput" placeholder="输入查询内容" value="Python SSE怎么用">
10    <button onclick="submitQuery()">提交查询</button>
11    <div id="result"></div>
12
13    <script>
14        let eventSource = null;
15        const resultDom = document.getElementById("result");
16
17        async function submitQuery() {
18            const query = document.getElementById("queryInput").value;
19            const sessionId = "query_" + Date.now(); // 自动生成会话ID
```

```
20         resultDom.innerHTML = "";
21
22         // 1. POST提交查询内容
23         await fetch("http://127.0.0.1:8001/submit_query", {
24             method: "POST",
25             headers: {"Content-Type": "application/json"},
26             body: JSON.stringify({query: query, session_id: sessionId})
27         });
28
29         // 2. 建立SSE连接
30         eventSource = new EventSource(`http://127.0.0.1:8001/stream/${sessionId}`);
31         eventSource.onmessage = function(event) {
32             resultDom.innerHTML += event.data + "<br>";
33         };
34     }
35 </script>
36 </body>
37 </html>
```

步骤3：运行演示

输入 “FastAPI 教程”， 点击提交， 页面显示：

```
1 【FastAPI教程】的第1段回答：xxx1
2 【FastAPI教程】的第2段回答：xxx2
3 ...
```

核心知识点

- 后端：用Pydantic模型接收 POST 请求体，解析查询内容；
- 前端：POST 请求传 JSON 数据，实现 “用户输入→后端处理→实时返回”。

场景五：SSE 事件属性说明

SSE 协议中，除了核心的 data 字段，还支持 event（自定义事件类型）、id（消息 ID）、retry（重连时间）等属性：

- event：用于给不同类型的消息标记自定义事件名，前端可以根据事件名区分处理不同消息（比如 “进度更新”“完成通知”）；
- 格式要求：event：事件名\n 必须在 data：内容\n\n 之前；
- 改造方向：在生成 SSE 响应时，为不同阶段的消息（进度、完成）添加不同的 event 属性。

步骤1：后端代码（sse_step5.py）

```
1 import asyncio
2 import uuid
3 from fastapi import FastAPI, BackgroundTasks
4 from fastapi.middleware.cors import CORSMiddleware
5 from fastapi.responses import StreamingResponse
6 from pydantic import BaseModel
7
8 app = FastAPI()
9 # 跨域配置（保持不变）
10 app.add_middleware(
11     CORSMiddleware,
12     allow_origins=["*"],
13     allow_methods=["*"],
14     allow_headers=["*"]
15 )
16
17 # ✅ 替换列表缓存：用异步队列存储每个会话的消息（key: session_id, value: asyncio.Queue）
18 task_queues = {}
19
20 class QueryRequest(BaseModel):
21     query: str
22     session_id: str = None
23
24 @app.post("/submit_query")
25 async def submit_query(req: QueryRequest, background_tasks: BackgroundTasks):
26     # 生成或使用用户传入的session_id
27     session_id = req.session_id or str(uuid.uuid4())
28     # 将耗时任务加入后台执行
29     background_tasks.add_task(long_task, session_id, req.query)
```

```
30     return {"message": "任务已经启动", "session_id": session_id}
31
32 async def long_task(session_id: str, query: str):
33     """模拟耗时的异步任务，分阶段往队列丢消息（替代列表累加）"""
34     # 为当前会话创建专属异步队列
35     queue = asyncio.Queue()
36     task_queues[session_id] = queue
37
38     # 模拟5个进度步骤，往队列丢进度消息
39     for i in range(5):
40         progress_msg = {
41             "event": "progress",
42             "data": f"【{query}】的第{i+1}段回答:xxx{i+1}"
43         }
44         await queue.put(progress_msg) # 进度消息入队
45         await asyncio.sleep(1)
46
47     # 任务完成，往队列丢完成消息
48     complete_msg = {
49         "event": "complete",
50         "data": f"【{session_id}】查询完成！所有结果已返回"
51     }
52     await queue.put(complete_msg)
53     # 关键：丢结束标记，告诉SSE可以停止监听
54     await queue.put(None)
55
56 @app.get("/stream/{session_id}")
57 async def stream(session_id: str):
58     """SSE流式返回任务结果，基于队列实现（无轮询）"""
59     async def event_generator():
60         # 等待当前会话的队列创建（防止SSE比任务先启动）
61         while session_id not in task_queues:
62             await asyncio.sleep(0.1)
63         queue = task_queues[session_id]
64
65         # 循环从队列取消息，有消息就推，收到结束标记就停
66         while True:
67             msg = await queue.get() # 异步阻塞等待消息（无轮询）
68             if msg is None: # 收到结束标记，退出循环
69                 break
70
71             # 拼接自定义Event的SSE格式（和你原逻辑一致）
72             yield f"event: {msg['event']}\n"
73             yield f"data: {msg['data']}\n\n"
74
75     return StreamingResponse(event_generator(), media_type="text/event-stream")
76
77 if __name__ == "__main__":
78     import uvicorn
79     uvicorn.run(app, host="127.0.0.1", port=8001)
```

long_task 中不再存储纯文本，而是存储包含 event 和 data 的字典，分别标记消息类型（progress/complete）和内容；

这样可以区分“进度更新”和“任务完成”两类消息，前端能针对性处理。

标准 SSE 格式要求：event：事件名\n + data：内容\n\n；

示例输出（前端收到的原始数据）：

```
1 event: progress
2 data: 【测试】的第1段回答:xxx1
3
4 event: complete
5 data: 【xxx-xxx-xxx】查询完成！所有结果已返回
```

步骤2：前端代码（sse_step5.py）

```
1 <!DOCTYPE html>
2 <html lang="zh-CN">
3 <head>
4     <meta charset="UTF-8">
5     <title>FastAPI SSE 测试</title>
```

```
6      <!-- 仅保留必要的基础样式，无复杂装饰 -->
7      <style>
8          body { padding: 20px; }
9          #response { margin-top: 20px; white-space: pre-wrap; }
10         .progress { color: #666; }
11         .complete { color: green; }
12         .error { color: red; }
13     </style>
14 </head>
15 <body>
16     <!-- 核心交互区域 -->
17     <input type="text" id="query" placeholder="输入查询内容" style="width: 400px; padding: 5px;">
18     <button onclick="submitQuery()">提交查询</button>
19
20     <!-- 流式响应展示区 -->
21     <div id="response"></div>
22
23     <script>
24         // 后端接口地址（和你的FastAPI启动地址一致）
25         const API_BASE = 'http://127.0.0.1:8001';
26         let eventSource = null; // 存储SSE连接实例
27
28         // 提交查询的核心函数
29         async function submitQuery() {
30             const query = document.getElementById('query').value.trim();
31             if (!query) {
32                 alert('请输入查询内容！');
33                 return;
34             }
35
36             // 清空之前的响应内容
37             document.getElementById('response').innerHTML = '';
38
39             // 1. 调用submit_query接口获取session_id
40             try {
41                 const res = await fetch(`${API_BASE}/submit_query`, {
42                     method: 'POST',
43                     headers: { 'Content-Type': 'application/json' },
44                     body: JSON.stringify({ query: query })
45                 });
46                 const data = await res.json();
47                 const sessionId = data.session_id;
48
49                 // 2. 建立SSE连接监听流式响应
50                 connectSSE(sessionId);
51             } catch (err) {
52                 document.getElementById('response').innerHTML = `<div class="error">提交失败：${err.message}</div>`;
53             }
54         }
55
56         // 建立SSE连接的函数
57         function connectSSE(sessionId) {
58             // 关闭已有连接，避免重复监听
59             if (eventSource) eventSource.close();
60
61             // 创建新的SSE连接
62             eventSource = new EventSource(`${API_BASE}/stream/${sessionId}`);
63
64             // 监听进度事件 ( progress )
65             eventSource.addEventListener('progress', (e) => {
66                 const responseDiv = document.getElementById('response');
67                 responseDiv.innerHTML += `<div class="progress">${e.data}</div>`;
68             });
69
70             // 监听完成事件 ( complete )
71             eventSource.addEventListener('complete', (e) => {
72                 const responseDiv = document.getElementById('response');
73                 responseDiv.innerHTML += `<div class="complete">${e.data}</div>`;
74                 eventSource.close(); // 完成后关闭连接
75             });
76
77             // 监听错误事件
```

```
78         eventSource.onerror = (e) => {
79             document.getElementById('response').innerHTML += `<div class="error">连接异常：${e.message} || '未知错误'</div>`;
80             eventSource.close();
81         };
82     }
83 </script>
84 </body>
85 </html>
```

event 是 SSE 协议的扩展属性，用于分类消息，前端可通过 addEventListener(事件名) 监听；

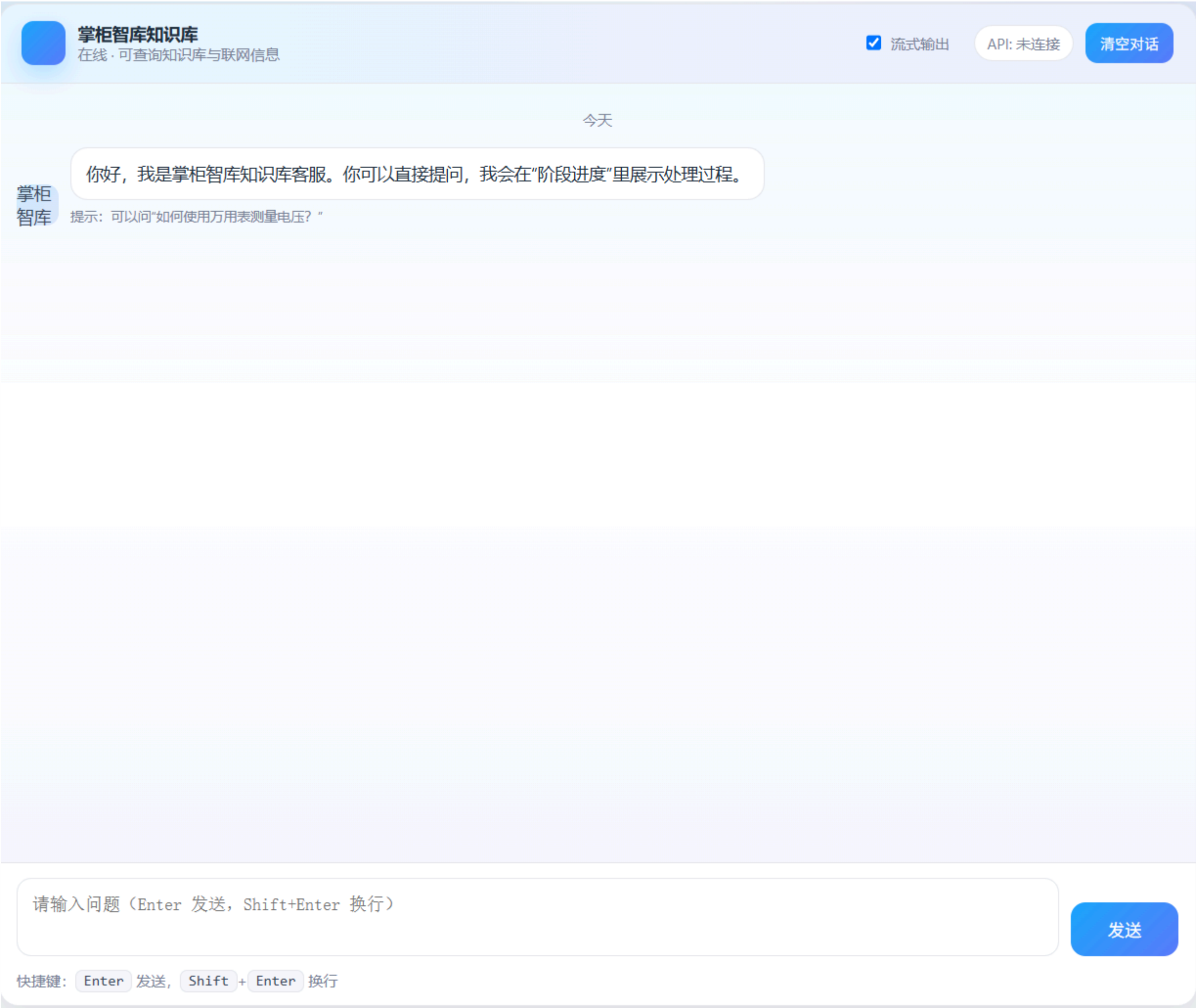
SSE 消息必须以 \n\n 结尾，多个字段（event/data）之间用 \n 分隔；

8.2 基础 Web 服务端搭建

本节介绍如何基于 FastAPI 搭建后端服务，提供页面访问和查询接口。

8.2.1 前端交互设计

检索 Web 聊天界面（chat.html）的核心结构与数据流转逻辑，该页面直接决定了后端接口的设计规范。



1) 页面核心组件

- **顶栏 (Topbar):** 展示服务连接状态与流式开关。
- **对话区 (Chat):**
 - **用户气泡:** 展示提问内容。
 - **系统气泡:** 集成**文本答案与处理进度**（折叠面板），实时展示后台（检索/重排/生成）的执行状态。
- **输入区 (Composer):** 支持快捷发送与多行输入。

2) 数据交互闭环

前端与后端的全双工交互流程如下：

1. **会话初始化**：加载页面时生成或读取 session_id，作为用户唯一标识。
2. **提交任务**：点击发送 -> POST /query（携带问题与流式标记） -> 获取 session_id 确认任务已接收。
3. **建立长连接 (SSE)**：立即通过 EventSource 监听 /stream/{session_id}，建立实时通信管道。
4. **事件驱动更新**：

▪ ready: 连接握手成功。

▪ progress: **实时更新进度条**（如：正在检索... -> 检索完成）。

▪ delta: **流式逐字输出**（打字机效果）。

▪ final: 接收完整答案与引用源。

基于此逻辑，后端需提供 /query（任务提交）与 /stream（事件推送）两个核心接口。

8.2.2 服务接口设计

本节详细定义查询服务的所有对外接口，包括页面访问、任务提交、流式推送及历史管理。

1) 页面访问接口

- **路径**: /chat.html (GET)
- **功能**: 返回前端聊天界面。
- **响应**: HTML 静态页面。

2) 检索查询接口

- **路径**: /query (POST)
- **功能**: 接收用户提问并启动后台处理图逻辑。
- **参数**:

```
1 {
2   "query": "万用表怎么测量电压？",
3   "session_id": "可选，未传则后台自动生成",
4   "is_stream": true // 是否启用流式推送
5 }
```

- **响应 (is_stream: true)**:

```
1 { "message": "结果正在处理中...", "session_id": "xxx-uuid" }
```

- **响应 (is_stream: false)**:

```
1 { "message": "处理完成！", "session_id": "xxx", "answer": "回答内容...", "done_list": [] }
```

3) 流式获取接口 (SSE)

- **路径**: /stream/{session_id} (GET)
- **功能**: 建立 SSE 长连接，实时推送任务进度与生成文本。
- **推送数据格式 (JSON)**:

▪ **progress 事件**: {"done_list": ["节点A", "节点B"], "running_list": ["节点C"]}

▪ **delta 事件**: {"text": "生成的增量字符"}

▪ **final 事件**: {"answer": "完整最终答案"}

▪ **error 事件**: {"error": "错误详情"}

4) 会话历史查询

- **路径**: /history/{session_id} (GET)
- **功能**: 从 MongoDB 中获取当前会话的历史聊天记录。
- **参数**: limit (可选，默认50条)
- **响应**:

```
1 {
2   "session_id": "xxx",
3   "items": [
4     {
5       "_id": str(r.get("_id")) if r.get("_id") is not None else "",
```

```
6         "session_id": r.get("session_id", ""),
7         "role": r.get("role", ""),
8         "text": r.get("text", ""),
9         "rewritten_query": r.get("rewritten_query", ""),
10        "item_names": r.get("item_names", []),
11        "ts": r.get("ts")
12    }
13 ]
14 }
```

5) 清空会话历史

- **路径:** /history/{session_id} (DELETE)
- **功能:** 删除 MongoDB 中该会话的所有记录。
- **响应:** { "message": "History cleared", "deleted_count": 10 }

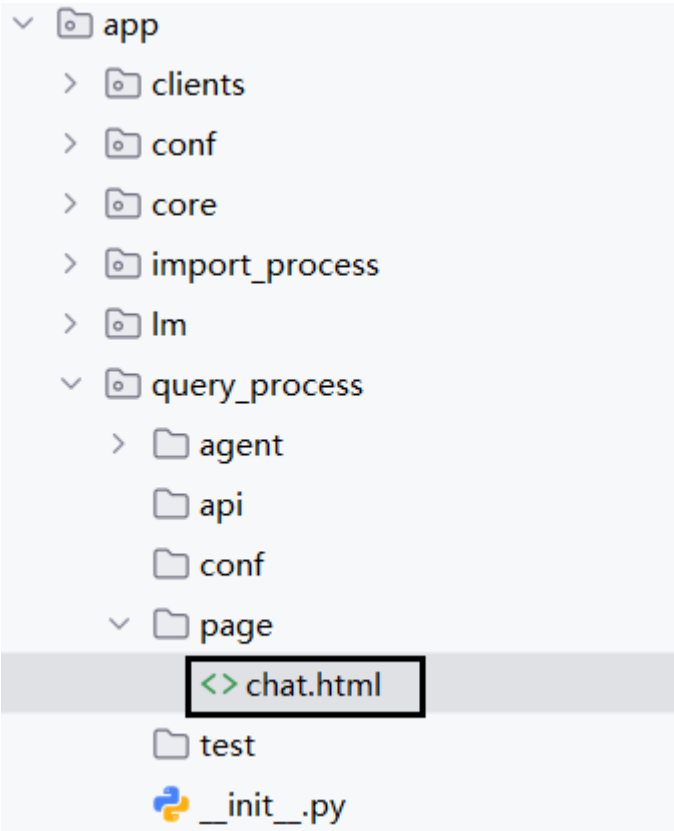
6) 健康检查接口

- **路径:** /health (GET)
- **功能:** 检查服务存活状态。
- **响应:** { "ok": True }

8.2.3 接口代码实现

1) 导入查询页面

将资料chat.html添加到app/query_process/page文件夹中！

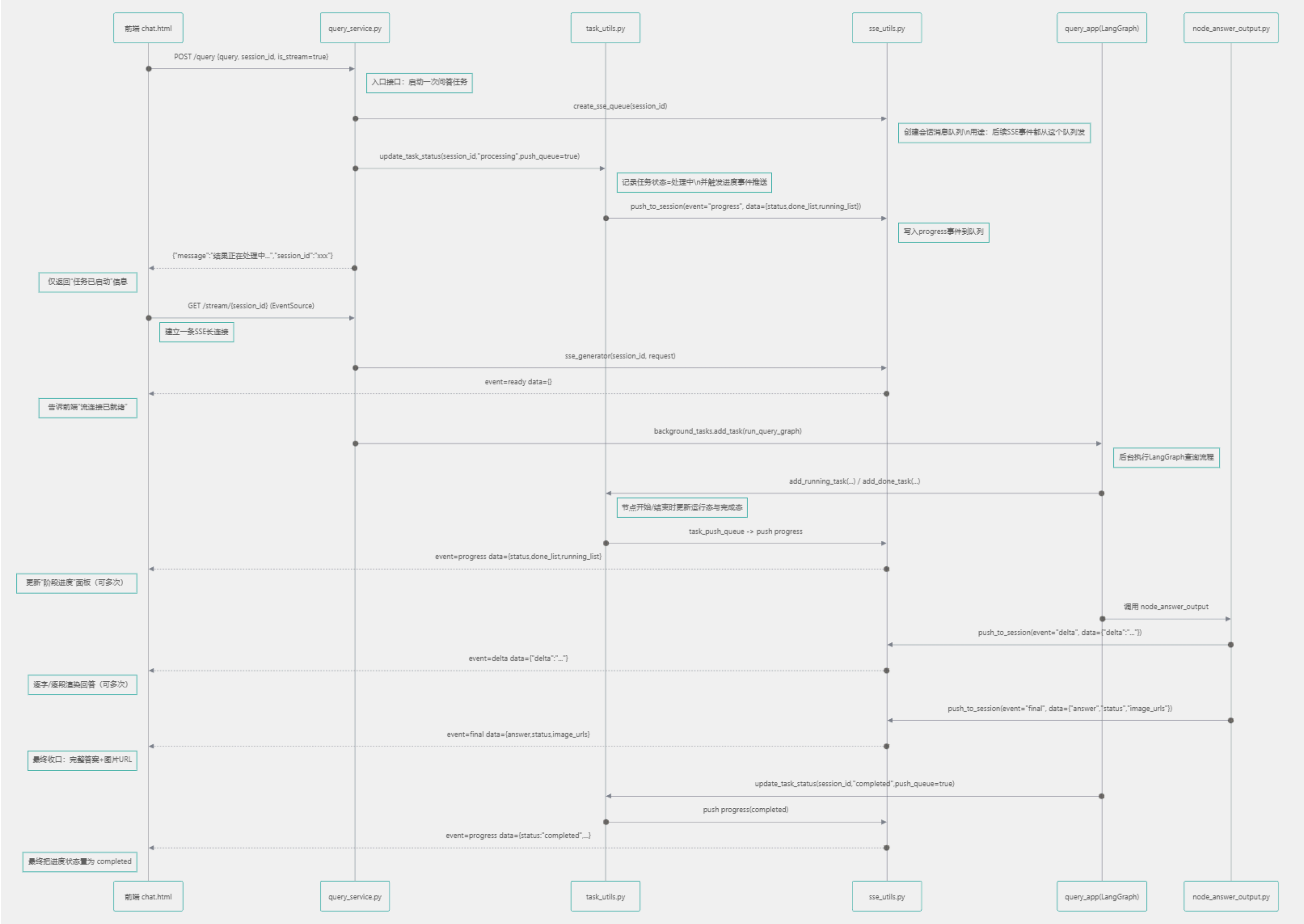


3) 前后端交互说明

知识库查询服务的流式响应流程，涉及到四个核心模块的协同工作：

1. Web 服务层 (query_service.py): 负责接收请求、建立 SSE 连接。
2. SSE 工具层 (sse_utils.py): 负责管理消息队列、打包和推送事件。
3. 任务状态层 (task_utils.py): 负责记录每个节点的执行进度，并自动触发 SSE 推送。
4. 图节点执行层 (query_process/): 实际的业务逻辑节点（如检索、Rerank、生成答案），它们通过更新状态来驱动进度条。

核心流程时序图：



2) 定义和实现api接口服务

在 app/query_process/api 目录下创建 query_service.py，我们将代码拆解为以下几个部分进行实现。

首先引入 FastAPI、Pydantic 以及项目内部的工具类。

```
1 from pathlib import Path
2 import uuid
3 import uvicorn
4 from fastapi import FastAPI, BackgroundTasks, HTTPException, Request
5 from fastapi.responses import FileResponse, StreamingResponse
6 from pydantic import BaseModel, Field
7 from starlette.middleware.cors import CORSMiddleware
8 from app.core.logger import logger
9
10 from app.utils.task_utils import *
11 from app.utils.sse_utils import create_sse_queue, SSEEvent, sse_generator
12 from app.clients.mongo_history_utils import *
13 from app.query_process.agent.main_graph import query_app
14
15
16 # 定义fastapi对象
17 app = FastAPI(title="query service", description="掌柜智库查询服务！")
18
19 # 跨域配置
20 app.add_middleware(
21     CORSMiddleware,
22     allow_origins=["*"],
23     allow_methods=["*"],
24     allow_headers=["*"],
25 )
26
27 # 返回chat.html页面
28 @app.get("/chat.html")
29 async def chat():
30     current_dir_parent_path = Path(__file__).absolute().parent.parent
31     chat_html_path = current_dir_parent_path / "page" / "chat.html"
```

```
32         if not chat_html_path.exists():
33             logger.error(f"页面不存在：{chat_html_path}")
34             raise HTTPException(status_code=404, detail=f"没有查询到页面，地址为：{chat_html_path}！")
35         logger.info("成功加载 chat.html 页面")
36         return FileResponse(chat_html_path)
```

3) 定义数据模型 (Pydantic)

定义前端请求的数据结构，确保参数类型安全，并增加兼容字段。

```
1 # 定义接口接收的数据结构
2 class QueryRequest(BaseModel):
3     """查询请求数据结构"""
4     query: str = Field(..., description="查询内容")
5     session_id: str = Field(None, description="会话ID")
6     is_stream: bool = Field(False, description="是否流式返回")
```

4) 实现核心查询逻辑

这是最关键的逻辑部分，包含后台任务处理函数 run_query_graph 和 API 接口 /query。

```
1 # 核心查询处理函数
2 def run_query_graph(session_id: str, user_query: str, is_stream: bool = True):
3     logger.info(f"[{session_id}] 开始执行查询流程，流式模式：{is_stream}")
4     default_state = {"original_query": user_query, "session_id": session_id, "is_stream": is_stream}
5     try:
6         query_app.invoke(default_state)
7         update_task_status(session_id, TASK_STATUS_COMPLETED, is_stream)
8         logger.info(f"[{session_id}] 查询流程执行完成")
9     except Exception as e:
10        logger.exception(f"[{session_id}] 查询流程异常：{str(e)}")
11        update_task_status(session_id, TASK_STATUS_FAILED, is_stream)
12        if is_stream:
13            push_to_session(session_id, SSEvent.ERROR, {"error": str(e)})
14
15 # 查询接口
16 @app.post("/query")
17 async def query(background_tasks: BackgroundTasks, request: QueryRequest):
18     """
19     1 解析参数
20     2 更新任务状态
21     3 调用处理流程图
22     4 返回结果
23     """
24     user_query = request.query
25     session_id = request.session_id if request.session_id else str(uuid.uuid4())
26     is_stream = request.is_stream
27
28     if is_stream:
29         create_sse_queue(session_id)
30         logger.info(f"[{session_id}] 已创建 SSE 消息队列")
31
32     update_task_status(session_id, TASK_STATUS_PROCESSING, is_stream)
33     logger.info(f"[{session_id}] 任务开始处理，查询内容：{user_query}")
34
35     if is_stream:
36         background_tasks.add_task(run_query_graph, session_id, user_query, is_stream)
37         logger.info(f"[{session_id}] 流式任务已提交至后台执行")
38         return {
39             "message": "结果正在处理中...",
40             "session_id": session_id
41         }
42     else:
43         run_query_graph(session_id, user_query, is_stream)
44         answer = get_task_result(session_id, "answer", "")
45         logger.info(f"[{session_id}] 非流式查询处理完成")
46         return {
47             "message": "处理完成！",
48             "session_id": session_id,
49             "answer": answer,
50             "done_list": []
51         }
```

```
52 |
53 | # SSE 流式推送接口
54 | @app.get("/stream/{session_id}")
55 | async def stream(session_id: str, request: Request):
56 |     logger.info(f"[{session_id}] 客户端已建立 SSE 流式连接")
57 |     return StreamingResponse(
58 |         sse_generator(session_id, request),
59 |         media_type="text/event-stream",
60 |         headers={
61 |             "Cache-Control": "no-cache",
62 |             "Connection": "keep-alive",
63 |             "X-Accel-Buffering": "no"
64 |         }
65 |     )
```

5) 健康检查接口

```
1 | # 健康检查
2 | @app.get("/health")
3 | async def health():
4 |     """服务健康检查"""
5 |     logger.info("健康检查接口调用成功")
6 |     return {"ok": True}
```

6) 启动查询服务

```
1 | if __name__ == "__main__":
2 |     uvicorn.run(app, host="127.0.0.1", port=8001)
```

8.3 历史对话记录管理

- 1. **上下文连续**：保留关键信息，支持追问、补充条件与多轮推理不断档。
- 2. **指代消歧**：如果只考虑本次的问题往往不足以让大模型理解用户的上下文含义，尤其是一些指代比如：“这个设备”，“它”等等。
- 3. **交互确认**：为明确问题的一些信息，Agent 可能会反问用户一些问题，经过几次确认后才能准确解答后续问题，比如商品的信号。
- 4. **识别用户**：长期记录对话可以记住用户偏好，最终形成用户画像。

咱们项目这里没有使用 LangChain 的 Checkpointer 方式管理会话，而是自定义了一套基于 MongoDB 的持久化管理策略。

这样做的有很多好处：更加灵活自主，可以根据条件范围查询对话，可以给对话自定义格式，管理查询写入内容。

但是相对来说需要做的开发工作也比较多。

MongoDB 是一种文档数据库，特别适用于存储海量数据，结构简单，关系简单的数据。

- **性能和单表容量**上都比 MySQL/PostgreSQL 等传统关系型数据库要好。虽比不上 Redis 这种内存数据库，但在持久化能力和检索功能又比内存数据库强上很多。
- **缺点**是不适合保持有复杂关联关系，查询复杂的场景。

所以对于这种**结构简单、查询简单但数据庞大**的对话信息特别合适。尤其是 MongoDB 的存储基本单元就是一份 JSON 文档，与对话也是非常契合。

8.4 MongoDB 快速准备

MongoDB 是一个基于分布式文件存储的数据库，由 C++ 语言编写。旨在为 WEB 应用提供可扩展的高性能数据存储解决方案。

8.4.1 Linux 下使用 Docker 安装 MongoDB

1. 拉取 MongoDB 镜像

```
1 | docker pull mongo:latest
```

2. 运行 MongoDB 容器

```
1 | docker run -itd --name mongo -p 27017:27017 mongo
```

- -p 27017:27017：将容器的 27017 端口映射到主机的 27017 端口。
- --name mongo：容器名称。

3. 常用管理命令

- 查看运行状态：

```
1 docker ps | grep mongo
```

- 停止 MongoDB 容器：

```
1 docker stop mongo
```

- 启动 MongoDB 容器（停止后再次启动）：

```
1 docker start mongo
```

- 重启 MongoDB 容器：

```
1 docker restart mongo
```

4. 进入 MongoDB 容器

```
1 docker exec -it mongo mongosh
```

8.4.2 MongoDB 客户端安装 (Windows)

推荐使用 MongoDB Compass，这是 MongoDB 官方提供的图形化管理工具。

1. **下载**：访问 [MongoDB Download Center](#)，选择 Windows 版本下载安装包。
2. **安装**：运行下载的 .exe 或 .msi 文件，按照提示完成安装。
3. **连接**：
 - 打开 MongoDB Compass。
 - 在连接页面输入连接字符串（URI），例如：mongodb://localhost:27017（如果是远程服务器，替换 localhost 为服务器 IP）。
 - 点击 **Connect** 按钮。

8.4.3 MongoDB 基本使用 (CRUD)

在 MongoDB 中，数据存储在 **数据库 (Database)** -> **集合 (Collection)** -> **文档 (Document)** 中。

以下操作可以在 mongosh 命令行或 MongoDB Compass 的 Mongosh 终端中执行。

1. 创建数据库和集合

MongoDB 不需要显式创建数据库，当往一个不存在的数据库中写入数据时，它会自动创建。

```
1 // 切换/创建数据库 'testdb'
2 use users
3 db.dropDatabase() // 删除当前库
```

2. 插入文档 (Create)

```
1 // 插入一条数据到 'users' 集合
2 // 创建 users 集合 db.createCollection("users")
3 // 语法: db.collection【集合名】.insertOne(document)
4 // _id字段必须是唯一的
5 在MongoDB集合中，每个文档都必须包含具有唯一值的 _id字段。
6 如果为 _id字段指定值，则必须确保该值在集合中是唯一的。如果不指定值，驱动程序会自动为该字段生成唯一的 ObjectId 值。
7 我们建议让驱动程序自动生成 _id 值以确保唯一性。重复的 _id 值违反了唯一索引约束，导致驱动程序返回错误。
8
9 db.users.insertOne({ name: "张三", age: 25, email: "zhangsan@example.com" })
10
11 // 插入多条数据
12 // 语法: db.collection.insertMany([document1, document2, ...])
13 db.users.insertMany([
14   { name: "李四", age: 30, email: "lisi@example.com", city: "Beijing" },
15   { name: "王五", age: 28, email: "wangwu@example.com", city: "Shanghai" },
16   { name: "赵六", age: 35, email: "zhaoliu@example.com", city: "Beijing" }
17 ])
```

```
17 ])
```

3. 查询文档 (Read)

```
1 // 1. 查询所有数据
2 // 语法: db.collection.find(query, projection)
3 db.users.find()
4
5 // 2. 等值查询: 查询 name 为 "张三" 的数据
6 db.users.find({ name: "张三" })
7
8 // 3. 比较操作符查询:
9 // $gt (大于), $lt (小于), $gte (大于等于), $lte (小于等于), $ne (不等于)
10 // 示例: 查询 age 大于 25 的数据
11 db.users.find({ age: { $gt: 25 } })
12
13 // 4. 逻辑操作符查询:
14 // $and (与), $or (或)
15 // 示例: 查询 city 为 "Beijing" 并且 age 小于 30 的数据
16 db.users.find({ city: "Beijing", age: { $lt: 30 } })
17
18 // 示例: 查询 city 为 "Beijing" 或者 "Shanghai" 的数据
19 db.users.find({ $or: [ { city: "Beijing" }, { city: "Shanghai" } ] })
20
21 // 5. 包含查询 ($in)
22 // 示例: 查询 age 在 [25, 30] 中的数据
23 db.users.find({ age: { $in: [25, 30] } })
```

4. 更新文档 (Update)

```
1 // 1. 更新一条数据
2 // 语法: db.collection.updateOne(filter, update, options)
3 // $set 操作符用于修改指定字段的值, 如果字段不存在则创建
4 db.users.updateOne(
5   { name: "张三" },           // 过滤条件: 找到 name 为 "张三" 的第一条文档
6   { $set: { age: 26 } }      // 更新操作: 将 age 设为 26
7 )
8
9 // 2. 更新多条数据
10 // 语法: db.collection.updateMany(filter, update, options)
11 // 示例: 将所有 city 为 "Beijing" 的用户的 status 字段设为 "active"
12 db.users.updateMany(
13   { city: "Beijing" },
14   { $set: { status: "active" } }
15 )
16
17 // 3. 自增/自减 ($inc)
18 // 示例: 将 name 为 "李四" 的 age 增加 1
19 db.users.updateOne(
20   { name: "李四" },
21   { $inc: { age: 1 } }
22 )
```

5. 删除文档 (Delete)

```
1 // 1. 删除一条数据
2 // 语法: db.collection.deleteOne(filter)
3 // 示例: 删除 name 为 "王五" 的第一条记录
4 db.users.deleteOne({ name: "王五" })
5
6 // 2. 删除多条数据
7 // 语法: db.collection.deleteMany(filter)
8 // 示例: 删除 age 大于等于 35 的所有用户
9 db.users.deleteMany({ age: { $gte: 35 } })
10
11 // 删除所有数据 (慎用)
12 // db.users.deleteMany({})
```

6. 了解索引(Index)

<https://www.mongodb.com/zh-cn/docs/manual/core/indexes/index-types/index-single/>

```
1 // 1. 创建【单字段索引】
2 // 单列索引 -》 根据单个字段进行查询
3 // where [ name = x ] and [ age = x ]
4 // 语法：db.集合.createIndex({字段: 1}) 1升序 / -1降序
5 // 索引本身就是排好序的结构（MongoDB 默认是 WiredTiger 索引树(B树)）
6 // 创建时写 1 或 -1，就是告诉 MongoDB：
7 // 这个索引按什么顺序存
8 db.users.createIndex({ name: 1 })           // 给 name 建索引
9 db.users.createIndex({ age: -1 })           // 给 age 倒序索引
10 // 1 索引数据 正序排列
11 // -1 索引数据 倒序排列 不影响主数据的排序！ 主数据也不会排序！
12 // 单列索引 1 和 -1 随便...
13
14
15 // 2. 创建【联合索引】（多字段组合）
16 // 最左匹配原则：顺序非常重要！ 用法： 1个联合索引顶多个单个索引（多）
17 // 先按 city 从小到大排序（1） 用法： 条件（1 -1 无所谓）+ 排序（排序规则 正 1 倒 -1）
18 // city 一样的，再按 age 从大到小排序（-1）
19 db.users.createIndex({ city: 1, age: -1 })
20 // db.users.find({ city: "北京" }).sort({ age: -1 })
21 // 数据已经按 city 分组、age 倒序排好了！直接返回，不用再排序。
22
23 // 3. 查看集合所有索引
24 db.users.getIndexes()
25
26 // 4. 查看查询是否使用索引（执行计划）
27 // 重点：executionStats 看是否走索引
28 db.users.find({ name: "张三" }).explain()
```

8.5 开发基于 MongoDB 的历史对话工具

在 MongoDB 服务中创建数据库和 Collection（kb002）。

8.5.1 对话数据结构

```
1 {
2   "session_id": "",
3   "role": "",
4   "text": "",
5   "rewritten_query": "",
6   "item_names": [],
7   "ts": 123
8 }
```

每条数据代表一条对话。

字段名	说明
session_id	session id
role	角色： user\assistant
text	对话文本【答案】
rewritten_query	改写后的问题
item_names	产品名称，可多个
ts	时间戳 秒

8.5.2 添加配置文件

根目录下建立 .env 文件：

```
1 #Mongodb的配置
2 MONGO_URL=mongodb://47.94.86.115:27017
3 MONGO_DB_NAME=kb002
```

8.5.3 历史对话工具代码实现

```
1  # 导入系统模块：用于读取环境变量
2  import os
3  # 导入日志模块：用于记录程序运行日志（成功/失败/错误信息）
4  import logging
5  # 导入类型注解模块：用于函数参数/返回值的类型提示，提升代码可读性和规范性
6  from typing import List, Dict, Any, Optional
7  # 导入时间模块：用于生成时间戳，记录对话的创建时间
8  from datetime import datetime
9  # 导入pymongo核心模块：MongoDB原生Python驱动，实现数据库连接和操作
10 # ASCENDING：表示升序排序，用于MongoDB索引和查询排序
11 from pymongo import MongoClient, ASCENDING
12 # 导入bson的ObjectId：MongoDB默认的主键类型，用于唯一标识文档
13 from bson import ObjectId
14 # 导入dotenv模块：用于从.env文件加载环境变量，避免硬编码敏感配置（如MongoDB连接地址）
15 from dotenv import load_dotenv
16
17 # 加载.env文件中的环境变量，使os.getenv能读取到配置
18 load_dotenv()
19
20
21 class HistoryMongoTool:
22     """
23     MongoDB 历史对话记录读写工具类（基于原生 PyMongo 实现）
24     核心功能：封装MongoDB的连接、集合初始化、索引创建，为上层提供统一的数据库操作入口
25     扩展功能：支持与LangChain消息对象的格式转换（原代码预留能力）
26     """
27     def __init__(self):
28         """
29         类初始化方法：完成MongoDB的连接、数据库/集合获取、索引创建
30         初始化失败会抛出异常并记录错误日志，确保程序感知连接问题
31         """
32         try:
33             # 从环境变量读取MongoDB连接地址（敏感配置，不硬编码）
34             self.mongo_url = os.getenv("MONGO_URL")
35             # 从环境变量读取要使用的数据库名称
36             self.db_name = os.getenv("MONGO_DB_NAME")
37
38             # 创建MongoDB客户端实例，建立与数据库的连接
39             self.client = MongoClient(self.mongo_url)
40             # 获取指定名称的数据库对象
41             self.db = self.client[self.db_name]
42             # 获取对话记录的集合（相当于关系型数据库的表），集合名：chat_message
43             self.chat_message = self.db["chat_message"]
44
45             # 为chat_message集合创建复合索引，提升查询性能
46             # 索引规则：session_id升序 + ts降序，适配"按会话查最新记录"的核心查询场景
47             # create_index自带幂等性：索引已存在时不会重复创建，无需额外判断
48             self.chat_message.create_index([("session_id", 1), ("ts", -1)])
49
50             # 记录成功日志，确认数据库连接和初始化完成
51             logging.info(f"Successfully connected to MongoDB: {self.db_name}")
52         except Exception as e:
53             # 捕获所有初始化异常，记录详细错误日志
54             logging.error(f"Failed to connect to MongoDB: {e}")
55             # 重新抛出异常，让调用方感知初始化失败，避免使用未初始化的实例
56             raise
57
58
59 # 定义全局变量：存储HistoryMongoTool的单例实例
60 # 作用：避免多次创建HistoryMongoTool实例，从而避免重复建立MongoDB连接
61 _history_mongo_tool = None
62 # 模块加载时尝试初始化单例实例，实现预加载
63 # 目的：将数据库连接的初始化提前到模块加载阶段，避免第一次调用接口时才建立连接（提升首次响应速度）
64 try:
65     _history_mongo_tool = HistoryMongoTool()
66 except Exception as e:
67     # 初始化失败时仅记录警告日志，不抛出异常
68     # 原因：模块加载阶段的异常可能导致整个程序启动失败，此处保留懒加载兜底（get_history_mongo_tool会再次尝试创建）
69     logging.warning(f"Could not initialize HistoryMongoTool on module load: {e}")
70
```

```
71 def get_history_mongo_tool() -> HistoryMongoTool:
72     """
73     获取HistoryMongoTool的单例实例（饿汉模式）
74     核心逻辑：全局实例为空时创建，不为空时直接返回，保证整个程序只有一个数据库连接实例
75     :return: HistoryMongoTool的单例实例
76     """
77     # 声明使用全局变量，避免函数内视为局部变量
78     global _history_mongo_tool
79     # 懒加载：仅当全局实例为空时，才创建新的实例
80     if _history_mongo_tool is None:
81         _history_mongo_tool = HistoryMongoTool()
82     # 返回单例实例
83     return _history_mongo_tool
84
85
86
87 def clear_history(session_id: str) -> int:
88     """
89     清空指定会话的所有历史对话记录
90     :param session_id: 会话唯一标识，用于筛选要删除的记录
91     :return: 实际删除的文档数量，删除失败返回0
92     """
93     # 获取全局的HistoryMongoTool实例，使用单例模式避免重复创建数据库连接
94     mongo_tool = get_history_mongo_tool()
95     try:
96         # 执行批量删除操作：删除所有session_id匹配的文档
97         result = mongo_tool.chat_message.delete_many({"session_id": session_id})
98         # 记录删除成功日志，包含删除数量和会话ID，便于问题排查
99         logging.info(f"Deleted {result.deleted_count} messages for session {session_id}")
100        # 返回实际删除的数量（delete_many的返回对象包含deleted_count属性）
101        return result.deleted_count
102    except Exception as e:
103        # 捕获删除异常，记录错误日志，包含会话ID
104        logging.error(f"Error clearing history for session {session_id}: {e}")
105        # 异常时返回0，标识删除失败
106        return 0
107
108
109 def save_chat_message(
110     session_id: str,
111     role: str,
112     text: str,
113     rewritten_query: str = "",
114     item_names: List[str] = None,
115     image_urls: List[str] = None,
116     message_id: str = None
117 ) -> str:
118     """
119     写入/更新单条会话记录到MongoDB
120     支持两种模式：无message_id时新增记录，有message_id时更新已有记录
121     :param session_id: 会话唯一标识，关联对话所属的会话
122     :param role: 消息角色，固定值：user（用户）/assistant（助手）
123     :param text: 对话核心内容，用户的提问或助手的回答
124     :param rewritten_query: 重写后的查询语句（可选，用于检索增强等场景，默认空字符串）
125     :param item_names: 关联的商品名称列表（可选，支持多商品，默认None）
126     :param image_urls: 关联的图片URL列表（可选，默认None）
127     :param message_id: 记录主键ID（可选，有值则更新，无值则新增）
128     :return: 插入/更新的记录唯一标识（新增返回ObjectId字符串，更新返回传入的message_id）
129     """
130     # 生成当前时间的时间戳（秒级），用于记录消息的创建时间，后续用于排序和查询
131     ts = datetime.now().timestamp()
132
133     # 构造要插入/更新的文档数据（MongoDB的基本数据单元是文档，类似Python字典）
134     document = {
135         "session_id": session_id, # 会话ID，关联维度
136         "role": role, # 消息角色
137         "text": text, # 消息内容
138         "rewritten_query": rewritten_query or "", # 重写查询，空值处理为空字符串
139         "item_names": item_names, # 关联商品名称列表
140         "image_urls": image_urls, # 关联图片URL列表
141         "ts": ts # 时间戳，排序和时间筛选维度
142     }
```

```
143
144 # 获取全局的HistoryMongoTool实例，使用单例模式
145 mongo_tool = get_history_mongo_tool()
146 # 判断是否传入主键ID，区分更新/新增逻辑
147 if message_id:
148     # 有message_id：执行更新操作（根据主键更新）
149     result = mongo_tool.chat_message.update_one(
150         {"_id": ObjectId(message_id)}, # 更新条件：主键匹配（需将字符串转为ObjectId类型）
151         {"$set": document} # 更新操作：$set表示只更新指定字段，保留其他字段
152     )
153     # 更新操作返回传入的message_id作为标识
154     return message_id
155 else:
156     # 无message_id：执行新增操作
157     result = mongo_tool.chat_message.insert_one(document)
158     # 新增操作返回插入的ObjectId并转为字符串，便于上层使用（避免直接返回ObjectId对象）
159     return str(result.inserted_id)
160
161
162 def update_message_item_names(ids: List[str], item_names: List[str]) -> int:
163     """
164     批量更新历史会话记录的关联商品名称
165     :param ids: 要更新的记录主键ID列表（字符串类型）
166     :param item_names: 要设置的新商品名称列表
167     :return: 实际更新的文档数量，更新失败返回0
168     """
169     # 获取全局的HistoryMongoTool实例，使用单例模式
170     mongo_tool = get_history_mongo_tool()
171     try:
172         # 将字符串类型的主键列表转为MongoDB的ObjectId类型（数据库中主键是ObjectId类型）
173         object_ids = [ObjectId(i) for i in ids]
174         # 执行批量更新操作
175         result = mongo_tool.chat_message.update_many(
176             # 更新条件：复合条件，同时满足
177             {
178                 "_id": {"$in": object_ids} # 主键在指定的ID列表中（批量筛选）
179             },
180             {"$set": {"item_names": item_names}} # 更新操作：设置新的商品名称列表
181         )
182         # 记录更新成功日志，包含更新数量和新的商品名称
183         logging.info(f"Updated {result.modified_count} records to item_names: {item_names}")
184         # 返回实际更新的数量（modified_count：真正被修改的文档数，区别于matched_count）
185         return result.modified_count
186     except Exception as e:
187         # 捕获批量更新异常，记录错误日志
188         logging.error(f"Error updating history item_names: {e}")
189         # 异常时返回0，标识更新失败
190         return 0
191
192
193 def get_recent_messages(session_id: str, limit: int = 10) -> List[Dict[str, Any]]:
194     """
195     查询指定会话的最近N条对话记录，返回原始字典格式
196     结果按时间正序排列，可直接喂给LLM作为上下文
197     :param session_id: 会话唯一标识，用于筛选指定会话的记录
198     :param limit: 条数限制，默认返回最近10条
199     :return: 对话记录列表（字典格式），查询失败返回空列表
200     """
201     # 获取全局的HistoryMongoTool实例，使用单例模式
202     mongo_tool = get_history_mongo_tool()
203     try:
204         # 构造查询条件：仅查询指定session_id的记录
205         query = {"session_id": session_id}
206
207         # 执行查询：按时间戳升序排序，限制返回条数
208         # find(query)：获取符合条件的游标（惰性加载，不立即查询）
209         # sort("ts", ASCENDING)：按ts字段升序（从旧到新），适配LLM上下文顺序
210         # limit(limit)：限制返回的最大条数
211         cursor = mongo_tool.chat_message.find(query).sort("ts", ASCENDING).limit(limit)
212         # 将游标转为列表，触发实际数据库查询，获取所有符合条件的文档
213         messages = list(cursor)
214
```

```
215         # 返回查询结果列表
216         return messages
217     except Exception as e:
218         # 捕获查询异常，记录错误日志
219         logging.error(f"Error getting recent messages: {e}")
220         # 异常时返回空列表，避免上层处理None报错
221         return []
222
223
224 # 主程序入口：仅当直接运行该脚本时执行，用于简单的功能测试
225 if __name__ == "__main__":
226     # 简单测试代码：验证数据库的写入和查询功能是否正常
227     # 测试会话ID，用于标识测试的对话记录
228     sid = "000015_hybrid"
229     # 1. 写入用户消息（手动指定ts=1000，便于测试排序）
230     save_chat_message(sid, "user", "你好（Hybrid）")
231     # 2. 写入助手回复（手动指定ts=1001，按时间顺序紧跟用户消息）
232     save_chat_message(sid, "assistant", "你好！我是基于原生 Mongo + LangChain 对象的助手。")
233     # 3. 写入带关联商品的用户消息（手动指定ts=1002，测试item_names字段）
234     save_chat_message(sid, "user", "这个万用表怎么换电池？", item_names=["混合万用表"])
235
236     # 4. 查询指定会话的最近5条记录，验证查询功能
237     print("--- 查询 LangChain 对象记录 ---")
238     messages = get_recent_messages(sid, limit=5)
239     # 打印查询到的记录数量
240     print(f"查询到的记录数: {len(messages)}")
241     # 遍历打印每条记录的详细内容
242     for m in messages:
243         print(f" {m} ")
```

8.5.4 调用历史对话

1. 在 query_service.py 中加入

```
1 @app.get("/history/{session_id}")
2 async def history(session_id: str, limit: int = 50):
3     """
4     查询当前会话历史记录
5     """
6     try:
7         records = get_recent_messages(session_id, limit=limit)
8         items = []
9         for r in records:
10             items.append({
11                 "_id": str(r.get("_id")) if r.get("_id") is not None else "",
12                 "session_id": r.get("session_id", ""),
13                 "role": r.get("role", ""),
14                 "text": r.get("text", ""),
15                 "rewritten_query": r.get("rewritten_query", ""),
16                 "item_names": r.get("item_names", []),
17                 "ts": r.get("ts")
18             })
19         return {"session_id": session_id, "items": items}
20     except Exception as e:
21         raise HTTPException(status_code=500, detail=f"history error: {e}")
22
23 @app.delete("/history/{session_id}")
24 async def clear_chat_history(session_id: str):
25     count = clear_history(session_id)
26     return {"message": "History cleared", "deleted_count": count}
```

2. 在 node_item_name_confirm.py 中增加保存消息的调用

```
1 def node_item_name_confirm(state):
2     print(f"---node_item_name_confirm 处理")
3
4     add_running_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))
5     add_done_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))
6
7     save_chat_message(state['session_id'], "user", state['original_query'], "", state.get("item_names", []))
8
9     print(f"---已保存对话 处理完成")
```

9. 检索数据节点实现与测试

9.1 产品确认节点 (node_item_name_confirm)

文件: app/query_process/agent/nodes/node_item_name_confirm.py

节点作用与实现思路

节点作用: 根据用户提出的问题，初步对齐要问的是**哪个产品**以便定位手册。
方式就是利用大模型从问题中提取产品名，与知识库中已有问题进行向量比对。
如果没法确定则返回用户把候选名单给**用户引导用户确认产品**。

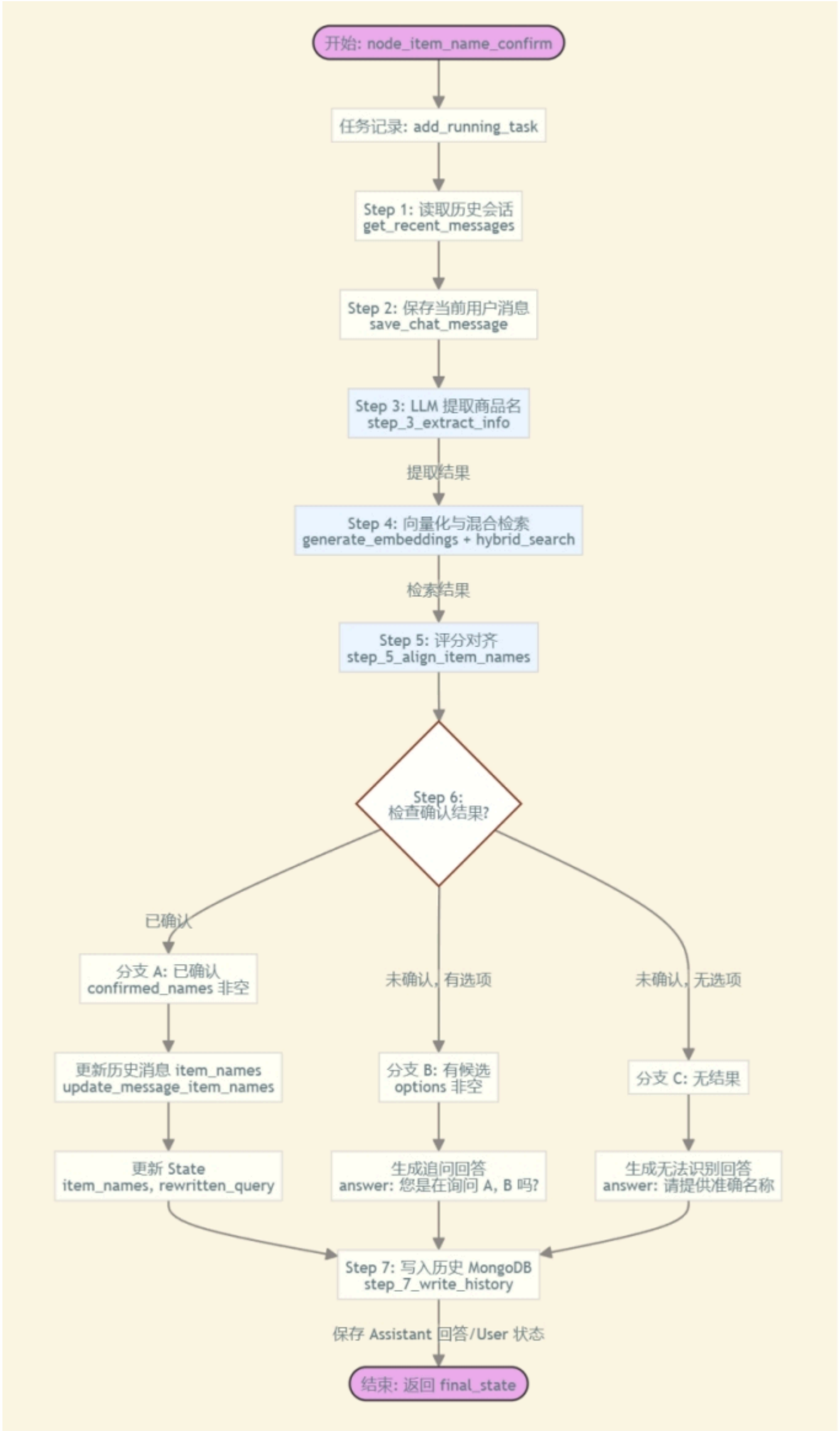
次要任务:

- 取出历史会话
- 保存当前问题至历史对话库
- 利用大模型讲当前问题进行改写，主要目的是处理代词问题。
- 一旦确定设备，则将当前和历史会话标记产品名称

实现思路:

- 路由分发:** 采用轻量级的条件判断逻辑，通过文件后缀 (.pdf / .md) 决定激活 is_pdf_read_enabled 还是 is_md_read_enabled 状态位，实现不同格式文件的差异化处理。
- 元数据提取:** 在入口处统一提取文件名 (file_title)，作为贯穿整个知识库构建流程的唯一标识，避免后续节点重复解析。
- 任务监控初始化:** 集成 task_utils，记录当前任务 ID 和初始状态，为前端提供实时的进度反馈。

步骤分解



流程说明

- 获取历史会话：**根据 session_id 从 MongoDB 中获取最近 10 条历史会话记录，用于构建上下文。
- 保存当前问题：**将用户当前的提问保存到数据库，确保对话历史的完整性。
- 意图理解与改写 (LLM)：**
 - 利用大模型分析当前问题及历史上下文。
 - 提取商品名：**识别用户询问的核心商品名称 (item_names)，支持提取多个，返回 JSON 列表。
 - 指代消歧与改写：**处理“它”、“这个”等代词，生成指代明确的重写问题 (rewritten_query)。
 - 输出结构：** {"item_names": [], "rewritten_query": ""}。
- 向量化评分 (BGE-M3)：**
 - 将提取出的 item_names 逐个进行向量化。
 - 在 Milvus 向量数据库中查询，与标准商品名进行比对，获取相似度评分 (Score) 。
- 商品名对齐策略 (核心逻辑)：**
 - > 0.95 (确信匹配)：**
 - 含义：**极高置信度，系统认为用户输入的名称与库中商品名几乎完全一致。
 - 操作：**直接锁定该商品。若有多个 > 0.95，取最高分的一个。
 - 0.8 ~ 0.95 (高概率候选)：**
 - 含义：**较高置信度，但存在细微差异（如型号别名、模糊拼写），系统不敢100%确定。
 - 操作：**将其作为“候选商品”，后续让用户确认。
 - 0.6 ~ 0.8 (疑似候选)：**
 - 含义：**中等置信度，可能是相关商品，也可能是误匹配。
 - 操作：**取前 5 个作为宽泛的“候选列表”。

- **< 0.6 (无关/噪音):**
 - **含义:** 低置信度，认为与库中商品无关。
 - **操作:** 直接丢弃，不作为有效商品名返回。

6. 确认状态检查与分支处理:

- **分支 A: 已明确确认 (Confirmed)**
 - **条件:** 有一个商品评分 > 0.95，或用户明确选定了一个商品。
 - **动作:**
 1. 更新状态 state，写入确定的 item_names 和 rewritten_query。
 2. **回溯更新:** 将历史记录中未识别出商品名的对话，补充标记为当前确认的商品名（利用上下文一致性）。
 3. 进入下一节点（如检索节点）。
- **分支 B: 未确认但有候选 (Ambiguous)**
 - **条件:** 没有 > 0.95 的商品，但有 > 0.8 的候选列表。
 - **动作:** 生成澄清话术（如“您是想问以下哪个产品：A, B, 还是 C? ”），直接返回给用户，**中断流程**等待用户回复。
- **分支 C: 未确认且无候选 (Unknown)**
 - **条件:** 所有商品评分均 < 0.6。
 - **动作:** 生成通用回复（如“抱歉，未找到相关产品，请提供准确型号”），直接返回给用户，**中断流程**。

7. 持久化状态:

- 将生成的答案（包括澄清追问或最终结果）写入 MongoDB 历史记录。
- 如果有确定的 item_names，同步更新到数据库记录中。

节点代码实现

本节点的核心逻辑是通过 node_item_name_confirm 函数协调多个子步骤函数完成。为了代码清晰和可维护性，我们将逻辑拆解为以下独立的方法：

步骤1：导入依赖

引入必要的系统库、LangChain 消息类型、以及项目中封装好的工具函数（如任务管理、MongoDB 操作、LLM 客户端、Milvus 客户端等）。

```
1 import sys
2 import os
3 import json
4 import logging
5 from typing import List, Dict, Any, Optional
6 from langchain_core.messages import SystemMessage, HumanMessage
7 from mpmath import limit
8
9 from app.core.load_prompt import load_prompt
10 from app.query_process.agent.state import QueryGraphState
11 from app.utils.task_utils import add_running_task, add_done_task
12 from app.clients.mongo_history_utils import get_recent_messages, save_chat_message, update_message_item_names
13 from app.lm.lm_utils import get_llm_client
14 from app.lm.embedding_utils import generate_embeddings
15 from app.clients.milvus_utils import get_milvus_client, create_hybrid_search_requests, hybrid_search
16 from dotenv import load_dotenv, find_dotenv
17 from app.core.logger import logger
18
19 load_dotenv(find_dotenv())
```

步骤2：主流程：节点入口函数 (node_item_name_confirm)

功能: 串联功能所有步骤，作为 LangGraph 的节点入口。

逻辑:

1. 标记任务开始 (add_running_task)。
2. 获取历史消息 (get_recent_messages) 并保存用户当前问题 (save_chat_message)。
3. 执行提取信息。
4. 若提取到商品名，依次执行（搜索）、（对齐）。
5. 执行更新状态和 写入历史。
6. 标记任务完成 (add_done_task) 并返回最终 State。

```
1 def node_item_name_confirm(state: QueryGraphState) -> QueryGraphState:
```

```
2 """
3 主节点函数：商品名称确认流程
4 """
5 logger.info(">>> node_item_name_confirm: 开始处理")
6
7 session_id = state["session_id"]
8 original_query = state.get("original_query", "")
9 is_stream = state.get("is_stream", False)
10
11 # 标记任务开始
12 add_running_task(session_id, "node_item_name_confirm", is_stream)
13
14 # 1. 获取历史记录
15 history = get_recent_messages(session_id, limit=10)
16 logger.info(f"Node: 获取到 {len(history)} 条历史消息")
17
18 # 2. 保存用户当前消息（初始保存，后续 step 7 会更新）
19 message_id = save_chat_message(session_id, "user", original_query, "", state.get("item_names", []))
20 logger.debug(f"Node: 用户消息已初始保存, ID: {message_id}")
21
22 # 3. 提取信息
23 extract_res = step_3_extract_info(original_query, history)
24 item_names = extract_res.get("item_names", [])
25 rewritten_query = extract_res.get("rewritten_query", original_query)
26
27 # 更新 State 中的 rewrite_query
28 state["rewritten_query"] = rewritten_query
29
30 align_result = {}
31
32 # 4. & 5. 如果有提取到商品名，进行搜索和对齐
33 if len(item_names) > 0:
34     query_results = step_4_vectorize_and_query(item_names)
35     align_result = step_5_align_item_names(query_results)
36 else:
37     logger.info("Node: 未提取到商品名，跳过向量检索")
38
39 # 6. 检查确认状态
40 state = step_6_check_confirmation(state, align_result, session_id, history, rewritten_query)
41
42 # 7. 写入最终历史
43 final_state = step_7_write_history(state, session_id, history, rewritten_query, message_id)
44
45 # 将 history 存入 state，供后续节点（如 node_answer_output）使用
46 final_state["history"] = history
47
48 # 标记任务完成
49 add_done_task(session_id, "node_item_name_confirm", is_stream)
50
51 logger.info(f"Node: 处理结束, Final State Item Names: {final_state.get('item_names')}")
52 return final_state
```

步骤3：提取商品名称 (step_3_extract_info)

功能：利用 LLM 从用户当前问题及历史会话中提取核心信息。

输入：

- query: 用户当前的问题字符串。
- history: 最近的对话历史列表。

逻辑：

- 构造包含历史对话和当前问题的 Prompt。
- 调用 LLM（开启 JSON 模式），要求提取 item_names（商品名列表）并生成 rewritten_query（改写后的独立问题）。
- 解析 LLM 返回的 JSON，处理可能的格式异常。

输出：包含 item_names 和 rewritten_query 的字典。

RAG 中重写用户提问（rewritten_query）的核心原因（重点环节）

- 修复用户输入的“不完美”：**用户提问常存在口语化、模糊、错别字、信息缺失（比如“这个怎么用”缺上下文）等问题，重写后可补全信息、修正错误，让查询更精准（例：“张三的产品”→“查询张三相关的 XX 产品使用说明”）。

2. **适配知识库检索规则**：RAG 的核心是“检索知识库找答案”，原始提问可能不符合知识库的索引 / 关键词体系（比如用户说“咋弄”，知识库用“如何操作”），重写后可对齐术语、补充检索关键词，提升召回率。
3. **增强上下文关联性**：多轮对话中，用户可能只说“再说说这个”，重写时可结合历史对话补全上下文（例：“再说说这个”→“再说说 XX 产品的使用步骤”），避免检索跑偏。

重写提问的核心目标是：**把用户“自然、模糊的原始问题”转化为“精准、适配检索规则的查询语句”**，最终提升 RAG 检索的准确性和答案的相关性。

先提取提示词：

位置：prompts/rewritten_query_and_itemnames.prompt

```
1 历史会话：
2 {history_text}
3
4 当前用户问题：{query}
5
6 请根据历史会话和当前问题，提取用户正在询问的商品名称（item_names）。
7 1. 如果用户明确提到了商品名称，请提取出来。可能有一个或多个，但不能重复。
8 2. 如果用户使用了代词（如"这个"、"它"），请结合历史会话指代消解，确定商品名称。
9 3. 如果无法确定商品名称，item_names 返回空列表。
10 4. 请重新改写用户的问题（rewritten_query），使其成为包含商品名称的独立完整问题。
11
12 请直接返回JSON格式结果，格式如下：
13 {{
14     "item_names": ["商品A", "商品B"],
15     "rewritten_query": "关于商品A和商品B，..."
16 }}
```

方法细化实现：

```
1 def step_3_extract_info(query, history) -> Dict:
2     """
3     利用LLM从当前问题以及历史会话中提取出主要询问的商品名称item_names（可多个，JSON列表形式）
4     若商品名不够明确则返回空列表，同时根据上下文重新改写问题，保证问题独立完整
5     :param query: 字符串 - 用户当前原始查询问题（如："这个多少钱？"）
6     :param history: 列表[字典] - 近期会话历史，每条消息含role/text等字段，格式：[{"role": "user/assistant", "text": "消息内容", "_id": "消息ID"}, ...]
7     :return: 字典 - 提取结果，固定包含2个字段，格式：
8         {
9             "item_names": ["商品名1", "商品名2", ...], # 提取的商品名列表，无则空列表
10            "rewritten_query": "改写后的完整问题"      # 包含商品名的独立问题，无则返回原始query
11        }
12    """
13    # 代码核心步骤总结：
14    # 1. 初始化准备：获取LLM客户端，拼接历史会话为文本格式，加载并拼接提示词，构造LLM调用的消息列表
15    # 2. LLM调用与响应处理：调用LLM客户端获取响应，清理响应内容中的JSON代码块格式，解析为JSON字典
16    # 3. 结果校验与异常处理：确保返回字典包含item_names/rewritten_query字段（缺失则补默认值），捕获所有异常并返回兜底结果
17
18    # 1. 先获取llm客户端（用户提起商品名称和重写查询）
19    logger.info("Step 3: 正在初始化 LLM 客户端...")
20    client = get_llm_client(json_mode=True)
21    # 构造历史对话文本，拼接为"角色：内容"的格式，供LLM做上下文理解
22    history_text = ""
23    for msg in history:
24        history_text += f"{msg['role']}: {msg['text']}\n"
25    logger.info(f"Step 3: 历史上下文准备完成（长度：{len(history_text)}）")
26    # print(f"{sys._getframe().f_code.co_name}: 历史消息对话文本：{history_text}")
27
28    # 2. 处理和动态拼接提示词
29    """
30    为了让 Python 把大括号当作“普通字符”保留下来，f-string 规定：用双大括号 {{ 表示普通的左大括号 {，双大括号 }} 表示普通的右大括号 }。
31    """
32    prompt = load_prompt("rewritten_query_and_itemnames", history_text=history_text, query=query)
33    logger.info(f"Step 3: 提示词加载成功")
34
35    # 构造LLM调用的消息列表，包含系统角色（定义助手身份）和用户角色（传入提示词）
36    messages = [
37        SystemMessage(content="你是一个专业的客服助手，擅长理解用户意图和提取关键信息。"),
38        HumanMessage(content=prompt)
39    ]
40
41    """
```

```
42 # 替换后的通用格式（兼容绝大多数LLM接口）
43 messages = [
44     {
45         "role": "system", # SystemMessage 对应 role: "system"
46         "content": "你是一个专业的客服助手，擅长理解用户意图和提取关键信息。"
47     },
48     {
49         "role": "user", # HumanMessage 对应 role: "user"（也可写 "human"，按接口要求调整）
50         "content": prompt # 原 HumanMessage 的 content 直接复用
51     }
52 ]
53
54 # 如果你需要外层包一层 "messages" 键（比如适配OpenAI API格式），则写成：
55 messages_dict = {
56     "messages": [
57         {"role": "system", "content": "你是一个专业的客服助手，擅长理解用户意图和提取关键信息。"},
58         {"role": "user", "content": prompt}
59     ]
60 }
61 """
62 try:
63     # 调用LLM客户端，发起请求获取提取结果
64     logger.info("Step 3: 正在调用 LLM...")
65     response = client.invoke(messages)
66     logger.info("Step 3: 收到 LLM 响应")
67     # 打印LLM原始响应，便于调试
68     # print("node_item_name_confirm response:", response)
69     # 提取响应中的文本内容
70     content = response.content
71     # 处理LLM可能返回的代码块格式（如```json ... ```），去除包裹符
72     if content.startswith("```json"):
73         content = content.replace("```json", "").replace("```", "")
74
75     # 将处理后的文本转为JSON字典，解析LLM返回结果
76     result = json.loads(content)
77     logger.info(f"Step 3: 解析 LLM 结果: {result}")
78     # 健壮性处理：确保返回结果包含item_names字段，无则设为空列表
79     if "item_names" not in result:
80         result["item_names"] = []
81     # 健壮性处理：确保返回结果包含rewritten_query字段，无则复用原始查询
82     if "rewritten_query" not in result:
83         result["rewritten_query"] = query
84     # 返回解析后的提取结果
85     return result
86 except Exception as e:
87     # 捕获所有异常（如LLM调用失败、JSON解析失败等），记录错误日志
88     logger.error(f"Step 3 LLM 提取失败: {e}")
89     # 异常时返回默认结果：空商品名列表+原始查询
90     return {"item_names": [], "rewritten_query": query}
```

步骤4：向量化并查询 (step_4_vectorize_and_query)

功能：将提取出的商品名转化为向量，并在 Milvus 库中搜索最相似的标准商品名。
输入：

- item_names: 提取出的商品名列表。

逻辑：

- 调用 generate_embeddings 批量生成 BGE-M3 稠密向量和稀疏向量。
- 遍历每个商品名，构建混合搜索请求（Dense + Sparse）。
- 调用 hybrid_search 在 ITEM_NAME_COLLECTION 集合中检索 Top 5 相似结果。
输出：每个商品名的搜索结果列表（包含提取名、匹配到的标准名及评分）。

```
1 def step_4_vectorize_and_query(item_names) -> List[Dict]:
2     """
3     把分析出的item_names逐个向量化（BGE-M3模型），并在Milvus向量数据库(kb_item_names)中执行混合搜索，获取匹配评分
4     :param item_names: 列表[字符串] - step3提取的商品名列表（如["苹果15", "华为P60"]）
5     :return: 列表[字典] - 每个商品名的向量化+搜索结果，格式：
6         [
7             {
```

```
8         "extracted_name": "提取的原始商品名", # 如"苹果15"
9         "matches": [ # 该商品名的TopN匹配结果，无则空列表
10             {
11                 "item_name": "数据库中的商品名", # Milvus中存储的标准化商品名
12                 "score": 0.98 # 混合搜索的相似度评分（0-1，越高越相似）
13             },
14             ...
15         ]
16     },
17     ...
18 ]
19 """
20 logger.info(f"Step 4: Starting vectorization and query for items: {item_names}")
21 # 初始化最终返回结果列表，存储每个商品名的向量化查询结果
22 results = []
23 # 获取Milvus向量数据库的客户端连接对象（已完成初始化和连接校验）
24 client = get_milvus_client()
25 # 校验Milvus客户端连接是否成功，失败则记录错误日志并返回空结果
26 if not client:
27     logger.error("Failed to connect to Milvus")
28     return results
29
30 # 从环境变量中获取Milvus中存储商品名称向量的集合名（表名）
31 # kb_item_names
32 collection_name = os.environ.get("ITEM_NAME_COLLECTION")
33 # 校验集合名是否存在，不存在则记录错误日志并返回空结果
34 if not collection_name:
35     logging.error("No collection name found in env")
36     return results
37
38 # 对所有商品名称批量生成BGEM3向量（稠密+稀疏），相比逐个生成提升处理效率
39 # embeddings格式：{"dense": [向量1, 向量2,...], "sparse": [向量1, 向量2,...]}
40 logger.info("Step 4: 正在生成向量...")
41 embeddings = generate_embeddings(item_names)
42 logger.info(f"Step 4: 已生成 {len(item_names)} 个商品名的向量。开始 Milvus 搜索...")
43
44 # 遍历每个商品名称，逐个执行向量搜索（保证结果与原始商品名一一对应）
45 for i in range(len(item_names)):
46     try:
47         logger.info(f"Step 4: 正在处理商品 {i+1}/{len(item_names)}: {item_names[i]}")
48         # 从批量生成的向量结果中，取出当前商品名对应的稠密向量（高维连续值，如[0.12, 0.35,...]）
49         dense_vector = embeddings.get("dense")[i]
50         # 从批量生成的向量结果中，取出当前商品名对应的稀疏向量（键值对，如{100:0.747, 205:0.664}）
51         sparse_vector = embeddings.get("sparse")[i]
52
53         # 构造Milvus混合搜索请求对象，传入稠/稀疏向量，指定返回Top5匹配结果
54         # reqs返回格式：[稠密向量搜索请求，稀疏向量搜索请求]，与混合搜索权重一一对应
55         reqs = create_hybrid_search_requests(
56             dense_vector=dense_vector,
57             sparse_vector=sparse_vector,
58             limit=5
59         )
60
61         logger.info(f"Step 4: 正在 Milvus 集合 '{collection_name}' 中执行混合搜索: '{item_names[i]}'")
62         # 执行BGEM3混合向量搜索，获取数据库中的匹配结果和评分
63         # 默认配置：稠/稀疏向量权重各0.8/0.2，开启评分归一化（将距离值转为0-1相似度评分）
64         search_res = hybrid_search(
65             client=client, # Milvus客户端连接实例
66             collection_name=collection_name, # 目标向量集合名（存储商品向量的表）
67             reqs=reqs, # 混合搜索请求对象列表
68             ranker_weights=(0.8, 0.2), # 稠/稀疏向量评分权重配比（和为1最佳）
69             limit=5, # 最终返回Top5匹配结果
70             norm_score=True, # 开启评分归一化，统一评分量级为0-1
71             output_fields=["item_name"] # 指定返回Milvus中存储的商品名字段（业务字段）
72         )
73         logger.info(f"Step 4: '{item_names[i]}' 搜索完成。找到 {len(search_res[0])} if search_res else 0 个匹配项。")
74
75         # 初始化当前商品名的匹配结果列表，存储匹配到的商品名+对应相似度评分
76         matches = []
77         # # [
78         #     [
79             #         {
```

```
80         #             "id": 551,
81         #             "distance": 0.08821295201778412,
82         #             "entity": {
83         #                 "color": "orange_6781"
84         #             }
85         #         },
86         # 校验搜索结果是否有效（非空且包含数据，适配Milvus批量搜索格式）
87         if search_res and len(search_res) > 0:
88             # 遍历当前商品名的Top5匹配结果（search_res[0]为该商品的独立搜索结果集）
89             for hit in search_res[0]:
90                 # 提取匹配结果中的商品名和评分，做防KeyError处理（设置默认空字典）
91                 # hit格式：{"id": 数据库ID, "distance": 相似度评分, "entity": {"item_name": "标准化商品名"}}
92                 matches.append(
93                     {
94                         "item_name": hit.get("entity", {}).get("item_name"), # 数据库标准化商品名
95                         "score": hit.get("distance"), # 0-1相似度评分
96                     }
97                 )
98
99             # 将当前商品名的原始名称+匹配结果，封装后加入最终结果列表
100         results.append({
101             "extracted_name": item_names[i], # step3提取的原始商品名称
102             "matches": matches # 该商品名的Top5匹配结果（含评分）
103         })
104
105         # 捕获单个商品名处理的异常（不中断其他商品名执行），仅记录错误日志
106         except Exception as e:
107             logger.error(f"Step 4: 查询商品名 '{item_names[i]}' 时出错: {e}")
108
109         # 返回所有商品名的向量化+搜索结果列表
110     return results
```

步骤5：对齐结果 (step_5_align_item_names)

功能：根据搜索评分（Score）判定商品名是“确认”、“候选”还是“无效”。

输入：

query_results: 步骤3输出的搜索结果。

逻辑：

- 规则a: 只有一个高置信度结果（>0.85）→ 直接确认该商品名
- 规则b: 如果多条匹配结果评分超过0.85 → 优先取与原始提取名相同的，无则取分数最高的
- 规则c: 如果无0.85分以上结果 → 取分数≥0.6的最高前5个作为候选
- 规则d: 如果无0.6分及以上结果 → 不返回任何商品名（确认+候选均为空）

```
1 def step_5_align_item_names(query_results) -> dict:
2     """
3     5 根据Milvus搜索评分，逐个对齐step3提取的item_names，生成「确认商品名」和「候选商品名」
4     对齐规则（优先级a>b>c>d）：
5         a 如果只有一个匹配结果评分高于0.85 → 直接确认该商品名
6         b 如果多条匹配结果评分超过0.85 → 优先取与原始提取名相同的，无则取分数最高的
7         c 如果无0.85分以上结果 → 取分数≥0.6的最高前5个作为候选
8         d 如果无0.6分及以上结果 → 不返回任何商品名（确认+候选均为空）
9     :param query_results: 列表[字典] - step4的返回结果，每个商品名的搜索匹配数据（格式同step4返回值）
10    :return: 字典 - 商品名对齐结果，包含确认列表和候选列表，格式：
11        {
12            "confirmed_item_names": ["确认商品名1", "确认商品名2"], # 去重后的确认商品名，无则空列表
13            "options": ["候选商品名1", "候选商品名2", ...] # 去重后的候选商品名，无则空列表
14        }
15    """
16    # 初始化确认商品名列表（符合高置信度规则的商品名）
17    confirmed_item_names: List[str] = []
18    # 初始化候选商品名列表（低置信度，需用户确认的商品名）
19    options: List[str] = []
20
21    logger.info(f"获得待处理的数据源：{query_results}")
22
23    for res in query_results:
24        # 提取原始的数据，商品名和匹配结果
25        extracted_name = (res.get("extracted_name", "")).strip()
26        # 获取匹配的商品名，无就获取空列表
```

```
27 matches = res.get("matches", []) or []
28 # 若无匹配结果，直接跳过当前商品名的对齐
29 if not matches:
30     continue
31 # {
32 #     "item_name": hit.get("entity", {}).get("item_name"), # 数据库标准化商品名
33 #     "score": hit.get("distance"), # 0-1相似度评分
34 # }
35 # 对匹配结果按评分**降序**排序（高分在前，优先取相似度高的）
36 matches.sort(key=lambda x: x.get("score", 0), reverse=True)
37
38 # 筛选高置信度匹配结果：评分>0.85
39 high = [m for m in matches if m.get("score", 0) > 0.85]
40 # 筛选中置信度匹配结果：评分≥0.6（仅高置信度为空时生效）
41 mid = [m for m in matches if m.get("score", 0) >= 0.6]
42
43 # 规则a：只有一个高置信度结果（>0.85）→ 直接确认该商品名
44 if len(high) == 1:
45     confirmed_item_names.append(high[0].get("item_name"))
46     continue # 匹配到规则a，跳过后续规则判断
47
48 # 规则b：多条高置信度结果（>0.85）
49 if len(high) > 1:
50     # 初始化选中结果为None，优先匹配原始提取名
51     picked = None
52     # 若原始提取名非空，优先取与原始名相同的匹配结果
53     if extracted_name:
54         for m in high:
55             if m.get("item_name") == extracted_name:
56                 picked = m
57                 break
58     # 如果没有与原始名相同的结果，则取分数最高的第一个结果
59     if not picked:
60         picked = high[0]
61
62     # 将选中的结果加入确认商品名列表
63     confirmed_item_names.append(picked.get("item_name"))
64     continue # 匹配到规则b，跳过后续规则判断
65
66 # 规则c：无0.85分以上结果，取≥0.6分的最高前5个作为候选
67 # 注：高置信度列表high为空时才会走到此处（规则a/b均不满足）
68 if len(mid) > 0:
69     # 取中置信度结果的前5个，加入候选列表
70     for m in mid[:5]:
71         options.append(m.get("item_name"))
72
73 # 规则d：无0.6分及以上结果 → 不做任何操作，确认+候选列表均为空
74 # 返回最终对齐结果：确认列表和候选列表均做去重处理（list(set())）
75 return {
76     "confirmed_item_names": list(set(confirmed_item_names)), # 去重，避免重复确认
77     "options": list(set(options)) # 去重，避免重复候选
78 }
```

步骤6：检查确认状态 (step_6_check_confirmation)

功能：根据对齐结果更新会话状态（State），决定后续流程分支。
输入：

- state: 当前会话状态。
- align_result: 步骤4的对齐结果。

逻辑：

1. **分支A（有确认商品）**：更新 State 中的 item_names，并批量回填历史消息中缺失的商品名关联。
2. **分支B（有候选选项）**：生成澄清反问句（如“您是指...吗？”），写入 State 的 answer 字段，清空 item_names。
3. **分支C（无结果）**：生成拒识回复（如“未找到相关产品...”），写入 State 的 answer 字段。

输出：更新后的 State。

```
1 def step_6_check_confirmation(state, align_result, session_id, history, rewritten_query):
2     """
3     6 检查step5对齐后的商品名状态，分3种分支更新会话状态（state），并同步更新历史消息的商品名关联
```

```
4      :param state: 字典 - 原始会话状态，包含session_id/original_query等核心字段
5      :param align_result: 字典 - step5的对齐结果（格式同step5返回值）
6      :param session_id: 字符串 - 会话唯一标识
7      :param history: 列表[字典] - 近期会话历史（格式同step3的history入参）
8      :param rewritten_query: 字符串 - step3改写后的完整问题
9      :return: 字典 - 更新后的会话状态，包含item_names/answer/rewritten_query等字段
10     """
11     # 从对齐结果中提取确认商品名列表，无则空列表
12     confirmed = align_result.get("confirmed_item_names", [])
13     # 从对齐结果中提取候选商品名列表，无则空列表
14     options = align_result.get("options", [])
15
16     # 分支A：有确认的商品名（高置信度，无需用户确认）
17     if confirmed:
18         # 收集历史消息中未关联商品名的消息ID（需批量更新关联）
19         ids_to_update = []
20         for msg in history:
21             if not msg.get("item_names"): # 仅更新item_names为空的历史消息
22                 mid = msg.get("_id") # 提取消息唯一ID
23                 if mid:
24                     ids_to_update.append(str(mid)) # 转为字符串，避免ID格式问题
25
26         # 若存在需更新的消息ID，批量更新历史消息的商品名关联
27         if ids_to_update:
28             update_message_item_names(ids_to_update, confirmed)
29
30         # 更新会话状态：设置确认商品名、改写后的查询
31         state["item_names"] = confirmed
32         state["rewritten_query"] = rewritten_query
33         # 若状态中存在旧答案，删除（避免干扰后续流程）
34         if "answer" in state:
35             del state["answer"]
36         # 返回更新后的状态
37         return state
38
39     # 分支B：无确认商品名，但有候选商品名（中置信度，需用户明确）
40     if options:
41         # 候选商品名拼接为字符串（取前3个，避免过长），格式："商品1、商品2、商品3"
42         options_str = "、".join(options[:3])
43         # 构造向用户确认的提示语
44         answer = f"您是想问以下哪个产品：{options_str}？请明确一下型号。"
45         # 更新会话状态：设置确认提示语、清空商品名列表
46         state["answer"] = answer
47         state["item_names"] = []
48         return state
49
50     # 分支C：无确认商品名，且无候选商品名（无匹配结果，需用户重新提供）
51     state["answer"] = "抱歉，未找到相关产品，请提供准确型号以便我为您查询。"
52     state["item_names"] = []
53     return state
```

步骤7：写入历史记录 (step_7_write_history)

功能： 将本次交互的核心数据持久化到 MongoDB。

输入：

- state: 更新后的状态。
- rewritten_query: 改写后的问题。

逻辑：

1. 如果生成了助手回答（分支 B/C），调用 save_chat_message 保存助手消息。
2. 更新用户当前消息记录，补充 rewritten_query 和关联的 item_names。

输出： State（无变化）。

```
1 def step_7_write_history(state, session_id, history, rewritten_query, message_id):
2     """
3     7 把本次处理的核心信息（用户问题、助手答案、商品名、改写查询）写入MongoDB的会话历史
4     包含2个核心操作：1. 写入助手答案（若有）；2. 更新用户原始问题的关联信息
5     :param state: 字典 - step6更新后的会话状态，包含answer/item_names等字段
6     :param session_id: 字符串 - 会话唯一标识
7     :param history: 列表[字典] - 近期会话历史（无实际业务逻辑，预留扩展）
```

```
8      :param rewritten_query: 字符串 - step3改写后的完整问题
9      :param message_id: 字符串 - 本次用户问题的消息唯一ID ( step2生成 )
10     :return: 字典 - 最终的会话状态 ( 无额外修改, 直接返回入参state )
11     """
12     # 若会话状态中有助手答案 ( 分支B/C ), 写入助手消息到历史
13     if state.get("answer"):
14         save_chat_message(
15             session_id=session_id, # 会话ID, 关联所属会话
16             role="assistant", # 消息角色: 助手
17             text=state["answer"], # 消息内容: 向用户确认的提示语/无结果提示语
18             rewritten_query="", # 助手消息无需改写查询, 设为空
19             item_names=state.get("item_names", []) # 关联的商品名列表 ( 分支B/C均为空 )
20         )
21
22     # 强制更新本次用户原始问题的关联信息 ( 核心: 补充改写查询、商品名 )
23     save_chat_message(
24         session_id=session_id, # 会话ID, 关联所属会话
25         role="user", # 消息角色: 用户
26         text=state["original_query"], # 消息内容: 用户原始查询
27         rewritten_query=rewritten_query, # 补充step3改写后的完整问题
28         item_names=state.get("item_names", []), # 补充关联的商品名列表
29         message_id=message_id # 消息ID, 指定更新已存在的用户消息 ( 而非新增 )
30     )
31
32     # 返回最终会话状态, 供下游节点使用
33     return state
```

主流程：测试代码 (node_item_name_confirm)

```
1 if __name__ == "__main__":
2     # 模拟输入状态
3     mock_state = {
4         "session_id": "test_session_001",
5         "original_query": "HAK 180 烫金机怎么用?",
6         "is_stream": False
7     }
8
9     print(">>> 开始测试 node_item_name_confirm...")
10    try:
11        # 运行节点
12        result_state = node_item_name_confirm(mock_state)
13
14        print("\n>>> 测试完成! 最终状态:")
15        print(json.dumps(result_state, indent=2, ensure_ascii=False))
16
17        # 简单验证
18        if result_state.get("item_names"):
19            print(f"\n[PASS] 成功提取并确认商品名: {result_state['item_names']}")
20        else:
21            print(f"\n[WARN] 未确认到商品名 (可能是向量库无匹配或LLM未提取)")
22
23    except Exception as e:
24        print(f"\n[FAIL] 测试运行出错: {e}")
```

扩展思考：关于 rewritten_query 和 item_name 的逻辑顺序考量？

"先重写了 rewritten_query 然后在确认的 item_name 如果 item_name不准，先重写问题是不是也不准了？"

目前的逻辑顺序是：

- Step 3 (LLM) : 基于历史对话，提取 item_names (原始提取) 和生成 rewritten_query (基于原始提取)。
 - 此时的 rewritten_query 确实是基于 LLM 当时 认为的商品名生成的。
- Step 4 & 5 (Milvus + 对齐) : 拿着 Step 3 提取的原始商品名去向量库搜索，并进行对齐（标准化/修正）。
 - 这里可能会把 "苹果15" 修正为 "iPhone 15 Pro"。
- Step 6 (Confirmation) : 根据对齐结果更新 state["item_names"]。
 - 关键点：此时 state["rewritten_query"] 依然是 Step 3 生成的那个。

结论：流程确实有调整空间。如果 Step 5 把商品名修正了（例如从“HAK”修正为“HAK 180 烫金机”），但 rewritten_query 仍然保留着旧的、可能不准确的商品名描述，确实会造成不一致。

通常的优化方案：

在 Step 6 或 Step 7 中，如果确认了新的标准商品名，理论上应该 再次 把 rewritten_query 中的旧商品名替换为新确认的标准名。

不过，在目前的实现中，保留 Step 3 的 rewritten_query 也是一种 折中策略：

- 成本考量：再次调用 LLM 去重写会增加延迟和 token 消耗。
- 检索影响：rewritten_query 主要用于后续的混合检索。如果后续检索同时使用了 item_names 字段做过滤（Filter），那么 rewritten_query 里商品名是否绝对精确可能影响没那么大（只要语义还在）。

9.2 查询节点搜索向量库 (node_search_embedding)

文件: app/query_process/agent/nodes/node_search_embedding.py

步骤分解

- 1) 获取查询上下文
- 从状态 (state) 中提取经过改写的问题 (rewritten_query) 以及在上一节点确认的商品名称列表 (item_names) 。
- 2) 文本向量化
- 调用嵌入模型接口，将改写后的问题转换为向量表示。这里同时生成两种向量：
- 稠密向量 (Dense Vector)：用于捕捉语义相似度。
 - 稀疏向量 (Sparse Vector)：用于捕捉关键词匹配。
- 3) 构建混合检索请求
- 准备 Milvus 混合搜索参数：
- 过滤条件：如果有确定的商品名，构造 item_name in [...] 过滤器，限定搜索范围。
 - 检索策略：结合稠密向量（COSINE 距离）和稀疏向量（IP 内积）进行多路召回。
- 4) 执行向量检索
- 连接 Milvus 数据库的 CHUNKS_COLLECTION 集合，执行混合检索。
- 加权融合：默认使用 0.8/0.2 的权重对稠密和稀疏得分进行重排序（Rerank）。
 - 结果截取：返回 Top 5 最相关的文档切片（Chunks）。
- 5) 更新状态
- 将检索到的文档切片列表 (embedding_chunks) 存入状态，供后续节点使用。

节点代码实现

```
1 import sys
2 import os
3 from app.utils.task_utils import add_running_task,add_done_task
4 from app.lm.embedding_utils import generate_embeddings
5 from app.clients.milvus_utils import create_hybrid_search_requests,hybrid_search,get_milvus_client
6 from app.core.logger import logger
7 from dotenv import load_dotenv,find_dotenv
8 load_dotenv(find_dotenv())
9
10
11 def node_search_embedding(state):
12     """
13     核心节点函数：基于已确认商品名+改写后的用户问题，执行Milvus向量数据库混合检索
14     流程：用户问题向量化 → 构造带商品名过滤的混合搜索请求 → 执行稠密+稀疏混合检索 → 返回检索结果
15     :param state: Dict - 会话状态字典，包含上游传递的核心信息，关键字段：
16         {
17             "session_id": str,          # 会话唯一标识
18             "rewritten_query": str,      # step3改写后的完整用户问题（含商品名）
19             "item_names": list[str],    # step6已确认的标准化商品名列表
20             "is_stream": bool/None      # 是否为流式响应，可选
21         }
22     :return: Dict - 检索结果字典，仅包含embedding_chunks字段，供下游节点使用：
23         {
24             "embedding_chunks": List[Dict] # Milvus检索结果列表，无结果则为空列表
25                                           # 每个元素为一条匹配的向量数据，含业务字段
26         }
27     """
```

```
28 logger.info("---search_milvus 开始处理---")
29 add_running_task(state["session_id"],sys._getframe().f_code.co_name,state["is_stream"])
30
31 # 1. 从会话状态中提取核心入参，为后续检索做准备
32 query = state.get("rewritten_query") # 提取改写后的用户问题（含商品名，独立完整）
33 item_names = state.get("item_names") # 提取已确认的标准化商品名列表（精准过滤用）
34
35 logger.info(f"核心入参提取：query='{query}', item_names={item_names}")
36
37 # 2. 对改写后的用户问题执行向量化，生成BGEM3稠密+稀疏向量
38 logger.info(f"开始为文本获取嵌入值：{query[:50]}..." if len(query) > 50 else f"开始为“{query}”文本获取嵌入值...")
39 # 调用向量化函数，入参为列表（支持批量，此处仅单条查询）
40 # 生成与商品名匹配的语义向量，用于后续相似性检索
41 embeddings = generate_embeddings([query])
42
43 dense_vec = embeddings.get("dense")[0]
44 sparse_vec = embeddings.get("sparse")[0]
45 # 打印稠密/稀疏向量日志，便于调试向量生成结果
46 logger.debug(f"向量生成成功：dense_dim={len(dense_vec)}, sparse_len={len(sparse_vec)}")
47
48 # 3. 准备Milvus向量数据库连接相关配置，指定检索的集合
49 # 从环境变量中获取Milvus中存储「文本片段向量」的集合名（表名），避免硬编码
50 collection_name = os.environ.get("CHUNKS_COLLECTION")
51 logger.info(f"正在连接到 Milvus 并准备集合 '{collection_name}'...")
52
53 # 4. 构造Milvus混合搜索请求对象（核心步骤）
54 # 先通过辅助函数生成商品名过滤表达式，精准过滤检索范围
55 # 'item_name in ["苹果15", "华为P60"]'
56
57 # 若无商品名，直接返回None（不做过滤）
58 if not item_names:
59     logger.warning("item_names 为空，跳过检索，返回空结果")
60     return {"embedding_chunks": []}
61
62 # 对每个商品名添加双引号，拼接为Milvus支持的in语法格式
63 quoted = ", ".join(f'"{v}"' for v in item_names)
64 # 构造最终过滤表达式
65 expr = f"item_name in [{quoted}]"
66 logger.info(f"创建搜索请求过滤表达式：{expr}")
67
68 # 构造稠密+稀疏混合搜索请求，整合向量、过滤条件、搜索参数
69 reqs = create_hybrid_search_requests(
70     dense_vector=dense_vec, # 取用户问题的稠密向量（单条，故取索引0）
71     sparse_vector=sparse_vec, # 取用户问题的稀疏向量（单条，故取索引0）
72     expr=expr, # 商品名过滤表达式，缩小检索范围（仅检索指定商品名的向量）
73     limit=10 # 底层检索返回数量（后续会再过滤为5，预留更多结果做重排序）
74 )
75
76 # 5. 执行Milvus稠密+稀疏混合向量检索（核心调用）
77 logger.info("开始执行 Milvus 混合检索...")
78 client = get_milvus_client()
79 res = hybrid_search(
80     client=client,
81     collection_name=collection_name, # 检索的目标集合名（文本片段向量集合）
82     reqs=reqs, # 构造好的混合搜索请求对象（稠密+稀疏）
83     ranker_weights=(0.8, 0.2), # 稠/稀疏向量评分权重配比，各占50%（可按业务调优）
84     norm_score=True, # 开启评分归一化，将距离值转为0-1区间的相似度评分
85     limit=5, # 最终返回的TOP5相似度最高结果
86     output_fields=["chunk_id", "content", "item_name"] # 指定返回的业务字段
87 )
88
89 # 打印节点处理成功日志，输出原始检索结果，便于调试
90 hit_count = len(res[0]) if res and len(res) > 0 else 0
91 logger.info(f"节点 search_embedding 处理成功，检索到 {hit_count} 条相关片段")
92 if hit_count > 0:
93     logger.debug(f"Top1 检索结果示例：{res[0][0]}")
94
95 # 标记当前任务完成，更新任务状态
96 add_done_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
97
98 # 6. 构造并返回结果：若检索结果非空，取res[0]（适配Milvus批量搜索格式），否则返回空列表
99 # res[0]为当前单条查询的检索结果，包含TOP5匹配的向量数据及业务字段
```

```
100 |         return {"embedding_chunks": res[0] if res else []}
```

主流程测试

```
1  if __name__ == "__main__":
2      # 模拟测试数据
3      test_state = {
4          "session_id": "test_search_embedding_001",
5          "rewritten_query": "HAK 180 烫金机使用说明", # 模拟改写后的查询
6          "item_names": ["HAK 180 烫金机"], # 模拟已确认的商品名
7          "is_stream": False
8      }
9
10     print("\n>>> 开始测试 node_search_embedding 节点...")
11     try:
12         # 执行节点函数
13         result = node_search_embedding(test_state)
14         logger.info(f"检索结果汇总：{result}")
15         # 验证结果
16         chunks = result.get("embedding_chunks", [])
17         print(f"\n>>> 测试完成！检索到 {len(chunks)} 条结果")
18
19         if chunks:
20             print("\n>>> Top 1 结果详情:")
21             top1 = chunks[0]
22             # 打印关键字段（注意：entity字段可能包含具体业务数据）
23             print(f"ID: {top1.get('id')}")
24             print(f"Distance: {top1.get('distance')}")
25             entity = top1.get('entity', {})
26             print(f"Item Name: {entity.get('item_name')}")
27             print(f"Content Preview: {entity.get('content', '')[:100]}...")
28         else:
29             print("\n>>> 警告：未检索到任何结果，请检查 Milvus 数据或 item_names 是否匹配")
30
31     except Exception as e:
32         logger.error(f"测试运行失败：{e}", exc_info=True)
```

9.3 假设性文档向量搜索 (node_search_embedding_hyde)

文件: app/query_process/agent/nodes/node_search_embedding_hyde.py

节点作用和实现思路

什么是HyDE: HyDE（Hypothetical Document Embeddings，“假设文档向量化”）是一种让检索更“聪明”的技巧。当用户只给出一句短问题时，系统先不急着想去向量库里硬搜，而是先让大模型“脑补”出一段可能的答案或相关描述——就像先写一段“假想的说明书”。这段文字通常更具体、更像一篇小文档，包含更多关键词、背景和表达方式。接着，把这段“假想文档”做成向量，再拿它去向量库里找相似内容。结果往往比直接拿那一句短问题去做向量检索更准，因为检索时用的是更“饱满”的语义信号。

它的**优点**是简单、通用、对很多场景都有效；**缺点**是也会“脑补过头”，生成的假设文本可能带偏方向，所以常见做法是配合过滤、重排或多路检索一起用。

实现思路:

- 1) 利用大模型根据用户查询生成假设性文档（Hypothetical Document）。
- 2) 利用“重写问题 + 假设性文档”生成 embedding，并到向量库检索切片。

节点代码实现

步骤1：导入依赖

```
1  # HyDE节点
2  import sys
3  from app.utils.task_utils import add_running_task, add_done_task
4  from app.lm.lm_utils import *
5  from app.lm.embedding_utils import *
6  from app.clients.milvus_utils import *
7  from app.core.logger import logger
```

```
8 from app.core.load_prompt import load_prompt
9 from dotenv import load_dotenv, find_dotenv
10 load_dotenv(find_dotenv())
```

步骤2：主流程：节点入口函数 (node_item_name_confirm)

功能：串联功能所有步骤，作为 LangGraph 的节点入口。

```
1 def node_search_embedding_hyde(state):
2     """
3     HyDE (Hypothetical Document Embedding) 检索节点
4     核心思想：通过LLM生成假设性答案（HyDE文档），将其向量化后用于检索，以解决短查询语义稀疏问题。
5
6     执行步骤：
7     1. 参数提取：从会话状态中获取改写后的查询（rewritten_query）和已确认的商品名（item_names）。
8     2. 生成假设文档（Step 1）：调用LLM，基于用户问题生成一段假设性的理想回答（即HyDE文档）。
9     3. 混合检索（Step 2）：
10        - 将“用户问题 + 假设文档”合并，生成BGE-M3稠密+稀疏向量。
11        - 在Milvus中执行混合检索（带商品名过滤），召回最相似的知识切片。
12     4. 结果封装：返回检索到的切片列表和生成的假设文档，更新会话状态。
13
14     :param state: 会话状态字典，包含 session_id, rewritten_query, item_names 等
15     :return: 包含 hyde_embedding_chunks（检索结果）和 hyde_doc（假设文档）的字典
16     """
17     logger.info("---HyDE（假设文档检索）节点开始处理---")
18     # 记录任务开始状态
19     add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
20
21     # 1. 参数提取与校验
22     # 优先使用改写后的查询，若无则降级使用原始查询
23     rewritten_query = state.get("rewritten_query")
24     if not rewritten_query:
25         rewritten_query = state.get("original_query")
26
27     if not rewritten_query:
28         logger.error("HyDE节点错误：未找到有效的用户查询（rewritten_query/original_query 均为空）")
29         return {}
30
31     item_names = state.get("item_names")
32     logger.info(f"HyDE检索入参: query='{rewritten_query}', item_names={item_names}")
33
34     # 阶段1：生成假设性文档
35     hyde_doc = ""
36     try:
37         logger.info("Step 1: 开始生成假设性文档（HyDE Doc）...")
38         hyde_doc = step_1_create_hyde_doc(rewritten_query)
39         logger.info(f"Step 1: 假设文档生成成功（长度: {len(hyde_doc)}）")
40         logger.debug(f"假设文档预览: {hyde_doc[:100]}...")
41     except Exception as e:
42         logger.error(f"Step 1（生成假设文档）发生异常: {e}", exc_info=True)
43         # HyDE生成失败属于非阻断性错误，可选择直接返回空或降级处理，此处直接返回空结果
44         return {}
45
46     # 阶段2：用“重写问题 + 假设文档”检索切片
47     try:
48         logger.info("Step 2: 基于假设文档执行 Milvus 混合检索...")
49         res = step_2_search_embedding_hyde(
50             rewritten_query=rewritten_query,
51             hyde_doc=hyde_doc,
52             item_names=item_names,
53             top_k=5,
54         )
55
56         hit_count = len(res[0]) if res and len(res) > 0 else 0
57         logger.info(f"Step 2: 检索完成，召回 {hit_count} 条相关切片")
58
59         if hit_count > 0:
60             # 打印第一条结果用于调试
61             first_hit = res[0][0]
62             score = first_hit.get("distance")
63             content_preview = first_hit.get("entity", {}).get("content", "")[:30]
64             logger.debug(f"Top1 结果: Score={score}, Content='{content_preview}...')"
```

```
65
66         return {
67             "hyde_embedding_chunks": res[0] if res else [],
68             "hyde_doc": hyde_doc,
69         }
70     except Exception as e:
71         logger.error(f"Step 2 (向量生成与检索) 发生异常: {e}", exc_info=True)
72         return {}
73     finally:
74         # 无论成功失败，均标记任务结束
75         add_done_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
76         logger.info("---HyDE 节点处理结束---")
```

步骤3：获取假设性答案 (step_1_create_hyde_doc)

先提取提示词：

位置：prompts/rewritten_query_and_itemnames.prompt

```
1 请基于以下用户查询生成一个简洁的回答范文。
2 用户查询: {rewritten_query}
3 要求：
4 1. 回答要简洁明了，包含核心信息即可
5 2. 假设你是该领域的专家，提供专业的解释
6 3. 不要使用"假设"、"可能"等不确定的词汇
7 4. 保持回答与查询主题高度相关
8 5. 使用中文回答且不超过300字
```

方法细化实现：

```
1 def step_1_create_hyde_doc(rewritten_query: str) -> str:
2     """
3     阶段1：利用大模型根据用户查询生成假设性文档（Hypothetical Document）。
4     HyDE的核心在于：利用LLM生成一个“虚构但相关”的文档，用该文档的向量去检索真实的文档，
5     从而缓解短查询（Query）与长文档（Document）在语义空间不匹配的问题。
6
7     :param rewritten_query: 用户改写后的查询语句
8     :return: LLM生成的假设性文档内容
9     """
10    if not rewritten_query:
11        logger.error("Step 1 Error: rewritten_query 为空")
12        raise ValueError("rewritten_query 不能为空")
13
14    logger.info(f"Step 1: 开始生成假设性文档（HyDE），Query: {rewritten_query}")
15
16    try:
17        llm = get_llm_client()
18        # 加载提示词模板，生成假设文档
19        # 提示词通常引导LLM："请为这个问题写一段专业的回答..."
20        hyde_prompt = load_prompt("hyde_prompt", rewritten_query=rewritten_query)
21        logger.debug(f"Step 1: Prompt加载成功，长度: {len(hyde_prompt)}")
22
23        # 调用LLM生成
24        response = llm.invoke(hyde_prompt)
25        hyde_doc = response.content
26
27        logger.info(f"Step 1: 假设文档生成完成，长度: {len(hyde_doc)} 字符")
28        logger.debug(f"Step 1: 文档预览: {hyde_doc[:50]}...")
29
30        return hyde_doc
31
32    except Exception as e:
33        logger.error(f"Step 1: 生成假设文档失败: {e}")
34        raise e
```

步骤4：假设性答案进行向量查询(step_2_search_embedding_hyde)

方法细化实现：

```
1 def step_2_search_embedding_hyde(
2     rewritten_query: str,
```

```
3     hyde_doc: str,
4     item_names=None,
5     req_limit: int = 10,
6     top_k: int = 5,
7     ranker_weights=(0.8, 0.2), # 调整默认权重以偏向稠密向量 (0.8, 0.2)
8     norm_score: bool = True, # 默认开启归一化
9     output_fields=["chunk_id", "content", "item_name"],
10 ):
11     """
12     阶段2：利用“重写问题 + 假设性文档”生成 embedding，并到向量库检索切片。
13
14     :param rewritten_query: 改写后的查询
15     :param hyde_doc: Step 1 生成的假设性文档
16     :param item_names: 商品名称列表，用于元数据过滤 (item_name in [...])
17     :param req_limit: Milvus 搜索时的候选召回数量
18     :param top_k: 最终返回的 Top K 结果数量
19     :param ranker_weights: 混合检索权重 (Dense, Sparse)
20     :param norm_score: 是否对分数进行归一化
21     :param output_fields: 返回结果中包含的字段
22     :return: 检索结果列表
23     """
24     if not rewritten_query:
25         raise ValueError("rewritten_query 不能为空")
26     if not hyde_doc:
27         raise ValueError("hypothetical_doc 不能为空")
28
29     # 1. 拼接查询与假设文档，形成更丰富的语义上下文
30     combined_text = rewritten_query + " " + hyde_doc
31     logger.info(f"Step 2: 拼接 Query + HyDE Doc, 总长度: {len(combined_text)}")
32
33     # 2. 生成向量 (Dense + Sparse)
34     logger.info("Step 2: 正在生成混合向量 (Embedding)...")
35     embeddings = generate_embeddings([combined_text])
36
37     # 3. 准备 Milvus 检索
38     collection_name = os.environ.get("CHUNKS_COLLECTION")
39     if not collection_name:
40         logger.error("Step 2 Error: 环境变量 CHUNKS_COLLECTION 未设置")
41         return []
42
43     logger.info(f"Step 2: 准备在集合 '{collection_name}' 中执行混合检索")
44
45     # 构造过滤表达式 (如果有商品名限制)
46     expr = None
47     if item_names:
48         # 处理 item_names 中的引号，防止注入或语法错误
49         quoted = ", ".join(f'"{v}"' for v in item_names)
50         expr = f"item_name in [{quoted}]"
51         logger.info(f"Step 2: 应用过滤条件: {expr}")
52     else:
53         logger.info("Step 2: 未指定商品名过滤，将全库检索")
54
55     try:
56         # 构造搜索请求
57         reqs = create_hybrid_search_requests(
58             dense_vector=embeddings.get("dense")[0],
59             sparse_vector=embeddings.get("sparse")[0],
60             expr=expr,
61             limit=req_limit,
62         )
63
64         client = get_milvus_client()
65         if not client:
66             logger.error("Step 2 Error: 无法连接到 Milvus")
67             return []
68
69         # 执行混合检索
70         logger.info(f"Step 2: 执行 Hybrid Search, Weights={ranker_weights}, TopK={top_k}")
71         res = hybrid_search(
72             client=client,
73             collection_name=collection_name,
74             reqs=reqs,
```

```
75         ranker_weights=ranker_weights,
76         norm_score=norm_score,
77         limit=top_k,
78         output_fields=list(output_fields),
79     )
80
81     hit_count = len(res[0]) if res and len(res) > 0 else 0
82     logger.info(f"Step 2: 检索完成, 找到 {hit_count} 个匹配切片")
83
84     return res
85
86 except Exception as e:
87     logger.error(f"Step 2: 检索过程发生异常: {e}")
88     return []
```

主流程：测试代码

```
1 if __name__ == "__main__":
2     # 本地测试代码
3     print("\n" + "="*50)
4     print(">>> 启动 node_search_embedding_hyde 本地测试")
5     print("="*50)
6
7     # 模拟输入状态
8     mock_state = {
9         "session_id": "test_hyde_session_001",
10        "original_query": "HAK 180 烫金机怎么操作?",
11        "rewritten_query": "HAK 180 烫金机的具体操作步骤是什么?",
12        "item_names": ["HAK 180 烫金机"],
13        "is_stream": False
14    }
15
16    try:
17        # 运行节点
18        result = node_search_embedding_hyde(mock_state)
19
20        print("\n" + "="*50)
21        print(">>> 测试结果摘要:")
22        print(f"HyDE Doc Generated: {bool(result.get('hyde_doc'))}")
23        if result.get("hyde_doc"):
24            print(f"Doc Preview: {result.get('hyde_doc')[:50]}...")
25
26        chunks = result.get("hyde_embedding_chunks", [])
27        print(f"Chunks Found: {len(chunks)} , chunks内容: {chunks}")
28        if chunks:
29            print(f"Top Chunk Score: {chunks[0].get('distance')}")
30        print("="*50)
31
32    except Exception as e:
33        logger.exception(f"测试运行期间发生未捕获异常: {e}")
```

9.4 网络搜索文档 (node_web_search_mcp)

文件: app/query_process/agent/nodes/node_web_search_mcp.py

9.4.1 mcp的调用的准备工作

MCP 服务需先在阿里云百炼平台完成开通与配置，才能通过代码调用，以下是完整的开通流程说明：

步骤1：前提条件

已注册阿里云账号，并完成实名认证（百炼 MCP 服务需实名认证后使用）。

步骤2：开通百炼 MCP 服务的详细步骤

1、进入百炼 MCP 广场

打开浏览器，访问百炼 MCP 服务市场链接：

(若链接失效，可通过阿里云官网→“产品”→“人工智能”→“百炼”→“MCP 广场” 进入)

2、搜索并选择目标 MCP 服务

在 MCP 广场的搜索框中输入关键词（如 “联网搜索”）；

在搜索结果中找到目标服务（如本场景的 “联网搜索”）， 点击服务卡片进入详情页。



步骤 3：开通 MCP 服务

进入服务详情页后，点击 “开通服务” 按钮，会弹出 “开通 MCP 服务” 配置窗口，需完成以下配置：



1. **选择 API Key**：从下拉框中选择已有的DASHSCOPE_API_KEY（若未创建，需先在百炼控制台的 “API 密钥管理” 中生成）；
2. 选择部署模式：
 - 推荐选择「个人 FC 资源部署」（资源独立、安全隔离，适合正式场景）；
 - 测试场景可选择「公共 FC 资源部署」（共享资源，启动更快）；
3. 选择计费模式（个人FC资源部署）：
 - 「基础模式」：按调用时长计费（0.000156 元 / 秒），调用后释放资源，成本低；
 - 「极速模式」：按部署时长计费（0.13 元 / 时），启动速度极快（冷启动≤5 毫秒），适合低延迟场景；
4. **选择部署地域**：建议选择与业务服务器同地域（如 “华东 2（上海）”），降低网络延迟；
5. 确认配置后，点击 “确认开通” 按钮，等待百炼平台完成服务部署（通常 1-2 分钟）。

步骤 4：获取 MCP 服务地址

开通完成后，在服务详情页的 “调用信息” 区域，复制对应的 **MCP 服务 SSE 地址**（如本场景的 https://dashscope.aliyuncs.com/api/v1/mcps/WebSearch/sse），后续代码中需将该地址配置到.env文件的MCP_DASHSCOPE_BASE_URL中。

注意事项

1. 现在开通阿里百炼的bailian_web_search，如果是新申请的则默认是streamablehttp，不再支持sse。
2. 如果是之前开通的sse模式需要重新开通一下，会自动变为streamablehttp。
3. 百炼官方mcp 链接：<https://bailian.console.aliyun.com/cn-beijing?spm=a2c4g.11186623.0.0.406e358bUXaXLY&tab=app#/mcp-market/detail/WebSearch>

9.4.2 处理策略

1) 获取查询词

从 LangGraph 全局状态对象state中提取**重写后的精准查询语句**（rewritten_query），为标准化搜索关键词；若该字段为空 / 未提取到，直接终止后续流程，避免无效 MCP 调用。

2) 初始化 MCP 连接

基于MCPServerSse创建 MCP 客户端实例，配置百炼 MCP 核心连接参数后，通过await search_mcp.connect()建立 SSE 流式连接（连接成功返回{"type": "connect", "success": true}）。

核心配置参数（JSON 格式）：

```
1 {
2   "url": "百炼MCP SSE接口地址 ( .env中MCP_DASHSCOPE_BASE_URL )",
3   "headers": {
4     "Authorization": "百炼/阿里云API密钥 ( .env中OPENAI_API_KEY )"
5   },
6   "timeout": 300,    // 客户端整体超时时间 ( 秒 )
7   "sse_read_timeout": 300  // SSE流式读取超时时间 ( 秒 )
8 }
```

3) 调用搜索工具

基于已建立的 MCP 连接，通过call_tool()调用百炼专属搜索工具bailian_web_search，**工具调用固定传参格式（JSON）**，参数不可随意修改：

```
1 {
2   "tool_name": "bailian_web_search",    // 固定值，百炼搜索工具唯一标识
3   "arguments": {
4     "query": "步骤1提取的rewritten_query",    // 必选，搜索查询词
5     "count": 5    // 可选，返回结果数量，默认5条（建议1-10）
6   }
7 }
```

4) 解析与格式化

接收 MCP 流式响应，提取有效数据并清洗，最终封装为统一格式文档列表，为后续节点提供标准化数据。

① MCP 原始返回值（核心有效片段，SSE 流式 JSON）

```
1 {
2   "type": "tool_call",
3   "content": [
4     {
5       "text": "{\"pages\": [{\"title\": \"结果标题\", \"url\": \"结果链接\", \"snippet\": \"核心摘要\", \"source\": \"数据源\"}]}"
6     }
7   ]
8 }
```

② 解析规则

- 1. 过滤出type: "tool_call"的响应，提取content[0].text并转为 JSON 对象；
- 2. 提取对象中pages数组，遍历后仅保留title/url/snippet三个核心字段；
- 3. 对所有字段做清洗（去首尾空格、过滤空值），剔除snippet为空的无效结果。

③ 最终格式化结果（列表嵌套字典，统一格式）

```
1 [
2   {
3     "title": "清洗后的结果标题",
4     "url": "清洗后的结果链接",
5     "snippet": "清洗后的核心摘要（非空）"
6   }
7 ]
```

5) 更新状态与资源清理

① 资源清理

无论调用成功 / 失败 / 中断，均通过await search_mcp.cleanup()关闭 MCP 连接，释放客户端资源，避免资源泄漏。

② 状态更新返回

将步骤 4 格式化后的文档列表，以web_search_docs为字段名更新到 LangGraph 全局状态并返回，供后续节点（重排序、大模型生成）使用；无有效结果则返回空字典。

最终返回状态（JSON）

```
1 {
2   "web_search_docs": [
3     {
4       "title": "HAK 180 烫金机官方操作手册",
5       "url": "https://xxx.com/hak180/manual",
6       "snippet": "HAK 180 顶部50-170mm局部烫金设置：操作面板【转印参数】 - 【区域设置】，选择顶部局部，输入起始50mm、结束170mm，保存生效"
7     }
8   ]
9 }
```

9.4.3 处理关键点

OpenAI MCP 官方 SDK 的核心方法（connect()、call_tool()、cleanup()等）均为**异步函数（async def）**，而本项目中使用的 LangGraph 框架，其节点函数默认采用**同步调用方式（invoke）**。

由于 Python 语法限制，**同步函数中无法直接调用异步方法**（会抛出SyntaxError异常），因此需要让 LangGraph 的搜索节点（node_web_search_mcp）以同步方式运行 MCP 的异步 API，核心解决方案是使用asyncio.run()做**同步 - 异步桥接**：通过该方法临时启动一个异步事件循环，执行 MCP 的所有异步代码，执行完成后自动关闭循环，回到同步逻辑，这是 Python 中同步代码调用异步代码的标准方案。

9.4.4 代码实现

步骤1： 准备和环境

需安装百炼 MCP 官方 SDK 和 Python 异步相关依赖，执行以下命令：

```
1 # 核心依赖：MCPSDK ( openai-agents )
2 uv add openai-agents
3 # 其他基础依赖（若未安装）：langgraph、requests、python-dotenv
4 uv add langgraph requests python-dotenv
```

配置文件和加载

环境变量配置（.env 文件）

在项目根目录创建.env文件，添加以下配置（替换为自身的百炼 API 密钥）：

```
1 # 百炼MCP WebSearch的SSE接口地址（固定值，无需修改）
2 MCP_DASHSCOPE_BASE_URL=https://dashscope.aliyuncs.com/api/v1/mcps/WebSearch/sse
3 MCP_DASHSCOPE_BASE_URL_STREAMABLE=https://dashscope.aliyuncs.com/api/v1/mcps/WebSearch/mcp
4 # 你的阿里云/百炼平台API密钥（从百炼控制台获取，必填）
5 OPENAI_API_KEY=sk-48849600c0fa4aa7bfd709e1f47415ff
```

定义读取配置文件

位置： app/config/bailian_mcp_config.py

```
1 # 导入核心依赖：数据类、环境变量读取、路径处理
2 from dataclasses import dataclass
3 import os
4 from dotenv import load_dotenv
5
6 load_dotenv()
7
8
9 # 定义mcp的服务配置
10 @dataclass
11 class McpConfig:
12     mcp_base_url: str
13     api_key : str
14
15 mcp_config = McpConfig(
16     mcp_base_url=os.getenv("MCP_DASHSCOPE_BASE_URL_STREAMABLE"),
17     api_key=os.getenv("OPENAI_API_KEY")
18 )
```

步骤2：导入基础依赖

```
1 import asyncio
2 import os
3 import json
4 import sys
5 from agents.mcp import MCPServerSse # pip install openai-agents
6 from agents.mcp import MCPServerStreamableHttp # pip install openai-agents
7
8 from app.conf.bailian_mcp_config import mcp_config
9 from app.utils.task_utils import add_running_task,add_done_task
10
11 DASHSCOPE_BASE_URL_SSE = mcp_config.mcp_base_url
12 DASHSCOPE_API_KEY = mcp_config.api_key
```

MCP三种方式对比:

- stdio
 - 连接形态：本地进程间通信（你的客户端拉起 MCP Server 子进程，用 stdin/stdout 传 JSON-RPC）
 - 适用场景：本机工具、桌面插件、开发调试
 - 优点：实现简单、延迟低、不走网络、部署门槛低
 - 缺点：不适合跨机器/云原生；进程生命周期管理复杂；扩展性一般
 - SSE
 - 连接形态：HTTP 下行流（Server → Client 推送事件）+ 通常再配一个上行请求通道
 - 适用场景：早期 Web 场景、需要流式返回但交互不复杂
 - 优点：浏览器友好、流式输出直观
 - 缺点：天然“偏单向”；上行/下行分离，协议拼装复杂；代理/网关兼容性、重连语义、会话一致性处理麻烦
 - Streamable HTTP
 - 连接形态：基于 HTTP 的统一可流式传输模式（请求/响应语义更完整，可流式返回）
 - 适用场景：云上 MCP、跨网络调用、生产系统
 - 优点：更标准化、易被网关/CDN/鉴权体系接纳；与现代 API 基础设施更兼容；实现与运维一致性更好
 - 缺点：比 stdio 多网络链路与时延治理成本
- 为什么很多场景从 SSE 转到 Streamable
- 本地工具/离线插件：选 stdio
 - 浏览器演示/轻量流式：可用 SSE
 - 云上生产、跨网络、要稳定：优先 Streamable HTTP

步骤3：定义mcp网络访问工具

<https://openai.github.io/openai-agents-python/mcp/>

```
1 async def mcp_call(query):
2     # 初始化 MCP
3     search_mcp = MCPServerSse(
4         name="search_mcp",
5         params={
6             "url": DASHSCOPE_BASE_URL_SSE,
7             "headers": {"Authorization": DASHSCOPE_API_KEY},
8             "timeout": 300,
9             "sse_read_timeout": 300
10        }
11    )
12
13    try:
14        await search_mcp.connect()
15        # 直接调用工具
16        result = await search_mcp.call_tool(
17            tool_name="bailian_web_search",
18            arguments={"query": query, "count": 5}
19            # arguments={"query": "今天北京的天气情况", "count": 5}
20        )
21        return result
22    finally:
23        await search_mcp.cleanup()
```

```
24
25 async def mcp_call_streamable(query):
26     search_mcp = MCPServerStreamableHttp(
27         name="search_mcp",
28         params={
29             "url": DASHSCOPE_BASE_URL_STREAMABLE,
30             "headers": {"Authorization": DASHSCOPE_API_KEY},
31             "timeout": 300,
32             "sse_read_timeout": 300,
33             "terminate_on_close": True,
34         },
35         max_retry_attempts=2,
36     )
37     try:
38         await search_mcp.connect()
39         result = await search_mcp.call_tool(
40             tool_name="bailian_web_search",
41             arguments={"query": query, "count": 5},
42         )
43         return result
44     finally:
45         await search_mcp.cleanup()
```

另外了解：

- MCPServerStreamableHttp 是 Agents SDK 里的 MCP 连接器，负责连 streamable-http 的 MCP Server。
- MCP Adapter 是“翻译层”，把 MCP 暴露的 tools 转成 LangChain 可调用对象。

mcpadapter使用伪代码：

```
1 import asyncio
2 from langchain.agents import create_agent
3 from langchain.chat_models import init_chat_model
4 from langchain_mcp_adapters.client import MultiServerMCPCClient
5
6 DASHSCOPE_BASE_URL_STREAMABLE = "https://dashscope.aliyuncs.com/api/v1/mcps/WebSearch/sse"
7 DASHSCOPE_API_KEY = "Bearer <YOUR_API_KEY>"
8
9 async def run_with_mcp_agent(query: str):
10     async with MultiServerMCPCClient(
11         {
12             "search_mcp": {
13                 "transport": "streamable_http",
14                 "url": DASHSCOPE_BASE_URL_STREAMABLE,
15                 "headers": {"Authorization": DASHSCOPE_API_KEY},
16                 "timeout": 300,
17                 "sse_read_timeout": 300,
18             }
19         }
20     ) as mcp_client:
21         tools = await mcp_client.get_tools()
22
23         model = init_chat_model(
24             "qwen-plus",
25             model_provider="openai",
26             base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
27             api_key="<YOUR_API_KEY>",
28         )
29
30         agent = create_agent(model=model, tools=tools)
31
32         result = await agent.ainvoke(
33             {"messages": [{"role": "user", "content": query}]}
34         )
35         return result
36
37 if __name__ == "__main__":
38     out = asyncio.run(run_with_mcp_agent("帮我搜索今天AI行业三条新闻并总结"))
39     print(out)
```

步骤4：主流程编写

```
1 def node_web_search_mcp(state):
2     print("---node_web_search_mcp处理---")
3     add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
4
5     query = state.get("rewritten_query", "")
6     docs = []
7     # 如果没有查询内容，直接返回
8     if query:
9         result = asyncio.run(mcp_call_streamable(query))
10        if result:
11            pages = json.loads(result.content[0].text).get("pages") or []
12            # 统一输出结构化结果，供后续 rerank/引用使用
13            # 每条:{title, url, snippet}
14
15            for item in pages:
16                snippet = (item.get("snippet") or "").strip()
17                url = (item.get("url") or "").strip()
18                title = (item.get("title") or "").strip()
19                if not snippet:
20                    continue
21                docs.append({"title": title, "url": url, "snippet": snippet})
22
23            print("MCP 搜索结果:", docs)
24        add_done_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
25        if docs:
26            return {"web_search_docs": docs}
27    return {}
```

9.4.5 主流程测试

```
1 from dotenv import load_dotenv
2
3 if __name__ == '__main__':
4     load_dotenv()
5     test_state = {
6         "rewritten_query": "HAK 180 在出厂默认状态下，若想在纸张上只把烫金膜转印到顶部 50 mm-170 mm 的局部区域，应在操作面板上如何设置"
7     }
8
9     # 调用 websearch_node 函数
10    result_state = node_web_search_mcp(test_state)
11
12    # 验证结果
13    print("测试结果:")
14    print(f"查询内容: {test_state.get('rewritten_query')}")
15
16    # 输出搜索结果
17    search_results = result_state.get('web_search_docs', [])
18    print(f"搜索结果数量: {len(search_results)}")
19    print("search_results", search_results)
```

9.5 结果融合重排 (node_rrf)

RRF介绍

融合重排 (Reciprocal Rank Fusion) 做**去重+融合打分**，输出一个统一的 TopN 列表 rrf_chunks 供后续回答/重排使用。

目标：为多路召回的同一数据源（向量数据库两路）的切片进行排序筛选。

核心策略：就是把统一切片每一路排名的倒数和该路权重相乘作为累加起来，然后把多路的切片汇总排序截取前 n 名。

也就是说出现次数越多分越高，排名越高分越多，该路权重越大分越高。

核心代码：

```
score_map.get(chunk_id, 0.0) + 1.0 / (k + pos) * weight
```

其中：

- score_map 保存每个切片的累计分数。
- pos 就是该路的当前排名。
- weight 是该路的权重。
- k 是一个衰减系数，越小则排名越重要，越大则出现次数越重要，这里默认值为 60，则偏大认为多次出现更加重要。

处理流程

- 1) 获取上游检索节点返回的文档
- 2) 为不同来源设置权重

节点代码实现

步骤1：导入基础依赖

```
1 import sys
2 from typing import List, Dict, Any
3 from app.utils.task_utils import add_running_task, add_done_task
4 from app.core.logger import logger
```

步骤2：主流程编写

```
1 # =====
2 # LangGraph RRF 融合节点
3 # 功能：接收多路向量检索结果 → 统一格式 → 加权融合 → 输出最终排序列表
4 # =====
5 def node_rrf(state):
6     """
7     RRF (Reciprocal Rank Fusion) 倒数排名融合节点
8
9     功能：
10    将来自不同检索源（如 Embedding 检索、HyDE 检索、知识图谱检索等）的结果进行融合排序。
11    RRF 是一种无需训练的算法，仅根据文档在不同列表中的排名来计算最终得分。
12
13    步骤：
14    1. 提取各路检索结果：从 state 中获取 embedding_chunks 和 hyde_embedding_chunks。
15    2. 结果标准化：将不同格式的检索结果统一转换为包含 chunk_id 的实体列表。
16    3. 设置权重：为不同来源分配权重（当前配置：Embedding=1.0，HyDE=1.0）。
17    4. 执行 RRF：计算融合分数并重新排序。
18    5. 结果截断：保留 Top K 个结果。
19    6. 更新状态：将融合后的结果存入 state["rrf_chunks"]。
20    """
21    logger.info("---RRF（倒数排名融合）开始处理---")
22    add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
23
24    # =====
25    # 步骤1：从 state 取出两路召回结果
26    # =====
27    embedding_chunks = _as_entity_list(state.get("embedding_chunks"))
28    hyde_embedding_chunks = _as_entity_list(state.get("hyde_embedding_chunks"))
29
30    logger.info(f"RRF 输入统计: Embedding源={len(embedding_chunks)}条, HyDE源={len(hyde_embedding_chunks)}条")
31
32    # Debug：打印前5条ID便于核对
33    if embedding_chunks:
34        logger.debug(f"Embedding源 chunk_ids (前5个): {[c.get('chunk_id') for c in embedding_chunks[:5]]}")
35    if hyde_embedding_chunks:
36        logger.debug(f"HyDE源 chunk_ids (前5个): {[c.get('chunk_id') for c in hyde_embedding_chunks[:5]]}")
37
38    # =====
39    # 步骤2：配置多路权重（可根据业务调整）
40    # =====
41    source_weights = [
42        (embedding_chunks, 1.0),
43        (hyde_embedding_chunks, 1.0)
44    ]
45
46    # =====
47    # 步骤3：执行 RRF 融合排序
48    # =====
49    rrf_res = reciprocal_rank_fusion(source_weights, k=60, max_results=10)
```

```
50
51 # =====
52 # 步骤4：提取最终文档列表
53 # =====
54 rrf_chunks = [doc for doc, score in rrf_res]
55
56 # 任务完成标记
57 add_done_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))
58
59 # 把融合结果存入 state
60 return {"rrf_chunks": rrf_chunks}
```

步骤3：编写解析向量数据库结果函数

```
1 # =====
2 # 工具函数：统一格式化检索结果
3 # 功能：将不同来源（Milvus Hit/字典/自定义对象）统一转为标准实体列表
4 # =====
5 def _as_entity_list(state_list) -> List[Dict[str, Any]]:
6     """
7     将上游节点输出统一规整为 entity dict 列表。
8     兼容：
9     - dict: {"entity": {...属性名和对应的字.}, "distance": ...} 或直接就是 {...}
10    - pymilvus Hit: 不是 dict，但通常支持 hit.get("entity") 或 hit.entity
11    - 其他：当作 chunk_id
12    """
13    out: List[Dict[str, Any]] = []
14    for doc in (state_list or []):
15        if not doc:
16            continue
17
18        final_ent = {}
19
20        # =====
21        # 情况A：处理 Milvus 返回的 Hit 对象（含 entity、id、distance）
22        # =====
23        if hasattr(doc, "entity") and hasattr(doc, "id"):
24            # 提取 entity 内容（支持对象转字典 / 直接是字典）
25            entity_content = doc.entity
26            if hasattr(entity_content, "to_dict"):
27                final_ent = entity_content.to_dict()
28            elif isinstance(entity_content, dict):
29                final_ent = entity_content.copy()
30            else:
31                # 尝试强转字典，兼容不同 SDK 版本
32                try:
33                    final_ent = dict(entity_content)
34                except:
35                    pass
36
37            # 补充唯一 ID（优先用内部 chunk_id，没有则补外层 id）
38            if "id" not in final_ent and "chunk_id" not in final_ent:
39                final_ent["id"] = doc.id
40
41            # 补充相似度分数
42            if hasattr(doc, "distance"):
43                final_ent["score"] = doc.distance
44
45        # =====
46        # 情况B：doc 已经是字典（模拟数据 / 已格式化数据）
47        # =====
48        elif isinstance(doc, dict):
49            # 子情况：字典嵌套 entity 结构 {entity:{...}, id:...}
50            if "entity" in doc:
51                ent = doc["entity"]
52                if isinstance(ent, dict):
53                    final_ent = ent.copy()
54            # 补充 ID 和分数
55            if "id" in doc and "id" not in final_ent:
56                final_ent["id"] = doc["id"]
57            if "distance" in doc:
58                final_ent["score"] = doc["distance"]
```

```
59         else:
60             # 扁平字典，直接使用
61             final_ent = doc
62
63         # =====
64         # 情况C：支持 .get() 方法的其他对象
65         # =====
66         elif hasattr(doc, "get"):
67             ent = doc.get("entity") or doc
68             if isinstance(ent, dict):
69                 final_ent = ent
70
71         # 只保留合法非空字典
72         if final_ent and isinstance(final_ent, dict):
73             out.append(final_ent)
74
75     return out
```

步骤4：通用带权重的RRF算法实现

该代码实现的是**带权重的倒数排名融合（Reciprocal Rank Fusion, RRF）** 算法，核心作用是将**多个不同来源的文档排序结果**，结合各来源的权重进行融合，最终输出一个综合所有来源排序信息、按融合得分降序排列的统一文档列表，解决多来源排序结果的融合与重排序问题。

输入参数

1. source_weights：核心输入，列表类型，每个元素是(来源文档列表, 权重)的元组。其中：
- 来源文档列表：单个来源对文档的排序结果（有序，靠前的文档在该来源中相关性更高）；

▪ 权重：该来源的重要性系数（权重越高，该来源的排序结果对最终融合的影响越大）。
2. k：RRF 算法专属常数（默认 60），用于平衡文档排名对得分的影响，避免排名过前的文档得分无限制偏高，是 RRF 的标准超参数。
3. max_results：可选参数，限制最终返回的文档数量，None表示返回全部融合结果。

输出结果

列表类型，每个元素是(文档对象，RRF融合得分)的元组，**按融合得分降序排列**，得分越高表示文档在综合所有来源后的相关性越强。

```
1  # =====
2  # RRF 核心算法：倒数排序融合
3  # 作用：把多路召回结果按排名加权融合，自动去重、重新排序
4  # =====
5  def reciprocal_rank_fusion(
6      source_weights: list,
7      k: int = 60,
8      max_results: int = None,
9  ) -> List[tuple]:
10     """
11     通用带权重的RRF算法实现
12     :param source_weights: 列表，每个元素是(来源文档列表，权重)的元组
13                           例如: [[doc1, doc2], 1.0), ([doc2, doc3], 0.8)]
14     :param k:      RRF 常数，默认 60。用于平滑排名影响，避免高排名文档占据过大优势。
15     :param max_results: 只返回前 N 个，None 表示全部
16     :return:      [(元素, RRF 得分), ...] 按得分降序排列
17     """
18     # 存储每个文档的总得分
19     score_map = {}
20     # 存储每个文档完整内容
21     chunk_map = {}
22
23     # =====
24     # 遍历每一路召回结果，计算 RRF 分数
25     # =====
26     for docs, weight in source_weights:
27         # rank 从 1 开始（第一名=1，第二名=2...）
28         for rank, item in enumerate(docs, start=1):
29             # 获取文档唯一标识（chunk_id 优先，否则用 id）
30             chunk_id = item.get("chunk_id") or item.get("id")
31
32             if not chunk_id:
33                 logger.warning(
34                     f"RRF Warning: item missing chunk_id/id: {list(item.keys()) if isinstance(item, dict) else item}")
35                 continue
36
```

```
37         # =====
38         # RRF 公式核心
39         # score += 权重 * (1 / (k + rank))
40         # =====
41         score_map[chunk_id] = score_map.get(chunk_id, 0.0) + weight * (1.0 / (k + rank))
42
43         # 只保存第一次出现的文档（去重）
44         chunk_map.setdefault(chunk_id, item)
45     # =====
46     # 按 RRF 总分排序
47     # =====
48     merged = []
49     for chunk_id, score in score_map.items():
50         doc_item = chunk_map[chunk_id]
51         merged.append((doc_item, score))
52
53     # 得分从高到低排序
54     merged.sort(key=lambda x: x[1], reverse=True)
55
56     # 截断最多返回 N 条
57     if max_results is not None:
58         merged = merged[:max_results]
59
60     return merged
```

关键细节：

- 1. 排名pos从 1 开始：符合实际排序逻辑（第 1 名、第 2 名...），而非程序默认的 0 索引；
- 2. 唯一标识chunk_id：作为文档的“唯一键”，实现**跨来源的文档匹配**（不同来源的同一文档，通过chunk_id累计得分）；
- 3. 核心得分公式： $1.0 / (k + pos) * weight$
 - 基础项 $1/(k+pos)$ ：RRF 算法的核心，排名越靠前（pos 越小），该项值越大，且通过k平滑排名的影响（避免 pos=1 时得分过大）；
 - 权重项* weight：实现“带权重融合”，重要来源的排序结果，对文档最终得分的贡献成比例放大。
- 4. 得分累加逻辑：同一文档出现在多个来源中，或在单个来源中出现多次（极少情况），其得分会通过score_map.get(chunk_id, 0.0)持续累加。

k 的核心作用

k 是 RRF 算法的**核心超参数**（行业通用默认值 60，也可根据业务调整），**核心作用是对排名的影响做「平滑 / 缓冲」，避免排名的微小差异导致得分剧烈波动，同时防止极端值（如 pos=1）主导总得分**，具体解决 2 个关键问题：

问题 1：避免pos=1时得分无限制偏高，压制其他来源的贡献

- 如果没有k，公式会变成 $1/pos$ ，此时：pos=1：得分 = 1.0；pos=2：得分 = 0.5；pos=3：得分≈0.33；
第 1 名的得分是第 2 名的 2 倍、第 3 名的 3 倍，单个来源的第 1 名会直接主导总得分，其他来源的排名信息几乎被忽略，失去“多来源融合”的意义。
- 加入k=60后，公式为 $1/(60+pos)$ ：pos=1：≈0.0164；pos=2：≈0.0162；pos=3：≈0.0160；

前几名的得分差异被大幅缩小，单个来源的排名无法独断总得分，必须结合多个来源的排名才能获得高总得分，符合多来源融合的核心目标。

问题 2：平衡排名的“边际效应”，让排名靠后的文档也有合理贡献

- 没有k时，排名靠后的文档得分会快速趋近于 0（如pos=100， $1/100=0.01$ ），几乎没有贡献；
- 加入k=60后，pos=100的得分 = $1/(60+100)=0.00625$ ，与pos=50的得分（ $1/110≈0.0091$ ）差异更小，**排名靠后的文档仍能为总得分提供合理贡献**，避免直接被“抛弃”。

简单总结k的作用：**让排名对得分的影响更“温和”，保证多来源排名信息都能有效参与融合，而非少数高排名文档垄断得分。**

主流程测试

```
1  # =====
2  # 本地测试入口
3  # =====
4  if __name__ == "__main__":
5      print("\n" + "=" * 50)
6      print(">>> 启动 node_rrf 本地测试")
7      print("=" * 50)
8
9      mock_state = {
10         "session_id": "test_rrf_session",
```

```
11         "is_stream": False,
12         "original_query": "HAK 180 烫金机怎么操作？",
13         "rewritten_query": "HAK 180 烫金机的具体操作步骤是什么？",
14         "item_names": ["HAK 180 烫金机"]
15     }
16
17     try:
18         from app.query_process.agent.nodes.node_search_embedding import node_search_embedding
19         from app.query_process.agent.nodes.node_search_embedding_hyde import node_search_embedding_hyde
20
21         emb_res = node_search_embedding(mock_state)
22         hyde_res = node_search_embedding_hyde(mock_state)
23         mock_state['embedding_chunks'] = emb_res.get("embedding_chunks") or []
24         mock_state['hyde_embedding_chunks'] = hyde_res.get("hyde_embedding_chunks") or []
25
26         result = node_rrf(mock_state)
27         rrf_chunks = result.get("rrf_chunks", [])
28
29         emb_cnt = len(mock_state.get("embedding_chunks") or [])
30         hyde_cnt = len(mock_state.get("hyde_embedding_chunks") or [])
31
32         print("\n" + "=" * 50)
33         print(">>> 测试结果摘要:")
34         print(f"输入数量: Embedding={emb_cnt}, HyDE={hyde_cnt}")
35         print(f"输出数量: {len(rrf_chunks)}")
36         print("-" * 30)
37
38         print("最终排名:")
39         for i, doc in enumerate(rrf_chunks, 1):
40             doc_id = doc.get("chunk_id") or doc.get("id")
41             content = (doc.get("content") or "")[:20]
42             print(f"Rank {i}: ID={doc_id}, Content={content}...")
43
44         print("=" * 50)
45
46     except Exception as e:
47         logger.exception(f"测试运行期间发生未捕获异常: {e}")
```

9.6 Rerank重排序(node_rerank)

9.6.1 重排序介绍

- 核心目标：**将来自不同来源的检索结果合并，根据语义利用专业的打分模型进行重新评分，并根据评分再次进行筛选。
- 场景：**由于 RRF 只能对于同源的检索结果进行打分筛选，但应对**不同源**的（比如：切片与网页搜索结果）就力不从心了。这时就需要 Rerank 出场了。
- 机制：**相对 RRF 这种只依靠一个公式就能打分，Rerank 就比较复杂耗时了，Rerank 的是一种依靠 AI 模型精读打分，它会仔细阅读文章与问题得出相关性评分，这与大模型的机制非常接近。

9.6.2 Rerank模型的下载

<https://modelscope.cn/collections/BGE-955c53c53ef34c>

在 tools 中加入 app/tool/download_reranker.py 并执行，将模型提取下载到指定位置。

```
1 #modelscope：字节跳动旗下的魔搭 AI 社区（国内主流的开源模型仓库，类似 Hugging Face，适配国内网络环境）
2 from modelscope.hub.snapshot_download import snapshot_download
3
4 local_dir = r"D:\ai_models\modelscope_cache\models\rerank"
5
6 snapshot_download(
7     #北京人工智能研究院（BAAI）发布的大尺寸 BGE 重排序模型，专为文本相关性排序设计，精度较高
8     model_id="BAAI/bge-reranker-large",
9     cache_dir=local_dir,
10 )
11
12 print("下载完成，模型目录：", local_dir)
```

9.6.3 处理步骤

- 1) 合并文档：把 RRF 得到的切片集合和搜索结果进行合并。
- 2) 利用 Reranker 模型对文档进行打分、重排。
- 3) 根据相关性评分，用动态 TopK 算法进行截取。

动态TopK：与固定的 TopN 排名最大的区别就是不会定死录取名次。而是通过一个分数断崖落差把好坏区分，选择优势明显的前 N 名。比如前 5 名分别为 95、93、89、77、71。明显前三名断崖领先后两名所以就只取前三名，避免低分文档混入候选。

9.6.4 代码实现

9.6.4.1 定义reranker_utils.py

BAAI/bge-reranker-large 是北京人工智能研究院（BAAI）发布的**重排序模型本体**（包含模型权重、配置文件等核心文件），而 FlagReranker 是 FlagEmbedding 库中专门为**BGE 系列重排序模型**设计的**封装运行类**，用于简化该模型的加载、初始化和推理调用流程。

更通俗的理解：

- BAAI/bge-reranker-large：相当于“一台高性能的发动机”（模型核心，提供重排序的核心能力）；
- FlagReranker：相当于“发动机的专用启动 / 控制装置”（封装了模型加载、设备适配、精度设置、推理逻辑等，让开发者无需关注模型底层细节，直接调用即可实现文本相关性打分）。

使用 FlagReranker 加载 BAAI/bge-reranker-large，**核心只需安装 FlagEmbedding 包**，该包会自动依赖安装模型运行所需的 PyTorch、Transformers、Sentence-Transformers 等底层依赖，无需手动逐个安装。

```
1 # 基础安装（官方源，国内可替换为清华/阿里镜像）
2 uv add FlagEmbedding
3
4 # 国内镜像提速安装（推荐，解决下载慢问题）
5 uv add FlagEmbedding -i https://pypi.tuna.tsinghua.edu.cn/simple
```

准备配置文件 .env

```
1 # 重排序模型配置
2 BGE_RERANKER_LARGE=D:\ai_models\modelscope_cache\models\rerank\BAAI\bge-reranker-large
3 #BGE_RERANKER_DEVICE=cuda:0
4 BGE_RERANKER_DEVICE=cpu
5 #BGE_RERANKER_FP16=1
6 BGE_RERANKER_FP16=0
```

加载配置参数

位置：app/conf/reranker_config.py

```
1 # 导入核心依赖：数据类、环境变量读取、路径处理
2 from dataclasses import dataclass
3 import os
4 from dotenv import load_dotenv
5
6 # 提前加载.env配置文件（保持和原代码一致，只需执行一次）
7 load_dotenv()
8
9 @dataclass
10 class RerankerConfig:
11     bge_reranker_large: str # 本地模型路径
12     bge_reranker_device: str # 模型仓库标识
13     bge_reranker_fp16: bool # 是否开启半精度（1=True/0=False）
14
15 # 实例化配置对象，和原代码lm_config风格保持一致
16 reranker_config = RerankerConfig(
17     bge_reranker_large=os.getenv("BGE_RERANKER_LARGE"),
18     bge_reranker_device=os.getenv("BGE_RERANKER_DEVICE"),
19     # 特殊处理：将.env中的1/0转为布尔值，兼容常见的数字/字符串格式
20     bge_reranker_fp16=os.getenv("BGE_RERANKER_FP16") in ("1", "True", "true", 1)
21 )
```

定义工具方法

```
1 from FlagEmbedding import FlagReranker
```

```
2 from app.conf.reranker_config import reranker_config
3
4 _reranker_model = None
5
6 def get_reranker_model():
7     global _reranker_model
8     if _reranker_model is None:
9         _reranker_model= FlagReranker(
10             model_name_or_path=reranker_config.bge_reranker_large,
11             device=reranker_config.bge_reranker_device,
12             use_fp16=reranker_config.bge_reranker_fp16
13         )
14     return _reranker_model
```

了解两种主流远程仓库配置示例

示例 1：从Hugging Face Hub加载（官方源，模型标识：BAAI/bge-reranker-large）

```
1 from FlagEmbedding import FlagReranker
2
3 # 直接指定Hugging Face仓库标识，自动远程下载+缓存
4 model = FlagReranker(
5     model_name_or_path="BAAI/bge-reranker-large", # HF远程模型标识
6     device="cuda",
7     use_fp16=True
8 )
```

示例 2：从魔搭 ModelScope加载（国内源，模型标识：damo/nlp_bge_reranker_large）

魔搭上的 BGE 重排序模型标识为 damo/nlp_bge_reranker_large（与 HF 的 BAAI/bge-reranker-large 是同一模型，权重完全一致），国内网络下载速度更快：

```
1 from FlagEmbedding import FlagReranker
2
3 # 直接指定魔搭远程模型标识，国内自动提速下载
4 model = FlagReranker(
5     model_name_or_path="damo/nlp_bge_reranker_large", # 魔搭远程模型标识
6     device="cuda",
7     use_fp16=True
8 )
```

远程模型的本地缓存路径

首次远程下载的模型会自动保存到以下默认缓存目录，后续调用直接加载：

- Windows：C:\Users\你的用户名\.cache\FlagEmbedding
- Linux/Mac： ~/.cache/FlagEmbedding

9.6.4.2 导入与常量定义

```
1 import sys
2 from app.utils.task_utils import *
3
4 from dotenv import load_dotenv
5 import sys
6 from app.lm.reranker_utils import get_reranker_model
7 from app.utils.task_utils import add_running_task
8
9 load_dotenv()
10
11 # -----
12 # Rerank / TopK 全局常量（不从 state 读取）
13 # -----
14 # 动态 TopK 硬上限：最多取前 N 条（<=10）
15 RERANK_MAX_TOPK: int = 10
16 # 最小 TopK：至少保留前 N 条（>=1，且 <= RERANK_MAX_TOPK）
17 RERANK_MIN_TOPK: int = 1
18 # 断崖阈值（相对）
19 RERANK_GAP_RATIO: float = 0.25
20 # 断崖阈值（绝对）
21 RERANK_GAP_ABS: float = 0.5
```

9.6.4.3 节点主流程 (node_rerank)

```
1 # Rerank节点 ( 工作流入口 )
2 def node_rerank(state):
3     """
4     Rerank节点
5     对检索到的文档进行重新排序，提高相关性
6     """
7     print("---Rerank---")
8     add_running_task(state["session_id"], sys._getframe().f_code.co_name, state.get("is_stream"))
9
10    # 阶段一：合并文档
11    doc_items = step_1_merge_docs(state)
12    # 阶段二：对文档进行重排序
13    scored_docs = step_2_rerank_docs(state, doc_items)
14    # 阶段三：动态 TopK
15    topk_docs = step_3_topk(scored_docs)
16    print("最终文档:", topk_docs )
17
18    add_done_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))
19    return {"reranked_docs": topk_docs}
```

9.6.4.4 合并文档 (step_1_merge_docs)

```
1 def step_1_merge_docs(state):
2     """
3     阶段一：文档合并与标准化
4
5     目标：将多路召回（本地知识库 + 联网搜索）的异构数据，统一合并为 Reranker 模型可处理的标准格式。
6
7     输入来源：
8     1. rrf_chunks (List[Dict]): 本地知识库检索结果（经 RRF 融合排序）。
9         - 结构：包含 Milvus entity 信息的复杂字典或对象。
10        - 关键字段：chunk_id, content, title/item_name。
11    2. web_search_docs (List[Dict]): 联网搜索结果（经 MCP 搜索返回）。
12        - 结构：包含搜索摘要的扁平字典。
13        - 关键字段：snippet, title, url。
14
15    输出结果 (List[Dict]):
16    - 标准化文档列表，每项包含：
17        - text: 用于重排序的核心文本（content 或 snippet）
18        - title: 标题（用于增强语义或展示）
19        - doc_id/chunk_id: 唯一标识（本地文档有，联网文档为 None）
20        - url: 来源链接（本地为空，联网文档有）
21        - source: 来源标记（"local" 或 "web"）
22    """
23
24    # 1. 提取输入源
25    rrf_docs = state.get("rrf_chunks") or []
26    web_docs = state.get("web_search_docs") or []
27
28    logger.info(f"Step 1: 开始合并文档 - 本地RRF源: {len(rrf_docs)}条, 联网Web源: {len(web_docs)}条")
29    doc_items = []
30    # -----
31    # 2. 处理本地知识库文档 (rrf_chunks)
32    # -----
33    for i, doc in enumerate(rrf_docs):
34        # 简化：直接使用 dict(doc) 转换，如果 doc 本身是 dict 则无损，如果是对象则尝试转换
35        # 由于上游 RRF 节点已经做了 _as_entity_list 处理，这里 doc 极大概率已经是纯字典
36        # 因此可以移除繁琐的 try-except 和 entity 嵌套判断，直接取值
37
38        # 兼容性处理：优先取 'entity' 字段（防守式编程），若无则视为 doc 本身即 entity
39        # 注意：这里的 doc 应当已经是字典（由上游 _as_entity_list 保证）
40        entity = doc.get("entity") if isinstance(doc, dict) and "entity" in doc else doc
41
42        # 提取核心文本（content），这是重排序的依据
43        # 如果不是字典或无 content，则跳过
44        if not isinstance(entity, dict):
45            logger.warning(f"本地文档格式异常 (index={i}): {type(entity)}")
46            continue
47
48        content = entity.get("content")
```

```
49         if not content:
50             # 仅在 debug 模式记录，避免生产环境日志刷屏
51             logger.debug(f"跳过无内容文档 (index={i}, keys={list(entity.keys())})")
52             continue
53
54         # 提取元数据（使用 .get 链式回退，简洁明了）
55         doc_id = entity.get("chunk_id") or entity.get("id")
56         title = entity.get("title") or entity.get("item_name") or ""
57
58         # 组装标准化对象
59         doc_items.append({
60             "text": content,
61             "doc_id": doc_id,
62             "chunk_id": doc_id, # 兼容旧逻辑保留字段
63             "title": title,
64             "url": "",
65             "source": "local",
66         })
67
68         # -----
69         # 3. 处理联网搜索文档 (web_search_docs)
70         # -----
71         for i, doc in enumerate(web_docs):
72             # 兼容不同字段名：优先取 snippet（摘要），其次 content
73             text = (doc.get("snippet") or doc.get("content") or "").strip()
74             url = (doc.get("url") or "").strip()
75             title = (doc.get("title") or "").strip()
76
77             if not text:
78                 logger.debug(f"跳过无内容联网结果 (index={i})")
79                 continue
80
81             doc_items.append({
82                 "text": text,
83                 "doc_id": None, # 联网结果无固定 ID
84                 "chunk_id": None,
85                 "title": title,
86                 "url": url,
87                 "source": "web",
88             })
89
90         logger.info(f"Step 1: 文档合并完成，共输出 {len(doc_items)} 条标准化文档")
91         return doc_items
```

9.6.4.5 重排序 (step_2_rerank_docs)

```
1 def step_2_rerank_docs(state, doc_items):
2     """
3     阶段二：对文档进行重排序
4     - 输入 doc_items：[{ text,doc_id}, ...]（由第一阶段产出）
5     - 输出：在 state 中写入 reranked_docs（结构化列表）
6     """
7     question = state.get("rewritten_query") or state.get("original_query") or ""
8
9     # 如果没有文档或问题，直接返回
10    if not doc_items or not question:
11        logger.warning("Step 2: 跳过重排序（无文档或无问题）")
12        return []
13
14    logger.info(f"Step 2: 开始重排序（Rerank），待排序文档数：{len(doc_items)}")
15
16    # 初始化重排序模型（这里以使用 BGE 重排序模型为例）
17    texts = [x["text"] for x in doc_items]
18    try:
19        reranker = get_reranker_model()
20
21        # 构建查询-文档对（必须是 str）
22        """
23        格式：列表，每个元素是二元元组 / 列表，严格遵循（query, passage）顺序（即你的「问题、答案」）：
24        第 1 个元素（query）：用户的问题 / 检索词（如“什么是 RRF 算法？”）；
25        第 2 个元素（passage）：候选答案 / 待匹配文档（如你之前 RRF 融合后的文档内容）；
26        支持单组匹配和批量匹配：
```

```
27         # 单组匹配：1个问题+1个候选答案
28         sentence_pairs = [("什么是RRF算法？", "RRF是倒数排名融合算法，用于多来源排序结果融合")]
29         # 批量匹配：1个问题+多个候选答案（重排序核心场景，推荐）
30         sentence_pairs = [
31             ("什么是RRF算法？", "RRF是倒数排名融合算法，用于多来源排序结果融合"),
32             ("什么是RRF算法？", "FP16是半精度推理，能降低模型显存占用"),
33             ("什么是RRF算法？", "FlagReranker是BGE重排序模型的封装类")
34         ]
35         注意：顺序不可颠倒（必须是「问题在前，答案在后」），模型对输入顺序有严格要求，颠倒会导致打分结果失真。
36     2. 输出结果：scores 分数含义与格式
37     格式：列表，元素为浮点数，列表长度与 sentence_pairs 完全一致，一一对应（第 n 个分数对应第 n 个（问题，答案）元组的相关性）；
38     分数含义：数值越高，代表「问题」与「答案」的语义匹配度 / 相关性越强（BGE 重排序模型的分数无固定取值范围，核心看相对大小，用于排序即可）；
39
40     核心用途：将分数与候选答案绑定，按分数降序排列，即可得到「与问题最相关→最不相关」的答案排序，实现重排序。
41     """
42     # 格式：列表，每个元素是二元元组 / 列表，严格遵循（query，passage）顺序
43     sentence_pairs = [[question, t] for t in texts]
44     # 计算相关性得分
45     logger.info("Step 2: 正在计算相关性得分...")
46     scores = reranker.compute_score(sentence_pairs)
47     # 将得分与文档配对并排序（按 score 降序）
48     scored_docs = []
49     for item, text, score in zip(doc_items, texts, scores):
50         # 保留两位小数便于日志查看
51         score_val = float(score)
52         scored_docs.append(
53             {
54                 "text": text,
55                 "score": score_val,
56                 "source": item.get("source") or "",
57                 "chunk_id": item.get("chunk_id"),
58                 "doc_id": item.get("doc_id"),
59                 "url": item.get("url") or "",
60                 "title": item.get("title") or "",
61             }
62         )
63     # 按分数降序排序
64     scored_docs.sort(key=lambda x: x["score"], reverse=True)
65     return scored_docs
66 except Exception as e:
67     logger.error(f"Step 2: 重排序过程发生异常：{e}", exc_info=True)
68     # 出错时降级：返回原始文档顺序，分数置为 0 或 None
69     # 避免整个流程中断
70     fallback_docs = [
71         {
72             "text": x.get("text"),
73             "score": 0.0, # 降级分数
74             "source": x.get("source") or "",
75             "chunk_id": x.get("chunk_id"),
76             "doc_id": x.get("doc_id"),
77             "url": x.get("url") or "",
78             "title": x.get("title") or "",
79         }
80         for x in doc_items
81     ]
82     # 在这里我们不直接修改 state，而是返回结果让主流程处理
83     # 但为了兼容原有逻辑（虽然函数签名是返回 scored_docs），我们记录异常并抛出或返回降级结果
84     # 这里选择返回降级结果，保证流程继续
85     return fallback_docs
```

9.6.4.6 动态 TopK (step_3_topk)

```
1 def step_3_topk(scored_docs):
2     """
3     阶段三：动态 TopK（最多 10）
4     基于 scored_docs（已按 score 降序排序）进行智能截断，
5     核心逻辑：结合固定上下限+断崖阈值判断，避免机械取前N条，保留语义相关的连续文档集合
6     :param scored_docs: 列表，元素为带score的文档字典，已按score降序排列，格式如[{"doc": 文档对象, "score": 相关性分数}, ...]
7     :return: 列表，动态截断后的TopK文档列表，数量≤10
8     """
9
10    # 硬上限：最多取前10条，取全局常量与实际文档数的较小值（避免索引越界）
11    # 注：max_topk从全局常量读取，不依赖外部状态，保证逻辑一致性
```

```
11 max_topk = min(RERANK_MAX_TOPK, len(scored_docs))
12 min_topk = RERANK_MIN_TOPK # 硬下限：至少保留的文档数量（全局常量配置）
13 gap_ratio = RERANK_GAP_RATIO # 相对断崖阈值：分数下降的相对比例阈值（全局常量配置）
14 gap_abs = RERANK_GAP_ABS # 绝对断崖阈值：分数下降的绝对差值阈值（全局常量配置）
15
16 # 1) 断崖截断核心逻辑：从min_topk之后开始检测分数断崖，出现则提前截断
17 topk = max_topk # 默认值：无断崖时取满硬上限（最多10条）
18 # 仅当实际可取值超过硬下限时，才触发断崖检测（否则直接取满min_topk）
19 if topk > min_topk:
20     # 遍历范围：从min_topk-1到max_topk-2（索引从0开始），检测相邻两个文档的分数差
21     # 例：min_topk=3, max_topk=10 → 遍历i=2,3,4,5,6,7,8（对应第3~9条文档，检测与下一条的差距） # 不是从 0 开始！因为：前 min_topk
是个强制保留的，不参与断崖判断！
22     for i in range(min_topk - 1, max_topk - 1):
23         s1 = scored_docs[i].get("score") # 当前位置文档的分数
24         s2 = scored_docs[i + 1].get("score") # 下一个位置文档的分数
25
26         gap = s1 - s2 # 计算相邻文档的分数绝对差距（因已降序，gap≥0）
27         # 计算相对差距：绝对差距 / 当前文档分数（+1e-6避免除数为0/极小值，防止程序报错）
28         # 1e-6 是 Python 中科学计数法的写法，等价于 0.000001（10 的负 6 次方，也就是百万分之一）。
29         rel = gap / (abs(s1) + 1e-6)
30         # 触发断崖截断条件：绝对差距≥绝对阈值 OR 相对差距≥相对阈值
31         # 满足任一条件，说明下一条文档相关性骤降，截断在当前位置
32         if gap >= gap_abs or rel >= gap_ratio:
33             logger.info(f"Step 3: 触发断崖截断 @ index={i} (Score {s1:.4f} -> {s2:.4f}, Gap={gap:.4f})")
34             topk = i + 1 # 最终取前i+1条（索引转实际数量，如i=2 → 取前3条）
35             break # 触发截断后立即退出循环，不再检测后续位置
36
37 # 按最终计算的topk值，截取前topk条文档
38 topk_docs = scored_docs[:topk]
39
40 logger.info(f"Step 3: 截断完成，保留前 {len(topk_docs)} 条文档 (TopK={topk})")
41
42 if topk_docs:
43     preview = ", ".join([f"{d.get('chunk_id') or 'Web'}({d.get('score'):.3f})" for d in topk_docs[:3]])
44     logger.debug(f"Step 3: Top3 文档预览: {preview}")
45
46 # 返回动态TopK处理后的文档列表
47 return topk_docs
```

9.6.5 节点主流程测试 (node_rerank)

```
1 if __name__ == "__main__":
2     print("\n" + "=" * 50)
3     print(">>> 启动 node_rerank 本地测试")
4     print("=" * 50)
5
6     # 1. 模拟数据
7     # 1.1 RRF 本地文档数据
8     mock_rrf_chunks = [
9         {"entity":{"chunk_id": "local_1", "content": "RRF是一种倒数排名融合算法", "title": "算法介绍", "score": 0.9}},
10        {"entity":{"chunk_id": "local_2", "content": "BGE是一个强大的重排序模型", "title": "模型介绍", "score": 0.8}},
11        {"entity":{"chunk_id": "local_3", "content": "无关的测试文档内容", "title": "测试文档", "score": 0.1}} # 预期低分
12    ]
13
14    # 1.2 MCP 联网搜索数据
15    mock_web_docs = [
16        {"title": "Rerank技术详解", "url": "http://web.com/1", "snippet": "Rerank即重排序，常用于RAG系统的第二阶段"},
17        {"title": "无关网页", "url": "http://web.com/2", "snippet": "今天天气不错，适合出去玩"} # 预期低分
18    ]
19
20    mock_state = {
21        "session_id": "test_rerank_session",
22        "rewritten_query": "什么是RRF和Rerank？", # 查询意图：想了解这两个算法
23        "rrf_chunks": mock_rrf_chunks,
24        "web_search_docs": mock_web_docs,
25        "is_stream": False
26    }
27
28    try:
29        # 运行节点
30        result = node_rerank(mock_state)
31        reranked = result.get("reranked_docs", [])
```

```
32
33     print("\n" + "=" * 50)
34     print(">>> 测试结果摘要:")
35     print(f"输入文档总数: {len(mock_rrf_chunks) + len(mock_web_docs)}")
36     print(f"输出文档总数: {len(reranked)}")
37     print("-" * 30)
38
39     print("最终排名:")
40     for i, doc in enumerate(reranked, 1):
41         print(f"Rank {i}: Source={doc.get('source')}, Score={doc.get('score'):.4f}, Text={doc.get('text')[:20]}...")
42
43     # 验证逻辑：
44     # 预期 "local_1", "local_2", "Rerank技术详解" 分数较高
45     # 预期 "local_3", "无关网页" 分数较低，可能被截断或排在最后
46
47     top1_score = reranked[0].get("score")
48     if top1_score > 0:
49         print("\n[PASS] Rerank 打分正常")
50     else:
51         print("\n[FAIL] Rerank 打分异常（均为0或负数）")
52
53     print("=" * 50)
54
55 except Exception as e:
56     logger.exception(f"测试运行期间发生未捕获异常：{e}")
```

9.7 答案输出节点(answer_output)

9.7.1 输出答案节点介绍

- 1) **检查答案**： 判断state 中的answer是否已经存在，如果存在直接输出answer中的答案，注意判断是否需要流式输出需要则流式输出
- 2) **生成提示词**： 根据state中的问题、重新问题、历史对话、提问商品（item_names）、 重排内容 组织prompt 并调用llm
- 3) **生成答案**： 调用大模型输出答案， 注意判断是否需要流式输出需要则流式输出
- 4) **保存答案**： 把答案写入到mongodb的history中 利用clients/mongo_history_utils.py中的save_chat_message方法
- 5) **流操作的final push：做最后一次push操作**（主要是为了触发前端图片渲染）

9.7.2 代码实现

9.7.2.1 导入依赖和准备

```
1 import sys
2 from app.utils.task_utils import add_running_task, add_done_task, set_task_result
3 from app.utils.sse_utils import push_to_session, SSEEvent
4 from app.query_process.agent.state import QueryGraphState
5 from app.core.logger import logger
6 from app.core.load_prompt import load_prompt
7 from app.lm.lm_utils import get_llm_client
8 from app.clients.mongo_history_utils import save_chat_message
9 import re
10
11 _IMAGE_BLOCK_MARKER = "【图片】"
12 MAX_CONTEXT_CHARS = 12000
```

9.7.2.2 节点主流程 (node_answer_output)

```
1 def node_answer_output(state: QueryGraphState) -> QueryGraphState:
2     """
3     1 判断state 中的answer是否已经存在，如果存在直接输出answer中的答案，注意判断是否需要流式输出需要则流式输出
4     2 根据state中的问题、重新问题、历史对话、提问商品（item_names）、 重排内容 组织prompt 并调用llm 生成答案
5     3 阶段三：调用大模型输出答案 注意判断是否需要流式输出需要则流式输出
6     4 把答案写入到mongodb的history中 利用utils/mongo_history_utils.py中的save_chat_message方法
7     5 做最后一次push操作（主要是为了触发前端图片渲染）
8     {
9         "answer": "HAK 180 烫金机的操作面板位于...（大模型生成的纯文本）...",
10        "status": "completed",
```

```
11         "image_urls": [  
12             "http://local-server/images/panel_view.jpg",  
13             "http://local-server/images/button_detail.jpg"  
14         ]  
15     }  
16     """  
17     logger.info("---node_answer_output（答案生成）节点开始处理---")  
18     add_running_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))  
19  
20     # 阶段一：检查answer是否存在,如果存在直接输出answer中的答案  
21     answer_exists = step_1_check_answer(state)  
22  
23     # 阶段二 如果没有answer则 构建 Prompt  
24     if not answer_exists:  
25         prompt = step_2_construct_prompt(state)  
26         state["prompt"] = prompt  
27  
28     # 阶段三： 如果没有answer则 调用大模型输出答案  
29     step_3_generate_response(state, prompt)  
30  
31     # 提取图片URL（用于历史记录和前端展示）  
32     image_urls = _extract_images_from_docs(state.get("reranked_docs") or [])  
33  
34     # 阶段四：把答案写入到mongodb的history中  
35     if state.get("answer"):  
36         logger.info("---写入MongoDB历史记录---")  
37         step_4_write_history(state, image_urls=image_urls)  
38  
39     add_done_task(state['session_id'], sys._getframe().f_code.co_name, state.get("is_stream"))  
40  
41     # 阶段五：流式输出结束，发送 final 事件 [最后兜底，确保图片都能争取渲染和结束]  
42     logger.info(f"---发送 final 事件---图片为：{image_urls}")  
43     if state.get("is_stream"):  
44         push_to_session(  
45             state['session_id'],  
46             SSEEvent.FINAL,  
47             {  
48                 "answer": state["answer"],  
49                 "status": "completed",  
50                 "image_urls": image_urls # 发送图片URL给前端  
51             }  
52         )  
53  
54     logger.info("---node_answer_output 节点处理结束---")  
55     return state
```

9.7.2.3 检查答案 (step_1_check_answer)

```
1 def step_1_check_answer(state) -> bool:  
2     """  
3     阶段一：检查 state 中是否已有 answer。  
4     - 若已存在：按需推送流式 delta（用于 SSE），并返回 True  
5     - 若不存在：返回 False  
6     """  
7     answer = state.get("answer", None)  
8     is_stream = state.get("is_stream")  
9     if answer:  
10         if is_stream:  
11             logger.info("---Step 1: 发现已有答案，执行流式推送---")  
12             push_to_session(state["session_id"], SSEEvent.DELTA, {"delta": answer})  
13         else:  
14             set_task_result(state["session_id"], "answer", answer)  
15         return True  
16     else:  
17         return False
```

9.7.2.4 构建 Prompt (step_2_construct_prompt)

定义外部提示词文件：

位置：prompts/answer_out.prompt

```
1 你是一个智能助手，请根据参考内容回答用户的问题。
2 要求：
3 1 尽量基于【参考内容】和【用户问题】 作答，不要编造不存在的事实。
4 2 如果用户的问题需要通过图片来辅助说明（例如：外观、结构、接线、示意图等），图片只能来自于本地切片文本中的图片，请在答案最后追加一个独立的图片区
   块，格式严格如下：
5  【图片】
6  <图片URL1>
7  <图片URL2>
8  （ 每行一个URL；如果没有合适图片则不要输出【图片】区块）
9
10 【参考内容】
11 {context}
12
13 【历史对话】
14 {history}
15
16 【相关商品/实体】
17 {item_names}
18
19 【用户问题】
20 {question}
21
22 请回答：
```

定义提示词方法实现：

这段代码的核心功能是“**为大模型准备食材**”：

- 挑选食材：从已经按相关性排好序的文档列表 (reranked_docs) 中，依次取出文档。
- 清洗与摆盘：
 - 提取文档的正文 (text)。
 - 贴上标签 (meta_parts)：比如这是第几条、来自哪里（本地/网络）、相关性得分多少、标题是什么。这样大模型在引用时知道出处。
 - 拼装成一段完整的文本块。
- 控制分量：累加所有拼装好的文本长度，一旦超过最大限制 (MAX_CONTEXT_CHARS ，比如 12000 字)，就停止添加。这是为了防止把大模型“撑死”（超出 Token 限制）。
- 打包：最后把所有选中的文档用换行符拼成一个长字符串 (context_str)，塞进 Prompt 里发给大模型。

```
1  # 目标结构
2  # HAK 180 烫金机的操作面板位于机器正前方。开启电源后，您需要先设置温度，默认建议设置在 110℃ 左右。
3  # 具体的按键位置请参考下图：
4  # 【图片】
5  # http://local-server/images/panel_view.jpg
6  # http://local-server/images/button_detail.jpg
7  def step_2_construct_prompt(state: QueryGraphState) -> str:
8      """
9      第一阶段：构建 Prompt
10     根据state中的问题、重新问题、历史对话、提问商品 ( item_names )、 重排内容 组织prompt
11     """
12     # 1. 获取相关信息
13     original_query = state.get("original_query", "")
14     rewritten_query = state.get("rewritten_query", "")
15     # 优先使用重写后的问题
16     question = rewritten_query if rewritten_query else original_query
17     history = state.get("history", [])
18     item_names = state.get("item_names", [])
19     reranked_docs = state.get("reranked_docs") or []
20
21     # 2 从重排内容中，提取为资料字符串，不可超过限额
22     # 优先使用结构化 reranked_docs ( 包含 source/chunk_id/url/score )，便于约束与引用
23     # -----
24     # 逻辑解释：
25     # 1. 遍历重排序后的文档列表 (reranked_docs)，这些文档已经按相关性从高到低排序。
26     # 2. 对每个文档提取关键信息 (text, source, chunk_id, url, title, score)。
27     # 3. 构造 "元数据头 + 正文" 格式的字符串，例如：
28     #     "[1] [local] [chunk_id=123] [score=0.95] [title=操作手册]
29     #     这里是文档的正文内容..."
30     # 4. 累加字符长度，如果超过 MAX_CONTEXT_CHARS (如 12000 字符)，则停止添加，
31     #     确保 Prompt 长度在 LLM 的处理范围内，避免 Token 溢出。
32     # -----
```

```
33 docs = []
34 used = 0
35 for i, doc in enumerate(reranked_docs, start=1):
36     text = (doc.get("text") or "").strip()
37     if not text:
38         continue
39     source = doc.get("source") or ""
40     chunk_id = doc.get("chunk_id")
41     url = (doc.get("url") or "").strip()
42     title = (doc.get("title") or "").strip()
43     score = doc.get("score")
44
45     meta_parts = [f"[{i}]"]
46     if source:
47         meta_parts.append(f"[{source}]")
48     if chunk_id:
49         meta_parts.append(f"[chunk_id={chunk_id}]")
50     if url:
51         meta_parts.append(f"[url={url}]")
52     if score is not None:
53         # 保留四位小数
54         meta_parts.append(f"[score={float(score):.4f}]")
55     if title:
56         meta_parts.append(f"[title={title}]")
57     doc = " ".join(meta_parts) + "\n" + text
58     if used + len(doc) > MAX_CONTEXT_CHARS:
59         break
60     docs.append(doc)
61     # 计算使用长度！ + 2 两个\n\n
62     used += len(doc) + 2
63 context_str = "\n\n".join(docs) if docs else "无参考内容"
64
65
66 # 3. 格式化 History (历史对话)
67 # -----
68 # 逻辑解释：
69 # 1. 遍历历史对话记录 (history)。
70 # 2. 将每轮对话格式化为 "用户：... \n 助手：..." 的文本块。
71 # 3. 同样进行长度累加判断 (used)，确保历史记录+参考文档的总长度不超过 MAX_CONTEXT_CHARS。
72 # 注意：这里的 used 变量是接着上面处理文档后的长度继续累加的，
73 # 意味着如果文档占用了太多 Token，历史记录可能会被截断或完全丢弃。
74 # -----
75 history_str = ""
76 if history:
77     for msg in history:
78         # 修正：MongoDB存储格式为 {"role": "user"/"assistant", "text": "..."}
79         role = msg.get("role")
80         text = msg.get("text")
81         if role == "user" and text:
82             history_str += f"用户：{text}\n"
83         elif role == "assistant" and text:
84             history_str += f"助手：{text}\n"
85
86         used += len(history_str) + 2
87         if used > MAX_CONTEXT_CHARS:
88             break
89 else:
90     history_str = "无历史对话"
91
92 # 4. 格式化 Item Names (提问商品)
93 item_names_str = ", ".join(item_names) if item_names else "无指定商品"
94
95 # 5. 组装 Prompt
96 prompt = load_prompt("answer_out",
97     context=context_str,
98     history=history_str,
99     item_names=item_names_str,
100     question=question
101 )
102
103 logger.info(f"组装后的提示词为：{prompt}")
104
```

9.7.2.5 生成回答 (step_3_generate_response)

```
1 def step_3_generate_response(state: QueryGraphState, prompt: str) -> QueryGraphState:
2     """
3     第二阶段：生成回答
4     调用llm生成答案，支持流式输出
5     """
6     logger.info("---Step 3: 开始生成回答 (LLM Generation)---")
7     logger.debug(f"最终Prompt内容: {prompt}")
8
9     # 获取 LLM 客户端
10    # 注意：这里我们使用统一的 get_llm_client 获取实例
11    llm = get_llm_client()
12
13    # 判断是否需要流式输出
14    # 通常 state 中会注入 stream_queue 用于 SSE 推送
15    session_id = state.get("session_id")
16    is_stream = state.get("is_stream")
17
18    if is_stream:
19        logger.info(f"模式: 流式输出 (Streaming), Session: {session_id}")
20        final_text = ""
21        try:
22            # 使用 stream 方法进行流式生成
23            for chunk in llm.stream(prompt):
24                delta = getattr(chunk, "content", "") or ""
25                if delta:
26                    final_text += delta
27                    # 将增量内容放入队列
28                    push_to_session(session_id, SSEvent.DELTA, {"delta": delta})
29
30            logger.info(f"流式输出完成，总长度: {len(final_text)}")
31
32        except Exception as e:
33            logger.error(f"流式生成出错: {e}", exc_info=True)
34            # 发生错误时，尝试推送到前端
35            push_to_session(session_id, SSEvent.ERROR, {"error": str(e)})
36
37        state["answer"] = final_text
38    else:
39        # 非流式直接调用
40        logger.info(f"模式: 非流式输出 (Blocking), Session: {session_id}")
41        try:
42            response = llm.invoke(prompt)
43            content = response.content
44            state["answer"] = content
45            set_task_result(session_id, "answer", content)
46            logger.info(f"生成回答完成，长度: {len(content)}")
47        except Exception as e:
48            logger.error(f"生成回答出错: {e}", exc_info=True)
49            state["answer"] = "抱歉，生成回答时出现错误。"
50
51    return state
```

9.7.2.8 提取参考内容中的图片链接(_extract_images_from_docs)

在 node_answer_output.py 中，虽然 Prompt 要求大模型输出图片区块，但实际上代码里有一段“双保险”逻辑：

```
1 def _extract_images_from_docs(docs):
2     # ... 正则提取 markdown 图片链接 ...
```

这段代码并没有去解析大模型生成的那个【图片】区块，而是 直接从源文档（reranked_docs）里用正则提取图片链接。

为什么要这样做？

- 1. 大模型不可控：大模型可能会编造 URL，或者格式写错（比如漏了换行）。
 - 2. 源数据最准：直接从排好序的文档里提取图片，肯定是真的存在的图片。
- 所以，最终给前端的数据结构是：push_to_session 发送的 JSON：

```
1 {
2     "answer": "HAK 180 烫金机的操作面板位于...（大模型生成的纯文本）...",
3     "status": "completed",
4     "image_urls": [
5         "http://local-server/images/panel_view.jpg",
6         "http://local-server/images/button_detail.jpg"
7     ]
8 }
```

- answer : 大模型的回答文本。
 - image_urls : 代码自己从文档里提取出来的图片列表（不是大模型生成的）。
- 结论： Prompt 里的【图片】要求主要是为了让大模型在 文字回答中能够提及图片（比如“请看下图”），起到引导作用。但最终传给前端展示图片的，其实是代码里那个靠谱的 _extract_images_from_docs 函数提取逻辑

```
1 def _extract_images_from_docs(docs):
2     """
3     辅助方法：从文档列表中提取图片URL
4
5     核心逻辑：
6     1. 遍历所有相关文档（包括本地知识库切片和联网搜索结果）。
7     2. 策略一：直接检查文档的 'url' 字段（常见于联网搜索结果）。
8         - 验证后缀名是否为图片格式（.jpg，.png 等）。
9     3. 策略二：使用正则表达式扫描文档 'text' 正文内容（常见于本地 Markdown 文档）。
10        - 匹配 Markdown 图片语法: ![alt text](image_url)。
11    4. 对提取到的 URL 进行去重处理，返回唯一图片列表。
12
13    :param docs: 文档列表，每个文档为字典格式
14    :return: 图片 URL 字符串列表
15    """
16    images = []
17    seen = set() # 用于去重，避免同一张图片重复出现
18    if not docs:
19        return []
20
21    # -----
22    # 正则表达式解释: r'!\[.*?\]\((.*?)\)'
23    # 1. !\[    -> 匹配 Markdown 图片语法的开头 "![["（注意 [ 需要转义）
24    # 2. .*?    -> 非贪婪匹配图片描述文本 (Alt Text)，即 [] 中间的内容
25    # 3. \]     -> 匹配描述文本的结束符 "]"
26    # 4. \(     -> 匹配 URL 部分的开始符 "("
27    # 5. (.*?) -> 捕获组 (Group 1)：非贪婪匹配括号内的实际 URL 内容
28    # 6. \)     -> 匹配 URL 部分的结束符 ")"
29    # ( ... ) （不带反斜杠）：这就是 捕获组 。
30    # 它的作用是告诉程序：“虽然我匹配了整个 ![...](...) 结构，但我 只要 这括号里的内容”。
31    # -----
32    md_img_pattern = re.compile(r'!\[.*?\]\((.*?)\)')
33
34    logger.info(f"开始提取图片，待处理文档数: {len(docs)}")
35
36    for i, doc in enumerate(docs):
37        # 1. 优先检查 url 字段（主要针对 Web Search 结果）
38        url = (doc.get("url") or "").strip()
39        if url:
40            # 简单后缀判断：确保是静态图片资源
41            if url.lower().endswith(('.png', '.jpg', '.jpeg', '.gif', '.webp', '.bmp', '.svg')):
42                if url not in seen:
43                    logger.debug(f"文档[{i}] 发现图片 URL (字段): {url}")
44                    seen.add(url)
45                    images.append(url)
46
47        # 2. 检查 text 字段中的 Markdown 图片（主要针对 Local Chunk）
48        text = (doc.get("text") or "").strip()
49        if text:
50            # findall 机制解释：
51            # 正则表达式 r'!\[.*?\]\((.*?)\)' 中包含一个捕获组 (.*?)
52            # 当存在捕获组时，findall 只返回括号内匹配到的内容（即 URL），而不是整个 ![...](...) 字符串
53            # 示例：
54            # 输入 text: "参考图片 ![面板图](http://img.com/1.jpg) 如下"
55            # 返回 matches: ['http://img.com/1.jpg']
56            matches = md_img_pattern.findall(text)
57            for img_url in matches:
58                img_url = img_url.strip()
```

```
58         if img_url and img_url not in seen:
59             logger.debug(f"文档[{i}] 正文发现 Markdown 图片: {img_url}")
60             seen.add(img_url)
61             images.append(img_url)
62
63     logger.info(f"图片提取完成, 共找到 {len(images)} 张唯一图片: {images}")
64     return images
```

9.7.2.9 写入历史 (step_4_write_history)

```
1 def step_4_write_history(state: QueryGraphState, image_urls = None) -> QueryGraphState:
2     """
3     阶段四：把本轮答案写入 MongoDB history。
4     利用 utils/mongo_history_utils.py 中的 save_chat_messages 方法。
5     """
6     session_id = state.get("session_id", "default")
7     answer = (state.get("answer") or "").strip()
8     item_names = state.get("item_names") or []
9
10    try:
11        if answer:
12            save_chat_message(
13                session_id=session_id,
14                role="assistant",
15                text=answer,
16                rewritten_query="",
17                item_names=item_names,
18                image_urls=image_urls,
19                message_id=None
20            )
21    except Exception as e:
22        # 写历史失败不应影响主链路
23        logger.error(f"写入Mongo历史记录失败: {e}")
24
25    return state
```

9.7.3 节点主流程测试 (node_answer_output)

```
1 if __name__ == "__main__":
2     print("\n" + "="*50)
3     print(">>> 启动 node_answer_output 本地测试")
4     print("="*50)
5
6     # 1. 构造模拟数据
7     # 模拟重排序后的文档列表 (reranked_docs)
8     # 包含：本地文档（带Markdown图片）、联网结果（带URL字段）、纯文本文档
9     mock_reranked_docs = [
10         {
11             "chunk_id": "local_101",
12             "source": "local",
13             "title": "HAK 180 烫金机操作手册_v2.pdf",
14             "score": 0.95,
15             "text": """
16             HAK 180 烫金机的操作面板位于机器正前方。
17             开启电源后，您需要先设置温度，默认建议设置在 110°C 左右。
18             具体的操作面板布局请参考下图：
19             ![操作面板布局图](http://local-server/images/panel_view.jpg)
20
21             如果是进行局部烫金，请调节侧面的旋钮。
22             ![侧面旋钮细节](http://local-server/images/knob_detail.png)
23             """
24         },
25         {
26             "chunk_id": None,
27             "source": "web",
28             "title": "HAK 180 常见故障排除 - 官网",
29             "score": 0.88,
30             "url": "http://example.com/hak180_troubleshooting.jpeg", # 这是一个直接指向图片的URL（虽然少见，但用于测试提取）
31             "text": "如果机器无法加热，请检查保险丝是否熔断..."
32         },
33         {
```

```
34         "chunk_id": "local_102",
35         "source": "local",
36         "title": "安全注意事项",
37         "score": 0.82,
38         "text": "操作时请务必佩戴隔热手套，避免高温烫伤。"
39     }
40 ]
41
42 # 模拟历史记录
43 mock_history = [
44     {"role": "user", "text": "你好，这款机器怎么用？"},
45     {"role": "assistant", "text": "您好！请问您具体指的是哪一款机器？"},
46     {"role": "user", "text": "HAK 180 烫金机"}
47 ]
48
49 # 模拟输入状态
50 mock_state = {
51     "session_id": "test_answer_session_001",
52     "original_query": "HAK 180 烫金机怎么操作？",
53     "rewritten_query": "HAK 180 烫金机的具体操作步骤和面板设置方法",
54     "item_names": ["HAK 180 烫金机"],
55     "history": mock_history,
56     "reranked_docs": mock_reranked_docs,
57     "is_stream": False, # 测试非流式
58     # "is_stream": True, # 若要测试流式，需确保 SSE 环境或 mock 相关函数
59     "answer": None # 初始无答案
60 }
61
62 try:
63     # 运行节点
64     result = node_answer_output(mock_state)
65
66     print("\n" + "="*50)
67     print(">>> 测试结果摘要:")
68
69     # 1. 验证 Prompt 构建
70     if "prompt" in result:
71         print(f"[PASS] Prompt 构建成功 (长度: {len(result['prompt'])})")
72         # print(f"Prompt 预览:\n{result['prompt'][:200]}...")
73     else:
74         print("[FAIL] Prompt 未构建")
75
76     # 2. 验证答案生成
77     answer = result.get("answer")
78     if answer and len(answer) > 10:
79         print(f"[PASS] 答案生成成功 (长度: {len(answer)})")
80         print(f"答案预览: {answer[:50]}...")
81     else:
82         print(f"[WARN] 答案生成可能异常 (Content: {answer})")
83
84     # 3. 验证图片提取
85     # 我们期望提取到 3 张图片：
86     # 1. http://local-server/images/panel_view.jpg (来自 local_101)
87     # 2. http://local-server/images/knob_detail.png (来自 local_101)
88     # 3. http://example.com/hak180_troubleshooting.jpeg (来自 web 结果的 url 字段)
89
90     # 注意：这里我们没办法直接从 result state 里拿到 image_urls，因为它是作为 SSE 推送出去的，或者存库了
91     # 但我们可以通过日志观察 _extract_images_from_docs 的输出
92     # 如果需要验证，可以临时修改 node_answer_output 返回 image_urls
93     print("\n[INFO] 请检查上方日志中是否包含 '图片提取完成' 及以下 URL:")
94     print(" - http://local-server/images/panel_view.jpg")
95     print(" - http://local-server/images/knob_detail.png")
96     print(" - http://example.com/hak180_troubleshooting.jpeg")
97
98     print("="*50)
99
100 except Exception as e:
101     logger.exception(f"测试运行期间发生未捕获异常: {e}")
```

9.7.4 实现效果

掌柜智库知识库

在线 · 可查询知识库与联网信息

流式输出

API: 未连接

清空对话

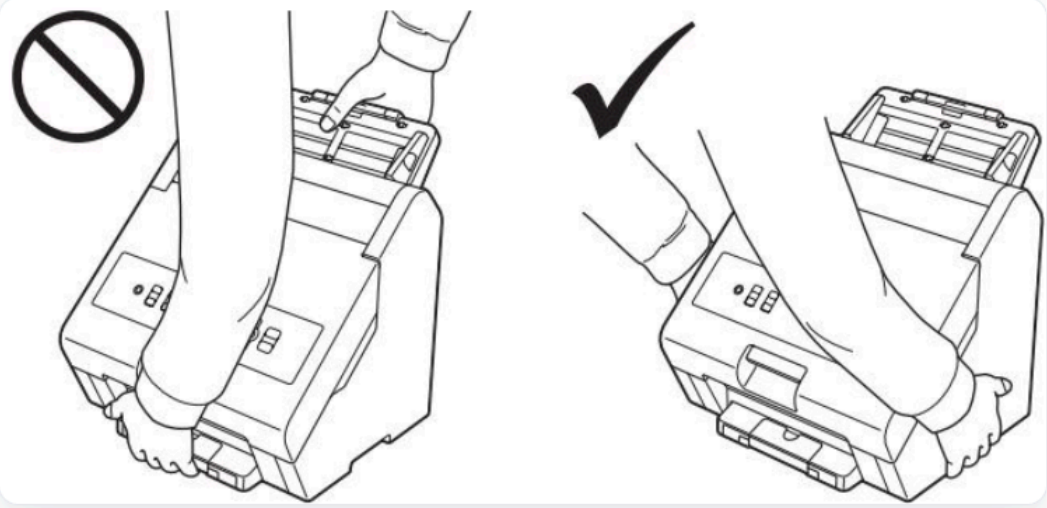
烫金机 警告标签 请勿撕下或损坏设备上的任何注意/ 警告标签以及序列号标签。 # 设备保修和责任 本手册中的任何内容都将不会影响现有设备保修，也不应被视为授予任何其他设备保修

我

根据【参考内容】，设备上的警告标签和序列号标签若被撕下或损坏，将与设备保修和责任直接相关。具体关联如下：

本手册中的任何内容都不会影响现有设备保修，也不应被视为授予任何其他设备保修。然而，不遵循本手册中的安全说明（包括撕下或损坏警告标签、序列号标签等），可能导致设备的保修失效。

因此，保持警告标签和序列号标签完整，是确保设备在正常使用条件下享有有效保修的重要前提。一旦标签被破坏，可能被视为用户未遵守安全操作规范，从而导致保修条款不适用。



<http://47.94.86.115:9000/knowledge-base-files/upload-images/hak180产品安全手册/7db61f15d3ae0e28247ed4a75f5db9f745349d4f1a8deb653c501496a1f06f88.jpg>

请输入问题（Enter 发送，Shift+Enter 换行）

发送

快捷键:

Enter

 发送,

Shift + Enter

 换行